

RAFAEL CHINELATO DEL NERO

Java



CHALLENGERS

CONTAINS
JAVA 16
CHALLENGES



70+ CODE CHALLENGES

Java Challengers

Challenge Yourself With Fun Java Code Challenges

Rafael Chinelato del Nero

This book is for sale at <http://leanpub.com/javachallengers>

This version was published on 2022-01-07



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2021 - 2022 Rafael Chinelato del Nero

Tweet This Book!

Please help Rafael Chinelato del Nero by spreading the word about this book on [Twitter](#)!

The suggested tweet for this book is:

I just bought the Java Challengers book! It's time to have fun challenging myself with the Java Code Challenges and refine my Java skills!

The suggested hashtag for this book is [#JavaChallengers](#).

Find out what other people are saying about the book by clicking on this link to search for this hashtag on Twitter:

[#JavaChallengers](#)

Contents

1 Introduction	1
1.2 Why Java?	1
1.3 What kinds of concepts are in this book?	2
1.4 Who is this book for?	2
1.5 What are the Java Challenges?	3
1.6 Disclaimer about the Java Challenges	3
1.7 Java versioning and tools	5
1.8 Will this book prepare you for the software development market?	6
2 Variable types and flow control (Review)	8
2.1 A Brief Review about Algorithms	8
2.2 Arithmetic Operators	15
2.3 Flow Control	18
2.4 Loops	28
2.5 Arrays	32
2.6 Local variable restrictions	37
2.7 Candy Price Code Challenge	43
2.8 Summary	44
3 Basic object-oriented programming (Review)	46
3.1 Classes	48
3.2 Methods	54
3.3 The Keyword this	57
3.4 Constructors	61
3.5 Object references	65
3.6 Arrays are objects in Java	69
3.7 Bank Deposit/Withdraw Challenge	72
3.8 Summary	74
4 Encapsulation access modifiers, and package structure	75
4.1 Programming to interfaces	75
4.2 What are and how to use getters and setters?	76
4.3 The Builder Design Pattern	78
4.4 Organizing Packages in a Real Project	81

CONTENTS

4.5 Encapsulation Patterns	84
4.6 Access Modifiers	86
4.7 Inner Class	93
4.8 The static keyword	97
4.9 Instance and static block	104
4.10 Small Real-World Project Challenge	107
4.11 Summary	109
5 Overloading	111
5.1 Overloading basics	111
5.2 Wrappers and autoboxing	118
5.3 Varargs	122
5.4 Overloading Real-World Usage Challenge	126
5.5 Summary	129
6 Inheritance and Polymorphism	130
6.1 When to use inheritance	131
6.2 Every Class is an Object	135
6.3 Checking the type of an object	137
6.4 The toString method	139
6.5 Access modifiers and inheritance	140
6.6 Object composition	142
6.7 Multiple inheritance in Java	145
6.8 Using super to access a parent class's methods	146
6.9 Constructors and inheritance	147
6.10 Class casting	150
6.11 Preventing inheritance	152
6.12 Abstract classes	154
6.13 Factory Method Strategy Discount Real-world Challenge	159
6.14 Summary	160
7 Interfaces	162
7.1 Interfaces and Polymorphism	162
7.2 Covariant return types	164
7.3 Default methods	167
7.4 Abstract classes vs. interfaces	168
7.5 Static methods in interfaces	171
7.6 Simulating multiple inheritance with default methods	172
7.7 Private methods in interfaces	177
7.8 Command Design Pattern Challenge	179
7.9 Summary	179
8 Exceptions	181
8.1 Checked and unchecked exceptions	181

CONTENTS

8.2 Stack Trace	185
8.3 Handling or Declaring Checked Exceptions	187
8.4 try, catch, finally	190
8.6 try with resources	194
8.7 Multi catch	199
8.8 Creating a customized exception	204
8.9 Throw early, catch late	209
8.10 Real World Exception Creation Challenge	211
8.11 Summary	211
9 Lambdas and Functional Interfaces	213
9.1 What is Lambda?	213
9.1.3 Mysterious Door Lambda Challenge	215
9.2 Functional Interfaces	217
9.3 Method Reference	227
9.4 Lambda Method Reference Matcher Challenge:	234
9.5 Summary	238
10 Optional	239
10.1 Wrapping a value into Optional	239
10.2 isPresent and isEmpty	240
10.3 Optional Antipatterns	243
10.4 ifPresent and ifPresentOrElse	247
10.6 Handling Exceptions with orElseThrow	252
10.7 Transforming Optional Data	256
10.8 Final Yoshi Food Optional Challenge	262
10.9 Summary	264
11 Generics and Object Comparison	265
11.1 Comparing objects with equals and hashCode	265
11.2 Generics	272
11.4 Upper and Lower Bound Generics	281
11.5 Summary	288
12 Collections	289
12.1 Collections API	289
12.2 List	290
12.3 ArrayList	295
12.4 Vector	299
12.5 Deque & Stack	300
12.6 ConcurrentModificationException	306
12.7 Using Comparable	309
12.8 Set	316
12.9 HashSet	317

CONTENTS

12.10	LinkedHashSet	320
12.11	TreeSet	322
12.12	Map	325
12.16	TreeMap	338
12.17	Elements Searching	344
12.18	Summary	346
13	Streams	348
	Creating a Stream	348
	Intermediate Operations	351
	Terminal Operations	359
	Transforming Streams	364
	Processing Order	364
	Parallel Stream	364
	Summary	364
13	Newest Features of Java (In Development)	365
13.1	Introduction to Modules Java 9	365
13.2	Reserved word var from Java 10	365
13.3	New switch case statement from Java 14	365
13.4	Using record from Java 14	365
13.5	Using Sealed classes Java 15	365
13.6	Using Pattern Matching Type Checks Java 15	365
13.7	Text Blocks Java 15	365
13.8	Summary	365

1 Introduction

Java is one of the most used programming languages in the world, and it's also very powerful. Developers who know Java in depth are highly valued by the market and able to create high-quality, maintainable code. Java is not a simple language to master, but this book will provide valuable guidance for developers who want to harness its full power and be a reference as a software engineer.

Even senior developers may not be familiar with some key concepts of the Java language, and that will impact the quality of the code they write. But reading a book is not enough to really acquire a skill. Developers have to put their knowledge into practice by applying what they've learned in some way.

This book provides at least 5 Java Code Challenges per chapter to test your understanding of the and trully absorb the concepts it introduces. You'll get up-to-date with the most recent Java versions and will learn Java from real project situations so that when you come across similar situations you will know how to respond effectively.

As the name suggests, Java Challengers is not an easy book, but it will guide you through the concepts and won't leave gaps that will make learning difficult. Although the subjects the book presents are complex, the concepts are explained in a practical way, aiming to facilitate the learning process of the reader.

After you read, consider, and try to solve each challenge, you can test the code provided and run your own tests by changing it and analyzing the output. This will help you connect the dots with the content you consume in the book.

You'll then be able to use the concepts you've learned in your day-to-day work, which is the whole point of going deep into the concepts of a programming language.

To produce highly maintainable Java code you have to use the most suitable features of the language. Then bugs have a really hard time hiding. This increases the satisfaction of developers, managers, and customers with the delivered product, bringing more profit to the company. Everyone wins.

It's definitely worth it to invest time in a deep dive into the Java language, and that's exactly what this book will provide. So, what are you waiting for? Are you ready for all the challenges? How many of them can you get right? Are you prepared to take your Java skills to a whole new level?

1.2 Why Java?

Java is the world's most popular programming language, and a developer who knows Java will rarely be unemployed. Many companies opt to use this language to make hiring new developers an easier task—its popularity means it's easier to find professionals who know Java.

Reliability, maturity, and robustness are the essence of the language, and companies of all sizes choose Java because of those qualities.

Java also has a large ecosystem of open source projects that help programmers with their day-to-day development, many of which are free.

With Java it's possible to create organized, structured code with reliable unit tests that help developers build robust software that's resistant against bugs.

1.3 What kinds of concepts are in this book?

You will learn about core concepts from the Java language that will be useful in your day-to-day work. The book covers crucial concepts such as polymorphism, encapsulation, overloading, inheritance, composition, exceptions, Optionals, generics, concurrency, equals and hashCode, collections, streams, functional programming with lambdas and method references, and the main concepts from the newest Java versions (since this book will be updated from time to time).

It makes a big difference if a Java developer really understands these concepts, and that's why they are explored in this book.

1.4 Who is this book for?

Java Challengers is aimed at developers who want to create high-quality code in their day-to-day work. With a consolidated knowledge of the core concepts you will be able to use the full power of Java, applying the most suitable features to the situations you are facing.

Developers who like solving challenging puzzles will also enjoy the book—the challenges aren't easy, but they are fun.

You don't have to be an experienced developer to get value out of this book. Developers who have at least some knowledge of the basics of programming logic and algorithms should be able to understand the book, although familiarity with object-oriented programming is also recommended.

1.4.1 Who is this book not for?

Java Challengers covers most of the core certification content, but this book is not focused on certification. You can use it as supporting material, and if your goal is Java certification you can get extra material such as mock exams, check tutorials, articles, or other books. It's actually a good idea to mix your learning sources, we learn much better a subject when reading, listening, watching and doing something useful with the learned concepts.

This also may not be the right book for developers who are looking for a general introduction but aren't focused on creating high-quality code and becoming experts. The point of going deep into

Java concepts is to give developers the power to use the full capabilities of Java by applying the most suitable features of the language to the problem at hand.

Finally, developers who just want to write "code that works" without going any deeper shouldn't read this book.

1.4.2 What is covered in the book?

Many features that are infrequently used in real projects have not been included in this book. For example, the Java JDBC (Database library) are not discussed because it's likely that in a real project you will be using some kind of persistence framework (a set of libraries that facilitates day-to-day work with a predefined and simpler structure) such as Hibernate, Spring Data, or others.

Instead, core features of Java are explored in a way that they can be totally absorbed and used as best as possible. Reinventing the wheel is one of the worst things that can happen in software development. If you know the core features of Java well, you won't take the risk of using the wrong feature to solve a problem. You will be using Java masterfully.

1.5 What are the Java Challenges?

The Java Challenges are challenging code puzzles that require you to know the relevant Java concepts very well. Most of the challenges include the `main` method, and your goal is to guess what will happen when running the code.

The Java code challenges are carefully designed to explore the most important Java concepts that will make a difference in your day-to-day work as a Java developer. Many of them feature well-known shows and characters, and expect to see funny `class/method/variable` names.

If you get the challenges right, that's great! But if not, don't get frustrated; try to understand why, and eventually you will absorb the concepts. The point of the book is to deepen your understanding of Java by having fun with the challenges.

You will also find a real-world challenge at the end of each chapter which will get you better prepared for applying suitable solutions to problems you might face in real projects. These will help you with interview problem-solving questions too.

The real-world challenge will usually be a situation where you must apply the concepts learned in the chapter to solve a common problem. Many of them involve established design patterns and unit tests, and solutions are available so you can check if your code is correct.

1.6 Disclaimer about the Java Challenges

The intention of the Java challenges is to test your understanding of the concepts explained throughout the book, so don't expect clean code in every challenge. Instead, see the challenges as a way to fix the concepts in your mind and practice your code analyzing skills.

The better you are able to analyze code, the faster and sharper programmer you will become. These exercises will enable and empower you to produce flexible and meaningful code using the most appropriate Java concepts. The more features and concepts from Java you master, the better able you will be to track down bugs and fix them efficiently. In addition, you will be able to produce highly optimized solutions for many different situations.

1.6.1 Running a Java application without a class and main method

As of Java 9, you don't need to create a class or a `main` method for simple code tests. You can use the command line with `jshell`. It's a powerful tool when you want to run something quickly to test syntax or code logic. You can implement the code examples in this book using `jshell` if you wish.

Make sure you have Java 9 or later installed on your machine by running the following command on your command terminal:

```
java --version
```

Then configure the environment variable `JAVA_HOME` and add `JAVA_HOME\bin` to your operating system environment variable `PATH`.

Just a brief review about the `PATH` environment variable, this path will enable you to run commands globally on your command terminal.

We have two options when running the java command lines, either we point our current directory in the command terminal to `JAVA_HOME\bin` or we inform our OS that we want to use the commands located in `JAVA_HOME\bin` globally without having to change your current directory to the `JAVA_HOME\bin` every time we want to use java commands.

Once you've done that, you can start JShell using the following command:

```
jshell
```

Then you can write Java code without the obligation of declaring a class and a `main` method, and execute it immediately from the command line. It's a great option to practice the examples in the book, but any way you choose to write your code is fine. What really matters is you doing your own tests with as many examples as you can from this book.

1.6.1 Where to run the Java applications

You can write Java code in a Notepad file, at the command line, and so on, but of course that wouldn't be productive for big, complex projects. It would be very hard to organize and produce all the Java files required by the system!

Fortunately, there are tools that can help us in our day-to-day work, known as integrated development environments (IDEs). Of the top Java IDEs, the one

I like the most is IntelliJ. It's practical and easy to use, has great integration with most common Java technologies, and is very performant.

Another option is `Eclipse`, which has the best set of programming keyboard shortcuts but in my opinion is not so good at integrating and building complex systems. A third popular IDE is `NetBeans`; I haven't used it much but it's generally regarded as a good IDE for `Java` and it will do the job.

IDE choice is very personal, so pick the IDE you like the most—after all, what matters most is the `Java` code, not where the code was written!

1.6.2 Golden tips to solve the Java Challenges

The challenges are tricky, and they will demand that you analyze the code carefully so that you can guess the output.

Before deciding on a solution to a challenge, you should consider every alternative to see if there is another one that makes sense. If so, analyze the code again and make your best guess.

It can be tempting to go straight to the obvious answer. Before all the challenges there will be a detailed explanation covering everything you need to answer the problem correctly.

If you feel confident that you already know the concept you can try the challenge before reading through the description; if you get the answer wrong, you can always go back to the concept's explanation to try to find out what happened.

Another crucial action for taking full benefit of this book is to run the code in a challenge and change it so that you can test the outputs by yourself. This is a good way to be sure that you've absorbed the main concepts.

Some persistence will be required, but if you run the code enough times, the concepts will be clear and consolidated.

1.7 Java versioning and tools

As you read the book, it's highly recommended that you test the code to see what happens with each example. All the code is stored in `GitHub`. You can download the project if you don't have knowledge of `Git`, or you can simply use the `Git` commands if you know them.

`Git` is a widely used tool for storing and versioning code, and familiarity with it is a **must** for every software engineer. If you don't know `Git` yet, it's time to practice it. You can learn the basic commands and run the `Java Challenges` code.

You will find all the code for this book, from examples to the challenges, at <https://github.com/rafadelnero/javachallengersbook>.

You'll see that the project includes a build manager tool called `Maven`. `Maven` provides an automated way to build, test, deploy, and manage code dependencies that makes our lives as software engineers easier.

You will have to import the project as a Maven project, and then you will be able to run the code from the book. There are also some unit tests included so you can learn a bit about how tests should be applied.

In working through the challenges, you will be able to use and learn about a variety of popular tools.

1.8 Will this book prepare you for the software development market?

A deep knowledge of Java is the most important knowledge every Java developer should have in their toolbox, because it's from code that all the business logic will be built. If developers write high-quality code it will require less maintenance, which translates to more profit for the company and less stress for everyone.

The Java knowledge on its own is great but you need to expand more your knowledge-base to be highly effective. You don't need to know every other single technology very well, instead, you need to know enough to solve a problem in a real-project.

Also, be careful to not try to embrace the world. If you start learning several technologies at the same time, it might be overwhelming and you will be very likely to be frustrated and eventually give up.

One culture that is expanding in the market is DevOps. That's because many projects are being moved to a cloud platform, and many automation/container technologies come along with it (Jenkins, GitHubActions, Docker, Kubernetes). Instead of manually making deployments, testing, and checking code quality, we can simply use DevOps tools to automate the whole process of software delivery.

When we talk about DevOps, a crucial skill to have is the Linux OS, I highly encourage you to use Linux on your machine or if you have a Mac OS it will be better for you to get familiarized with command lines. If you are a Windows user though, you can still use a Linux subsystem, a dual-boot or a virtual machine with Linux too. Also, if you are a beginner on Linux, I recommend you to start with Ubuntu because it's a more user-friendly platform and at the same time a robust one.

The cloud era is in full swing, so we need to know at least a bit about technologies such as top cloud providers such as AWS, Google Cloud Platform, Azure, Oracle Cloud, a little bit about Infrastructure as Code technologies Terraform, Cloud Formation, or any other scripts to provision cloud services. A working knowledge of databases is also essential for every developer, because we often have to build complex SQL scripts to bring in the necessary data for the business. NoSQL is a big thing too.

It's also important to know the main concepts of the Microservices architecture. You will be asked about these concepts in an interview, so at least have an idea of the advantages and disadvantages of using Microservices.

Many projects in the market rely on full stack developers, which means that developers have to know at least the basics of a front-end technology.

Therefore frameworks such as Angular, React, and Vue are some of the big ones; this might change with time. All these technologies are based on Javascript, so it's a good idea to have a basic knowledge of the language as well.

The good thing is that by mastering Java concepts, you will be able to learn any other language far more easily because in the end you will simply use the same language concepts and paradigms you have from Java to learn another language and this process will be incredibly faster.

It's a lot to learn, isn't it? But don't worry, you don't need to be a specialist in all those technologies. You need a basic understanding of how to use them, and then you can choose one specific technology that you would like to explore in depth. If you're reading this book it means you've selected Java, which I think is a powerful choice.

Let's start going deep into Java!

2 Variable types and flow control (Review)

This chapter covers:

- Creating basic Java algorithms based on a problem
- Using suitable types of primitive variables
- Using different types of flow control instructions
- Using different types of loops
- Using unidimensional and bidimensional arrays
- Declaring and using local variables

The core purpose of every programming language is to control the flow of logic, making computations with different values and organizing information in a way that makes sense to the final user of the software. This chapter introduces several concepts that you can use to accomplish this objective, including variables (primitives and special types), conditions, loops, and arrays.

Using the information in this chapter, you will be able to develop your own algorithms to solve specific problems. For example, you could build a mathematical formula to make a calculation, construct logic to check if a password is correct with a condition, or create a program that will receive two numbers, multiply them, and print the result. Your imagination is the limit.

Your code might not have the robustness required by a real project, but you will be able to design the logic to solve specific problems. We'll start with an in-depth exploration of variables.

2.1 A Brief Review about Algorithms

Code algorithm is an organized and logical way to solve a particular problem through instructions where we get the output or result expected.

We are dealing with algorithms every day in our lives. When we go to a shop to buy what we need, we have to give our brain logical instructions to get the job done. We can go beyond that, when we are going to sleep, we also have to take some logical actions to sleep properly, any other common action we take every day can be seen as an algorithm.

Mathematical functions are another example of algorithms, in the computer, the difference is the way we pass the instructions. The instructions have to be in the language the computer understand that would be binary numbers but binary numbers are too hard to manipulate in a pure way, that's why we have programming languages that have powerful features to solve real-world problems

about financial businesses, entertainment, modern machines, mobile devices and the list goes on and on.

Let's then review the universal code concepts so that we can apply a logic algorithm effectively with Java or any other language.

2.1.1 Code Syntax

Code syntax is the way the compiler will understand the code, it's a predefined way that we must write our code. If we don't follow the pattern the JVM is expecting, the code won't compile.

We can try to use a variable without declaring the type for example, which Java won't accept:

```
intVariable = 7;
```

We have to always follow the code syntax to work with any programming language. Java for example has a very similar syntax from the C language.

2.1.2 Code Semantic

Everything in the real-world has to be done with a defined logic, when we want to brush our teeth we open the bathroom cabinet, then get the toothbrush, then put on the toothpaste, move the toothbrush into our mouth, brush it until our teeth get clean, wash the toothbrush, wash the mouth, then put the toothbrush back in the cabinet.

Notice that it wouldn't be possible to brush our teeth without following that logic, if we try to brush our teeth without the toothpaste for example, that wouldn't work.

Another example is when we are going to cook something, we have to follow a logic with the receipt, if we cook the ingredients in the wrong order or with the wrong ingredients, the receipt won't work as well.

Simply put, code semantic is the logic we have to implement to perform tasks.

2.1.3 Variables in Java: Primitive types and values

A variable in programming is a way to store a value into a named allocated space in the computer's memory. We can store text, binary, and numerical values in a variable and then make calculations, perform comparisons, and so on.

Java has two main types of variables: primitives and objects. Many of the primitive variable types are also used in other programming languages. If you've worked with C, for example, you will find that many of the variable types in Java are the same. We'll focus on the primitive types for now; Java objects will be introduced in the next chapter.

2.1.4 Primitive types

As mentioned previously, it's possible to store and manipulate characters, numbers, and binary values such as true and false with the variety of primitive variable types defined in Java. The eight Java primitives are listed in table 2.1. The table contains the following information:

- **Type:** The variable type, defined by the size of the allocated space in memory and the data structure (for example, if it's text or numbers).
- **Range:** The range of values that can be used for the variable type.

Default instance value—The default value of the variable when no value is assigned to it upon creation of a new instance. We will explore instances in the next chapter.

- **Size:** The size that will be allocated in memory for the variable type.

Example literals—Examples of values that can be used in this type of variable.

Type	Range	Default value	Size	Example literals
boolean	true or false	false	1 bit	true, false
byte	-128 .. 127	0	8 bits	1, -90, 128
char	Unicode character or 0 to 65,536	\u0000	16 bits	'a', '\u0031', '\201', '\n', 4
short	-32,768 .. 32,767	0	16 bits	1, 3, 720, 22000
int	-2,147,483,648 .. 2,147,483,647	0	32 bits	-2, -1, 0, 1, 9
long	- 9,223,372,036,854,775,808 .. 9,223,372,036,854,775,807	0	64 bits	-4000L, -900l, 10L, 700L
float	3.40282347 x 10 ³⁸ .. 1.40239846 x 10 ⁻⁴⁵	0.0	32 bits	1.67e200f, -1.57e-207f, .9f, 10.4F
double	1.7976931348623157 * 10 ³⁰⁸ .. 4.9406564584124654 * 10 ⁻³²⁴	0.0	64 bits	1.e700d, -123457e, 37e1d

Table 2.1.4 The primitive variable types defined in Java

Note that the primitive types in Java can never have the value null; if you don't assign them a value when you create them the JVM will assign them the default value listed in this table. This will only happen with an instance variable, a concept we will explore more deeply in future chapters. As you'll see later in this chapter, local variables are not assigned default values if one is not provided; you must assign them values when you create them, or your code will not compile.

2.1.5 Manipulating a variable in Java

Java is a strongly typed language and it's necessary to define the variable type before using it.

After declaring the variable we must assign some value, otherwise the variable existence doesn't make much sense. The most basic way to assign a value to a variable is by using the simple assignment operator, `=`. When we are assigning a value to a variable for the first time we say we are initializing it.

If we are using the primitive types for example, we need to use the following syntax to declare and assign the values to those variables.

```
1 int luckNumber = 7;
2 double weight = 70.0;
3 boolean isJavaRocker = true;
```

In Java 10 we also have the possibility to define a non-typed variable that can have any value type within it.

For example:

```
var luckNumber = 7;
```

The `luckNumber` variable will be inferred as an `int` type with the value of 7.

We can't try to declare a variable without giving a type like Javascript, the following code, for example, won't be compiled:

```
luckNumber = 7;
```

2.1.6 Variable Names Rules

There are some rules in order to give variable names. In Java, ideally we should simply follow the JavaBeans convention using the camelCase pattern. But it's also good to at least know what name we can use for variables.

Let's see at first valid examples of variable names in Java:

Case scenario	Valid variable names (but not recommended)
A variable name can start with A to Z	<code>int HomerAge = 35;</code>
A variable name can start with <code>_</code>	<code>int _homerAge = 35;</code>
A variable name can start with <code>\$</code>	<code>int \$homerAge = 35;</code>
After the first character, we can use numbers.	<code>int homerAge007 = 35;</code>

Let's see now invalid variable names in Java:

Case scenario	Invalid variable name
A Java keyword can't be used as a variable name.	<code>int double = 7;</code>
A variable can't start with a number.	<code>int 007HomerAge = 35;</code>
Special Characters other than \$ and _ for variable names.	<code>int *homerAge = 35;</code>
Special characters that are not \$ or _ in the middle of the name	<code>int homer-age = 35;</code>

One other very important detail about variable names with Java is that they are case-sensitive, which means that if the variable name has a capital letter when declared, we have to use the variable in the exact way, otherwise it won't compile.

```
double homerGrade = 0;
```

If we try to use the variable `homerGrade` as the following code, the code won't compile:

```
System.out.println(homergrade);
```

To use variables in Java, the naming case has to exactly match, if we use the `homerGrade` variable as the following, the code will compile successfully:

```
System.out.println(homerGrade);
```

Even though those are the rules to name Java variables, it's not a good practice to start a variable with a capital letter, with underscore (`_`), or a currency symbol (`$`).

Instead, we should follow the JavaBeans conventions which we will explore in the next session.

2.1.7 Variable naming conventions

Java uses a standard naming convention for variables so that it's clear to anyone viewing the code what the variables are. The convention it uses is `camelCase`, where the first word is lowercase and the first letter of each following word in the name is uppercase. For example:

```
1 simpsonName
2 beerFlavor
3 misterBurnsAge
```

Every Java developer should follow the JavaBeans convention for naming different elements of code. Following a defined and agreed-upon standard ensures your code is readable and understandable by a wide audience.



Note: Java Beans Definition

The JavaBeans specification is a set of guidelines agreed upon by the community in order to standardize Java code so that developers can understand one another's code more easily. When reading code, this helps a lot because we know instantly whether we are dealing with classes, methods, or variables.

If you wish to take a look at the Java Beans convention document, you can find it at <https://www.oracle.com/technetwork/articles/javaee/spec-136004.html>.

2.1.8 Incrementing and Decrementing Variables

There are different ways to assign a value to a variable in Java, and to increment or decrement the assigned value. Let's take a look at some of the operators that are available.

You can change a variable's value by assigning it a new value. Suppose you need to add a specific number to the value. One option is to do the following:

—

Note: `System.out.println` Purpose

We will be using the `System.out.println` method many times in the examples. This statement has many object oriented concepts that we will explore much deeper in further chapters. For now when you see this statement just keep in mind that it's used to print something on the console.

—

```
1 homerGrade = homerGrade + 2;
2 System.out.println(homerGrade);
```

The code above will print 4

You can also use Java's addition assignment operator (`+=`), specifying the amount to add to the variable's value. This will do exactly what was done in the previous example, adding 2 to the value of `homerGrade`:

```
1 int homerGrade = 2;
2 homerGrade += 2; // #1
3 System.out.println(homerGrade); // #2
```

- #1: This is the same as `homerGrade = homerGrade + 2`
- #2: This will print 4

Other versions of the assignment operator exist for decreasing the value of a variable (`-=`), multiplying or dividing it by another value (`*=` and `/=`), and more.

2.1.9 Increment operator

A common operation is to increment the value of an integer variable by one. In Java, you can do this with the following code:

```
1 int homerGrade = 0;
2 homerGrade = homerGrade + 1;
```

But there's a more concise option: using the increment operator, `++`. You can add one to a variable's value by including `++` before or after the variable's name in an expression. These are known as the pre-increment and post-increment operators, respectively.

Let's see how the post-increment operator works:

```
1  int homerGrade = 0;
2  System.out.println(homerGrade++);
3  System.out.println(homerGrade);
```

By running the above code with jshell, it will print:

```
1  0
2  1
```

As you can see, the value of the homerGrade variable is only incremented in the line following the increment operation.

The pre-increment operator acts in a different way—the variable’s value is incremented by one on the same line where the operator appears. For example:

```
1  int homerGrade = 0;
2  System.out.println(++homerGrade);
```

When running the code above with jshell, it will print:

```
1
```

With the pre-increment operator, the value is incremented first and then the result is computed. With the post-increment operator, the result is computed and then the value is incremented.

2.1.10 Decrement operator

The same rules are applied with subtraction, using Java’s decrement operator (–). Again, there are two versions. In the following code example, which uses the post-decrement operator, the value of the homerGrade variable will only be decremented at the next line:

```
1  int homerGrade = 2;
2  System.out.println(homerGrade--);
3  System.out.println(homerGrade);
```

Running this code will print:

```
1  2
2  1
```

When using the pre-decrement operator the value of the homerGrade variable will be decremented right away, before the result is computed:

```
1 int homerGrade = 2;
2 System.out.println(--homerGrade);
3 System.out.println(homerGrade);
```

This will print:

```
1 1
2 1
```

2.2 Arithmetic Operators

The arithmetic operators are one of the most core concepts for every programming language. When developing code, we are manipulating information all the time.

Let's see the most basic arithmetic operators in Java:

Operator	Description
*	Multiplication operator
/	Division operator
%	Remainder operator
+	Additive operator (also concatenates String)
-	Subtraction operator

Let's explore then the operators precedence. Like in math, multiplication and division have precedence over addition and subtraction. Let's see some examples:

```
System.out.println(5 + 5 * 5);
```

The result of the code above will be 30 because the multiplication operation is the priority and will be executed first. Considering that $5 * 5$ will be 25 and then will be added to 5.

Then let's analyze a similar example with division that is also a priority:

```
System.out.println(5 + 5 / 5);
```

The output will be:

```
6
```

The result will be 6 because $5 / 5$ will be executed first giving the result of 1 and then will be added to 5.

But what happens when there is division and multiplication in the same operation? Let's see that in the following example:

```
System.out.println(5 / 5 * 1);
```

The output will be:

```
1
```

In the code above, the priority will be decided by the order the operators were introduced. Therefore $5 / 5$ will be executed first with the result of 1 and then will be multiplied by 1 which will give us the output of 1.

Let's see now what happens when we use multiplication first:

```
System.out.println(5 * 5 / 1);
```

The output will be:

25

In the code above, $5 * 5$ will be executed first with the result of 25 and then will be divided by 1, therefore the final result will be 25.

There is also the remainder operator, let's quickly review how it works:

```
System.out.println(5 % 5);
```

The output will be:

0

The remainder of $5 / 5$ is 0, when we divide 5 by 5 there is no remainder value.

A very common use case for algorithm is to check whether a number is even or odd and we can accomplish that by using the remainder operator, let's see an example to check if a number is even:

```
System.out.println(5 % 2);
```

The output will be:

1

The calculation above returns 1 because that's the remainder number of the division between 5 and 2 which means that $2 * 2$ is 4 and 1 is the remainder. Therefore the number 5 is odd.

If we do a similar operation with an even number we have the remainder of 0.

```
System.out.println(4 % 2);
```

In code above $4 / 2$ is 2, therefore there is no remainder and also means that 4 is an even number.

The precedence of the remainder operator is the same as the multiplication and division, let's explore the following example:

```
System.out.println(5 + 4 % 2);
```

The output will be:

5

The first operation to be executed is $4 \% 2$, then we will have the result of 0 and will be added to 5. Therefore the final result will be 5.

There is a way to force the precedence of the calculations too, we can accomplish that by using parenthesis '()'. If we want for example to perform an addition operation before a multiplication, division or remainder operators, we can do the following:

```
System.out.println( (5 + 5) * 5 );
```

The output will be:

50

When running the code above, the first operation to be executed will be $5 + 5$ that will be 10 and then will be multiplied by 5 which means that the final result will be 50. If the parenthesis wasn't there, the output would be 30.

Important points to remember about the arithmetic operators:

- Multiplication, division and the remainder operators have priority over addition and subtraction.
- If we are using only the multiplication, division and the remainder operators, the priority will be decided by what operator was used first.
- Parenthesis will always be the highest priority over any operator.

2.2.1 Commenting code

As you've seen in the previous examples, it's possible to add comments to your Java code to explain the details of what is happening. It's very common to see code comments in real-world projects. Sometimes comments are overused and just make code confusing, but in general they are still useful.

The simplest way to comment code is by using `//`, which tells the JVM (Java virtual machine) compiler to ignore everything that follows up to the end of the line. Let's look at an example:

```
1 // The following code will calculate the sum from number1 and number2
2 int number1, number2 = 5;
3 System.out.println(number1 + number2); // The JVM will print 10
```

In real-world projects it's not recommended to pollute code with several comments between code instructions. According to Uncle Bob (Robert Martin), author of the popular handbook Clean Code (Prentice Hall, 2008), commenting code is a way to say that we've failed at making our code simple to understand.

Comments like these are used in the examples in this book for the purpose of clarity and to ensure that the reader can follow along, but keep in mind that I would avoid using such comments in a real-world project.

Another popular use for comments is to provide descriptions of variables, classes, and methods, as a formal way to document your code. (Don't worry, we'll study all those concepts in depth in the next chapter.)

You've already seen the main Java method, so let's use that as an example:


```
1  /* This is the main Java method that will bootstrap the program */
2  public static void main (String[] args) { }
```

The compiler ignores everything between the `/*` and the `*/`, so these comments can span multiple lines. Documentation comments achieve the same effect:

```
1  /**
2   * This is the main Java method that will bootstrap the program
3   */
4  public static void main (String[] args) { }
```

You can also use commenting features known as tags in your comments, like these:

```
1  @author Homer Simpson, @since 1.0
```

To specify additional information such as the author and what software version the code became available in.

You can use HTML (Hypertext Markup Language) to format your code comments too, applying tags such as `<p>` for a new paragraph, `<b>` for bold text, `<table>` to add a table, and so on. You can explore classes from the Java Development Kit (JDK) to see some examples, the `String` class for example is a good one to take a look at.

For the purposes of this book you don't need to know all the possible tags and options for commenting code; you can learn the main tags and apply them where appropriate.

2.3 Flow Control

Flow control is a concept we use every day, in our day-to-day lives. We define conditions to check whether we can perform a particular action. For example, if it's a warm, sunny day, we can go to the beach. If we have time off from work and some money to spare, we can go on vacation. If you solve the Java Challenges, you can call yourself a Java rockstar! :)

Flow control is basically the ability to test a condition and perform one action if it's true and another if it's false. It's a way to control the programming flow based on conditions we are expecting.

We often use conditions in programming, and knowing how to manipulate conditions in depth makes a big difference to the quality of the code we create. Many developers aren't familiar with all the details of conditions in Java, so let's explore them in a little more depth.

Before analyzing our first flow control example, take a look at table 3.2, which summarizes the equality and relational operators we use to compare variables and values in programming.

Table 2.3 Equality and relational operators in Java

Relational operators	Explanation	Example	Returns
-----	-----	1 == 1	true
!=	Not equal to	1 != 1	false
>	Greater than	2 > 1	true
>=	Greater than or equal to	1 >= 1	true
<	Less than	3 < 2	false
<=	Less than or equal to	1 <= 3	true

Let's start with the most basic flow control statement in any programming language, the if statement:

```

1  int homerAge = 35;
2  if (homerAge >= 21)
3      System.out.println("Homer can drink beer");

```

When this command is executed, "Homer can drink beer" will be printed.

Notice that in this example braces ({}) are not used around the condition for the if statement. That's perfectly allowed in Java, but only when there's a single instruction for the if statement. If we do this, for example, the second message will always be printed, even if (as in this case) the condition `homerWeight > 340` is not true:

```

1  int homerWeight = 240;
2
3  if (homerWeight > 340)
4      System.out.println("Homer is obese");
5      System.out.println("Homer needs to go on a diet"); // #1

```

The point `// #1` in the code will not be considered part of the if statement and will always be executed, because there are no braces.

Not using braces might be fine for some cases, but you might forget to add the braces when adding a new instruction to the if condition, so you must be careful if you decide to leave them out. The safest practice is to always include curly braces.

2.3.1 Logical operators

When we need to compare two or more conditions, we make use of logical operators (AND, OR, NOT). Developers use these frequently, and we use them all the time in the real world too.

2.3.2 The AND logical operator

In Java, the operator to join two conditions (AND) looks like this: `&&`. The main rule with this operator is that both conditions must be true to return true.

Here's an example:

```
1  if (atLeast12YearsOld && hasMoney) {
2      System.out.println("Watch horror movie in the cinema");
3  }
```

In this case, if a person is at least 12 AND has money for a ticket, then they will be able to watch a horror movie in the cinema. Note that both conditions must be true for the person to be able to go to the cinema.

There is another important rule with the && operator that makes sense in terms of performance. Because both conditions must be true, we know that if the first condition is false, the second condition doesn't need to be evaluated. This is exactly what happens: the JVM won't bother to evaluate the second condition if the first is false because it already knows that the && operator will return false. This is known as short-circuit evaluation.

Let's look at an example to illustrate how this works. In this example the first condition, `bartAge >= 12`, won't be fulfilled, so the second condition, `bartMoney >= 10`, won't even be evaluated:

```
1  int bartAge = 10;
2  double bartMoney = 20;
3
4  if (bartAge >= 12 && bartMoney++ >= 10) { // The second condition won't be executed
5      System.out.println("Bart can go to the cinema"); // This code won't be executed
6  }
7
8  System.out.println(bartMoney); // This will print 20
```

You can prove this by checking the value of the `bartMoney` variable, which will be incremented by one when the second condition is executed. When you print this variable after the if statement it prints 20, which shows that the JVM did not execute this condition.

2.3.3 The OR logical operator

The OR logical operator verifies whether at least one of the conditions is true. It's represented by `||` in Java. Let's look at an example:

```
1  boolean isBartWithCinemaTicket = false;
2  boolean isBartWithMoney = true;
3
4  if (isBartWithCinemaTicket || isBartWithMoney) {
5      System.out.println("Bart can watch a movie at the cinema");
6  }
```

In this case, only one condition has to be true: if Bart has a ticket or money to buy a ticket, he will be able to watch a movie at the cinema. Even though only the `isBartWithMoney` variable is true, the `||` operator will return true.

Again for performance reasons, the `||` operator works in a similar way to the `&&` operator. If the first condition is true, the second condition won't even be checked because the JVM already knows the operator will return true.

You can see this in code using a similar example to the one in the previous section:

```

1  int bartAge = 12;
2  double bartMoney = 20;
3
4  if (bartAge >= 12 || bartMoney++ >= 10) { // The second condition won't be executed
5      System.out.println("Bart can go to the cinema"); // This code will be executed
6  }
7
8  System.out.println(bartMoney); // This will print 20
```

Because the first condition is true, the second condition won't be executed and the `bartMoney` variable won't be incremented. Also note that the if condition is met because we are using the `||` operator and at least one condition is true. Therefore, Bart can go to the cinema.

The NOT logical operator

The final logical operator is NOT, represented in Java with `!`. This operator takes a Boolean value as its operand and reverses it—if the condition is true the `!` operator will transform it to false, and vice versa. For example:

```

1  System.out.println(!true); // This prints false
2  System.out.println(!false); // This prints true
```

2.3.4 Boolean variables and conditions

Sometimes we have to handle complex conditions in our code. One way to improve the readability of the code is by creating a boolean variable with a meaningful name to indicate whether or not the condition is true.

Suppose we need to check if there is enough money in an account to make a withdrawal:

```
if (moneyToWithdraw <= accountBalance) { ... }
```

We can assign this condition to a boolean variable:

```
boolean isMoneyAvailableInBalance = moneyToWithdraw <= accountBalance;
```

And then use the variable in the if statement instead:

```
if (isMoneyAvailableInBalance) { ... }
```

This technique is useful for making code more readable, especially when there are multiple conditions to compare inside a single if statement. A very powerful book to read if you'd like to dig deeper into code quality and code refactoring is Martin Fowler's *Refactoring: Improving the Design of Existing Code* (Addison-Wesley).

Another option for improving the readability of code where there are several conditions in a single if or other Java statement is to use nesting, placing one if statement inside another. It's best to avoid excessive nesting because it adds complexity and degrades the quality of the code, but considering that you are highly likely to see code nesting in real projects, it's good to know how it works. Here's an example:

```
1  int simpsonAge = 18;
2
3  if (simpsonAge >= 18) {
4      if (simpsonAge <= 50) {
5          System.out.println("The Simpson age is from 18 to 50");
6      }
7  }
```

Even better, we could improve the code in this case by using &&, and the result would be exactly the same:

```
1  if (simpsonAge >= 18 && simpsonAge <= 50) {
2      System.out.println("The Simpson age is from 18 to 50");
3  }
```

Can you see how much simpler your code can be if you know the different features of Java? This is just one simple example of how an in-depth knowledge of what the language offers can help you write much clearer, better, and more meaningful code.

2.3.5 Naming conventions for boolean variables

The JavaBeans convention for naming boolean variables is used with the following pattern is <YourConditionName>. It's also common to prefix the names of booleans with has, and in many cases this may seem more suitable (for example, hasMoneyToSpend). The JavaBeans specification does not mention this alternative, however, it's better to stick to the standard when possible, so always try to use the prefix is.

2.3.6 The ternary operator

When the condition you need to check is simple, you don't always need to use an if statement. Instead, you can use Java's ternary operator, `?`, which functions like a simplified form of if statement. It takes three operands, as follows:

Condition ? Return a value when condition is true : Return a value when condition is false

The following code checks whether the value of the `beerQuantity` variable is greater than or equal to 50. If so, the value “Homer” will be returned. If the condition is false, the value “Moe” will be returned instead:

```
1 int beerQuantity = 50;
2 String beersOwner = beerQuantity >= 50 ? "Homer" : "Moe";
3 System.out.println(beersOwner); // This will print "Homer"
```

The value “Homer” will be printed because the condition of the ternary operator is true.



Note: String

String is a class that represents characters. You don’t need to worry about the details for now; this will be explored further in a later chapter.

In the same way it is possible to nest a normal if statement inside another one, it is also possible to nest ternary operators—but this is far worse. Nesting ternary operators will make code maintenance a nightmare.

Here’s an example:

```
1 String beerOwner = beerQuantity >= 100 ? beerQuantity >= 200 ? beerQuantity >= 300 ? \
2   "Barney" : "Homer" : "Lenny" : "Carl";
```

It’s very confusing and difficult to understand, and you should avoid doing this at all costs.

2.3.7 Homer Beers Flow Control Challenge

In the following code, we will explore the concepts of the `if` statement, ternary operator, and logical operators. Homer will consume two batches of beer, `batchOfBeer1` and `batchOfBeer2`. How many beers do you think Homer will have had at the end of this challenge?

```
1 public class HomerBeersBatch {
2
3     public static void main(String... doYourBest) {
4         int batchOfBeer1 = 5;
5         int batchOfBeer2 = 5;
6
7         if (++batchOfBeer2 > batchOfBeer1 || batchOfBeer1++ == batchOfBeer2++) {
8             batchOfBeer1++;
9         }
```

```

10
11     batchOfBeer1 = (batchOfBeer1++ > batchOfBeer2 &&
12         ++batchOfBeer2 < batchOfBeer1++ ? ++batchOfBeer1 : ++batchOfBeer2);
13
14     System.out.println("Homer will have: " + batchOfBeer1-- +
15         " and " + batchOfBeer2++ + " beers.");
16 }
17
18 }

```

- A. Homer will have: 7 and 7 beers.
- B. Homer will have: 7 and 8 beers.
- C. Homer will have: 5 and 7 beers.
- D. Homer will have: 5 and 6 beers.

Explanation:

The first key for this challenge is to know that the pre-increment operator increases a variable's value at the time the variable is used, and the post-increment operator increases the value after the variable is used. The other important thing to know is that the short-circuit logical operator `||` will only check what is necessary to determine whether the condition will be fulfilled or not. That means in the first if:

```

1  if (++batchOfBeer2 > batchOfBeer1 || batchOfBeer1++ == batchOfBeer2++) {
2      batchOfBeer1++;
3  }

```

Only the first condition will be checked. Because it's true, the second condition doesn't need to be checked. Therefore, only `batchOfBeer2` will be incremented. Then, the `batchOfBeer1` variable will be incremented inside the if statement.

So, these will be the values of the two variables:

```

1  batchOfBeer1 = 6;
2  batchOfBeer2 = 6;

```

The next statement is the ternary operator:

```

1  batchOfBeer1 = (batchOfBeer1++ > batchOfBeer2 && ++batchOfBeer2 < batchOfBeer1++ ? \
2  ++batchOfBeer1 : ++batchOfBeer2);

```

The condition `batchOfBeer1++ > batchOfBeer2` will be false because the `batchOfBeer1` variable will be incremented only after this condition is checked. `batchOfBeer1` and `batchOfBeer2` will have the same value, 6, so the “else” condition will be fulfilled and the third operand will be evaluated. Because we are using the pre-increment operator the `batchOfBeer2` variable will be incremented to 7, and this value will be assigned to the `batchOfBeer1` variable.

The values of the variables at that stage will be:

```

1  batchOfBeer1 = 7
2  batchOfBeer2 = 7
3
4  Finally, in the last System.out.println the variables will each be incremented, but \
5  only after this statement has executed. Therefore, the result printed will be:
6
7  batchOfBeer1 = 7
8  batchOfBeer2 = 7

```

In conclusion, the right alternative is A, with the final output:

Homer will have: 7 and 7 beers.

2.3.8 The switch case statement

The switch case statement achieves a similar result to the if statement, but its syntax might be more appropriate when you want to check which of several possible values a variable has. Doing that with an if statement wouldn’t be suitable; the code would get confusing.

In the following code value of the `variableToCompare` is passed to the switch statement and is compared to each case value, and if a match is found that block of code is executed. If not, the default condition is executed:

```

1  int variableToCompare = 3;
2
3  switch (variableToCompare) {
4      case 1: System.out.println("First condition");
5      case 2: System.out.println("Second condition");
6      default: System.out.println("Default condition");
7  }

```

This prints:

Default condition

Let’s look at an example:


```
1 String barneyBeerPreference = "Duff";
2
3 switch (barneyBeerPreference) {
4     case "Guinness": System.out.println("Barney prefers Guinness");
5     case "Heineken": System.out.println("Barney prefers Heineken");
6     case "Duff": System.out.println("Barney prefers Duff");
7     default: System.out.println("Some other beer");
8 }
```

As you can see, the case "Duff" will fulfill the condition, so "Barney prefers Duff" will be printed. But then "Some other beer" will also be printed, and that's not what we want, right? We want only the first message to be printed, not the default.

To only fulfill the "Duff" condition, we need to add the break keyword after each case statement so that we break the switch case flow after doing what we want.

2.3.9 The break keyword

When you want to break the execution of the current process, you can use the break keyword. By adding this keyword in the switch case statement, for example, you can stop execution and avoid printing the information from the default statement:

```
1 String barneyBeerPreference = "Duff";
2
3 switch (barneyBeerPreference) {
4     case "Guinness": System.out.println("Barney prefers Guinness");
5     case "Heineken": System.out.println("Barney prefers Heineken");
6     case "Duff": {
7         System.out.println("Barney prefers Duff");
8         break;
9     }
10    default: System.out.println("Some other beer");
11 }
```

It's important to know the nuances of the switch case statement, because at times you may find yourself maintaining code that is not so clear. If you don't know how the core Java features work, you will have a hard time understanding or fixing bugs.

There are more optimized and less verbose ways to use the switch case statement in Java; we will explore those in future chapters because they involve more advanced concepts.

2.3.10 Breaking Bad Switch Case Challenge

In this challenge we will be exploring the following code:

```
1 public class BreakingBadChallenge {
2
3     public static void main(String... doYourBest) {
4         int formulaNumber = 10;
5
6         String heisenbergFormula = "";
7
8         switch (formulaNumber) {
9             case 1:
10                 heisenbergFormula += "H";
11             case 10:
12                 heisenbergFormula += "Ne";
13             case 30:
14                 heisenbergFormula += "Zn";
15             case 25:
16                 heisenbergFormula += "Mn";
17                 break;
18             default:
19                 heisenbergFormula += "He";
20         }
21
22         System.out.println(heisenbergFormula);
23     }
24
25 }
```

- A. Ne
- B. NeZnMnHe
- C. NeZnMn
- D. NeHe

Explanation:

The formulaNumber with the value of 10 will fulfill the condition, so Ne will be printed. But note that we are not using the break keyword after each case. The switch statement will continue executing until it finds this keyword. This means Zn and Mn will be concatenated to Ne before the switch statement is interrupted with the break keyword, giving us the final output of NeZnMn.

In conclusion, the correct alternative is:

C - NeZnMn

2.4 Loops

Looping in programming is a way to repeat certain logic until a condition is fulfilled. You can probably think of several examples in the real world: when we are brushing our teeth, we repeat the brushing action until they are clean; when we study for a test, we keep studying until we're confident we know enough to pass it; when we need to drive somewhere, we keep driving until we get where we want to go.

As in real life, in programming loops are used all the time. But not all developers know the full range of possibilities with loops in Java. We are challenged every day with different problems, and knowing the nuances of loops will help developers to produce the best code possible.

2.4.1 The for loop

The for loop is the most commonly used loop in Java. There are three sections in a for loop:

```
1  for (/* Initialize variable */; /* Condition to continue the loop */; /* Incrementer\  
2  */) { ... }
```

This is the typical way to use it:

```
for (int i = 0; i < 10; i++) { ... }
```

You can initialize as many variables as you want in a for loop:

```
for (short i = 0, y = 0; i < 10; i++) { ... }
```

You can create an infinite for loop by specifying a condition that will never be false, or omitting the condition:

```
for ( ; ; ) { System.out.println("This will print forever"); }
```

You can even use a `System.out.println` in the incrementer area:

```
for ( ;; System.out.println("Incrementer area")) { ... }
```

2.4.2 The break keyword in the for loop

As you saw in the switch case example, if you want to break out of the current process and exit the loop for some reason, you can use the `break` keyword to do so.

Suppose you want to stop the loop execution when the index is greater than or equal to 2:

```
1  for (int i = 0; i < 10; i++) {  
2      if (i >= 2) {  
3          System.out.println(i);  
4          break;  
5      }  
6  }
```

This code will print only 2. Because the break keyword is inside the for loop, the loop will be broken and the following numbers won't be printed.

2.4.3 The continue keyword

Sometimes you want to have control over the continuation of a loop's execution. For this goal, there is the continue keyword.

In the following code we will explore an example where we want logic to not be executed when a certain condition is fulfilled:

```
1  for (int i = 1; i <= 5; i++) {  
2      if (i >= 3) {  
3          System.out.println("Goku beats one Majin Buu with " + i + " hits");  
4          continue;  
5      }  
6      System.out.println("Goku hits Majin Buu");  
7  }
```

There is a code instruction at the end of the for loop that prints the text "Goku hits Majin Buu" to the console. But we only want this text to be printed when the value of the variable i is less than 3.

If the i value is 3 or greater, we don't want instruction outside of the if statement to be executed; instead, we want to print the message "Goku beats one Majin Buu with " + i + " hits". A perfect solution for that is to use the continue keyword to go to the next loop iteration.

When the if condition is fulfilled, the continue statement will send control back to the top of the loop body, skipping any statements below it; this means the second print statement will not be executed.

Keep in mind, though, that you should generally avoid using the continue keyword because having multiple flows in code makes it harder to read and maintain. Use this feature only if it is particularly suited to the situation you are coding for.

2.4.4 The use of labels

Sometimes we need to control outer loops from an inner loop. In these cases, labels can be useful. It's extremely rare to see this language feature in real-world code—indeed, it's best to avoid it when

possible because most developers don't know about it and it will make the code harder to maintain—but still it's useful to at least know that the feature exists with Java.

Here's a code example that uses labels:

```
1  outer:
2  for (int i = 0; i < 5; i++) {
3      inner:
4      for (int j = 0; j < 2; j++) {
5          if (j == 2)
6              continue inner;
7          if (i >= 3)
8              break outer;
9
10         System.out.println(i + ":" + j);
11     }
12 }
```

In this code, we're manipulating the outer loop (labeled with `outer:`) and an inner loop (labeled with `inner:`).

When the value of the `i` variable declared in the inner loop is 2, the inner loop will force the continuation of the inner loop avoiding the execution of the following instructions within the inner loop.

In the second `if` statement we are checking if the value of the `i` variable is greater or equals to 3, if this condition is fulfilled we will break the outer loop which will break the inner loop as well. Therefore, the final output of this program will be:

```
1  0:0
2  0:1
3  1:0
4  1:1
5  2:0
6  2:1
```

2.4.5 The while loop

The while loop is much simpler than the for loop. It consists of just the while keyword, the condition, and the loop body. With the while loop, we execute an action as long as the condition is true. When the condition is false, the loop breaks (stops executing).

The following while condition will print the numbers 0 to 10:

```
1  int i = 0;
2  while (i <= 10) { System.out.println(i++); }
```

It's also important to mention that the while loop might not be executed at all. This is the case when the while condition is false the first time through the loop, as in this example:

```
1  int i = 20;
2  while (i <= 10) {
3      System.out.println("This while loop instruction won't be executed");
4  }
```

2.4.6 The do while loop

The main difference between the do while and while loops is that a do while loop will always be executed at least once. The check comes at the end of the loop execution, to determine whether it should continue or not.

In the following code example the numbers 0 to 10 will be printed:

```
1  int i = 0;
2  do {
3      System.out.println(i++);
4  } while (i <= 10);
```

In this example, even though the condition is not fulfilled, the loop will still be executed once, printing 99:

```
1  int i= 99;
2  do {
3      System.out.println(i++);
4  } while (i <= 10);
```

2.4.7 The for-each loop

The for-each loop is widely used because it's a practical and easy way to loop into an array of elements, for example (we'll study arrays in the next section).

The for-each loop will return each element of an array, and we can then manipulate them. The following code will print 0 1 2 3:

```
1  int [] homerBeers = { 0, 1, 2, 3 };
2  for (int homerBeer : homerBeers) {
3      System.out.println(homerBeer);
4  }
```

2.5 Arrays

When you need to use multiple values in one variable, the most efficient way to do so is by creating an array.

An alternative is to do the following:

```
1  String simpsonCharacterName1 = "Moe";
2  String simpsonCharacterName2 = "Burns";
3  String simpsonCharacterName3 = "Bart";
```

But as you can see, the variable names are repetitive and that makes the code confusing. If you use the array type instead, you can use a single variable name with multiple values:

```
String [] simpsonCharacterNames = { "Moe", "Burns", "Bart" };
```

This representation is much more compact, taking up just one line. You can then access each element of the array by index. Array indexes in Java start with 0, so if to print “Burns” you would do the following:

```
System.out.println(simpsonCharacterNames[1]);
```

You can print all the information in the array using any kind of loop, such as a standard for loop:

```
1  for (int i = 0; i < simpsonCharacterNames.length; i++) {
2      System.out.println(simpsonCharacterNames[i]);
3  }
```

A while loop:

```
1  int i = 0;
2  while (i < simpsonCharacterNames.length) {
3      System.out.println(simpsonCharacterNames[i++]);
4  }
```

Or a for-each loop:

```
1  for (String eachCharacterName : simpsonCharacterNames) {  
2      System.out.println(eachCharacterName);  
3  }
```

You could even use streams and method references (these concepts will be explained in future chapters, but here's an idea of what this would look like):

```
Arrays.stream(simpsonCharacterNames).forEach(System.out::println);
```

Each of the loops will print the elements of the array:

```
1  Moe  
2  Burns  
3  Bart
```

If we want only to declare an array and create an instance of this array, we can use the following syntax:

```
int [] simpsonCharacterAges = new int[3];
```

Note that the keyword `new` here is related to the array instance and not the primitive type `int`.

Now that we have an array instance with a size of three elements, we can populate it with values, as follows. Note also that in Java the initial index of an array is 0, not 1:

```
1  simpsonCharacterAges[0] = 35;  
2  simpsonCharacterAges[1] = 102;  
3  simpsonCharacterAges[2] = 10;
```

If we use a for-each loop to show the values of the array, as follows:

```
1  for (int eachCharacterAge : simpsonCharacterAges) {  
2      System.out.println(eachCharacterAge);  
3  }
```

the output will be:

```
1  35  
2  102  
3  10
```

2.5.1 Arrays Array Default Values

Remember the primitive variables with their default values we analyzed in the start of this chapter labeled as Table 2.1 The primitive variable types defined in Java?

An important detail from arrays in Java is that if we declare an array and don't populate any value in it, the values of each element will contain the default value of each variable. Let's see some examples.


```
1 int[] intNumbers = new int[3];
2 System.out.println(intNumbers[0]);
```

It will print 0.

```
1 double[] doubleNumbers = new double[3];
2 System.out.println(doubleNumbers[0]);
```

It will print 0.0.

Also consider that objects, a subject we will go deeper in the next chapter, will have the default value of null when used in an array. Let's see the example of the String type:

```
1 String[] names = new String[3];
2 System.out.println(names[0]);
```

It will print null.

2.5.2 Mario Bros Index Array Challenge

This challenge will test your understanding of array creation syntax, how array indexes work, and how to iterate through each element of the array. Your goal is to figure out what will happen after running the main method just by analyzing the code. When you think you know the answer, try running the code.

```
1 public class MarioBrosIndexChallenge {
2
3     public static void main(String[] args) {
4         String[] marioCharacters = new String[3];
5         marioCharacters[1] = "Mario ";
6         marioCharacters[2] = "Luigi ";
7         marioCharacters[3] = "Peach ";
8
9         for (String marioCharacter: marioCharacters) {
10             System.out.print(marioCharacter);
11         }
12     }
13 }
```

- A. Mario Luigi Peach
- B. nullMario Peach
- C. java.lang.ArrayIndexOutOfBoundsException will be thrown

D. nullMario Luigi

The key to this challenge is the array index. As mentioned earlier, array indexes start from 0. Here we are creating an array with three elements. Because every array in Java starts with the index of 0, not 1, this means it's impossible to access the index 3, and if we try that the JVM will throw an `ArrayIndexOutOfBoundsException`.

This exception name is very clear: it tells us that we can't access an element whose array index is out of bounds. For now, don't worry too much about understanding exceptions in depth; we will explore this concept in detail in future chapters.

In conclusion, the correct alternative is:

C - `java.lang.ArrayIndexOutOfBoundsException` will be thrown

2.5.3 Multidimensional arrays

Java also offers multidimensional arrays. This is basically an array with two or more dimensions, or an array of arrays. If you were developing the logic for a board game, for example, a multidimensional array would be required.

You can imagine a bidimensional array like a set of information laid out in rows and columns. This type of array would be well suited to developing a tic-tac-toe game, for example. The indexes of the array would be:

```
1  0, 0 | 0, 1 | 0, 2
2  1, 0 | 1, 1 | 1, 2
3  2, 0 | 2, 1 | 2, 2
```

In each pairing you can think of the first index as the row number and the second as the column number.

To create a tic-tac-toe board, you could use the following code:

```
1  String [][] tictactoeBoard = new String[3][3]; // #1
2
3  for (int row = 0; row < 3; row++) {           // #2
4      for (int column = 0; column < 3; column++) { // #2
5          tictactoeBoard[row][column] = row + "," + column + "|"; // #3
6          System.out.print(tictactoeBoard[row][column]); // #4
7      }
8
9      System.out.println(); // #5
10 }
```

- #1 Declare a bidimensional array with three rows and three columns.
- #2 Declare two for loops, one to iterate through the rows and one to iterate through the columns.
- #3 Inside the loops, iterate through the rows and columns and populate the tictactoeBoard array with the row and column indexes.
- #4 While populating the tictactoeBoard variable, also print its value.
- #5 Add a line break at the end of each row.

The output of this program will be:

```

1  0,0|0,1|0,2|
2  1,0|1,1|1,2|
3  2,0|2,1|2,2|

```

2.5.4 Bidimensional Simpson Array Challenge

This code challenge explores several of the concepts you've learned about in this and the preceding sections: bidimensional arrays, a for loop declaring two variables, a while loop, and if statements. Your job is to predict the information contained in some of the array elements at the end.

This challenge is not so code-friendly—as mentioned in the introductory chapter, some of the challenges focus more on exploring the concepts and less on developing clean code.

```

1  public class BidimensionalArrayChallenge {
2
3      public static void main(String... doYourBest) {
4          String[][] simpsonsFoods = new String[2][2];
5
6          simpsonsFoods[0][0] = "Donuts";
7          simpsonsFoods[1][0] = "Broccoli";
8          simpsonsFoods[1][1] = "Krusty burger";
9
10         for (int lineIndex = 0, columnIndex; lineIndex < 2; lineIndex++) {
11             columnIndex = 0;
12
13             while (columnIndex < 2) {
14                 if (simpsonsFoods[lineIndex][0] == "Broccoli") {
15                     simpsonsFoods[lineIndex][columnIndex++] = "Healthy";
16                 } else {
17                     simpsonsFoods[lineIndex][columnIndex] = "Junk";
18                     break;
19                 }
20             }
21         }

```

```

22
23         System.out.println(simpsonsFoods[0][0] + "|" + simpsonsFoods[0][1] +
24             "|" + simpsonsFoods[1][0] + "|" + simpsonsFoods[1][1]);
25     }
26 }

```

- A. Junk|null|Healthy|Junk
- B. Junk|Junk|Junk|Healthy
- C. Junk|Junk|Healthy
- D. Junk|Healthy|Junk|

Let's analyze what happens in each iteration.

In the first iteration, the `lineIndex` and `columnIndex` variables are 0.

In the first if statement, the array element value will be "Donuts". Then "Junk" will be assigned in the place of "Donuts". Because we are using the post-increment operator, the `columnIndex` will be incremented only after this instruction has been executed. Also, note that the `break` keyword is used in this condition; therefore, it will break out of the inner loop that is iterating through the columns.

In the next iteration, the `lineIndex` variable will be incremented to 1 and the `columnIndex` will be reseted to 0, then the value to be compared now is "Broccoli" which will fulfill the 'if' condition. Therefore the value from `simpsonsFoods[1][0]` will be "Healthy".

Then, in the last iteration, the `lineIndex` will be 1 and `columnIndex` also 1. The fulfilled condition will be the second one. Therefore the value assigned to `simpsonsFoods[1][0]` will be "Junk".

Notice also that the `simpsonsFoods[0][1]` position of the array wasn't populated. In that case the default value of a String will be assigned which is null.

In conclusion, the final values of the selected array indexes will be:

A - Junk|null|Healthy|Junk

2.6 Local variable restrictions

The part of your program in which a variable can be seen and used is known as its scope. In Java, variables are scoped to the block in which they are defined. A code block is any code snippet surrounded by curly braces (`{ }`):

```
1 public static void main(String[] theSimpsons) {  
2     // The code inside the main method is a code block  
3  
4     if (true) { /* This is a code block too */ }  
5  
6     for (;;) { /* This is another code block */ }  
7 }
```

Variables that are declared inside a method can only be used within that method block. These are called local or method variables. Local variables effectively come into existence when a method starts its execution and cease to exist when the method ends its execution. They're accessible anywhere within the method block, including in inner/nested blocks, but there is no way to access a local variable outside of the method in which it was declared.

In the following example, the `simpsonName` variable is declared in the main method and used inside the enclosed for loop:

```
1 public static void main(String[] theSimpsons) {  
2     String simpsonName = "Homer"; // This variable can be used anywhere in the main \  
3     block  
4     for (int i = 0; i <= 10; i++) {  
5         System.out.println(simpsonName); // simpsonName can be used here  
6     }  
7 }
```

It's possible to declare a variable in any Java statement. If you declare the variable in a for loop, if statement, or any other non-method block, the local variable will only be accessible there. This is known as block scope.

In the following example, you won't be able to access the `allanHangoverAge` variable outside the if block. Because the variable was declared inside the if block, it can only be used there:

```
1 public static void main(String[] hangover) {  
2     if (true) {  
3         int allanHangoverAge = 40; // This variable is only visible in the if block  
4     }  
5  
6     // System.out.println(allanHangoverAge); We can't access allanHangoverAge here  
7 }
```

Attempting to run this code with the print statement uncommented would result in a compile-time error, and most IDEs would highlight this line and variable as being problematic.

We could even create a block without a statement (an "anonymous block"), and the same rule would be applied:

```
1 public static void main(String... starWars) {  
2     {  
3         int jediForce = 0; // This variable can only be used inside this block  
4     }  
5  
6     // System.out.println(jediForce); jediForce can't be used here  
7 }
```

Note that in practice, these blocks are rarely used outside of switch case statements.

As these examples show, a local variable is accessible only within the block (set of braces) in which it was declared, and any nested blocks.

It's also possible for a variable to be local to a loop. In a real project, you may come across code like this:

```
1 public static void main(String... starWars) {  
2     for (int outerLooping = 0; outerLooping < 10; outerLooping++) {  
3         System.out.println(outerLooping);  
4         // System.out.println(innerLooping); innerLooping can't be used here  
5  
6         for (int innerLooping = 0; innerLooping < 10; innerLooping++) {  
7             System.out.println(innerLooping); // Both variables can be used here  
8             System.out.println(outerLooping);  
9         }  
10  
11     }  
12  
13     // System.out.println(innerLooping); This variable can't be used here  
14     // System.out.println(outerLooping); This variable can't be used here  
15 }
```

Here, the outerLooping variable is declared and accessible in the outer for loop and in the inner for loop, but not outside both loops.

The innerLooping variable is declared and accessible only inside the inner for loop.

Another corner case in most Java statements is when we don't use braces. If we attempt to include multiple instructions without braces, only the first one will be executed and any variables we declared within the loop will not be accessible beyond that first instruction. For example:

```

1  for (int i = 0; i < 3; i++)
2      System.out.print(i);
3      System.out.print("\nThis statement is not part of the for loop");

```

**Note: Escape sequence**

Notice that we are using the escape sequence `\n`, which means that we are breaking a line.

**** The output of this code is**:**

```

1  012
2
3  This statement is not part of the for loop

```

2.6.1 Simpson Variable Scope Challenge

In the following Java challenge you will see some variables being declared. Your goal is to replace the `// Replace code` placeholder with the right alternative so that the code runs and compiles fine.

```

1  public class ScopeChallenge {
2
3      public static void main(String... doYourBest) {
4          boolean isHomerFat = true;
5
6          {
7              String homerFavoriteBeer = "Duff";
8          }
9
10         if (isHomerFat) {
11             String homerFavoriteFood = "Doughnuts";
12             for (int beersQuantity = 0; beersQuantity <= 10; beersQuantity++)
13                 if (isHomerFat)
14                     // Replace code
15             }
16         }
17
18     }

```

- A. `System.out.println(beersQuantity);`
`System.out.println(homerFavoriteFood);`
`System.out.println(isHomerFat);`
- B. `System.out.println(homerFavoriteBeer);`

- C. `int homerAge = 35;`
 `System.out.println(isHomerFat);`
- D. `System.out.println(homerFavoriteFood);`
 `System.out.println(beersQuantity);`

Explanation:

Let's first analyze the alternatives that won't compile.

Alternative B can't be right because `homerFavoriteBeer` is only accessible inside the anonymous block.

Alternative C can't be right because it begins by initializing a variable that can't be used in just one line. Because the `if` statement does not contain braces, it will be interpreted by the compiler as containing only one instruction (the variable initialization). It will be impossible to do anything with this variable afterwards because its scope will be local to that statement, so this code does not make sense and doesn't compile.

Alternative D is incorrect for a similar reason. Because the `for` loop does not contain braces, it will be interpreted as containing only one instruction: in this case, `if (isHomerFat) System.out.println(homerFavoriteFood);`. The `beersQuantity` variable is initialized in the `for` statement, but we are trying to use it after the `if` statement. This code won't compile because that variable will not be accessible in the second line of code.

Finally, alternative A is correct. In this case the variable `beersQuantity` that was declared in the `for` statement is accessible in the `if` statement. The `homerFavoriteFood` variable is also accessible: it was declared in the outer `if` statement that encloses the `for` loop and inner `if` statement. The `isHomerFat` variable was declared at the beginning of the main method and therefore is accessible throughout the whole main block.

2.6.2 Local variable initialization

If you declare a local variable without initializing it with a value, the code won't compile. Local variables aren't assigned default values, so if they don't get initialized there is no way to use them.

If you try to use a local variable you've declared without initializing it, like this:

```
1 int moeAge;  
2 System.out.println(moeAge);
```

the JVM will generate a compilation error like the following:

variable `moeAge` might not have been initialized

There's one exceptional case that it's good to be aware of. If you use an explicit `true` condition in the `if` statement, the variable will be considered as initialized by the JVM. Therefore, the following code will compile:


```
1  int beerCount;  
2  
3  if (true) {  
4      beerCount = 0;  
5  }  
6  
7  System.out.println(beerCount);
```

This code will also work:

```
1  int beerCount;  
2  
3  if (1 == 1) {  
4      beerCount = 0;  
5  }  
6  
7  System.out.println(beerCount);
```

If you try to use a boolean variable in the if condition, you'll get a compilation error because the JVM can't predict if the variable will be true or false, and therefore the beerCount variable might or might not be initialized:

```
1  int beerCount;  
2  boolean test = true;  
3  
4  if (test) { // The JVM can't predict the beerCount variable will be initialized  
5      beerCount = 0;  
6  }  
7  System.out.println(beerCount); // Compilation error here
```

If we try to run the program above we will get the following compilation error message:
variable beerCount might not have been initialized

The solution for this code is to use the else clause to ensure that the variable is initialized in every case. The following code will compile fine:

```
1  int beerCount;  
2  boolean test = true;  
3  
4  if (test) {  
5      beerCount = 0;  
6  } else {  
7      beerCount = 1;  
8  }  
9  
10 System.out.println(beerCount); // This will print 0
```

Points to remember

- A local variable can be only used in the declared block of code.
- A local variable in Java must be always initialized.
- If there is no certainty that the local variable will be initialized, there will be a compilation error.

2.7 Candy Price Code Challenge

Software engineers deal with algorithms all the time, and many companies ask job applicants to solve them during interviews. It's crucial to be able to analyze a situation and solve the problem with code. Understanding the problem you want to solve is a vital skill, and you should be constantly working on that.

In real-world projects, this is always the first thing you should try to do. Only then you will be able to provide the best possible solution and use your full potential as a software developer. To understand more about the problem and the system you're working with, you should talk to people, come up with questions, make notes—get curious. Then, by consequence, you will acquire knowledge about your project.

That said, let's take a look at this algorithm problem!

The boss of a candy company has a certain amount of money to spend, and they want to maximize the number of candies they can buy with this money to give to their employees.

Given an array of candy prices [4, 5, 3, 6, 7, 9] and supposing the boss has 30 dollars to spend, what is the maximum number of candies the boss can buy? Assume that only one of each item is available.

Your goal is to calculate the largest number of candies the boss can buy with their available funds.

Let's consider a simpler example first. If the boss has 10 dollars to spend and the candy prices are [5, 2, 1, 3, 4], the array will first have to be sorted: [1, 2, 3, 4, 5]. Then we have to iterate through the elements of this array and verify when the available amount of money is going to be

totally used. If we count the number of candies that can be purchased, there will be 4 items for this test case ($1 + 2 + 3 + 4 = 10$).

Array sorting wasn't addressed in this chapter, but for now you can use the bubble sorting strategy, which is one of the simplest ways to sort an array.

You create the `candyPrices` variable with the prices you wish, and then the following code should work to sort your array:

```
1  int lengthOfPrices = candyPrices.length;
2  for (int i = 0; i < lengthOfPrices-1; i++)
3      for (int j = 0; j < lengthOfPrices - i-1; j++)
4          if (candyPrices[j] > candyPrices[j+1]) {
5              int temp = candyPrices[j];
6              candyPrices[j] = candyPrices[j+1];
7              candyPrices[j+1] = temp;
8          }
9      }
10 }
```

Once you have the array sorted, it's up to you to print the information on the largest number of candies that can be bought with the money the boss has available.

**Note: Sorting an Array**

You could use an existing Java API method to sort the array, but because this hasn't been introduced in the book yet the solution presented here is to sort the array manually. If you already know the API solution, you can use that instead for this challenge.

Try the following test case scenario: you receive the array of candy prices `[8, 4, 1, 3, 10, 11]`, and the company boss has 20 dollars to spend on candies. The count of maximum possible candies to be bought should be 4. You can use any kind of loop or flow control statement that you find suitable for your logic.

2.8 Summary

- You learned about the different primitive types available in Java, what kinds of values they can store, and how to name them.
- How to assign values to variables and how to increment and decrement the values of numerical variables.
- Different flow control options available in Java were introduced, such as the `if` statement, ternary operator, and `switch` case statement.
- How to use logical operators for comparing conditions and boolean variables to simplify code dealing with complex conditions.

- The core types of loops are: for, while, do while, and for-each.
- Multiple-valued variables, also called arrays, and you learned that you don't need to declare several different variables to handle multiple values.
- Declare, iterate, and manipulate bidimensional arrays.
- The restrictions of local variables were discussed, including variable scope, where to declare them, and when to initialize them.

By applying the concepts in this chapter, you were able to solve an algorithm of reasonable difficulty.

Congratulations, you've finished the first chapter of Java Challengers! You are on your way to becoming a better programmer and learning how to analyze and improve complex code.

3 Basic object-oriented programming (Review)

This chapter covers:

- Creating, naming, and instantiating classes
- Creating and using class methods, and manipulating instance variables
- Accessing instance attributes and methods with the keyword `this`
- Understanding how to use constructors in a flexible way
- Manipulating object references
- Working with arrays

Before the advent of object-oriented programming there was the procedural paradigm, which made it very difficult to reuse code. With the procedural paradigm code easily gets spread throughout the whole system, and as a result it's hard to maintain. For example, every time it was necessary to validate a certain type of ID, developers would have to invoke the same function to do this. Any change to that function then required changes to numerous parts of the codebase. Developers also had to write extensive documentation to ensure functions were used correctly.

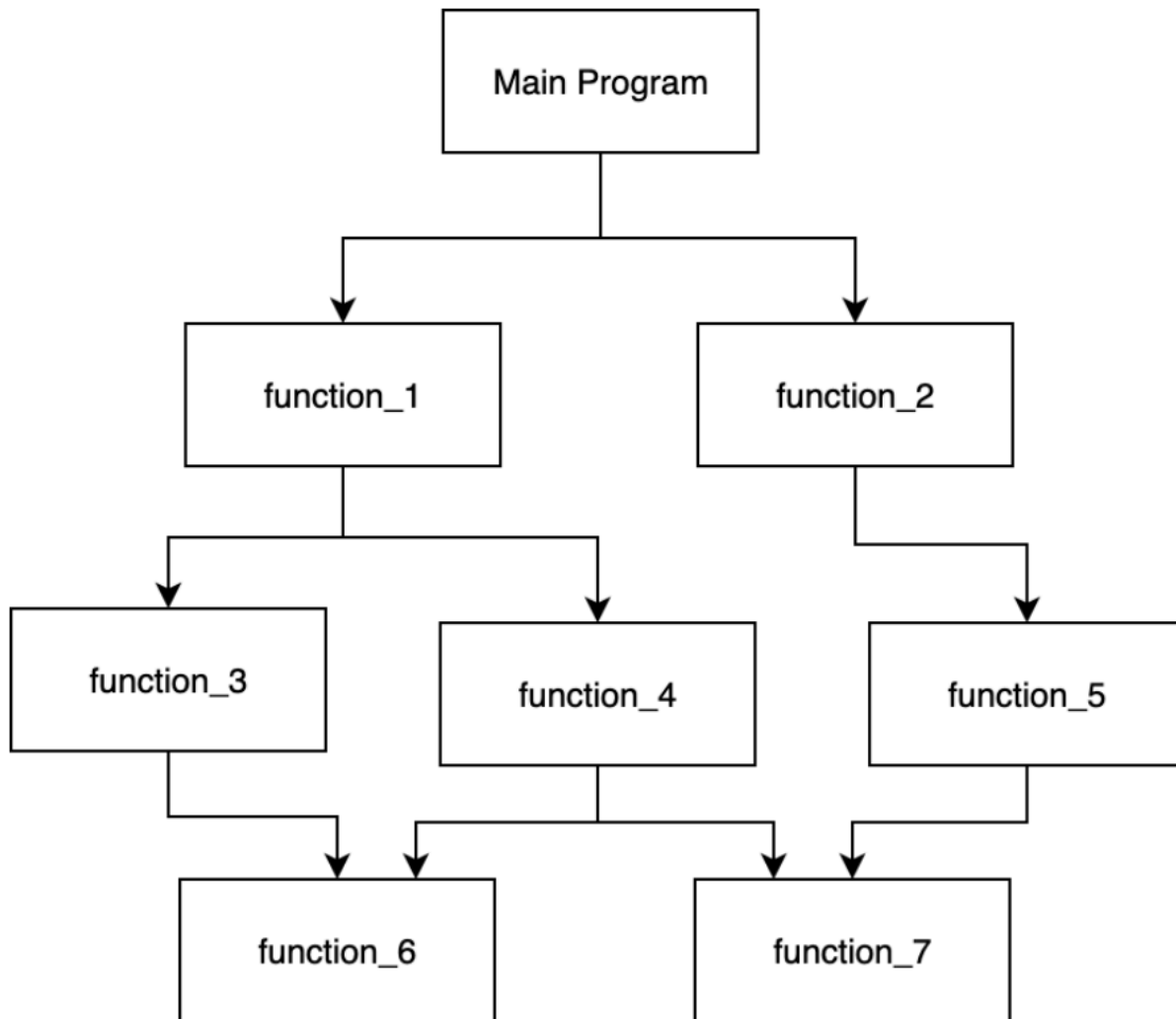
To solve these problems, the object-oriented paradigm was created. The main intention of this paradigm is to minimize differences between software code and the real world, making it easier to solve business problems in a more concise way. With object orientation we can better express real entities and actions from real-world problems, making it easier to reuse code.

Object orientation also makes it easier to organize the code structure, write less code, and centralize code features, which makes applications more flexible and with encapsulated logic.

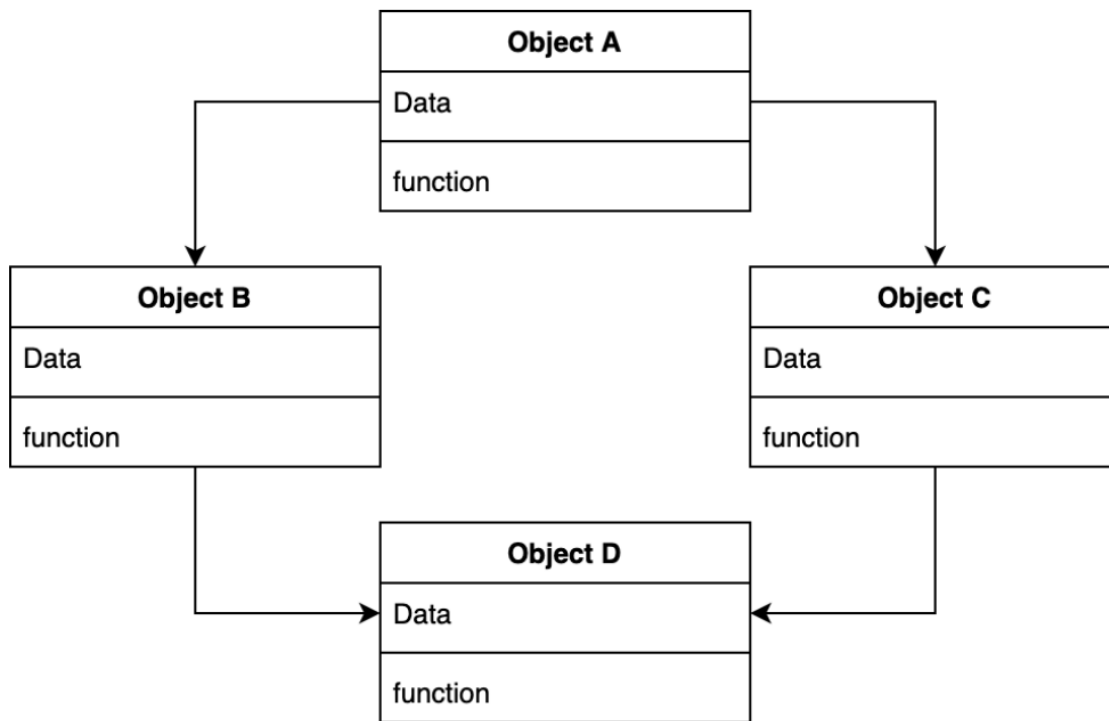
When you've mastered object-oriented concepts, it's possible to create easy-to-maintain code whose features are well organized and simple to track down.

To make the difference between both paradigms, procedural and object-oriented programming clearer, let's analyze diagrams describing them.

Let's first analyze how difficult it is to organize code when working with the procedural paradigm:



Now let's analyze how easier it is to understand the communication between different objects instead of only functions:



It's also important to emphasize that even when working in a language that uses this paradigm, like Java, it's possible to write code that does not fully take advantage of the power of object orientation. It's crucial to fully understand object-oriented principles so that we can develop flexible code that uses the full power of the key concepts and tools.

In this chapter, we will explore the most fundamental concepts and why they are important in real-world applications.

3.1 Classes

A class in the object-oriented programming paradigm is a way to abstract and organize some element from the real world: a computer, game, person, animal, character, discount, payment, house, ship, city, and so on. Can you spot the similarity between all these diverse items? That's right, they're all nouns—people, places, or things. To make sense in a project (for simple systems, at least), a class should be abstracted from something that exists in the real world.

An easy way to check for this is to verify that the class's name is a noun. A class with an action name (a verb), like `PerformAction`, should never be used.

Abstracting elements from the real world to create classes that make sense in the project you're working on is not an easy task. You need to analyze what the nouns are in the problem you want to solve, and condense that information into classes that will be meaningful in the solution you want to develop.

You can also think of a class like a blueprint or a factory for creating particular objects. For example, if you create a class with the name `Car`, you will be able to create a specific car with its own characteristics.

Here are some examples of class declaration in Java:

```
1 class Car { }
2 class Animal { }
3 class Discount { }
```

As you can see in these examples, we can create a car, an animal, or even a discount.

A class contains state with variables and behavior, or actions. For example, a `Car` class might define variables such as `maximumSpeed`, `color`, `brandName` and actions such as `accelerate` and `brake`. You might also create a class named `Calculator` with actions for performing different operations with numbers (add, subtract, multiply, and so on) and printing the result.

Now that we know that a class is a blueprint, in the remainder of this chapter, we'll explore how to make things happen using this blueprint.

3.1.1 Naming classes

Naming is a crucial skill we have to develop as software developers. Sometimes it's hard to define a name. We have to first abstract the concepts we're working with and make sure we understand what we're trying to do with them, which can require a considerable amount of effort.

In the previous chapter, I mentioned that Java developers should follow the JavaBeans convention for naming different elements of code. For naming classes the convention is to use the `PascalCase` pattern, which means to start each word in the class name with a capital letter.

Example class names include `Account`, `Discount`, `Person`, `CustomerAddress`, `BillingAddress`, and `PaymentService`.

3.1.2 Instantiating classes

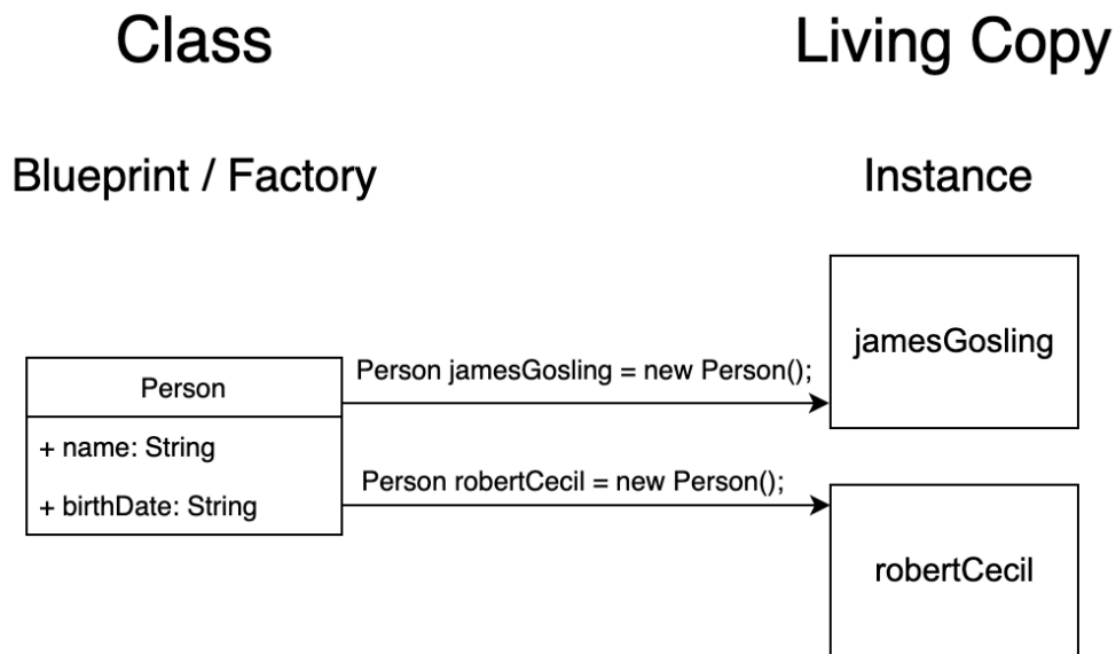
A class instance is a living copy of the class blueprint that is capable of having its own state and set of actions. These instances are also known as objects.

For example, we can create different instances of a `Person` class and give them specific names and behaviors. We can specify that one instance of the `Person` class has a name of "James Gosling" and a `birthDate` of May 19, 1955, then we can then create another instance of `Person` that has a name of "Robert Cecil Martin" and a `birthDate` of December 5, 1952.

To create, or instantiate, an object of the `Person` class we would do the following:


```
1 Person jamesGosling = new Person();  
2 Person robertCecil = new Person();
```

Figure 3.1 illustrates this concept of creating living copies of the class from the blueprint it provides.



The `jamesGosling` and `robertCecil` instances are individual copies of the `Person` class with distinct names and behaviors.



Constructor

Notice that when we are instantiating our class we use `()` at the end of the class name. This is actually the invocation of a constructor, which we will explore in detail later in this chapter. For now keep in mind that the constructor is responsible for initializing a class or instance constructing the object with the essential information.

Similarly, if we have a `Car` class we can create an instance representing a `Porsche 911` with a maximum speed of 227 miles per hour, and another instance representing a `Ferrari SF90 Stradale` with a maximum speed of 212 miles per hour. Once we've created these instances, we can manipulate each of them in an isolated way. The `Porsche` and `Ferrari` can perform different actions without affecting each other.

To create these instances of a `Car`, the Java syntax is the following:

```

1 Car porsche = new Car();
2 Car ferrari = new Car();

```

Figure 3.2 illustrates this concept of the `Car` class as a factory for creating instances with different attribute values.

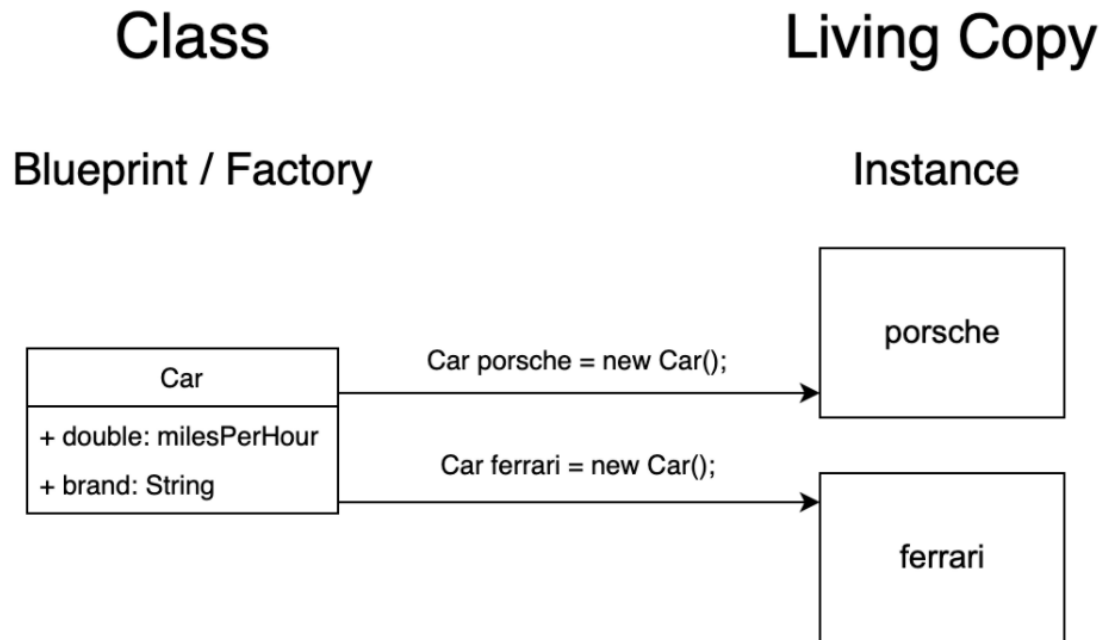


Figure 3.2 porsche and ferrari are instances of the `Car` class.

The following is a small but complete code example where we are instantiating a Java object:

```

1 public class Car {
2
3     public static void main(String[] args) {
4         Car porsche = new Car();
5     }
6
7 }

```

3.1.3 Class attributes

Now that you know what an instance is, it's time to explore the use of attributes. In the `Animal` instance example, I mentioned attributes such as the animal's type and sound. Let's develop this idea further. The following example defines the class `Animal`, which has two attributes, `breed` and

emittedSound. We can instantiate two instances of this class, a cat and a dog, and specify the attributes for each appropriately: :

```
1  class Animal {
2      String breed;
3      String emittedSound;
4
5      public static void main(String [] args) {
6          Animal dog = new Animal();
7          dog.breed = "Husky";
8          dog.emittedSound = "woof";
9
10         System.out.println(dog.breed);
11         System.out.println(dog.emittedSound);
12
13         Animal cat = new Animal();
14         cat.breed = "Persian";
15         cat.emittedSound = "meow";
16
17         System.out.println(cat.breed);
18         System.out.println(cat.emittedSound);
19     }
20 }
```

The output of the Animal class's main method is:

```
1  Husky
2  woof
3  Persian
4  meow
```

3.1.4 Default values

What happens when you declare an attribute in a Java class without a value? In this case the JVM will initialize it automatically with a default value.

Table 3.1 lists the default values for Java primitive and object types.

Type	Default value
boolean	false
byte, short, int, long	0
float, double	0.0
char	'\u0000'
Reference type (object reference)	null

It's important to emphasize that these default values are assigned only to variables declared at the class or instance level (inside a class, but outside a method). As you'll recall from chapter 2, local variables that are not declared at the class level, such as those declared inside a method—must be initialized manually upon creation.

Let's see in practice how this works in code:

```

1  public class DefaultAttributeValues {
2
3      boolean isHomerThin;
4      int homerGrade;
5      double nedFlandersBeerCount;
6      char maggieWords;
7      String margeSenseOfHumor;
8
9      public static void main(String[] defaultAttributesValues) {
10         DefaultAttributeValues values = new DefaultAttributeValues();
11         System.out.println(values.isHomerThin);           // #1
12         System.out.println(values.homerGrade);            // #2
13         System.out.println(values.nedFlandersBeerCount);  // #3
14         System.out.println(values.maggieWords);           // #4
15         System.out.println(values.margeSenseOfHumor);     // #5
16     }
17 }
```

- #1 Prints false
- #2 Prints 0
- #3 Prints 0.0
- #4 Prints \u0000
- #5 Prints null

It's important to know how default values in Java work because we deal with variables all the time.

Data that is being passed in the wrong way is a common cause of bugs. Knowing that default values will be assigned when you don't do this manually will make a huge difference when you're developing code, fixing bugs, or adding new features, enabling you to optimize your code design.

3.2 Methods

In the object-oriented programming paradigm, a method is an action or behavior of a class or instance. In Java a method has to declare the data type of the value it returns (we'll explore this concept further later in this section). If it doesn't return a value—for example, if the method just performs an action—it must be declared void.

Let's see how a dog emits a sound in Java code. The emitSound method doesn't return a value (it simply prints “woof”), so we declare it with the void keyword:

```
1  class AnimalMethodInvocation {
2
3      String emittedSound;           //#1
4
5      void emitSound() {             //#2
6          System.out.println(emittedSound);
7      }
8
9      public static void main(String[] args) {
10         AnimalMethodInvocation dog = new AnimalMethodInvocation();
11         dog.emittedSound = "woof";  //#3
12         dog.emitSound();             //#4
13     }
14
15 }
```

- #1 Instance variable declaration
- #2 emitSound method declaration
- #3 Assign the value “woof” to the emittedSound instance variable
- #4 Invoke the emitSound method from the dog instance

3.2.1 Passing parameters to methods

A parameter is a variable that can be passed in from outside a method and will live and die inside the method. Depending on what logic you are developing, it will often make more sense to create parameters than to manipulate instance variables.

To illustrate this concept, let's consider an example where we pass a message as a parameter and print it in a method:

```
1 class MessageParameter {
2
3     void print(String message) {                // #1
4         System.out.println(message);            // #2
5     }
6
7     public static void main(String[] args) {
8         MessageParameter messageParameter = new MessageParameter(); // #4
9         messageParameter.print("You are a Java Challenger");           // #5
10    }
11
12 }
```

- #1 Declare the print method with the message parameter.
- #2 Print the message parameter.
- #3 Create an instance of MessageParameter and assign it to the messageParameter variable.
- #4 Invoke the print method from the messageParameter instance variable, passing an argument value of "You are a Java Challenger".

When the print method is executed, "You are a Java Challenger" will be printed to the console.



Parameter VS Argument

Notice that I've used the terms "parameter" and "argument". Parameters are used when we are declaring a method. For example, we declare the print method with the message parameter.

Arguments are used when we are invoking a method. For example, we pass the argument "You are a Java Challenger" to the print method.

3.2.2 Returning primitive types

You'll often want to return values from a method so that you can use the result of the method call in another method process. When you want to return a value from code logic, you can declare that in your method. You specify the method's return type in the method declaration, and you use the return statement in the method body to return the value. For example, the following code demonstrates how you can return an int value from an addition operation:

```

1  class CalculatorSum {
2      int sum(int number1, int number2) {           // #1
3          return number1 + number2;                // #2
4      }
5
6      public static void main(String[] args) {
7          CalculatorSum calculator = new CalculatorSum(); // #3
8          int sumResult = calculator.sum(2, 2);        // #4
9          System.out.println(sumResult);              // #5
10     }
11 }

```

- #1 Declare the sum method with two parameters, number and number2, and specify that it will return an int primitive type.
- #2 Return the sum of the number1 and number2 variables.
- #3 Instantiate a CalculatorSum instance and assign it to the calculator variable.
- #4 Invoke the sum method from the calculator instance, passing the argument value 2 for both number1 and number2, and return the result to the sumResult variable.
- #5 Print the value of the sumResult variable, which is 4.

You can return any type of variable from a method; you just need to choose the most appropriate one for your needs. Let's see, for example, how we would return a float variable from a multiplication operation with fractional numbers:

```

1  class CalculatorMultiplication {
2
3      float multiply(float number1, float number2) { // #1
4          return number1 * number2;                 // #2
5      }
6
7      public static void main(String[] args) {
8          CalculatorMultiplication calculator = new CalculatorMultiplication(); // #3
9          float multiplicationResult = calculator.multiply(2.50, 2.50); // #4
10         System.out.println(multiplicationResult); // #5
11     }
12
13 }

```

- #1 Declare the multiply method with the number1 and number2 parameters and specify that it will return a double.
- #2 Return the result of multiplying the values of the number1 and number2 variables.
- #3 Create a Calculator instance and assign it to the calculator variable.

- #4 Invoke the multiply method from the calculator instance, passing the argument values of 2.50 and 2.50; return the result of the multiplication to the multiplicationResult variable.
- #5 Print the value of the multiplicationResult variable, which is 6.25.

The value you return from a method can be any of the primitive types: byte, char, short, int, long, float, double, or boolean.

3.2.3 Returning objects

In addition to the primitive types, a method can return an object of any type (for example, a String object). For now, just keep in mind that when you instantiate any class in Java you are creating an object. We will go deeper into the why of that later in the book, so don't worry too much about it right now.

In the following code, we concatenate (join) the value of two String object values and then return a String in the concatenate method:

```

1  class TextHandler {
2      String concatenate(String firstText, String secondText) { // #1
3          return firstText + secondText;                        // #2
4      }
5
6      public static void main(String []args) {
7          TextHandler textHandler = new TextHandler(); // #3
8          String concatenatedText = textHandler.concatenate("Java", "Challengers"); // \
9  #4
10         System.out.println(concatenatedText); // #5
11     }
12 }

```

- #1 Declare the concatenate method with the firstText and secondText parameters and specify that it will return a String object.
- #2 Concatenate the firstText and secondText values and return them as a String.
- #3 Instantiate Concatenator and assign it to the concatenator variable.
- #4 Invoke the concatenate method and pass the argument values “Java” to firstText and “Challengers” to secondText; then assign the result to the concatenatedText variable.
- #5 Print the value of the concatenatedText variable.

3.3 The Keyword this

The keyword `this` in Java provides a way of referencing the current instance, so that we can access its state (attributes) or behavior (methods). In the following example, we effectively assign the values

"Lamborghini" to the brand instance variable and "black" to the color instance variable by invoking the setupCar method. The this keyword tells the JVM that we are accessing an instance variable, although because there are no local variables declared with the same names (we'll consider that scenario in section 3.3.1) in this case using this is optional:

```
1  class Car {
2
3      String brand; // #1
4      String color; // #1
5
6      void setupCar() {
7          this.brand = "Lamborghini"; // #2
8          color = "black";           // #3
9      }
10
11     public static void main(String[] args) {
12         Car car = new Car();
13         car.setupCar();
14         System.out.println(car.brand); // #4
15         System.out.println(car.color); // #5
16     }
17
18 }
```

- #1 brand and color are declared as instance variables.
- #2 By using the keyword this, we are accessing an instance variable.
- #3 Using the keyword this is optional in this case because there is no local variable named color; the value "black" will be assigned to the instance variable anyway.
- #4 Prints "Lamborghini".
- #5 Prints "black".

It's also possible to invoke an instance method using the keyword this:

```
1  public class InvokingMethod {
2
3      public static void main(String[] args) {
4          InvokingMethod invokingMethod = new InvokingMethod();
5          invokingMethod.accelerate();
6      }
7
8      void accelerate() {
9          System.out.println("Accelerate!"); // #1
10     }
```

```

10         this.turnLeft();           // #2
11     }
12
13     void turnLeft() {
14         System.out.println("Turn left"); // #3
15     }
16 }

```

- #1 This prints “Accelerate!”
- #2 Invoke the turnLeft method.
- #3 This prints “Turn left”.

As this example demonstrates, you can invoke instance methods from another method.

Note that you always need to instantiate a class to invoke an instance method. Also note that the use of the keyword `this` is optional when invoking an instance method: you can use it or not.

3.3.1 Local variables vs. instance variables

A common use of the keyword `this` is to differentiate between instance and local variables with the same names, so that it’s clear which ones we want to access. In the following example, we explore what happens if we don’t make use of `this` in a case like this.

Here, the `color` variable is declared both as an attribute of the class `Car` and as a local variable in the method `changeColor`. Because we are not using the keyword `this` in the assignment, the effect here is to assign the value of the local variable to itself. This operation is pointless, because a local variable lives and dies inside the method in which it was declared:

```

1  public class Car {
2
3      String color = "";           // #1
4
5      void changeColor(String color) { // #2
6          color = color;           // #3
7      }
8
9      public static void main(String[] args) {
10         Car car = new Car();      // #4
11         car.changeColor("black"); // #5
12         System.out.println(car.color); // #6
13     }
14 }

```

- #1 Initialize the `color` instance variable with an empty `String`, `""`.

- #2 This parameter is a local variable.
- #3 The parameter value will be assigned to the local variable and will be lost because we are dealing with a local variable that lives and dies when the `changeColor` method is done executing.
- #4 Create an instance of `Car`.
- #5 Invoke the `changeColor` method, passing the argument "black" (this value will be lost).
- #6 This will print "".

By using the keyword `this`, we can indicate that we want to access the instance variable `color` instead of the local variable from inside the `changeColor` method, avoiding any confusion. Invoking the following method changes the value of the `color` variable of the `Car` instance to "black":

```

1  public class Car {
2
3      String color;                // #1
4
5      void changeColor(String color) {    // #2
6          this.color = color;           // #3
7      }
8
9      public static void main(String[] args) {
10         Car car = new Car();           // #4
11         car.changeColor("black");       // #5
12         System.out.println(car.color); // #6
13     }
14 }
```

- #1 Declare `color` as an instance variable.
- #2 This `color` parameter is a local variable.
- #3 The parameter value will be assigned to the instance variable because the keyword `this` is used here.
- #4 Create an instance of `Car`.
- #5 Change the value of the `color` instance variable to "black".
- #6 This prints "black".

To sum up, we use the keyword `this` when we want to access an instance member instead of a local variable or parameter. When a local variable and an instance variable with the same name exist, the local variable will take precedence over the instance variable if we don't use the keyword `this`. We need to explicitly indicate that we want to access the instance variable by using the keyword `this`.

3.4 Constructors

A constructor in Java is a special type of method whose main responsibility is to construct (create) an object of a class with certain attribute values. You can also invoke methods in the constructor, but its primary use is to define the object's state by providing initial values for its attributes .

In the following example we instantiate a `Car` object and pass the values "Porsche" and "black" to the constructor, initializing the attributes of this class:

```
1  public class Car {
2
3      String brand;
4      String color;
5
6      public Car(String brand, String color) { // This is the constructor method
7          this.brand = brand;
8          this.color = color;
9      }
10
11     public static void main(String[] args) {
12         Car porsche = new Car("Porsche", "black");
13         System.out.println(porsche.brand); // This will print Porsche
14         System.out.println(porsche.color); // This will print black
15     }
16
17 }
```

If you don't declare a constructor, the JVM will create one automatically for you. When you instantiate a new object, you are always invoking a constructor. For example, in this case:

```
1  public class SimpsonCharacter {
2
3      public static void main(String[] args) {
4          new SimpsonCharacter(); // The () of this instantiation is the constructor
5      }
6
7  }
```

the JVM will create the following default constructor:

```
1 public SimpsonCharacter() {  
2 }
```

If you do declare a customized constructor, however, the default constructor won't be created automatically by the JVM. In this case, because we create a specific constructor, there will be no default constructor:

```
1 public class SimpsonCharacter {  
2     String name;  
3  
4     public SimpsonCharacter(String name) { // Constructor method  
5         this.name = name;  
6     }  
7  
8     public static void main(String[] args) {  
9         new SimpsonCharacter("Homer"); // You need to pass a String here  
10    }  
11  
12 }
```

This means that you can't instantiate the `SimpsonCharacter` class with the following code:

```
new SimpsonCharacter();
```

The only way to instantiate this class is by invoking the customized constructor by passing a name:

```
new SimpsonCharacter("Homer");
```

If you want to make the class flexible, so it can be instantiated with either a custom or a default constructor, you need to declare both explicitly:

```
1 public class SimpsonCharacter {  
2     String name;  
3  
4     public SimpsonCharacter() { }  
5  
6     public SimpsonCharacter(String name) {  
7         this.name = name;  
8     }  
9  
10    public static void main(String[] args) {  
11        new SimpsonCharacter(); // You can instantiate without a parameter  
12        new SimpsonCharacter("Homer"); // You can also pass a String  
13    }  
14  
15 }
```

This concept demonstrated here is called `overloading`. Simply put, this is a way of making different versions of constructors or methods with the same name that receive different types or numbers of parameters. It's a core concept that will be explored in depth in future chapters.

3.4.1 Invoking constructors with the keyword `this`

It's also possible to invoke a constructor from another constructor in the same class. This feature is useful because it allows us to reuse code from other constructors; we don't need to rewrite the same piece of code.

It's important to keep in mind that you can only invoke one constructor from another in the first line of the constructor. Otherwise, there will be a compilation error.

Let's explore a common real-world project scenario where you can use this concept. Suppose you want to be able to instantiate a class named `BatMobile` either with two parameters that you specify, `versionName` and `modelYear`, or using the default constructor with two constant values, "Batman Forever" and 1995.

The only way to make it possible to instantiate this class either with or without parameters in the constructor is by declaring two constructors:

```
1  public class BatMobile {
2
3      String versionName;
4      int modelYear;
5
6      BatMobile(String versionName, int modelYear) {
7          this.versionName = versionName;
8          this.modelYear = modelYear;
9      }
10
11     BatMobile() {
12         this("Batman Forever", 1995);
13     }
14
15     public static void main(String[] batMobileRocks) {
16         BatMobile batmanForever = new BatMobile();
17         batmanForever.printBatMobileDescription();    // #1
18
19         BatMobile batmanArkhamKnight = new BatMobile("Arkham Knight", 2014);
20         batmanArkhamKnight.printBatMobileDescription(); // #2
21     }
22
23     void printBatMobileDescription() {
```

```

24         System.out.println("Version name: " + this.versionName + " Model year: " + t\
25 his.modelYear);
26     }
27 }

```

- #1 This will print “Version name: Batman Forever Model year: 1995”
- #2 This will print “Version name: Arkham Knight Model year: 2014”

As you can see, instead of passing the values "Batman Forever" and 1995 outside our BatMobile class, we've done this inside the class. This default instantiation doesn't need to be exposed outside of this class, and that's what we want to accomplish when we are using the concepts of object-oriented programming.

Another great advantage of using this approach to invoke a constructor inside the class is the flexibility we gain; it makes our code much easier to maintain.

3.4.2 Resident Evil Variable Change Challenge

The following challenge explores the concepts of instances, constructors, and methods. The instance pointer keyword `this` will be also used, and this is the biggest key to this problem. Are you ready to solve this challenge? Try it out!

```

1  public class ResidentEvilNameChange {
2
3      String name = "Nemesis";
4
5      ResidentEvilNameChange(String name) {
6          name = name;
7      }
8
9      public static void main(String[] doYourBest) {
10         ResidentEvilNameChange residentEvilNameChange = new ResidentEvilNameChange("\
11 Leon");
12         System.out.println(residentEvilNameChange.name);
13
14         residentEvilNameChange.changeName("Claire");
15         System.out.println(residentEvilNameChange.name);
16     }
17
18     void changeName(String name) {
19         this.name = this.name;
20     }
21 }

```

- A. Leon
 Claire
- B. Nemesis
 Claire
- C. Nemesis
 Nemesis
- D. Leon
 Nemesis



Important: Try out the Java Challenge **before** seeing the answer.

Explanation:

As mentioned previously, the key to this challenge is to understand how the keyword `this` works. By using that keyword, we are pointing to the current instance.

The first action we take in this code is to instantiate `ResidentEvilNameChange`. Then we pass the `String` "Leon" to its constructor—but notice that we are using the local variable here because we are not explicitly using the keyword `this`. You must remember that when you have a local variable that has the same name as an instance variable, the local variable will have higher priority and will be used instead of the instance variable if you don't use `this`.

The constructor assigns the value "Leon" to the local variable `name`, which ceases to exist right after the constructor is invoked (recall that a local variable is alive only as long as the method in which it is declared is executing). The instance variable `name` will keep the value of "Nemesis".

Next we invoke the `changeName` method, which assigns the instance variable `name` to itself because we are using the keyword `this` twice. We are basically passing "Nemesis" to the `name` instance variable again, so its value is not changed.

In conclusion, the final output of this challenge is:

- C. Nemesis
 Nemesis

3.5 Object references

When we instantiate an object in Java by calling its constructor, space is allocated in the memory heap to store the object. The `new` operator returns a reference to that object, which is assigned to a variable of the appropriate type.

We can think of variables that we instantiate with any Java object as pointers or controllers that are able to access the real objects in the memory heap. In the scenario in figure 3.1, for example, the

duke variable we create when instantiating an object with `new Duke("Java rocks!");` will act as a controller, allowing us to manipulate the real Duke object by changing its properties and passing it to other methods as an argument.

Figure 3.1 The duke variable we create when we instantiate the Duke object is able to access that object in memory.

Let's see how this works in code. In the following example, we first create an instance of the Duke class, then assign it to the variable `alternativeDuke`, which will point to the same Duke object in the memory heap. When we change any of this object's instance variables (in this case, the `preferredJavaVersion` variable), the object itself will actually be changed:

```
1 public class Duke {
2
3     int preferredJavaVersion;
4
5     public static void main(String[] dukeWantsToCode) {
6         Duke duke = new Duke();           // #1
7         duke.preferredJavaVersion = 11;    // #2
8
9         Duke alternativeDuke = duke;       // #3
10        alternativeDuke.preferredJavaVersion = 15; // #4
11
12        System.out.println(duke.preferredJavaVersion); // #5
13        System.out.println(alternativeDuke.preferredJavaVersion); // #6
14    }
15
16 }
```

- #1 Create the Duke object in the memory heap and assign the object to the local duke reference variable.
- #2 Assign the value 11 to duke's `preferredJavaVersion` instance variable.
- #3 Pass the object's reference to `alternativeDuke`.
- #4 Assign the value 15 to `alternativeDuke`'s `preferredJavaVersion` instance variable.
- #5 This prints 15.
- #6 This prints 15.

Both `println` statements print 15, because it's the `preferredJavaVersion` instance variable of the actual object stored in memory that is changed with each assignment. The `duke` and `alternativeDuke`'s reference variables both point to the same object, so the second change overwrites the first one.

3.5.1 Level Object Reference Challenge

This challenge explores how variables point to objects in the memory heap. You're creating two variables, `challenger` and `alternativeChallenger`, then manipulating their values. Can you guess what the output is?

```
1 public class Challenger {
2
3     int level;
4
5     public static void main(String[] doYourBest) {
6         Challenger challenger = new Challenger();
7         challenger.level = 7;
8
9         challenger.changeLevel();
10        Challenger alternativeChallenger = challenger;
11
12        alternativeChallenger.level = 12;
13
14        System.out.print(challenger.level);
15        System.out.print(alternativeChallenger.level);
16    }
17
18    void changeLevel() {
19        this.level = 9;
20    }
21 }
```

- A. 1212
- B. 77
- C. 912
- D. 99



Important: Try out the Java Challenge **before** seeing the answer.

Explanation:

Your first action in the main method is to instantiate the `Challenger` object. Then you assign the value 7 to the level variable.

The second important action is to invoke the `changeLevel` method from the challenge instance and then change the value of the level variable to 9. In doing this, you change the real object in the memory heap.

The key part of this challenge is when you assign the challenge instance to the `alternativeChallenger` variable. Remember that a variable is just a controller that points to the real object in the memory heap. Therefore, both variables will be pointing to the same object and will be changing properties of the same object.

When you change the value of `alternativeChallenger`'s level variable to 12, you are changing the object you created. `challenger`'s level variable will then have a value of 12 as well.

In conclusion, the correct answer for this challenge is:

A - 1212

If you got this challenge right, it means you have a good understanding of the concept of object references.

3.5.2 Metal Gear Null Reference Challenge

In this challenge you will explore object references in more detail. You will begin by instantiating the `MetalGearReference` class, and then you will invoke the `changeSoldierName` method with the object reference and perform some actions. Your goal is to guess what the output of the following program is.

```
1 public class MetalGearReference {
2
3     String name;
4
5     public static void main(String... doYourBest) {
6         MetalGearReference solidSnake = new MetalGearReference();
7         solidSnake.changeSoldierName(solidSnake, "Solid Snake");
8
9         MetalGearReference liquidSnake = new MetalGearReference();
10        liquidSnake.changeSoldierName(liquidSnake, "Liquid Snake");
11
12        System.out.println("1 soldier=" + solidSnake.name + " 2 soldier=" + liquidSn\
13ake.name);
14    }
15
16    void changeSoldierName(MetalGearReference metalGearReference, String name) {
17        metalGearReference.name = name;
18        this.name = "Big Boss";
19    }
```

```
20     metalGearReference = null;  
21 }  
22  
23 }
```

- A. 1 soldier=Solid Snake 2 soldier=Liquid Snake
- B. 1 soldier=Big Boss 2 soldier=Big Boss
- C. 1 soldier=null 2 soldier=null
- D. 1 soldier= 2 soldier=



Important: Try out the Java Challenge **before** seeing the answer.

Explanation:

The first point to make about this challenge is that passing the `solidSnake` instance to the instance method `changeSoldierName` is redundant. Because you're using the keyword `this` inside the `changeSoldierName` method, you are already accessing the instance that invoked this method.

In the `changeSoldierName` method, you are first changing the `solidSnake` instance's name to "Solid Snake", then referencing the invoking instance, which in this context is `solidSnake`, with the keyword `this`. Therefore, the `solidSnake` instance will have the name "Big Boss" at this point.

Finally, you assign the `metalGearReference` variable the value `null`—but note that `metalGearReference` is only a reference variable, not the real object. Note also that you are not changing the object itself; you are assigning `null` directly to this local variable. Only the local variable will change, and nothing will happen to the real object in the memory heap.

Because the `solidSnake` instance is still accessible in the main method, the object is still alive. The last value assigned to the `solidSnake` object was "Big Boss", so `solidSnake.name` will have this value stored into it.

The same process will happen with the instance `liquidSnake`.

In conclusion, the final answer to this challenge is:

**** B - 1 soldier=Big Boss 2 soldier=Big Boss ****

3.6 Arrays are objects in Java

As you learned in the previous chapter, an array is a variable type that can store multiple values. An important concept to keep in mind is that arrays in Java are objects. This means that when you create an array you're creating a reference to a real object stored in the JVM's memory heap. When a method changes an array value, the change will be reflected in the real object because the real object was changed.

You can imagine an object in Java as a controller that has access to the real object. When you declare an array in Java the array variable is not an object itself; it's a pointer to an object. Every variable that you instantiate as an object in Java is just a reference pointing to the real object in memory.

In the following example, we declare the array `homerSweets` and pass it to the `changeFirstElementToPudding` method. We then change the value of the first element of the array from "Chocolate" to "Pudding". Note that because we are working with an object, the change will be reflected even outside the `changeFirstElementToPudding` method:

```
1 public class HomerSweets {
2
3     public static void main(String... homerEatsSweets) {
4         String[] homerSweets = { "Chocolate", "Cake", "Ice Cream"};
5         changeFirstElementToPudding(homerSweets);
6         System.out.println(homerSweets[0]); // This will print "Pudding"
7     }
8
9     static void changeFirstElementToPudding(String[] sweets) {
10         homerSweets[0] = "Pudding";
11     }
12
13 }
```

In this example, because the `homerSweets` array is pointing to an object in the memory heap, the value "Pudding" will be assigned to the real `homerSweets` object from the main method. Because the `homerSweets` object is still accessible in the main method and the real object was changed, the `println` instruction will print "Pudding".

3.6.1 Link's Adventure Array Reference Challenge

In this challenge an array object with a size of 5 is created and populated, and then this array is assigned to another array variable and manipulated in a method. Can you guess what the output will be?

```

1  public class LinkAdventure {
2      public static void main(String... doYourBest) {
3          int[] linkOcarinaOfTimeLevels = new int[5];
4          linkOcarinaOfTimeLevels[0] = 0;
5          linkOcarinaOfTimeLevels[1] = 2;
6          linkOcarinaOfTimeLevels[2] = 4;
7          linkOcarinaOfTimeLevels[3] = 6;
8          linkOcarinaOfTimeLevels[4] = 8;
9
10         int[] linkMajorasMaskLevels = linkOcarinaOfTimeLevels;
11
12         LinkAdventure linkAdventure = new LinkAdventure();
13         linkAdventure.slayMonsters(linkOcarinaOfTimeLevels);
14         linkAdventure.slayMonsters(linkMajorasMaskLevels);
15
16         for (int eachLinkLevel : linkOcarinaOfTimeLevels) {
17             System.out.print(eachLinkLevel + " ");
18         }
19     }
20     private void slayMonsters(int[] linkLevels) {
21         for (int i = 0; i < linkLevels.length; i++) {
22             linkLevels[i] = linkLevels[i] + 2;
23         }
24     }
25 }

```

- A. 2 4 6 8 10
- B. 0 2 4 6 8
- C. 0 0 0 0 0
- D. 4 6 8 10 12



Important: Try out the Java Challenge **before** seeing the answer.

Explanation:

The key to solving this challenge is to know that an array in Java is actually an object. When you instantiate an object and assign it to a variable, the object itself will be stored in the memory heap of the JVM; the variable will simply store a reference that the JVM will use to locate the object and retrieve it for the Java application.

As long as the object is still accessible in the application, it will remain alive in the memory heap. How do we know whether an object is still accessible? When there is a variable that is still able to access the object.

Let's analyze the challenge step by step.

1 - First we declare an array of int variables, then we instantiate an array of int with a total size of 5, and finally we assign the new array object to the linkOcarinaOfTimeLevels variable. Then we populate the numbers 0, 2, 4, 6, and 8 into the array indexes from 0 to 4.

```
int[] linkOcarinaOfTimeLevels = new int[5];
```

2 - The following line of code is another big key to this challenge. We are assigning the reference to the array object, linkOcarinaOfTimeLevels, to linkMajorasMaskLevels. This means that both variables will be pointing to the same object—they just have different names.

```
int[] linkMajorasMaskLevels = linkOcarinaOfTimeLevels;
```

3 - Then we invoke the slayMonsters method and pass the linkOcarinaOfTimeLevels array to it. During the method invocation, we basically add 2 to each element in the array. Therefore, in this iteration the array values will be 2 4 6 8 10.

```
linkAdventure.slayMonsters(linkOcarinaOfTimeLevels);
```

4 - Finally, we invoke the slayMonsters method with the other array variable reference, linkMajorasMaskLevels, which points to the same object in the memory heap. Because both array variables are pointing to the same object, the effect of invoking the slayMonsters method twice will be to have 2 added to each element's value twice. Therefore, we will have the values 4 6 8 10 12.

```
linkAdventure.slayMonsters(linkMajorasMaskLevels);
```

In conclusion, the correct alternative is:

D - 4 6 8 10 12

3.7 Bank Deposit/Withdraw Challenge

Your goal in this challenge is to make use of the object-oriented programming concepts you've learned in this chapter. The challenge is based on a banking theme and is in two parts. First you will create an Account class and methods to withdraw and deposit funds in an Account instance. Then you will create a method to transfer money between Accounts.

3.7.1 Deposit Bank's Money

Your first step is to create a class with the name Account, the attribute balance, and a constructor receiving the initial value from balance and initializing the balance instance variable.

Then you'll create the deposit method, which will receive an amount to add to the balance. This method can't receive a negative number—if that happens you have to print the message "Negative numbers are not accepted" and you can't execute the deposit logic.

3.7.2 Withdraw Bank's Money

Next, create a `withdraw` method that will receive an amount and subtract this amount from the balance. You have to make sure the available balance is large enough to withdraw the requested amount; if the amount is greater than the balance you must print "The amount exceeds the balance" and not perform the withdraw action.

You'll also need to create a method called `getBalance` that will return the value of the balance variable.

Finally, create a class named `AccountTest` with a main method that performs the following steps:

- 1 Instantiate an `Account` object with a value of 1000 for the balance.
- 2 Print the value of the balance using the `getBalance` method.
- 3 Invoke the `withdraw` method, passing the value 100.
- 4 Print the balance value and make sure it's 900.
- 5 Try to withdraw an amount of 1000.
- 6 Print the balance and make sure it's 900 again, because it's not possible to withdraw 1,000 from 900.
- 7 Invoke the `deposit` method with a negative number, -100.
- 8 Make sure the balance is still 900, because negative numbers are not accepted.
- 9 Try to withdraw an amount of 900.
- 10 Print the balance and make sure it is 0.

3.7.3 Transfer Bank's Money

You can use the `Account` class from the first part of this challenge to complete this part. Your goal here is to develop the `transfer` method of the `Account` class. The `transfer` method will receive a `transferAmount` and another `Account`, the recipient, as parameters.

The logic of this method is to subtract the `transferAmount` from the current `Account`'s balance and add the `transferAmount` to the recipient `Account`'s balance.

If the `transferAmount` is greater than the balance of the origin `Account` (the current instance), print "The amount exceeds the balance" and do not perform the transfer action.

Finally, create a class named `TransferTest` with a main method that performs the following actions:

Instantiate an `Account` with the name `origin`, passing a balance of 1000 into the constructor.

Instantiate another `Account` with the name `recipient`, again passing a balance of 1000 into the constructor.

Invoke the origin `Account`'s `transferAmount` method, passing an amount of 100 and the recipient `Account` you created.

Print the balance of the origin `Account` and make sure it's 900, then check the recipient `Account` to make sure its balance is 1100.

Invoke the `transferAmount` method again from the origin `Account`, passing `1000` and the recipient `Account` as parameters.

Make sure your validation message is printed and check that the balance of the origin `Account` is still `900` and the balance of the recipient `Account` is still `1100`.

Disclaimer

This chapter's real-world challenge was purposefully designed in a simple way to test how well you've understood the concepts introduced here. In a real project you would likely have to add more validation checks, externalize these into different classes, and so on, but for now that's not relevant.

You would also want to create unit tests to test the code and make sure nothing gets broken when, for example, you refactor it (make it cleaner and better). You could then simply run the automated test after making any changes to make sure everything still works. Another great benefit of using unit tests is that it encourages you to create better and more organized code from the outset, because you know you'll be testing it later.

As a bonus, you can try to create your own unit test—it's out of the scope of this chapter, but there is a unit test in the source code that you can check out. You're basically doing the same as you did in the main methods; if you take a look at the test code you'll understand.

Congratulations! You've completed a very important part of your Java education. Java relies strongly on object-oriented concepts, and by absorbing the concepts of classes, instances, attributes, and object references you've taken a crucial step in your software engineering career.

3.8 Summary

In this chapter you learned how to abstract the real world and transform it into code by using the object-oriented programming paradigm.

- You know how to create classes with meaningful names.
- You know how to create instances of those classes (objects), and how to manipulate the current instance by using the keyword `this`.
- You practiced working with methods and learned that they accept parameters and return values (primitive types or objects) or perform actions (these methods are declared `void`).
- You learned how to work with instance attributes and references.
- You learned how to create flexible constructors in case you need a way to instantiate objects either with or without attributes by declaring a default constructor and a customized constructor, and you know how to make efficient use of constructors by initializing important attributes.
- You also know that object variables are not really the objects themselves, but references pointing to the objects in the memory heap of the JVM.

4 Encapsulation access modifiers, and package structure

This chapter covers:

a class has too many responsibilities and does too many things, we would be also breaking the concept of encapsulation.

The concept of encapsulation is vastly misunderstood by many software engineers.

Understanding the concept of encapsulation is a game-changer for any software engineer. Just by

Restricting attributes and methods are not enough to create a well-encapsulated system.

Instead, it's necessary to think carefully about the code structure and how different parts of the system will communicate with each other.

If a class has too many responsibilities and does too many things, we would be also breaking the concept of encapsulation.

Understanding the concept of encapsulation is a game-changer for any software engineer. Just by using this concept, the code gets very flexible and maintainable.

4.1 Programming to interfaces

Simply put, programming to interfaces is to program in such a way we will be communicating different classes through meaningful methods, so that when required code changes will be easier.

A real example of encapsulation is a car, we don't need to know how a car works behind the scenes to drive one, instead, we should access the 'interface' (methods) from the car in order to drive, such as `drive`, `switchGear`, `break`, `turnLeft`, `turnRight`. We don't need to know all the specific components of a car. We need to know what the car does and not, how the car works.

The parts of a car are already encapsulated and we only have to understand it in detail if we have to maintain or fix some specific part of the car.

In the object-oriented paradigm, we can use the same analogy, we shouldn't expose an important attribute of a class, we should instead create an interface (meaningful methods) that will be clear on what is supposed to accomplish, not how it works.

The interface of a car is very intuitive, we know for example that if we access the `accelerate` method the car will move and if we access the `turnRight` method we will turn right as well.

By applying the same principles of hiding internal functionalities from a class and externalizing methods that are meaningful to other parts of the system, we will be using encapsulation.

Let's see the example of the Car class in code then:

```
1 public class Car {
2
3     // Attributes here, example: engine, wheels, radiator bean, fuel distribution pi\
4     pe...
5
6     public void accelerate() {
7         System.out.println("Accelerating..");
8     }
9     public void breakAcceleration() {
10        System.out.println("Breaking...");
11    }
12    public void turnLeft() {
13        System.out.println("Turning left...");
14    }
15    public void turnRight() {
16        System.out.println("Turning right...");
17    }
18
19 }
```

As you could see in the code above, we hid the attributes and only exposed the interface methods that we should interact with. And that should be enough, we shouldn't need to know a class works behind the scenes but only know what it does.

4.2 What are and how to use getters and setters?

The use of getters and setters have the purpose of exposing private attributes to be read and written.

Using getters and setters for everything in real applications is in some way common and at the same time wrong because often developers are breaking the concept of encapsulation.

We are not supposed to create getters and setters for every attribute, instead, we should consider if it's really necessary to expose.

Let's see an example of a Customer class exposing all fields with getter and setter:

```
1  public class Customer {
2
3      private String name;
4      private String dateOfBirth;
5      private String address;
6
7      public String getName() {
8          return name;
9      }
10
11     public void setName(String name) {
12         this.name = name;
13     }
14
15     public String getDateOfBirth() {
16         return dateOfBirth;
17     }
18
19     public void setDateOfBirth(String dateOfBirth) {
20         this.dateOfBirth = dateOfBirth;
21     }
22
23     public String getAddress() {
24         return address;
25     }
26
27     public void setAddress(String address) {
28         this.address = address;
29     }
30
31 }
```

Even better would be to make the Customer class more restrictive and not allow the use of setter methods and transform every attribute in final. For doing that, we have to pass the values on the constructor:

```
1 public class MoreRestrictiveCustomer {
2
3     private final String name;
4     private final String dateOfBirth;
5     private final String address;
6
7     public MoreRestrictiveCustomer(String name, String dateOfBirth, String address) {
8         this.name = name;
9         this.dateOfBirth = dateOfBirth;
10        this.address = address;
11    }
12
13    public String getName() {
14        return name;
15    }
16
17    public String getDateOfBirth() {
18        return dateOfBirth;
19    }
20
21    public String getAddress() {
22        return address;
23    }
24
25 }
```

Note that the approach of passing values into the constructor is only a good one if we have at the maximum three parameters, more than that, it gets confusing.

Let's see what approach we can adopt to create complex objects in the next section.

4.3 The Builder Design Pattern

For developing simple code, we don't need too much sophistication but when code gets complex and connecting pieces get harder, then we need to use more elaborated techniques and one of the techniques that are very powerful is design patterns.

Design Patterns are common code patterns used to solve different code situations in an elegant way that is flexible and maintainable.

For solving the 'too many parameters in the constructor problem' and many others, the Builder Design Pattern was created by highly experienced developers.

The `Builder` creational pattern is very famous and useful, in real-projects it's vastly used. A powerful way to use the `Builder` pattern is by making the Object immutable, this means that once the object is created, the state (attributes) can't be changed.

The advantages of using the `Builder` pattern are:

- Immutability, there is no way to change the object values (To have setters in a data object may be a root-cause of bugs since it's easy to change the value)
- Readability, it's far clearer to build a data object by using the builder pattern
- Flexibility, we can easily change the builder structure since it's encapsulated only in the data object we are using it.

4.3.1 When to use the `Builder` pattern?

When a data object is complex to build and requires a lot of initializations

When there is a constructor receiving more than 3 parameters

Where there are many setters in a data object.

When we don't want data to be changed all over the system

Now that we know the advantages and when to use the builder pattern, let's analyze a code example where we will see how the pattern works in practice.

In the following code, we will have a class named as `BigBoss`, with 5 attributes declared as private, then we will use a static inner class named as `Builder` with the same 5 attributes from `BigBoss`. Then we will create public methods into the `Builder` inner class that will take all the values, assign to the builder instance and then return 'this' builder instance.

After assigning values to the 5 `Builder` attributes, we will create the `build` method that will return and build the `BigBoss` object. To accomplish that in an immutable way, we need to create a private constructor into the `BigBoss` class that will receive the builder and then we will assign the values we received in the builder to the `BigBoss` object. Therefore, we will return an instance from `BigBoss` in the `build` method passing 'this' that is the current instance from the builder and we will finally have the Object fully created, ready to be used.

If the explanation is still a bit difficult to understand, by reading the code it will get easy:

```
1 public class BigBoss {
2
3     private final String stealthCloth;
4     private final String primaryWeapon;
5     private final String secondaryWeapon;
6     private final String meleeWeapon;
7     private final String[] medicineItems;
8
9     public static class Builder {
```

```
10
11     private String stealthCloth;
12     private String primaryWeapon;
13     private String secondaryWeapon;
14     private String meleeWeapon;
15     private String[] medicineItems;
16
17     public Builder(String stealthCloth, String primaryWeapon) {
18         this.stealthCloth = stealthCloth;
19         this.primaryWeapon = primaryWeapon;
20     }
21
22     Builder withExtraWeapons(String secondaryWeapon, String meleeWeapon) {
23         this.secondaryWeapon = secondaryWeapon;
24         this.meleeWeapon = meleeWeapon;
25         return this;
26     }
27
28     Builder withMedicineItems(String[] medicineItems) {
29         this.medicineItems = medicineItems;
30         return this;
31     }
32
33     public BigBoss build() {
34         return new BigBoss(this);
35     }
36 }
37
38 private BigBoss(Builder builder) {
39     this.stealthCloth = builder.stealthCloth;
40     this.primaryWeapon = builder.primaryWeapon;
41     this.secondaryWeapon = builder.secondaryWeapon;
42     this.meleeWeapon = builder.meleeWeapon;
43     this.medicineItems = builder.medicineItems;
44 }
45
46 // Getters
47 }
```

In the following code, we will basically print the information we constructed by using the Builder pattern. Then, note that we are also using the `printf` method. If you are not familiar with this method yet, it will print a String where we use the placeholder `%s` and at the end of this method, you can see that the parameters have to be declared there.

```
1 public class BuilderExecutor {
2
3     public static void main(String[] args) {
4         BigBoss bigBoss = new BigBoss.Builder("Military uniform", "Mk22")
5             .withExtraWeapons("AK-47", "CQC knife")
6             .withMedicineItems(new String[] {"First Aid Kit"})
7             .build();
8
9         System.out.printf("%s , %s , %s , %s , %s", bigBoss.getStealthCloth(),
10             bigBoss.getPrimaryWeapon(), bigBoss.getSecondaryWeapon(),
11             bigBoss.getMeleeWeapon(), bigBoss.getMedicineItems()[0]);
12     }
13
14 }
```

To exercise your knowledge about the Builder pattern, create a builder for the `IronMan` class with the following attributes, name, age, car and specialWeapon. Then create a static inner class `Builder` within it and follow the same process from the `Builder` above.

Then create another class named as `IronManBuilderExecutor` and make use of your builder. Try it out and if you want to check, this exercise will be in the source-code named as `IronManBuilder`, you can check it out also but it's important that you practice it before.



Note: Lombok

Alternatively, to create getters, setters, builder and other boiler-plate code for data objects you can use Lombok. With Lombok, you use annotations, e.g. `@Getter`, `@Setter`, `@Builder`, and the code you need will be generated during compilation time. For more about Lombok, check the following link: <https://projectlombok.org/>¹

4.4 Organizing Packages in a Real Project

The main purpose of packages in a project is to organize a group of classes into packages that are named with a clear name so classes can be found more easily.

Imagine if we had to put all classes in just one package, it would be terrible to find classes and understand how the system is structured. In real Java projects, it's easy to find at least 100 classes and that's why it's so useful to create packages and organize our classes there.

¹<https://projectlombok.org/>

4.4.1 Why packaging?

Packaging is extremely important because in big projects, it gets really hard to find classes without organization. Even in small projects it's always a good practice to divide and organize the code in a clear way that is grouped in different sections where files can be easily found.

We can use the analogy of categories when we visit a blog website, if we look for categories, it's far easier to find what we want. If we go to the `javachallengers` website for example and look for the category Java streams, we will find only posts that are related to Java streams. The same happens with packages, if we want to look for a service for example, we go to the service package.

4.4.2 How to name packages?

But how to name a package? The universal pattern for naming packages in Java is the following, the domain of a company reversed, the name of the project and then the name from the package to group classes:

```
com.javachallengers.service
```

You must be wondering why do we have to use this extensive name? In Java, it's very common to reuse third-party libraries (a group of useful classes to solve specific necessities) and for this reason, it's very likely that we will find classes with the same names. But if we use a domain name and the project name we should never find a duplicate class name, therefore there won't be any class conflicts.

Let's suppose we have to handle a library to handle special operations with dates and the operation with an external library such as Yoda Time and for coincidence there the classes names are the same from Java and the Yoda Time library, then what would happen? The code wouldn't compile, there would be a conflict and the application wouldn't work.

4.4.3 Creating packages structure in a real-project

In a real-world Java application usually we will have three main layers of abstraction. The resource layer that will have the API endpoints that will usually connect to the front-end layer.

Then the resource layer will invoke the service layer which contains the business requirements logic.

Finally the service that will interact with the database layer code, it's commonly named as `dao` or `repository`.

Those three main packages that will have the core code in a Java application usually would be named as:

```
1 com.company-name.domain-name.system-package
2
3 com.javachallengers.payment.resources
4 com.javachallengers.payment.services
5 com.javachallengers.payment.dao or
6 com.javachallengers.payment.repository
```

Then we would have other packages to assist our three main packages. Some of them are the config package which will configure a web application with initial values we need, it might load resources classes from our application, it will basically initialize the whole application.

Then we also need an exceptions package where we will declare our specific Exceptions for business requirements. We will go deep into exceptions in future chapters:

```
com.javachallengers.payment.exceptions
```

The dtos (Data Transfer Object) package that are the objects used to pass data through the network. We send a dto to the front-end of our application:

```
com.javachallengers.payment.dtos
```

The model layer that are the objects from requirements that represent how is the domain of our application or how the core business data is abstracted.

```
com.javachallengers.payment.model
```

Another package that is common and useful in real-world projects is the `util`. Usually, in the `util` package we can create generic methods for services and avoid code repetition.

“com.javachallengers.payment.util”

The mentioned packages won't be exactly the same for all projects. If we are working with micro-services for example sometimes the feature-name would be removed from the package name since the domain is the micro-service already, in other applications we would have the feature name after the system package. Let's see how those variations would be:

```
1 com.company-name.brief-project-name.system-package.domain-name
2 com.company-name.brief-project-name.system-package
```

I personally like the following package structure when we are using a micro-service that is responsible for only one domain:

```
com.company-name.brief-project-name.system-package
```

The other option I like for projects with more than one domain is the following:

```
com.company-name.brief-project-name.feature-name.system-package
```

If you've been working with Java, you have probably seen many similarities with your project, if you never seen a real-world Java application, now you know how it's usually organized and that will help you to understand a real-world project.

4.5 Encapsulation Patterns

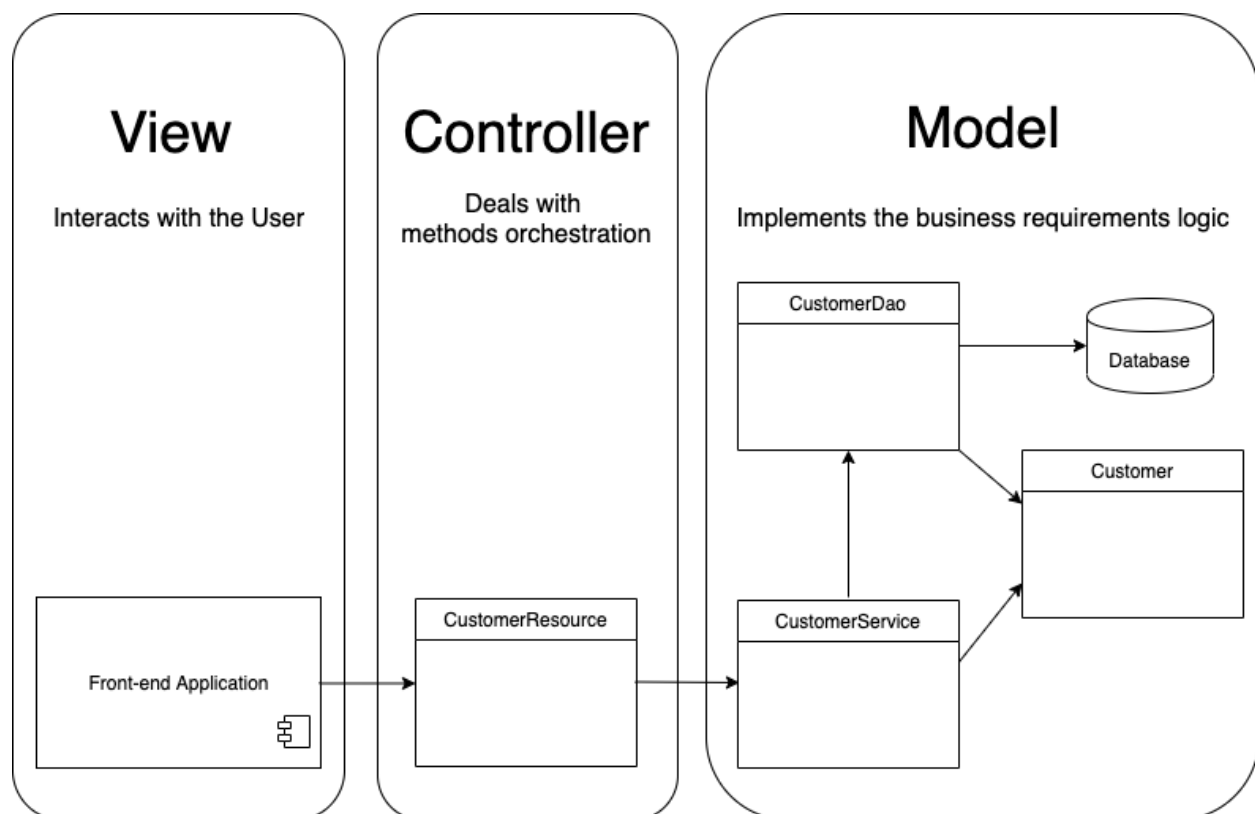
There are many important patterns that make good use of the encapsulation concept. Those patterns help to reduce coupling (how much classes know about each other) and enhance cohesion (how specialized is the class).

Let's explore some of the main encapsulation concepts so that we can also learn how to create encapsulated code.

4.5.1 The MVC Architectural Pattern

The MVC pattern divides the code in three layers, Model which is the layer where there is the business data class, we can also say model class or even domain class. Then the View layer that is the front-end application which will have the user interface where the user will be able to interact with the system. Finally there is the Controller layer that is an intermediate between the View and Model layers which will orchestrate invocations.

Let's analyze the following diagram to make the MVC pattern clear:



As you can see in the diagram above, we are using the MVC pattern that basically separates the application by different layers.

This is a great example of encapsulation, each layer is responsible for dealing with one responsibility, the view layer will interact with the user, the controller layer will orchestrate method invocations between the view and the model layer. The model layer will deal with the business logic and database interaction.

4.5.2 The use of DTO (Data Transfer Object)

When we are dealing with web projects, we have to send an Object via a network protocol, in current applications we very often use the HTTP protocol.

A network request and response is an expensive resource that we should pass only the necessary information to the front-end for example. We also don't need to expose details from the business requirements to every screen of a system.

The DTO pattern solves this problem very well, we can pass an Object with the information we need to the screen all ready to be consumed. With a DTO, we shouldn't include business requirements into it.

To summarize the use of a DTO, let's suppose we have our domain Object named as Customer.

```
1  public class Customer {
2
3      private String name;
4      private String dateOfBirth;
5      private String email;
6      private String login;
7
8      private Product product;
9      private Address address;
10     private PaymentAgreement paymentAgreement;
11
12     // Builder for constructing the object
13     // Getters
14
15 }
```

Then, let's suppose we have a screen in the system that we need to only show the customer's basic information but can you notice the amount of data we have in the Customer domain Object? We have a product, address and the paymentAgreement objects which will have a lot of unused information, we will also use a lot more network resources to traffic this object.

That's why the DTO pattern is very handy to send information to the screen. On this case we just want to send the Customer's basic information we can create the following object:

```
1 public class CustomerDTO implements Serializable {
2     private static final long serialVersionUID = 1L;
3
4     private String name;
5     private String dateOfBirth;
6     private String email;
7     private String login;
8
9     // Builder for constructing the object
10    // Getters
11
12 }
```

As you can notice in the code above, we removed the extra information we don't want and just kept the Customer basic information into the DTO.



Object Serialization:

It's also important to realize that when we send an object in the network, we need to implement `Serializable`. We won't cover serialization in this book but keep in mind that you need to serialize a DTO. Notice also that we need to declare a `serialVersionUID` to avoid data inconsistency when changing the class structure.

4.6 Access Modifiers

In order to apply the concept of encapsulation, we need to learn how to use the access modifiers. There are 4 types of access modifiers:

`public` = It's all opened, all classes can access another `public` class/method/attribute in the project

Let's see a code example with the `public` modifier:

```
1 // Notice the package name
2 package com.javachallengers.encapsulation.accessmodifiers;
3
4 public class ThePublicModifier {
5
6     public String publicAttribute;
7
8     public void thisMethodIsOpenedToBeAccessed() {
9         System.out.printf("Public method!");
10    }
11
12 }
```

Then we can use the class/method/attribute from another package since all of the members from ThePublicModier are public:

```

1  // Notice that the package is different here
2  package com.javachallengers.encapsulation.accessmodifiers.accesstopublic;
3
4  import encapsulation.accessmodifiers.ThePublicModifier;
5
6  public class ThePublicModifierTest {
7
8      public static void main(String[] args) {
9          ThePublicModifier publicModifier = new ThePublicModifier();
10         publicModifier.thisMethodIsOpenedToBeAccessed();
11         System.out.printf(publicModifier.publicAttribute);
12     }
13 }

```

protected = The class/method/attribute is restricted to the package they are in and when there is inheritance, they can be accessed in other packages.

Let's create a class that has a protected attribute and method to see in practice how that works in code:

```

1  package com.javachallengers.encapsulation.accessmodifiers.accesstoprotected;
2
3  public class ProtectedModifier {
4
5      protected String protectedAttribute;
6
7      protected void print() {
8          System.out.println("Protected method! This method is accessible in this pa\
9  ckage or outside with inheritance");
10     }
11 }

```

In the following code as we are working in the same package of ProtectedModifier class, we can access everything.

```

1  package com.javachallengers.encapsulation.accessmodifiers.accesstoprotected;
2
3  public class ProtectedModifierTest {
4
5      public static void main(String[] args) {
6          ProtectedModifier protectedModifier = new ProtectedModifier();
7          System.out.println(protectedModifier.protectedAttribute);
8          protectedModifier.print();
9      }
10 }
11
12 package com.javachallengers.encapsulation.accessmodifiers;    // #1
13
14 import encapsulation.accessmodifiers.accesstoprotected.ProtectedModifier;
15
16 public class ProtectedModifierOutsidePackage extends ProtectedModifier { // #2
17
18     public static void main(String[] args) {
19         ProtectedModifier protectedModifier = new ProtectedModifier();
20         // protectedModifier.print();                // #3
21
22         // We are using inheritance here
23         ProtectedModifierOutsidePackage outsidePackage = new ProtectedModifierOutsi\
24 dePackage();                // #4
25         outsidePackage.print();
26         System.out.println(outsidePackage.protectedAttribute); // #5
27     }
28 }

```

- #1: Notice we are in a different package
- #2: We are using inheritance when we are extending the ProtectedModifier class
- #3: We can't access it here without inheritance
- #4: Since we are using inheritance here, we have access to the method
- #5: We also have access to the attribute

default or when we don't specify one = The class/method/attribute will be only accessible in the package they are created.

```

1 package com.javachallengers.encapsulation.accessmodifiers.accesstodefault;
2
3 public class DefaultModifier {
4
5     String defaultAttribute; // #1
6
7     void print() {           // #2
8         System.out.println("This method can only be accessed inside the accesstodefa\
9 ult package");
10    }
11
12 }

```

- #1: This attribute can only be accessed inside the accesstodefault package
- #2: We don't declare explicitly that we are using the default access modifier. If we are not using public, protected or private we are using the default access modifier.

Since we are in the same package, we can access everything:

```

1 package com.javachallengers.encapsulation.accessmodifiers.accesstodefault;
2
3 public class DefaultModifierTest {
4
5     public static void main(String[] args) {
6         DefaultModifier defaultModifier = new DefaultModifier();
7         System.out.println(defaultModifier.defaultAttribute);
8         defaultModifier.print();
9     }
10 }
11
12 private = The class/method/attribute can be only accessed in its own class:
13
14 package com.javachallengers.encapsulation.accessmodifiers.accesstoprivate;
15
16 public class PrivateModifier {
17
18     private String privateAttribute; // #1
19
20     private void print() {           // #2
21         System.out.println("This method can only be accessed inside the PrivateModif\
22 ier class");
23     }

```



```

24
25 }

```

- #1: This attribute can only be used within the `PrivateModifier` class
- #2: This method can only be accessed inside the `PrivateModifier` class

4.6.1 Less Restrictive Modifiers Challenge

In the following challenge, your goal is to make the classes compile and run successfully. We have the `LessRestrictiveModifiers` class which will have all the access modifiers in instance variables in one package and the `LessRestrictiveModifiersExecutor` class which will use the attributes.

Your goal is to replace the `// Insert code here` comment with code that will work and run successfully in the main method from the `LessRestrictiveModifiersExecutor` class using the attributes from the `LessRestrictiveModifiers` class.

Which of the following alternatives will compile and run? (More than one alternative might be correct)

```

1  package com.javachallengers.encapsulation.challengers.access;
2
3  public class LessRestrictiveModifiers {
4
5      public String publicVariable = "public";
6      protected String protectedVariable = "protected";
7
8  }
9
10 package com.javachallengers.encapsulation.challengers;
11
12 import encapsulation.challengers.access.LessRestrictiveModifiers;
13
14 public class LessRestrictiveModifiersExecutor {
15
16     public static void main(String[] args) {
17         LessRestrictiveModifiers challenger = new LessRestrictiveModifiers();
18         // Insert code here
19     }
20 }

```

- `System.out.println(challenger.publicVariable);`
`System.out.println(challenger.protectedVariable);`
- Extend the `LessRestrictiveModifiers` in the `LessRestrictiveModifiersExecutor` class and insert the following code on the main method:

```
1      System.out.println(challenger.publicVariable);
2      System.out.println(new LessRestrictiveModifiersExecutor().protectedVariable);
```

C. Move the `LessRestrictiveModifiers` class to the `com.javachallengers.encapsulation.challengers` package, rename the package declaration to the moved package and insert the following code on the main method:

```
1      System.out.println(challenger.publicVariable);
2      System.out.println(challenger.protectedVariable);
```

D. Extend the `LessRestrictiveModifiers` in the `LessRestrictiveModifiersExecutor` class and insert the following code on the main method:

```
1      System.out.println(challenger.publicVariable);
2      System.out.println(challenger.protectedVariable);
```

Let's analyze first the incorrect alternatives, the alternative A is incorrect because we are trying to access our `protectedVariable` from another package without inheritance, therefore the code won't compile. Also note that we are using the `publicVariable` which can be accessed everywhere in the project so we won't have a compilation error with the public access modifier.

Then there is the alternative D that is tricky, the `publicVariable` will be accessed normally. The problem is the `protectedVariable`. Even though we are using inheritance, we are accessing the `LessRestrictiveModifiers` class directly so we are not even using inheritance in our code implementation. We are just declaring it in the `LessRestrictiveModifiersExecutor` but we are not using it at all.

Let's now analyze the correct alternatives, the alternative B is correct. Again, when accessing the `publicVariable` will be all fine and the interesting point of this alternative is that we are making use of inheritance in the code implementation. Even though we are in a different package, we can use the `protectedVariable` because we are creating an instance from the class that extends the `LessRestrictiveModifiers` class, therefore we can access the `protectedVariable` with no problem.

Finally, the alternative C is also correct. The `publicVariable` will be accessed normally. The key point of this alternative is that we are moving the `LessRestrictiveModifiersExecutor` class to the same package of the `LessRestrictiveModifiers` class, therefore we will be able to access the `protectedVariable` normally.

4.6.2 Restrictive Modifiers Challenge!

In the following Java Challenge, we will have a `private` and `default` variables in the `MoreRestrictiveModifiers` class and then we will try to use those variables in the

MoreRestrictiveModifiersExecutor and your goal is to figure out what code will compile by replacing it in the main method from the MoreRestrictiveModifiersExecutor class.

What of the code alternatives you can insert in the main method from the MoreRestrictiveModifiersExecutor class that will compile and run with no errors? (**More than one alternative might be correct**)

```
1 package com.javachallengers.encapsulation.challengers.access;
2
3 public class MoreRestrictiveModifiers {
4
5     private String privateVariable = "private";
6     String defaultVariable = "default";
7
8     public static void main(String[] args) {
9         // Insert code here
10    }
11
12 }
13
14 package com.javachallengers.encapsulation.challengers;
15
16 import encapsulation.challengers.access.MoreRestrictiveModifiers;
17
18 public class MoreRestrictiveModifiersExecutor {
19
20     public static void main(String[] args) {
21         MoreRestrictiveModifiers challenger = new MoreRestrictiveModifiers();
22         // Insert code here
23     }
24 }
```

A) Insert the following code in the MoreRestrictiveModifiersExecutor class:

```
System.out.println(challenger.defaultVariable);
```

B. Insert the following code in the MoreRestrictiveModifiersExecutor class:

```
System.out.println(challenger.privateVariable);
```

C. Change encapsulation.challengers to challengers.access from the MoreRestrictiveModifiersExecutor class and move it to the com.javachallengers.encapsulation.challengers package. Then insert the following code in the main method from MoreRestrictiveModifiersExecutor:

```
System.out.println(challenger.defaultVariable);
```

D. Insert the following code in the MoreRestrictiveModifiers class:

```
System.out.println(challenger.privateVariable);
```

Explanation:

Let's start with the wrong alternatives, in alternative A we are trying to access the `defaultVariable` from a different package from where `defaultVariable` was created. The `default` modifier only allows access in the same package.

The alternative B is also wrong because we are trying to access the most restricted access modifier in Java, the `private` one that can be only accessed within the class the variable was created. Therefore, when trying to access a `private` attribute from another class we will have a compilation error.

The alternative C is correct because first moving the `MoreRestrictiveModifiersExecutor` class to the same package as `MoreRestrictiveModifiers` and then we are accessing the `defaultVariable` which will be fine.

The alternative D is also correct because the `privateVariable` is being accessed within the class it was created, therefore it will compile and run successfully.

4.6.3 Main Takeaways of this section

The following table is organized from the less restrictive `public` to the more restrictive `private`.

Before looking at the table, let's analyze each modifier concept.

The `public` access modifier can be accessed everywhere in the project.

The second less restrictive access modifier is `protected` because the `protected` modifier can be accessed outside its package with inheritance. The `default` modifier can't, `protected` is less restrictive than `default`.

Then the `default` access modifier is more restricted than `protected` because we can only access members that are in the same package.

Finally, the `private` access modifier can only be accessed in the same class.

	public	protected	default	private					
different pack- age access	yes	yes, only with inheri- tance	no	no	same pack- age access	yes	yes	yes	no
In the same class	yes	yes	yes	yes					

4.7 Inner Class

Nested classes represent a special type of relationship that is it can access all the members (data members and methods) of outer class including `private`.

When we want to use the concept of encapsulation effectively, inner classes are crucial to be understood. One example where inner classes are vastly used is with the builder pattern.

Sometimes we want to create fake data for our tests, and inner classes come in handy to accomplish this goal.

Let's check the main types of inner classes:

When we want the enclosing class to be also instantiated when we want to utilize the instance inner class, then we should create an instance inner class. Let's see an example of how to work with it:

```

1  public class EnclosingInstanceInnerClass {
2
3      public static void main(String[] args) {
4          EnclosingInstanceInnerClass enclosingInstanceInnerClass = new EnclosingInsta\
5 nceInnerClass(); // #1
6          enclosingInstanceInnerClass.new InstanceInnerClass().print(); // #2
7      }
8
9      private void print() {
10         System.out.println("Text from the enclosing class");
11     }
12
13     class InstanceInnerClass {
14
15         public void print() {
16             new EnclosingInstanceInnerClass().print();
17         }
18     }
19 }
20
21 }
```

- #1: We have to first create an instance from the enclosing class
- #2: Then we are creating a new instance from the enclosing instance. We are using a strange syntax with `EnclosingInstanceInnerClass.new` but that's how it is. We have to do that because we need a new instance from the enclosing class to access the instance inner class.

4.7.1 Local Inner class

The only difference from an instance inner class to a local inner class is that the local inner class will be created in a method and will die after this method is executed. Let's then create an inner class inside a method:

```
1 public class EnclosingLocalInnerClass {
2
3     public static void main(String[] args) {
4         // new CustomPrinter(); It's not possible // #1
5         // new LocalInnerClass().new CustomPrinter(); // #2
6
7         new EnclosingLocalInnerClass().print();
8     }
9
10    void print () {
11        class LocalInnerClass {
12            void customPrint() {
13                System.out.println("Customized print");
14            }
15        }
16
17        new LocalInnerClass().customPrint(); // #3
18    }
19 }
```

- #1: We can't access our CustomPrinter inner class since it's only accessible within the print method.
- #2: We can't create an instance from our enclosing LocalInnerClass to access
- #3: It's only possible to access the inner class in the print method

A situation that can be useful to create an inner class in a method is when we want to rewrite or create a specific functionality, maybe only one method to accomplish one task and we don't want to make it external in the class for example because this inner class should only be visible within this method.

4.7.2 Static inner class

The static inner class is vastly used in real-world projects, mainly with the builder design pattern. It's a very useful inner class because it doesn't depend on the enclosing class to be instantiated, since it's static, we can instantiate it right away.

```

1  public class EnclosingStaticInnerClass {
2
3      public static void main(String[] args) {
4          StaticInnerClass staticInnerClass = new EnclosingStaticInnerClass.StaticInne\
5 rClass(); // #1
6          staticInnerClass.print();           // #2
7      }
8
9      static class StaticInnerClass {
10         private void print() {
11             System.out.println("Static inner class");
12         }
13     }
14 }

```

- #1: As you can see, we don't need to create an instance from the EnclosingStaticInnerClass, we just need to reference the enclosing class to access the StaticInnerClass.
- #2: Reinforcing that we can access private methods since it's an inner class

4.7.3 Anonymous inner class

If we don't want to specify an instance name for our inner class and just want to use some specific methods, a way to deal with this situation is by using an anonymous inner class.

Let's see an example which we want to declare a new method and use it straight away:

```

1  public class AnonymousInnerClass {
2
3      public static void main(String[] args) {
4          new AnonymousInnerClass() {           // #1
5              private void printSomethingElse() { // #2
6                  System.out.println("Printing from the anonymous Inner class");
7              }
8          }.printSomethingElse();                 // #3
9
10
11     }
12
13     private void print() {
14         System.out.println("Print something");
15     }
16 }

```

- #1: We are instantiating the `AnonymousInnerClass` object and then opening brackets to specify we are creating an anonymous inner class
- #2: Then we are creating the `printSomethingElse` method in this anonymous inner class
- #3: Finally, we are invoking the `printSomethingElse()` method and then it will print: "Printing from the anonymous Inner class".

The biggest advantage of an anonymous inner class is that we don't need to specify an object name nor externalize the class. Anonymous inner classes are handy when we want to use a few methods but don't want to create a whole structure of a class.

We will explore the subject of anonymous inner class further in future chapters when we see inheritance and interfaces.

4.8 The static keyword

The purpose of the static keyword is to make classes/methods/attributes static in memory, this means that it's not necessary to create an instance to make use of a static class, method or attribute.

Therefore, when we don't want to instantiate a class and use attributes or methods straightaway we can declare our attributes and methods by using the static keyword.

Methods and attributes declared as static will be ready to be used only by referencing the enclosing class.

One simple and clear way to see the static keyword in Java is to see static attributes or methods as class attributes or class methods.

```
1 public class StaticAttribute {
2
3     static String staticEnergy;           // #1
4
5     public static void main(String... staticAttributes) {
6         StaticAttribute.staticEnergy = "Very Strong"; // #2
7         System.out.println(staticEnergy);           // #3
8     }
9 }
```

- #1: This attribute is already static in the memory-heap.
- #2: It's possible to use the `staticEnergy` variable straight away without the need of instantiating the `StaticAttributes` class. In this line we basically assign "Very Strong" to the `staticEnergy` variable.
- #3: Then we will print "Very Strong".

A static method behaves in a very similar way from a static variable. We also don't need to create an instance from a class to use a static method, we simply use it right away.

Let's see then a code example where we can use a method instantly without the need of an instance by using a static method:

```
1 public class StaticMethod {
2
3     public static void main(String... staticAttributes) {
4         StaticMethod.print();    // #1
5     }
6
7     static void print() {        // #2
8         System.out.println("Static method, no instance required");
9     }
10 }
```

- #1: We are invoking the static method right away, without the necessity of having an instance, that's why a static method is also called as a class method
- #2: We only need to specify the static keyword in the print method to make it static

4.8.1 Limitations of a static method

A static method can only access static members, this means that a static method can't access the current instance by using the keyword `this`. This makes sense since a static method is a class method and not an instance class, therefore we can only access the class members.

Let's see how above example works in code, when we try to execute the following code:

```
1 public class StaticAccessToThis {
2
3     private String name;
4
5     public static void main(String[] args) {
6         print();
7     }
8
9     static void print() {
10        System.out.println(this.name); // #1
11    }
12 }
```

- #1: There will be a compilation error at this line since we are trying to access the current instance in a static context. In a static context there is not any instance.

4.8.2 How to use an instance in a static context?

The only way to use an instance in a static method is to use an instantiated object. Since we are manipulating our instance, there is no need to have an instance context, we can simply use our instance.

In the following code we will see an example with a static method that will receive an object instance to demonstrate how we can use instances in a static method.

```
1 public class InstanceInStaticContext {
2
3     String value;
4
5     public static void main(String[] args) {
6         InstanceInStaticContext instance = new InstanceInStaticContext();
7         instance.value = "It's an instance";
8         print(instance);
9     }
10
11     static void print(InstanceInStaticContext instance) {
12         System.out.println(instance.value);
13     }
14 }
```

4.8.3 Static attribute values are all the same for instances

Since a static attribute is actually a class attribute, when we create a new instance, there won't be a specific value for this instance, instead, the static value will be the same for all instances.

As mentioned before a static variable or method are not connected to instances, therefore it doesn't matter if we create a new instance or not.

Let's see a code example where we create different instances and change a static value to see what will happen:

```
1 public class StaticValuesWithInstances {
2
3     static String name;
4
5     public static void main(String[] args) {
6         StaticValuesWithInstances staticValues1 = new StaticValuesWithInstances();
7         staticValues1.name = "value1";           // #1
8
9         StaticValuesWithInstances staticValues2 = new StaticValuesWithInstances();
```

```
10         staticValues2.name = "value2";           // #2
11
12         System.out.println(staticValues1.name); // #3
13         System.out.println(staticValues2.name); // #4
14     }
15 }
```

- #1: This syntax works but it's not appropriate since we are not manipulating the instance values but the static value. The variable name will have the "value1" assigned
- #2: The same case from above, since we are accessing a static variable, the name variable is actually being shared between instances.
- #3: It will print "value2" since we are using a class attribute
- #4: It will print "value2" too

It's important to emphasize that we should never access a static member by using an instance, it's confusing to read a code like this. We should only know that it's possible to do that in Java but should never use this kind of syntax.

We should always reference the class name instead of the instance name when manipulating a static member like the following:

```
1  ClassName.staticVariable;
2  ClassName.staticMethod();
```

By following the mentioned pattern, it's very clear that we are dealing with static members. The code get more readable and easier to understand.

4.8.3 Main differences between instance and static

- Instance methods can access instance attributes and instance methods in the current instance
- Instance methods can access static variables and static methods
- Instance variable values can only be used with a created instance
- Instance methods can only be invoked with an instance
- Static methods can only access static variables and methods directly
- Static variable will share the same value to all instances

4.8.4 When to use a static attribute or method?

In a real-world project we usually use static attributes when we want to declare a constant attribute value. Since the value will be the same no matter if a new instance is created, it's a common approach to see constant attributes declared as static:

```
1 public class StaticAttributesRealWorld {
2     static final String USER_NAME = "javachallengers"; // #1
3
4     public static void main(String[] args) {
5         System.out.println(USER_NAME);                // #2
6     }
7 }
```

- #1: We used the Java Beans convention in the USER_NAME variable name and we are using the final keyword to specify it's a constant value, it can't be changed. Finally, it's a static variable, it means there is no need to create an instance to access it.
- #2: We used the static variable without an instance

4.8.4 Static Methods

Similarly to static attributes, static methods follow the same concept. A static method can be invoked without an instance.

In the following code, we will explore the invocation of a print static method. Note that we also don't need to create an instance to invoke the static method:

```
1 public class StaticMethod {
2
3     static void print() {           // #1
4         System.out.println("Static method, no instance required"); // #2
5     }
6
7     public static void main(String... staticAttributes) {
8         StaticMethod.print(); // #3
9     }
10
11 }
```

- #1: Declaring the static print method
- #2: Printing a message
- #3: Note that we are referencing the class name, this is a good practice because by looking at the code we can realize it's a static method. Then we are invoking the print method and finally printing "Static method, no instance required".

4.8.5 When to use static methods?

You can ask yourself, can this method be invoked without any instance variable state? Is this method simple enough? Only then you should make the decision of creating a static method.

One other very important point to consider about the use of static methods is that implementing Unit Tests gets harder. It's harder to test methods in an isolated way when we are using static methods. Unit Tests are not in the scope of this book but it's important to keep this principle in mind too so we can make the best decision.

There are many good JDK examples we can rely on. The Math class is a perfect example. Let's see some of the methods in action:

```
1 public class MathUsage {
2
3     public static void main(String[] args) {
4         System.out.println(Math.pow(2, 2));           // #1
5         System.out.println(Math.max(2, 7));           // #2
6         System.out.println(Math.min(1, 7));           // #3
7         System.out.println(Math.ceil(3.1));           // #4
8         System.out.println(Math.floor(3.6));          // #5
9         System.out.println(Math.decrementExact(2));   // #6
10        System.out.println(Math.random());            // #7
11    }
12
13 }
```

- #1: We are applying the power of 2 with 2, therefore it will print 4
- #2: We are verifying what number is greater, 2 or 7, therefore it will print 7
- #3: We are verifying what number is smaller 1 or 7, therefore it will print 1
- #4: We are rounding the number 3.1 to the next, therefore it will print 4
- #5: We are rounding the number 3.6 down, therefore it will print 3
- #6: We are decrementing a exact number to 2, therefore it will print 1
- #7: We are printing a random number

As you could see, in the examples of the Math class, we have simple and clear methods that don't need any kind of object state, we simply pass the variable to each method and each method returns what we want. If we have a similar case such as the Math class, it's a good idea to use static methods. Keep in mind that we will usually use static methods for creating utility methods.

This technique of looking at the JDK code is a great habit we can apply for the usage of anything we want in our application. We can learn a lot only by looking at the JDK code.

4.8.6 Static Simpson Java Challenge

In the following challenge we will explore the main principles from an instance and static variable. Can you guess what will be the output of the following challenge?

```
1 public class StaticSimpson {  
2  
3     String name;  
4     static int age;  
5  
6     public static void main(String[] args) {  
7         StaticSimpson homer = new StaticSimpson();  
8         homer.name = "Homer";  
9         homer.age = 35;  
10  
11        StaticSimpson bart = new StaticSimpson();  
12        bart.name = "Bart";  
13        bart.age = 10;  
14        bart.age++;  
15  
16        System.out.println(homer.name + bart.name);  
17        System.out.println(homer.age + bart.age);  
18    }  
19 }
```

- A. BartBart
45
- B. HomerBart
45
- C. HomerHomer
22
- D. HomerBart
22

Explanation:

The important points of this challenge is that an instance value will belong to an instance and a static value will belong to the class.

Note that the name variable is an instance variable which means that the instance of homer will have the value of "Homer" and bart will have the value of "Bart" in the name.

In the age variable, we are dealing with a class variable, therefore when we change the age variable in homer and bart, it doesn't matter, the static variable will be the same for all instances since it's a class variable. Therefore, when we change the class variable age to 10, it will be 10 for every instance.

Just keep in mind that accessing a static variable with an instance variable is a bad practice, never do that when developing your code, this challenge was created like this to help you to understand the instance and static concepts. You should instead use a static variable like this, 'StaticSimpson.age'.

The last detail of this challenge is that we are also adding one to the age variable and then it will be 11 for homer and bart.

In conclusion the correct alternative will be:

```
1 D) HomerBart
2     22
```

4.9 Instance and static block

There are alternatives for initializing variables or executing logic in Java. With the use of instance and static blocks, we can easily do that.

4.9.1 Instance block

Instance blocks might be useful for initializing variables. This concept is not very used in real applications but it's useful to know what and how to use it.

The instance block will be initialized when the instance is created.

Let's see first the practical use of an instance block and after this, we will understand when to use it.

An instance block has a similar purpose of a constructor, the difference is that the instance block will be executed before the constructor.

In the following example we will invoke the print method from the instance block and only then initialize the name variable, note that the listing numbers are in execution order:

```
1  class InstanceBlockPrint {
2
3      String name;
4      {                               // #2
5          this.print();               // #3
6      }                               // #4
7
8      InstanceBlockPrint() {          // #5
9          name = "Challenger";        // #6
10     }
11
12     public static void main(String[] args) {
13         InstanceBlockPrint instanceBlock = new InstanceBlockPrint(); // #1
14         instanceBlock.print();      // #7
15     }
```

```
16     }
17
18     void print() {
19         System.out.println(name);
20     }
21
22 }
```

- #1: The very first code execution is when we instantiate the InstanceBlockPrint class.
- #2: Then what is going to be initialized is the instance block, the { means that we are initializing the instance block.
- #3: We are invoking the print method that will print null here.
- #4: We are closing the instance block with }.
- #5: We are declaring our constructor, only now the constructor will be invoked.
- #6: Only now we will initialize the name variable with the value of "Challenger".
- #7: Finally we invoke the print method and "Challenger" is printed.

4.9.2 When to use instance blocks?

Usually, constructors are used for initializing instance variables but very often we need to add complex logic before initializing variables into the constructor.

Instance block is very handy when we want to execute logic operations before the constructor execution, we can encapsulate better our logic execution by making good use of instance blocks.

4.9.3 Static Block

Static blocks have the same purpose of instance blocks, initializing variables or executing logic operations. But the main difference between both blocks is when the static block is loaded.

At the moment we access the class the static block will be executed. Differently than the instance block that will be only invoked if we create an instance.

The static block will be initialized before instantiation and the constructor invocation.

Let's see a practical example of a static block. We will create a static block and will only access the class name in the main method, this will be enough to get the static block invoked.


```
1 public class StaticBlockPrint {
2
3     static String name;           // #1
4
5     static {                       // #2
6         name = "Challenger";      // #3
7     }
8
9     public static void main(String[] args) {
10         System.out.println(StaticBlockPrint.name); // #4
11     }
12 }
```

- #1: Declaration of the name static variable.
- #2: Declaration from the static block.
- #3: Initializing the name variable with "Challenger".
- #4: Printing the name variable, note that at the StaticBlockPrint class will be loaded at the moment we first access the class in some way. In our case we accessed the class variable which also triggered the variable declaration and then the static block execution. Therefore the output will be "Challenger".

4.9.4 Mario Bros Block Challenge

In the following challenge we will explore the concept of instance and static blocks when we are instantiating a new class. Your objective is to figure out what is the output of the following challenge when running the main method:

```
1 public class MarioBrosBlockChallenge {
2
3     {
4         System.out.print("instance ");
5     }
6
7     static {
8         System.out.print("static ");
9     }
10
11     MarioBrosBlockChallenge() {
12         System.out.print("constructor ");
13     }
14
15     public static void main(String... args) {
```

```
16         System.out.print("Jump into ");
17         new MarioBrosBlockChallenge();
18         new MarioBrosBlockChallenge();
19         System.out.print("blocks.");
20     }
21 }
```

- A. static Jump into instance constructor instance constructor blocks.
- B. Jump into static instance constructor static instance constructor blocks.
- C. Jump into static constructor instance constructor instance blocks.
- D. static Jump into constructor instance static constructor instance blocks.

Explanation:

When we are working with a static block remember that it will be executed only for the class and not for each instance. Note also that the static block will be always the first code to be executed because the class has to be the first to be loaded, without a class it's impossible to create an instance or even to invoke the main method. Therefore the first text to be executed will be static.

Then, after we get the `MarioBrosBlockChallenge` class loaded along with the static block, the first text from the main method is printed, then we get `Jump into`.

In the following line we are instantiating the `MarioBrosBlockChallenge` class which will invoke then our instance block printing instance.

After the instance block is executed, the constructor will be invoked therefore printing constructor.

Then we will create a `MarioBrosBlockChallenge` instance again and remember that the static block won't be executed again since the class was already loaded, a class is loaded only once. Therefore only the instance block and the constructor will be executed and the following will be printed, instance and constructor.

In conclusion, the correct alternative for this challenge is:

A - static `Jump into` instance constructor instance constructor blocks.

4.10 Small Real-World Project Challenge

Your objective in this challenge is to create a small real-world application by using concepts of encapsulation.

Before starting this challenge, keep in mind that the following example is fictitious and depends on future concepts to be 'perfect' for the real-world. The main focus of this challenge is to make the concept of encapsulation clear to you not to necessarily create a 'real-application'.

The key point of this challenge is to make you absorb the essence of a real application.

4.10.1 Step 1 Creating the Logic Classes:

You will create the three main layers we discussed in this chapter, `com.javachallengers.resources`, `com.javachallengers.services` and `com.javachallengers.dao`. Then you will create a `CustomerResource`, `CustomerService` and `CustomerDao` classes on their respective directories.

4.10.2 Connecting the Logic Classes

Now that you have all the logic classes created, you need to connect them. You will accomplish that by using the dependency injection concept which means basically that you will receive an instance from the class you need in the constructor.

You will then create a constructor in the `CustomerResource` class that will receive the `CustomerService` instance and then you will store this `CustomerService` instance in an instance variable named as `customerService`.

Then you will create another constructor in the `CustomerService` class that will receive the `CustomerDao` instance dependency into the instance variable you will create named as `customerDao`.

4.10.3 Creating the Data Class

Now you need to create the class that will contain the information when you make a search in the database, this class will be `Customer` which we will create in the `com.javachallengers.model`. In the `Customer` class you will have the following attributes, `id`, `name`, `birthDate`, `address`, `email`, `salary`, `login` and `passwordToken`. Remember to use the Builder pattern in the `Customer` class.

Then we are going to create a `CustomerDTO` class in the `com.javachallengers.dto` package which will contain the information we want to expose to the User Interface. We don't want to get the whole information from our domain `Customer` class, that's why we will use a `CustomerDTO` class.

Then we will create the `CustomerDTO` class with the following attributes, `name`, `birthDate`, `address`, `email` and `salary`. It's also recommended to use the Builder pattern here.

Finally, create a `toCustomerDTO` method that will receive a `Customer`, will populate the `CustomerDTO` attributes with the customer attributes and then will return a populated `customerDTO` object. Simply put, the goal of the `toCustomerDTO` method is to convert a `Customer` object to a `CustomerDTO` object.

4.10.4 Implementing the Logic Methods

Let's start by the `CustomerResource` class, we will create the `findById` method whose method signature will return a `CustomerDTO` and will receive a `customerId` as an `int` type. In this method you will basically invoke the `customerService.findById` method by passing the `customerId` you are receiving. Then you will convert the customer returned from the `findById` method and will use the `toCustomerDTO` method from the `CustomerDTO` class to convert the customer object to a `customerDTO` object.

Then we will implement the `findById` method in the `CustomerService` layer returning a `Customer` and receiving again the `customerId` which will have a very simple logic. Keep in mind that the service layer is the one that should deal with programming logic and business requirements. Therefore create a logic where if the `customerId` greater than zero you will invoke our `customerDao.findById` method passing the `customerId` otherwise you will return an empty `Customer` which would be an instance of a `Customer` with no values assigned to the attributes.

Finally, you will implement the `findById` method in the `CustomerDao` class which will return a new `Customer`.

You don't really need to implement anything sophisticated here, basically create a new `Customer` by using the `Builder` pattern, assign the `customerId` to the `id` field, then populate the other fields with the information you wish, remember that all fields must be populated.

4.10.5 Creating the Invoker Class

Now create a package named as `com.javachallengers.invoker` and then create a class named as `CustomerLogicInvoker` and you will create the main method that will invoke the `CustomerResource` class. In this class, you will chain the instantiation from the customer classes.

You will first instantiate the `CustomerDao`, then use the `customerDao` you created in the constructor of the `CustomerService`.

Then, you will instantiate the `CustomerResource` class which will take the `customerService` instance in the constructor.

Finally, you will invoke the `findById` method from the `customerResource` instance firstly passing an id that is negative, then make sure the `customerDTO` instance is empty.

Then invoke the `findById` method again passing the id of 1 and make sure that the `customerDTO` instance you are returning has the information you were expecting, you can simply print the information to do so.

4.11 Summary

- In this chapter you learned the core principles of encapsulation
- You learned how to use getters and setters
- You learned that we use encapsulation when we know what an Object does and not how
- You learned the MVC (Model View Controller) pattern that helps to encapsulate and organize layers of the system
- You learned the DTO (Data Transfer Object) pattern that encapsulates the data that should be used in the User Interface
- You know how and how and when to use access modifiers from the more restrictive to the less restrictive, private, default, protected and public.
- You learned how to use instance and static blocks

- You learned the differences between static and instance
- You learned how and when to use a static attribute or method
- You learned that a static attribute is shared between instances
- You also created a real-world challenge that contains the essence of a real-application divided in organized packages separated by responsibilities

5 Overloading

This chapter covers:

- Changing and extending legacy methods without breaking the code
- Creating flexible and easy-to-read APIs
- Creating better method names
- Creating concise and easy-to-use APIs and utility methods

Overloading refers to the ability to use the same method name with different parameters. It's a way to make your methods more flexible in Java. Overloading is a widely used technique in object-oriented programming languages, and you can see it in action in many classes of the JDK. It enables us to extend the possibilities of methods without breaking existing code. In this chapter you'll see several code examples that clarify the main concept and the details of this incredibly useful language feature.

5.1 Overloading basics

The following example is a simple illustration of method overloading. We are declaring the `learnAbility` method twice, with different parameters. In the second method invocation we receive the additional `String` parameter `weapon`:

```
1 public class NeoOverloading {
2
3     public static void main(String... matrixTheme) {
4         NeoOverloading neo = new NeoOverloading();
5         neo.learnAbility("Kung-Fu", 1010101);
6         neo.learnAbility("Taekwondo", 100010101);
7         neo.learnAbility("CloseQuartersCombat", 11100101, "Uzi");
8         neo.learnAbility("Specialized weapon training", 10101, "RPG");
9     }
10
11     void learnAbility(String ability, int binaryCode) {
12         System.out.printf("Neo has learned %s with the code %s \n",
13 ability, binaryCode);
14     }
15 }
```

```
16     void learnAbility(String ability, int binaryCode, String weapon) {
17         System.out.printf("Neo has learned %s with a %s and the code %s \n",
18 ability, weapon, binaryCode);
19     }
20
21 }
```

We can thus invoke the method with either two or three parameters, depending on the situation.

Overloading is widely used in real systems, and the knowledge of how to apply this powerful concept improves code quality by simplifying the API that developers use. Method overloading prevents developers from having to remember multiple method names, because we can simply create different versions of a method with the same name for different purposes.

5.1.1 What can and can't be done with overloading?

We can't use overloading to change a method's return type. This code won't compile:

```
1 public class MatrixNotWork {
2     void goToMatrix() {}
3     long goToMatrix() {}
4 }
```

Similarly, we can't use overloading just to change variable names. This code won't compile either:

```
1 public class MatrixNotWork {
2     void goToMatrix(String neo) {}
3     void goToMatrix(String morpheus) {}
4 }
```

To overload a method, their argument lists must differ in one of three ways. We can change the number of parameters, their types, their order or even mix all those three ways altogether will also work.

```

1  // Number of parameters:
2  public class MatrixOverloading {
3      void performAction(int number1) { }
4      void performAction(int number1, int number2) { }
5  }
6
7  // Type of parameters:
8  public class MatrixOverloading {
9      void performAction(int number1, int number2) { }
10     void performAction(float number1, float number2) { }
11 }
12
13 // Order of parameter types (Only valid if the variable types are different):
14 public class MatrixOverloading {
15     void performAction(double number1, int number2) { }
16     void performAction(int number2, double number1) { }
17 }

```

5.1.2 Type promotion with primitives

One of the most important concepts to keep in mind when working with overloading is the possibility of widening types, also known as type promotion. We can do this with any primitive type except boolean. Figure 5.1 shows the range of possibilities for type promotion, from left to right.

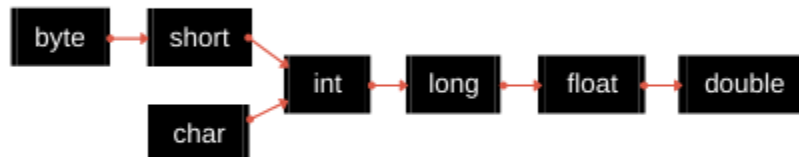


Figure 5.1.2 Type promotion possibilities in Java

A type can be widened to any other type that can accommodate its value without information loss (a “wider” type). For example, a byte can be widened to a short, int, long, float, or double, because

all of those types can accommodate larger values. This means if we declare a variable of type double and we want to assign a byte into it, it will work.

But if we want to assign a double into a byte, we will instead need to use Java's casting feature. This is a way of acknowledging to the compiler that you know you might be losing data on the conversion, and that you take full responsibility if that happens. It's an explicit type assignment, made by specifying the target type in parentheses before the value.

Let's take a look at some examples of widening and casting with primitives:

```
1 public class KeyMaker {
2
3     public static void main(String... wideningAndCastingPrimitives) {
4         char keyChar = 1;
5         double keyDouble = keyChar;
6         short keyShort = 1;
7         float keyFloat = keyShort;
8
9         int keyInt = (int) keyDouble;
10        byte keyByte = (byte) keyDouble;
11        long keyLong = (long) keyFloat;
12    }
13
14 }
```

As you can see, it's possible to assign a char to a double and a short to a float, but it's not possible to directly assign a double to an int, a double to a byte, or a float to a long.

It's important to understand type promotion in the context of method overloading because Java will promote types automatically when possible if the provided arguments don't match the parameters lists of any of the defined methods. For example, suppose you define two `performAction` methods, as follows:

```
1 public class MatrixOverloading {
2     void performAction(int number1, int number2) { }
3     void performAction(double number1, double number2) { }
4 }
```

If this method is called with a byte argument and an int argument, the method call will succeed because the byte value will automatically be promoted to an int. Similarly, if it's called with two float values, both will be promoted to double.

5.1.3 Using hardcoded primitive types

When you provide a hardcoded argument without specifying its type, the JVM will take care of determining the type for you. Here are a few examples:

```

1 public class MatrixHardcodedTypes {
2
3     public static void main(String... hardcodedPrimitiveTypes) {
4         System.out.println(1);    // It's an int
5         System.out.println(1.0);  // It's a double
6         System.out.println('1');  // It's a char
7         System.out.println(true); // It's a boolean
8     }
9
10 }

```

There is also the option of making numerical types explicit in hardcoded values to the JVM, to accomplish that we can use the correspondent type letters at the end of the numbers:

```

1 public class MatrixExplicitHardcodedTypes {
2
3     public static void main(String... hardcodedPrimitiveTypes) {
4         System.out.println(1L); // It's a long
5         System.out.println(1l); // It's a long
6         System.out.println(1F); // It's a float
7         System.out.println(1f); // It's a float
8         System.out.println(1D); // It's a double
9         System.out.println(1d); // It's a double
10    }
11
12 }

```

The suffix `L` or `l` is used to designate a long. The lowercase `'l'` is not recommended to use since it's similar to the number `'1'`, therefore, difficult to read. If we use `F` or `f` the number will be interpreted as a float, and `D` or `d` tells the compiler it's a double. As a general rule it's better to use capital letters if you need to do this, because lowercase letters are harder to read in the code.

An important detail about the char type is that when a char is converted to int or long, this is done by converting it to the ASCII table value of the provided character. For example, suppose we convert the char `'1'` to an int, like this:

```

1 char charMatrixConversion = '1';
2 int intMatrixConversion = charMatrixConversion;
3 System.out.println(intMatrixConversion);

```

The output will be 49 because the number 1 in the ASCII table is represented by 49. This same rule will be applied with any other character. For example, if we convert the char value `'a'` to an int, the

number that corresponds to a in the ASCII table will be assigned to the int variable and the output will be 97.



Definition of the ASCII table

The ASCII acronym stands for American Standard Code for Information Interchange. Since computers can only understand numbers, the ASCII table will represent a character by a number, therefore enabling the computer to be able to read each character.

You can consult further information from the ASCII table in the following link:
<http://www.asciitable.com/>

Like type promotion, it's important to understand this feature of the Java language in relation to method overloading. How the JVM interprets a method's arguments has an effect on whether the code compiles, and understanding these concepts will help you analyze your own and other people's code.

5.1.4 KeyMaker Overloading Challenge

The following challenge explores the concept of how primitive types are interpreted by the JVM when argument values are hardcoded. Can you figure out what will happen when this code is executed?

```
1 public class KeyMakerOverloading {
2
3     public static void main(String... primitiveOverloading) {
4         makeKey(1);
5         makeKey(1F);
6         makeKey('1');
7         makeKey(1.0);
8     }
9
10    static void makeKey(short shortCode) {
11        System.out.println("short:" + shortCode);
12    }
13    static void makeKey(long longCode) {
14        System.out.println("long:" + longCode);
15    }
16    static void makeKey(float floatCode) {
17        System.out.println("float:" + floatCode);
18    }
19    static void makeKey(double floatCode) {
20        System.out.println("double:" + floatCode);
21    }
22 }
```

- A. short:1
float:1.0
short:49
double:1.0
- B. long:1
float:1.0
short:1
float:1.0
- C. short:1
float:1.0
long:49
float:1.0
- D. long:1
float:1.0
long:49
double:1.0

Explanation:

Let's analyze the code. In the first invocation we pass the hardcoded value 1:

```
makeKey(1);
```

int is the default type interpreted to non-decimal numbers that fit within this type's range, so the JVM will convert this hardcoded number to int. But none of the overloaded methods takes an int, so the type will be promoted to a wider one. The first method the JVM finds that can accommodate this type is this one:

```
static void makeKey(long longCode) { ... }
```

Therefore, the first output will be:

```
long:1
```

In the second method invocation, we are passing an explicit float because we are using the letter F after the number:

```
makeKey(1F);
```

As we already have a corresponding method for the float type, the output for this invocation will be:

```
float:1.0
```

In the third method invocation we pass a char:

```
makeKey('1');
```

There's no method that takes a char, so type promotion will again be used here. As the diagram in figure 5.1 shows, a char can be promoted to an int, which as we've already seen will be promoted

further to a long in this example. The char value will be converted to the corresponding number in the ASCII table, which is 49. Therefore, the output will be:

```
long: 49
```

Finally, the number 1.0 will be automatically converted by the JVM to the type double, and then it will invoke the method that receives a double:

```
double:1.0
```

Therefore, the final answer will be:

```
1 D) long:1
2   float:1.0
3   long:49
4   double:1.0
```

5.2 Wrappers and autoboxing

As you know, in Java there are two kinds of types: objects and primitive types. But what if you want to use a primitive type as an object? Java has special classes that enable just that, known as wrapper classes. Each primitive type has a corresponding wrapper class: byte and Byte, short and Short, int and Integer, and so on.

Converting a primitive type into an object is called boxing. Before Java 1.5, we were obliged to box primitive types into wrapper types manually:

```
1 int primitiveInt = 7;
2 Integer matrixInt = new Integer(primitiveInt);
```

Fortunately, since Java 1.5 we don't need to do that anymore. We can just assign the primitive type directly into the wrapper type:

```
1 int primitiveInt = 7;
2 Integer matrixInt = primitiveInt;
```

This feature in Java is called autoboxing. The JVM does the work for us, which is great because it means our code is cleaner and more concise. This is what happens behind the scenes:

```
Integer matrixInt = Integer.valueOf(primitiveInt);
```

Auto-unboxing—converting in the other direction—works in the same way. We can write code like this:

```

1 Integer matrixInt = 7;
2 int newMatrixInt = matrixInt;

```

And the JVM will create this code behind the scenes:

```

1 Integer matrixInt = 7;
2 int newMatrixInt = matrixInt.intValue();

```

5.2.1 Things to be aware of when using wrappers

All wrapper objects are final, which means they are immutable—their attributes cannot be changed. Each time you assign a value to a wrapper type, the JVM will create a new wrapper object. This applies to autoboxing too: the JVM will create a new object in the memory heap every time, unless the wrapper type is cached.

Caching applies to the boolean values true and false and integer values in the range -128 to 127. For performance reasons, when we autobox an int type with a value in this range to the Integer wrapper type, the JVM will cache this value in the created instance. The same applies to autoboxing of byte, short, long, and char types (in the range \u0000 to \u007f, or 0 to 127). If we assign a value that is cached, a new object will not be created. Note that this caching occurs only with autoboxing, not when we use an explicit constructor to convert a primitive to a wrapper type.

You should also be aware that you can't directly assign one wrapper type to another, which makes sense because there is no direct relationship between the wrapper classes. Let's look at a few examples of these classes:

```

1 public final class Double extends Number
2     implements Comparable<Double>, Constable, ConstantDesc { ... }
3
4 public final class Integer extends Number
5     implements Comparable<Integer>, Constable, ConstantDesc { ... }

```

As you can see, the wrapper classes are very similar. But even though they both extend Number (as do the other numeric types), we can't assign a Double to an Integer or any other wrapper type. That's because of the widening rule—a type can be widened only to another type (class) that it inherits from. If we try to do this, the second line won't compile because Integer is not related to Double with inheritance:

```

1 Integer matrixInt = new Integer(7);
2 Double matrixDouble = matrixInt;

```

On the other hand, we can assign a Double, Integer, or any other numeric wrapper type to a Number:

```

1  Number matrixNumber = new Integer(7);
2  matrixNumber = new Double(7);

```

It's also not possible to assign a primitive type directly to a non matching wrapper type. For example, you can't assign an int type directly to a Double wrapper. The following code won't compile:

```
Double doubleMatrixNumber = 7;
```

The only way to accomplish this is by casting the value to the corresponding primitive type. Then the JVM will be able to auto-unbox it:

```
Double doubleMatrixNumber = (double) 7;
```

Finally, you should be aware that autoboxing has a cost for the JVM—wrappers use more memory resources than primitive types. The more you can use primitive types the better, because your logic operations will be executed faster. But if you need your value to be null or you need to work with Java collections, for example, which can only store objects, you will have to use wrappers.

5.2.2 Matrix Wrappers Challenge

This challenge will test your understanding of the concept of type widening. See if you can guess the final output. If not, don't worry; it's a tricky one!

```

1  import java.math.BigDecimal;
2
3  public class MatrixWrappers {
4
5      static String result = "";
6
7      public static void main(String... doYourBest) {
8          wrapBullets(5, 5D);
9          wrapBullets(1L, 5);
10         wrapBullets(new BigDecimal(4), 5D);
11         wrapBullets(5, 4);
12         wrapBullets((short) 5.0, 5.);
13
14         System.out.println(result);
15     }
16
17     static void wrapBullets(Integer integerWrapper, Double doubleWrapper) { \
18 result += "A";
19     }
20     static void wrapBullets(Short shortWrapper, Double doubleWrapper) { \
21 result += "B";
22     }

```

```

23     static void wrapBullets(Number numberWrapper, Double doubleWrapper) {           \
24 result += "C";
25     }
26     static void wrapBullets(double doublePrimitive, int intPrimitive) {           \
27 result += "D";
28     }
29 }

```

- A. ADCDB
- B. BCBAA
- C. BCDAC
- D. ACCAB

Explanation:

Let's analyze the method invocations. In the first one we are passing an int and a double value (indicated by the D):

```
wrapBullets(5, 5D);
```

Those two primitive types can't be widened to a broader type in another method, so the JVM will auto-box them for us. The invoked method will be:

```
static void wrapBullets(Integer integerWrapper, Double doubleWrapper)
```

In the second method invocation we are passing a long primitive type and an int:

```
wrapBullets(1L, 5);
```

The long can be widened to a double, so the JVM will do that rather than converting it to a wrapper type because widening consumes less memory resources. So in this case, it will invoke the following method:

```
static void wrapBullets(double doublePrimitive, int intPrimitive) { ... }
```

In the third method invocation we are using the wrapper type BigDecimal and a double primitive type:

```
wrapBullets(new BigDecimal(4), 5D);
```

BigDecimal is a Number, so this type can be widened to a Number, and the double will be autoboxed. The method invoked by the JVM is therefore:

```
static void wrapBullets(Number numberWrapper, Double doubleWrapper) { ... }
```

In the fourth method invocation we are using two int types:

```
wrapBullets(5, 4);
```

The strategy the JVM will choose is widening, so the following method will be invoked:

```
static void wrapBullets(double doublePrimitive, int intPrimitive)
```


Finally, in the fifth method invocation we are passing a double that is being cast to a short type, and another double (note that it's possible to use a double value like this, without the 0 after the decimal point; it's the same as 5.0):

```
wrapBullets((short) 5.0, 5.);
```

The only strategy the JVM can choose here is autoboxing. The short primitive type will be autoboxed to `Short` and the double primitive type will be autoboxed to `Double`. Therefore, the invoked method will be:

```
static void wrapBullets(Short shortWrapper, Double doubleWrapper) { ... }
```

In conclusion, the final answer will be:

A) ADCDB

5.3 Varargs

Some methods need to take a variable number of arguments. In Java, the varargs (variable arguments) feature makes it possible to pass as many parameters as we want in a very flexible manner. Let's see what this looks like in code. We define varargs using a standard type declaration followed by an ellipsis (...):

```
1 public class AgentSmithClones {
2
3     public static void main(String... varargs) {
4         createClones("smith1", "smith2", "smith3", "smith4");
5     }
6
7     static void createClones(String... agentSmiths) {
8         for (String agentSmith: agentSmiths) {
9             System.out.printf(agentSmith);
10        }
11    }
12
13 }
```

There are some rules about using varargs:

- Varargs can only be used as a method argument.
- It can only be declared as a parameter once in a method.
- When there is more than one method parameter, varargs must be the last argument in the list.
- When the JVM is matching overloaded methods, methods with varargs have the lowest method invocation priority.
- You can pass no arguments to a method with varargs (the equivalent of an empty array).

Let's see some of these rules in action with overloading:

```
1 public class OracleVision {
2
3     static String bits = "";
4
5     public static void main(String... varArgsOverloading) {
6         visionOf(1, 1);
7         visionOf(1.0, 1, 1);
8         visionOf();
9
10        System.out.println(bits);
11    }
12
13    static void visionOf(Integer neo, Integer morpheus) { bits += "A"; }
14    static void visionOf(double trinity, int... agentSmith) { bits += "B"; }
15    static void visionOf(int... keyMaker) { bits += "C"; }
16 }
```

What methods do you think are going to be invoked? If you followed the rules described previously, you probably got the answer right: it's ABC.

In the first method invocation, where we are passing two int primitive types, the JVM will prefer autoboxing to varargs because it consumes less memory resources.

In the second method invocation we are passing a double and two int primitives, so the only available option is the method receiving a double and an int varargs.

In the third invocation we pass nothing. Remember that when we use varargs we can pass as many parameters as we want, including no parameters at all. In this case, the third method will be invoked because it's the only method receiving one varargs type and nothing else.



More about Varargs

If you have more than one method with only a varargs parameter, how does the JVM choose between them if nothing is passed in the method invocation? In this case, it chooses the method with the most specific signature, following the rules of type promotion. For example, int... would be chosen over double... because int is a more specific (less wide) type.

Similarly, Exception... would be chosen over Object... because it's a more specific type (Object is Java's ultimate superclass, at the top of the class hierarchy tree—we'll explore this topic in much more depth in the next chapter).

Varargs is the most costly option in terms of resource usage, so the JVM will always choose it last. It's lazy and optimized to invoke the method that will consume the least memory. If multiple options are available, it will prefer (in this order):

- 1 Widening
- 2 Autoboxing
- 3 Varargs

Now that you've learned the main principles of varargs, let's try solving a challenge!

5.3.1 Neo Jump Overloaded varArgs Challenge

In this challenge you'll explore the concepts of varargs, widening, and autoboxing. See if you can guess the final result.

```
1 public class NeoJump {
2
3     static String finalResult = "";
4
5     public static void main(String... doYourBest) {
6         jump(1, true);
7         jump();
8         jump(new int[]{1, 2, 3});
9         jump(1L);
10        jump(1);
11        jump(Double.valueOf(1));
12
13        System.out.println(finalResult);
14    }
15
16    static void jump(Object o) { finalResult += "1"; }
17    static void jump(Object... o) { finalResult += "2"; }
18    static void jump(Exception... i) { finalResult += "3"; }
19    static void jump(Long l) { finalResult += "4"; }
20    static void jump(double d) { finalResult += "5"; }
21
22 }
```

- A. 322451
- B. 231551
- C. 231555
- D. 332455

Let's analyze each of the method invocations in turn. First we pass an int and a boolean type:

```
jump(1, true);
```

The only method capable of receiving these types and autoboxing is:

```
static void jump(Object... o) { finalResult += "2";}
```

In the next method call we pass nothing:

```
jump();
```

When we pass nothing, the varargs method with the most specific type will be invoked:

```
static void jump(Exception... i) { finalResult += "3";}
```

On the next call we pass a new array of int:

```
jump(new int[]{1, 2, 3});
```

Remember that an array in Java is an object, so in this case it's easier for the JVM to widen the array to an Object instead of invoking the varargs method. The invoked method will be:

```
static void jump(Object o) { finalResult += "1"; }
```



****Varargs is an Array ****

Varargs is an array behind the scenes. If there was a method receiving a varargs of int, it would be easier for the JVM to simply invoke it when passing an array of int. In this exceptional case using varargs would take priority over widening.

Next, we pass a long primitive type:

```
jump(1L);
```

The easiest strategy for the JVM is to widen a long to a double primitive type. Therefore, the invoked method will be:

```
static void jump(double d) { finalResult += "5";}
```

On the next invocation we pass an int:

```
jump(1);
```

The quickest solution for the JVM is again to widen the int type to double, so the same method will be invoked:

```
static void jump(double d) { finalResult += "5";}
```

Finally, we invoke the method passing a Double wrapper type:

```
jump(Double.valueOf(1));
```

The best strategy for the JVM is to widen the Double wrapper type to an Object, so the last method invoked by the JVM will be:

```
static void jump(Object o) { finalResult += "1"; }
```

In conclusion, the final output will be:

B) 231551

5.4 Overloading Real-World Usage Challenge

There are three parts to this chapter's real-world challenge, to test your understanding of the concept of method overloading.

5.4.1 Method overloading

Suppose you're working on a financial project and you need to extend a Java method that can't be changed directly. Fortunately, you know it's possible to extend a method by using overloading without breaking the existing method.

Your goal is to overload the existing method with a new method that will receive an extra `String` parameter. This is the existing method:

```
1 public class DiscountController {
2
3     boolean isDiscountAvailable(String customerId) {
4         return discountService.isDiscountAvailable(customerId);
5     }
6
7     // Implement the method with an extra String parameter here
8 }
```

You can use this simple `DiscountService` class for your implementation:

```
1 public class DiscountService {
2
3     boolean isDiscountAvailable(String customerId) {
4         System.out.println("Discount available to " + customerId);
5         return true;
6     }
7
8 }
```

You need to implement a new business requirement where you must pass an extra `String` containing the `userName` to log information about which the user is executing this action. The solution is to implement a new method with the extra `userName` parameter, log the `customerId` and the `userName`, and finally invoke the existing method, reusing its functionality.

5.4.1 Inserting by Customer Name

Now suppose you receive a task where you are only receiving the customer name from a front-end application and you need to insert this information.

But you already have a method that is inserting the `Customer` so the idea is to reuse it utilizing the overloading concept.

Then you need to make your `CustomerService` class flexible enough to be able to receive only the customer name and transform it into a `Customer` and then reuse the existent insert method that is receiving the `Customer` class as a parameter.

You don't want to pass the whole `Customer` object to the existing insert method you have on your `CustomerService` class.

Instead, you want to overload this method and pass in only a `String` with the customer name.

```
1 public class CustomerService {
2
3     CustomerDao dao;
4
5     void insert(Customer customer) {
6         dao.insert(customer);
7     }
8
9 }
```

To solve this part of the challenge you can use this pre created `CustomerDao` class:

```
1 public class CustomerDao {
2
3     void insert(Customer customer) {
4         System.out.println("Customer inserted");
5     }
6
7 }
```

And you can use this simple POJO (Plain Old Java Object) to pass the `String` into it:

```
1 public class Customer {
2
3     String name;
4
5     public Customer(String customerName) {
6         this.name = customerName;
7     }
8
9     public String getName() {
10         return name;
11     }
12 }
```

How could you redesign the `CustomerService` class to meet your needs using the power of method overloading?

5.4.2 Flexible overloading

For this part of the challenge suppose that you're working for a big retailer, and you need to create three methods that will receive either one, two, or three parameters: a `customerId`, an optional `discountId`, and an optional `productId`.

Your goal is to create a method, `processCustomerProduct`, that can be invoked with only a `customerId` or with a `customerId` and a `discountId`, or with a `customerId`, `productId`, and `discountId`.

The method that receives one or two parameters should invoke the method that receives three parameters, passing `null` for the missing values. This is a more efficient design than creating three methods with different names because the logic that must be used is very similar in each case, and instead of having this logic externalized, it's better to have this intelligence encapsulated in the service. How can you do that?

Insert your overloaded methods in the following code:

```
1 public class CustomerService {
2
3     // Variables and methods
4
5     void processCustomerProduct(String customerId) {
6         // Invoke the next method here
7     }
8
9     // Create the overloaded methods here
10 }
```

Don't worry about the logic of the method that will receive three parameters; simply print the three attributes with a default message. The important point of this challenge is to use overloading.

The knowledge you've gained in this chapter will enable you to create flexible and extensible method APIs that are easy to maintain and understand. The more Java you learn the better your code gets, and the more capable you will be to solve complex code problems like these.

5.5 Summary

- In this chapter you learned how to make effective use of method overloading to create flexible, reusable, and easy-to-read methods.
- You learned the widening order for primitive and object types.
- You learned how to use casting to convert a wider to lower types
- You learned how the JVM interprets hardcoded numbers and how to explicitly declare them with a specific type.
- You know how to make good use of wrappers with autoboxing, and you understand the drawbacks and tradeoffs of working with wrapper types.
- You learned how to pass multiple parameters to a method in a clean way by using varargs.
- You are now capable of using advanced overloading, and you know that the JVM is lazy and will choose the least memory-consuming method to invoke when different options are available (from widening to, autoboxing, and finally varargs).

6 Inheritance and Polymorphism

This chapter covers:

- Developing code using inheritance and polymorphism, and overriding methods from a superclass
- Understanding that every class in Java inherits from `Object`
- Understanding access modifiers
- Choosing between inheritance and composition
- Inheriting from interfaces (an alternative option for multiple inheritance)
- Developing flexible code by using the `super` keyword effectively, and invoking constructors with `super`
- Casting objects with the use of inheritance
- Preventing inheritance of classes and methods with the `final` keyword and the `private` modifier
- The differences between normal and abstract classes, and how to create an abstract class design that makes sense in a system

Inheritance is a very powerful concept in object-oriented programming that greatly facilitates code reuse.

Inheritance in Java is a way to create new child classes, or subclasses, based on the features of existing parent classes, or superclasses. The subclass inherits the behavior (methods) and state (fields) of the superclass, and can modify them as needed without having to reimplement all of the underlying logic. When used wisely, this powerful concept will bring great benefits to your projects.

When we talk about inheritance, polymorphism comes hand-in-hand because it's only possible to use it with inheritance. Polymorphism gives a great amount of flexibility in code and helps developers to create clean concise code design. The Collections api makes great use of polymorphism, every Java class can make use of polymorphism. Whenever we override a method from a parent class, we are using polymorphism. The Collections Api has many examples, when we declare a `List` and instantiate it with an `ArrayList` we are already using polymorphism. Most of design patterns make use of polymorphism, therefore, it's an extremely powerful feature to master in the Object-Oriented Programming paradigm.

Composition is another valuable code reuse mechanism, with lower coupling. With composition, one class can have an instance of another class as one of its fields—for example, a `House` can be composed of `Bedrooms`, a `Kitchen`, and so forth, and each of those component parts can be changed independently. This is an important concept in object-oriented programming, where our goal as developers is to abstract and translate real-world problems into code to develop solutions that will make people's lives easier.

6.1 When to use inheritance

Inheritance can be massively beneficial when used in the right way—and massively problematic when used in the wrong way. It generates a high level of coupling, which means any changes in the parent class’s code may become a nightmare. It’s important, therefore, not to overuse inheritance in your code.

How can you tell whether inheritance will be beneficial in your code? Ask yourself whether there is an is-a relationship between the class you’re creating and the one you want to inherit from. If the answer to this question is no, it’s not a good idea to use inheritance.

Good examples of inheritance include:

- 1 Car inherits from `Vehicle` (a Car is a Vehicle)
- 2 Dog inherits from `Animal` (a Dog is an Animal)
- 3 Gold inherits from `Metal` (Gold is a Metal)

Conversely, an inappropriate use of inheritance would be for Dog to inherit from Cat: it wouldn’t make sense to say “Dog is a Cat.”

Inheritance in Java is declared by using the `extends` keyword in the subclass’s definition. For example:

```
1 public class Car extends Vehicle {  
2     ...  
3 }
```

It works the same way with both concrete classes and abstract classes, which are classes that cannot be instantiated. The Java API uses these extensively to encapsulate functionality and state. We’ll look at abstract classes in depth later in this chapter.

6.1.1 Overriding inherited methods

One of the most powerful features inheritance provides is the ability to override a behavior from the parent class. A parent class can have a default behavior, and the child classes can have specialized behavior appropriate to them. This concept of having many similar classes that are related to a parent class through inheritance is known as polymorphism.

The following code demonstrates polymorphism and method overriding with the generic class `Hero` and the specialized classes `SpiderMan` and `Wolverine` that inherit from and extend it. See if you can figure out why executing the `HeroExecutor` class’s main method produces the results it does:

```
1  class Hero {
2      void usePower() {
3          System.out.println("The hero is going to use his power!");
4      }
5  }
6
7  class SpiderMan extends Hero {
8      @Override
9      void usePower() {
10         System.out.println("Hyper Web Throw");
11     }
12 }
13
14 class Wolverine extends Hero {
15     @Override
16     void usePower() {
17         System.out.println("Fatal Claw");
18     }
19 }
20
21 public class HeroExecutor {
22
23     public static void main(String... methodOverridingExample) {
24         Hero hero = new Hero();
25         hero.usePower();                // #1
26
27
28         Hero spiderMan = new SpiderMan();
29         spiderMan.usePower();           // #2
30
31
32         Hero wolverine = new Wolverine();
33         wolverine.usePower();           // #3
34     }
35 }
```

- #1 This prints “The hero is going to use his power!”
- #2 This prints “Hyper Web Throw”
- #3 This prints “Fatal Claw”

When we instantiate only Hero, we get the behavior specified for that class. Then we instantiate Hero again with SpiderMan. This class overrides the usePower method, so a different message is

printed ("Hyper Web Throw"). Finally, we instantiate Hero with Wolverine. This class also overrides usePower, so the output from Wolverine is printed ("Fatal Claw").



@Override

You may be wondering about the use of the @Override annotation in this example. This annotation indicates that the method is overriding a base class method. If there is no matching method in the base class (same name, parameter types, and so on), there will be a compilation error.

It's not required to use the @Override annotation, but it provides a useful check that your method overriding efforts will work correctly. Otherwise, in case of a mismatch the subclass method will simply behave as a new method, not an override of the existing method, possibly resulting in unexpected behavior.

6.1.2 Drink Beer Polymorphism Challenge

What Simpsons character drinks beer? Can you guess what will happen when this program is executed?

```

1  public class DrinkBeerChallenge {
2
3      public static void main(String... doYourBest) {
4          SimpsonCharacter bart = new Bart();
5          drinkBeer(bart);
6          drinkBeer(new Homer());
7          drinkBeer(new SimpsonCharacter());
8      }
9
10     static void drinkBeer(SimpsonCharacter simpsonCharacter) {
11         simpsonCharacter.drinkBeer(simpsonCharacter);
12     }
13
14 }
15
16 class Homer extends SimpsonCharacter {
17     void drinkBeer(SimpsonCharacter simpsonCharacter) {
18         System.out.println("Homer drinks");
19     }
20 }
21
22 class Bart extends Homer {
23     void drinkBeer(SimpsonCharacter simpsonCharacter) {

```

```
24         System.out.println("Bart can't drink!");
25     }
26 }
27
28 class SimpsonCharacter {
29
30     void drinkBeer(SimpsonCharacter simpsonCharacter) {
31         System.out.println("A character is drinking beer");
32     }
33
34 }
```

- A. Bart can't drink!
Homer drinks
A character is drinking beer
- B. A character is drinking beer
Homer drinks
A character is drinking beer
- C. A character is drinking beer
A character is drinking beer
A character is drinking beer
- D. Homer drinks
Homer drinks
A character is drinking beer

Explanation:

This challenge explores the concept of polymorphism, also known as virtual method invocation. When we override a parent class method in a subclass, the JVM will execute the version of the method defined in that subclass when instantiating it, enabling us to alter its behavior. This allows us to invoke methods in a flexible way.

Let's analyze the code step by step. In the main method, we begin by creating an instance of the Bart class and assigning it to the variable `bart`, of type `SimpsonCharacter`. We then pass the instance of Bart to the `drinkBeer` method:

```
1 SimpsonCharacter bart = new Bart();
2 drinkBeer(bart);
```

As a result the Bart class's `drinkBeer` method will be invoked, printing:

Bart can't drink!

Next, we instantiate Homer and pass this instance to the method directly:

```
drinkBeer(new Homer());
```

The Homer class's version of the method will be executed in this case, so the output will be:

Homer drinks

Finally, we invoke the drinkBeer method passing a SimpsonCharacter instance:

```
drinkBeer(new SimpsonCharacter());
```

Its corresponding method will be invoked, printing:

A character is drinking beer

Therefore, the correct response is:

```
1 A) Bart can't drink!
2   Homer drinks
3   A character is drinking beer
```

Note that we are not only using the power of inheritance here, but also polymorphism. Can you see how much flexibility we gain by just declaring the parent class in the method? The main method is responsible for providing the instance, which means that the drinkBeer method is flexible enough to receive any kind of SimpsonCharacter. If this method needs to be expanded in any way, it's easier because we can manipulate its behavior in the main method.

Creating decoupled code is crucial for code quality: it makes it easier to make changes to the code, improves testability, and makes the code more flexible and reusable.

6.2 Every Class is an Object

Every Java class is an object, which means every class extends Object. Have you ever noticed that every class has the equals and hashCode methods? That's not by coincidence—these methods are inherited from the Object class. It's Java's ultimate superclass, at the top of the tree.

Any new class you create will implicitly be extending Object. This means you'll be able to invoke methods from the Object class in your class, as shown here:

```
1 public class NewClassExtendsObject {
2
3     public static void main(String... thisClassExtendsObject) {
4         NewClassExtendsObject newClassExtendsObject = new NewClassExtendsObject();
5         newClassExtendsObject.equals("Test");
6         newClassExtendsObject.hashCode();
7         newClassExtendsObject.toString();
8         // Other methods...
9     }
10 }
```

You can also override any methods from the `Object` class in your classes, such as `equals`, `hashCode`, and `toString`. We will explore those methods more deeply in future chapters. For now, keep in mind that you can define the behavior you want for these `Object` methods.

6.2.1 SpiderMan Identity Checker Challenge!

This challenge explores the use of the `Object` class's `equals` method. You don't need to know exactly what this method does to solve it; you just need to know what a method override is and how it works. Look carefully at the code, then see if you can guess the output.

```
1 public class SpiderManIdentityChecker {
2
3     public static void main(String... doYourBest) {
4         System.out.println(new SpiderMan("Peter").equals(new SpiderMan("Peter")));
5         System.out.println(new SpiderMan("Peter").equals(new SpiderMan("Parker")));
6     }
7 }
8
9 class SpiderMan {
10     String name;
11     SpiderMan(String name) {
12         this.name = name;
13     }
14
15     @Override
16     public boolean equals(final Object obj) {
17         return this.equals(obj);
18     }
19
20 }
```

- A. true
false
- B. false
false
- C. java.lang.StackOverflowError
- D. true
true

Explanation:

The key to this challenge is in the implementation of the equals method.

Notice that this method was overridden incorrectly—because we are pointing to the current instance by using the keyword `this`, we are invoking the same equals method and passing the `obj`. This is an infinite recursive method invocation (resulting when a method invokes itself), and to avoid the infinite loop the JVM will throw a `java.lang.StackOverflowError`.

In conclusion, the correct alternative is:

C) `java.lang.StackOverflowError`

6.3 Checking the type of an object

Java's `instanceof` keyword is a useful feature that enables us to check whether an object (instance) is of a given type. It comes in very handy when we are working with inheritance. If we want to develop logic that requires us to know the type of a class, we can use the `instanceof` keyword to verify it.

We can also use the `instanceof` keyword to check whether a subclass extends a particular parent class, implements an interface, even if a declared parent type has the instance of a subclass.

As usual, let's examine some code to make these concepts clearer:

```
1 public class InstanceofExample {
2
3     public static void main(String[] args) {
4         Object xavier = new Xavier();
5         System.out.println(new Venom() instanceof Object);           // #1
6         System.out.println(xavier instanceof Xavier && xavier instanceof Object); // #2
7         System.out.println(new Wolverine() instanceof Hero);         // #3
8         System.out.println(new Hero() instanceof Wolverine);         // #4
9     }
10
11 }
12
```



```
13 class Hero {}
14 class Venom {}
15 class Xavier {}
16 class Wolverine extends Hero {}
```

- #1: This prints true
- #2: This prints true
- #3: This prints true
- #4: This prints false

Can you see how powerful the `instanceof` keyword is? Let's analyze each of the statements in turn. First we instantiate `Venom` and check whether it is an `Object`:

```
System.out.println(new Venom() instanceof Object);
```

All Java classes inherit from `Object`, so this returns true.

Next we instantiate `Xavier` and assign it to the variable `xavier`, of type `Object`. Then we check whether `xavier` is an instance of both `Xavier` and `Object`:

```
Object xavier = new Xavier();
```

```
System.out.println(xavier instanceof Xavier && xavier instanceof Object);
```

Because of polymorphic inheritance `xavier` is an instance of both `Xavier` and `Object`, so this will print true.

Then we check instantiate `Wolverine` and check whether it is a `Hero`:

```
System.out.println(new Wolverine() instanceof Hero);
```

The `Wolverine` class extends `Hero`, so this will also print true.

Finally, we check if a `Hero` is a `Wolverine`:

```
System.out.println(new Hero() instanceof Wolverine);
```

The `Hero` class doesn't know anything about the `Wolverine` class—only the opposite is true, because `Wolverine` extends `Hero`. Therefore, the result of this comparison is false.

6.2.3 Tricky Object Challenge

This challenge will test your understanding of the `instanceof` keyword, by presenting a series of class type checks. It also demonstrates the concept of inheritance.

```

1  public class TheTrickyObjectClass {
2
3      public static void main(String... doYourBest) {
4          System.out.print(new NoBeerException() instanceof Object ? "1" : "0");
5          System.out.print(new Barney() instanceof Object ? "1" : "0");
6          System.out.print(Barney.class instanceof Object ? "1" : "0");
7          System.out.print(Class.class instanceof Object ? "1" : "0");
8      }
9
10 }
11
12 class NoBeerException extends Exception { }
13 class Barney extends Object {} // Line 13

```

- A. 1111
- B. 1100
- C. 0000
- D. Compilation error at line 13

Explanation:

All Java classes extend `Object` with no exception, even the `Class` class is an object.

Note that we are using `Barney.class` which references a class literal. In other words, it means we are getting the class information such as class name, attributes names, types and the class metadata from `Barney`. `Barney.class` is the same as `Barney.getClass()`.

Therefore the correct answer is:

A - 1111

6.4 The `toString` method

The goal of the `toString` method is to reveal the state of an object. By overriding this method, it's possible to show the state of an object simply by printing the instance.

Let's take a look at an example:

```
1 public class SimpsonToString {
2
3     private String name;
4     private String weight;
5
6     private SimpsonToString(String name, String weight) {
7         this.name = name;
8         this.weight = weight;
9     }
10
11     public static void main(String... toStringExample) {
12         SimpsonToString homer = new SimpsonToString("Homer", "240");
13         System.out.println(homer);
14     }
15
16     @Override
17     public String toString() {
18         return "Simpson name:".concat(this.name).concat(", weight:").concat(weight);
19     }
20
21 }
```

The output of this program will be:

```
Simpson name:Homer, weight:240
```

The `toString` method can be handy for logging, for example. Instead of creating specific methods for showing the attributes of the object, we can simply override the `toString` method with what we want.

An important detail of the `toString` method is that we can simply print the whole object that the information of the `toString` method will be simply printed, we don't need to invoke the `toString` method explicitly.

6.5 Access modifiers and inheritance

You may have noticed the use of the keyword `private` in the previous example. Up to this point, all the classes and methods we've been working with have been `public`. These keywords are known as access modifiers, and they control the accessibility of classes and their members (instance variables, fields, and methods). The `public` access modifier means the members are accessible to all code, while the `private` modifier means they're only visible within that class. Methods declared `private` in a superclass are hidden from subclasses and cannot be accessed directly.

In addition to these, Java provides the `default` (or `package`) and `protected` access modifiers. `Default` access is what's provided if you don't specify a modifier: this code is accessible within the class

itself and any other classes in the same package (including subclasses, as long as they're in the same package). `protected` access is similar but allows access to all subclasses regardless of what package they're in.

When you're overriding a method, you can't use a more restrictive access modifier than was used in the superclass. If you do, there will be compilation errors. You can use a less restrictive modifier, though, such as `public` instead of `protected`.

Let's see what's possible and what isn't with a code example:

```

1  class Hero {
2      void attack() {}
3      public void jump() {}
4  }
5
6  class Wolverine extends Hero {
7      public void attack() {} // It's possible to use a more open access modifier
8      public void jump() {} // It's possible to use the same access modifier
9      // void jump() {} Compilation error, the default access modifier is more restric\
10 tive
11     // private void jump() {} Compilation error, the private access modifier is more\
12     restrictive
13 }

```

When overriding methods, keep in mind that they must have the same access restrictions as in the superclass or use a more open access modifier.

6.5.1 Inheritance Modifier Restriction Challenge

This challenge focuses on access modifiers. Can you guess what the output of this program is?

```

1  public class TheHiddenModifier {
2
3      public static void main(String... doYourBest) {
4          System.out.println(new TheHiddenModifier());
5          var hiddenModifier = new TheHiddenModifier();
6          System.out.println(hiddenModifier.equals(hiddenModifier));
7      }
8
9      @Override
10     String toString() { // Line 12
11         return "theHiddenModifier";
12     }

```

```
13
14     @Override
15     public boolean equals(Object obj) {
16         return this.equals(obj); // Line 18
17     }
18 }
```

- A. theHiddenModifier true
- B. theHiddenModifier false
- C. Compilation error at line 12
- D. Compilation error at line 18

Explanation:

The equals method works just fine since we are overriding it with the public access modifier.

However, let's analyze the Object class's toString method from the JDK code:

```
1 public class Object {
2     public String toString() {
3         return getClass().getName() + "@" + Integer.toHexString(hashCode());
4     }
5 }
```

Notice that it's public. This means it can't be overridden with a weaker access modifier, like the default modifier used at this code challenge; it has to be public.

Since at line 12 the default access modifier is more restrictive than public from the Object class, we will get a compilation error.

Therefore, the final answer will be:

C) Compilation error at line 12

6.6 Object composition

Composition is an alternative to inheritance that is widely used in real projects. One of its main benefits is that it avoids the strong coupling of inheritance, which means it's easier to make changes to the code.

When should we use composition? To determine this, you have to ask the question: is this object part of something? If the answer to that question is yes, it's a good candidate for composition. Let's take a look at a few examples:

- A Heart is part of a Person (a Person can't live without a Heart)

- An Engine is part of a Car (a Car can't run without an Engine)
- An Employee is part of a Company (a Company can't function without Employees)

Note also that the relationship works both ways. A Heart can't live outside of a Person (or similar entity), an Engine is no use without a Car to run, and an Employee needs to have a Company to employ them.

To better understand this type of relationship, consider the composition of a Person:

```
1 public class CompositionExample {
2     public static void main(String... personComposition) {
3         new Person(new Brain(), new Heart() /* other elements */);
4         // A Person is composed of a Brain, Heart, and many other elements
5     }
6     static class Person {
7         Brain brain;
8         Heart heart;
9         Person(Brain brain, Heart heart) {
10             this.brain = brain;
11             this.heart = heart;
12         }
13     }
14     static class Brain { }
15     static class Heart { }
16 }
```

Composition enables code reuse by allowing you to define objects that are made up of other objects, but without you having to inherit all the methods of those objects in your class.

6.6.1 Object aggregation

Aggregation is a weaker type of object dependency. Simply put, it's a type of object relationship in which both objects can survive by themselves. If we ask whether an object has an object and the answer is true, it's a good candidate for aggregation. For example:

- A Library has Books (Books will exist without a Library)
- A Person has Clothes (Clothes will exist without a Person)
- A Dog has a Collar (a Collar will exist without a Dog)

Here's what this looks like in code:

```
1 class Library {
2     List<Book> books = new ArrayList<>();
3 }
4 class Book { }
```

Note that composition and aggregation are abstract concepts. They depend on your point of view in the real world, and the rules you define for your system. In a real-world application you can simply use the term “composition”—you don’t need to define specifically if an object is a composition or an aggregation—but it’s still good to know the difference between them.

6.6.2 When to use composition instead of inheritance

The following is a really bad example of inheritance:

```
1 import java.util.ArrayList;
2
3 public class SimpsonCharacterInheritance extends ArrayList<String> {
4
5     public static void main(String... badExampleOfInheritance) {
6         SimpsonCharacterInheritance simpsonCharacters = new SimpsonCharacterInheritance();
7         simpsonCharacters.add("Homer");
8         simpsonCharacters.forEach(System.out::println);
9     }
10 }
11
12 }
```

Why? We’re creating a huge amount of coupling by extending `ArrayList`. If we ask if a `SimpsonCharacter` is an `ArrayList`, the answer is no, it definitely isn’t. We are inheriting every method from the `ArrayList` class into `SimpsonCharacter`, but a lot of those methods won’t be used.

This scenario would work much better with composition. Let’s see what it would look like if we were to instead compose the `SimpsonCharacter` class with an `ArrayList`:

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class SimpsonCharacterComposition {
5
6      static List<String> characters = new ArrayList<>();
7
8      public static void main(String... goodExampleOfComposition) {
9          characters.add("Homer");
10         characters.forEach(System.out::println);
11     }
12 }

```

We get all the functionality we need, and none of the functionality we don't.

6.7 Multiple inheritance in Java

Multiple inheritance refers to the scenario where a subclass inherits from more than one parent class. Unlike some languages, such as Python and C++, Java does not allow multiple inheritance with classes because it adds complexity—but we can simulate this behavior with interfaces. An interface in Java is a collection of abstract methods and fields that specify the behavior of a class. Interfaces don't keep state, which means they can't contain instance variables, only constants. We'll study them in much more detail in the next chapter, but for now this is all you need to know.

If you attempt to inherit from multiple classes, like in the following example, the code won't compile:

```

1  class Hero { }
2  class MarvelCharacter { }
3  class SpiderMan extends Hero, MarvelCharacter { }
4  One solution would be to inherit the classes one by one:
5  class MarvelCharacter { }
6  class Hero extends MarvelCharacter { }
7  class SpiderMan extends Hero { }

```

Another solution is to replace the classes with interfaces:

```

1  interface Hero { }
2  interface MarvelCharacter { }
3  class SpiderMan implements Hero, MarvelCharacter { }

```

6.7.1 Extending interfaces

What if we want to make an interface inherit methods from a parent interface? We can do that with Java! Here's an example:


```
1 public class InterfaceInheritance {
2
3     interface Human { void talk(); }
4     interface Hero { void rescue(Human human); }
5
6     interface Batman extends Hero, Human {}
7 }
```

In Java an interface can inherit from another interface just as a class can inherit from another class, so we can reuse the talk and rescue methods from the Human and Hero interfaces within the Batman interface. This is useful when we want to separate responsibilities into more generic interfaces and reuse them in a more effective way—we don't need to declare those methods again, which makes our code easier to maintain.

6.8 Using super to access a parent class's methods

It's possible to access methods of a parent class by using the keyword `super`. This is a very handy feature to know about when designing your classes, and it's widely used in design patterns. The `super` keyword gives amazing flexibility to our code because it enables us to reuse methods from the parent class. This is useful when a subclass contains members with the same names used in the parent class, and you want to distinguish between them.

In the following example, the class `Homer` invokes the `move` method from its superclass, `Character`:

```
1 public class SuperMethodExample {
2
3     public static void main(String[] args) {
4         new Homer().drinkBeer(); // It will print: Homer is walking...
5     }
6
7     static class Character {
8         void move(String name) {
9             System.out.println(name.concat(" is walking..."));
10        }
11    }
12
13    static class Homer extends Character {
14        void drinkBeer() {
15            super.move("Homer");
16        }
17    }
18
19 }
```

6.8.1 Using the super keyword with constructors

Like superclass methods, we can invoke constructors from the superclass by using the `super` keyword. In the following example, the `Moe` class is invoking the super constructor from the `Character` class:

```
1 public class SuperConstructor {
2     public static void main(String[] args) {
3         new Moe(); // This prints "Moe was created"
4     }
5
6     static class Character {
7         Character(String name) {
8             System.out.println(name.concat(" was created"));
9         }
10    }
11
12    static class Moe extends Character {
13        Moe() {
14            super("Moe");
15        }
16    }
17 }
```

The output of the main method in this example will be "Moe was created".

This is a particularly useful feature to be aware of because unlike other methods, constructors are not inherited by subclasses. There are circumstances where you might want to call a parent class's constructor from a subclass, however; for example, if you want to reuse the initialization code with some customization to reduce code duplication.

We'll consider other aspects of the relationship between constructors and inheritance in the next section.

6.9 Constructors and inheritance

Constructors are not members in Java, so they're not inherited by child classes. This makes sense: a constructor is used to create an object of the class in which it's defined, so each class should have its own constructor. A special case is the default constructor, as you'll see next.

6.9.1 The default constructor

In chapter 3, you learned that the JVM will create a default constructor for a class if no constructor is defined. It will also invoke the superclass's default constructor (the "super constructor")

automatically whenever a subclass is instantiated. This is required because the superclass might have private fields that are initialized in its constructor.

Because every Java class extends `Object`, this means the `Object` class's default constructor is implicitly invoked every time an object is instantiated. Imagine if we had to invoke the super constructor for all Java classes—remembering to do that would be a nightmare. That's one of the reasons that the JVM invokes the super constructor automatically when no parameters are being passed. In effect, it includes the line `super();` for you.

In the following example, the `Magneto` class will be invoking the super constructor of the `Villain` class with the default constructor created automatically by the JVM:

```

1  public class DefaultConstructorInheritance {
2      static class Villain {
3          Villain() {
4              System.out.println("A villain was created!");
5          }
6
7          public static void main(String ...defaultSuperConstructorInvocation) {
8              new Magneto();
9          }
10     }
11
12     static class Magneto extends Villain {
13         // The JVM will create the default constructor invoking the parent class con\
14     structor
15
16         /* This is the default constructor:
17         public Magneto() { super(); } */
18     }
19 }

```

This means when we instantiate the `Magneto` class, the JVM will invoke automatically the super constructor and print the message "A villain was created!".

Keep in mind that when using inheritance, the first constructor to be executed will be the one from the topmost parent class. In our case, the constructor execution order will be `Object`, `Villain`, then `Magneto`.

6.9.2 Parameterized parent class constructors

When the parent class defines a constructor with one or more parameters, the subclass must invoke this constructor explicitly. Otherwise, you'll get a compilation error. That's because the JVM only creates a default constructor when there is no constructor in a class.

In the following code, the Moe class explicitly invokes the super constructor from the Character class. This is required, because otherwise the code won't compile:

```
1  public class CustomizedConstructorSuper {
2
3      public static void main(String... customizedConstructor) {
4          new Moe();
5      }
6
7      static class Character {
8          Character(String name) {
9              System.out.println(name + " was invoked");
10         }
11     }
12
13     static class Moe extends Character {
14         // We'll get a compilation error if we don't invoke the constructor explicit\
15 ly
16         Moe () {
17             super("Moe Szyslak");
18         }
19     }
20
21 }
```

Be aware that if there is another constructor defined in the parent class that does not receive any parameters, you don't need to invoke the parent class constructor explicitly, as in this example. In the following code, we will see an example of overloaded constructors (constructors receiving different types of parameters). When there is a nontyped constructor in the parent class, we don't need to add the invocation of the super constructor explicitly.

In this example the Character class has one constructor without any parameters, and another one that takes a String. In this case the Moe subclass will invoke the superclass's nonparameterized constructor automatically, because the JVM will be able to create a default constructor invoking the super constructor with no parameters:

```
1 public class NonParameterizedConstructorSuper {
2
3     public static void main(String... nonParameterizedConstructor) {
4         new Moe();
5     }
6
7     static class Character {
8         Character() {
9             System.out.println("A Character was created");
10        }
11
12        Character(String name) {
13            System.out.println(name + "was created");
14        }
15    }
16
17    static class Moe extends Character {
18        // It's not necessary to add super here since the JVM adds super automatical\
19 ly
20        // when there is a constructor with no parameters in the parent class
21    }
22
23 }
```

Keep in mind that the `super` keyword will be added by the JVM automatically only for parent class constructors that don't take any parameters.

6.10 Class casting

When we need to convert a type to another one, we use the concept of casting. This is really useful when we want to deal with different compatible types. We saw some examples of casting with primitive types in the previous chapter, but it can also be done with classes and interfaces.

To illustrate this concept, let's take a look at a code example where we try to invoke a method from a subclass. Do you think this will work?

```
1 public class DrinkBeer {
2
3     public static void main(String... drinkBeer) {
4         drinkBeer(new Homer());
5     }
6
7     static void drinkBeer(Character character) {
8         character.drinkBeer();
9     }
10 }
11
12 class Character {
13 }
14
15 class Homer extends Character {
16     void drinkBeer() {
17         System.out.println("Homer drinks a beer");
18     }
19 }
```

If you think this code will print "Homer drinks a beer", think again. We're going to get a compilation error (a `ClassCastException`) because it's impossible for the JVM to know at compilation time what the method from the instance is. The JVM will only know about the methods from the `Character` class, and because the `Character` class doesn't have a `drinkBeer` method, it won't be possible to invoke this method directly from that class.

The only way to make this happen is through class casting. This is like saying, "Hey JVM, I know what I'm doing, so please cast this class with this type." When we do that explicitly, the JVM allows us to use the specific method from `Homer`, `drinkBeer`.

Let's see how that works in code:

```
1 public class DrinkBeer {
2     public static void main(String... drinkBeer) {
3         drinkBeer(new Homer());
4     }
5
6     static void drinkBeer(Character character) {
7         Homer homer = (Homer) character; // Cast Character to Homer
8         homer.drinkBeer();
9     }
10
11     static class Character {
12     }
```

```
13
14     static class Homer extends Character {
15         void drinkBeer() {
16             System.out.println("Homer drinks a beer");
17         }
18     }
19 }
```

This code will compile and produce the expected result—by using the power of casting, we can invoke the `drinkBeer` method from the `Homer` class. Bear in mind that you’ll have to use this technique in your code if a specific method exists only in the subclass. Casting from a parent class to a subclass like this is known as downcasting.

6.10.1 Casting a subclass to a superclass

If you want to cast a subclass to a superclass (known as upcasting), that’s perfectly possible:

```
1 public class SubclassToSuperClass {
2
3     public static void main(String... args) {
4         Barney barney = new Barney();
5         Character characterBarney = barney; // This works perfectly
6     }
7
8     static class Character { }
9     static class Barney extends Character { }
10
11 }
```

This example works because the `Barney` class extends the `Character` class, and therefore the `Barney` class knows everything about `Character`. With that in mind, the JVM will not complain about casting the `Barney` object to `Character`: `Barney` is a `Character`.

6.11 Preventing inheritance

We’ve considered various aspects of inheritance in this chapter, and by now you should have a good understanding of how useful it can be. But what if you want to prevent inheritance of your classes or methods? Let’s take a look at the options.

6.11.1 The final keyword

Sometimes we don’t want a class to ever be extended by another class. A great example from the JDK is the `String` class:

```
1 public final class String
2     implements java.io.Serializable, Comparable<String>, CharSequence,
3         Constable, ConstantDesc { ... }
```

Note that the `String` class is declared `final`. Can you guess why that is? `String` is an immutable class, which means that its main attributes will never be changed. The `final` keyword is used to ensure that this is the case—but that isn't the whole point of this keyword. The main goal of using `final` is to not allow any other class to extend the `String` class.

Have you ever tried to extend this class? If you do this:

```
class AnyClass extends String { }
```

the code won't compile, because the `String` class is declared as `final`.

Another important point to mention about a `final` class is that it's not possible to override any of its methods. This is because you can't inherit from such a class. But what if you want to allow inheritance in your classes, but prevent just certain methods from being overridden? You can use the `final` keyword for that too.

6.11.2 Preventing specific methods from being overridden

If you want a class to be extendable but you don't want certain methods of the class to be overridden, you can use the `final` keyword on those methods.

The following example shows what happens when you try to override a `final` method:

```
1 class Character { final void performAction() { }}
2 class Homer extends Character {
3     void performAction() { }
4 }
```

This code won't compile because it's not possible to override a `final` method.

The only way to make it work is by removing the `final` keyword from the method declaration, like in the following code:

```
1 class Character { void performAction() { }}
2 class Homer extends Character {
3     void performAction() { }
4 }
```


6.11.3 The private modifier with Inheritance

Another way to protect certain methods of a class is to use the private access modifier. As you learned earlier, private methods or fields won't be visible to subclasses or any external classes. They are accessible only to the class in which they are defined.

In the following example, the `Villain` class declares some private members that we then attempt to use in the `Joker` subclass:

```
1  class Villain {
2      private int age;
3      private void attack() {}
4  }
5
6  class Joker extends Villain {
7      void doTrick() {
8          System.out.println(super.age);
9          super.attack();
10     }
11 }
```

This code won't compile, because the `super.age` variable and the `super.attack` method won't be accessible in the subclass. The only way to make it work is by removing the private access modifier and using a less restrictive option, such as the default modifier:

```
1  class Villain {
2      int age;
3      void attack() {}
4  }
5
6  class Joker extends Villain {
7      void doTrick() {
8          System.out.println(super.age);
9          super.attack();
10     }
11 }
```

The code will now work because we are no longer restricting access with the private modifier. Declaring these members protected (or public) would also work.

6.12 Abstract classes

As mentioned earlier, classes that can't be instantiated directly are called abstract classes. They can contain a mix of concrete and abstract methods (methods with and without implementations). Many

JDK classes use this concept, because it wouldn't make sense to instantiate them. For example:

```

1 public abstract class AbstractList<E> extends AbstractCollection<E> implements List<
2 E> { ... }
3
4 public abstract class AbstractCollection<E> implements Collection<E> { ... }
5
6 public abstract class AbstractSet<E> extends AbstractCollection<E> implements Set<E> \
7 { ... }
8
9 public abstract class AbstractMap<K,V> implements Map<K,V> { ... }
10
11 public abstract class Number implements java.io.Serializable { ... }

```

Let's explore the abstract class `Number`. It doesn't make sense to instantiate that class, because it's too generic. Java is a strongly typed programming language, so instantiating the `Number` class won't help developers to work with numbers. What's this class good for, then? When we declare the `Number` class as abstract, it all makes sense!

We can reuse lots of code from `Number` in the Java wrapper classes. Here are some examples of JDK classes that extend this one:

```

1 public final class Integer extends Number
2     implements Comparable<Integer>, Constable, ConstantDesc { ... }
3
4 public final class Double extends Number
5     implements Comparable<Double>, Constable, ConstantDesc { ... }
6
7 public final class Long extends Number
8     implements Comparable<Long>, Constable, ConstantDesc { ... }
9
10 public class BigDecimal extends Number implements Comparable<BigDecimal> { ... }

```

In fact, all the numeric wrapper classes extend `Number`. Note that they all pass the is a test: an `Integer` is a `Number`, as are a `Double`, `Byte`, and so on. All of these classes can reuse the methods declared in the `Number` class, providing their own implementations for the abstract ones—this is a perfect example of a good use of inheritance.

A good example to illustrate the use of an abstract method is the abstract class `OutputStream` because this class doesn't make sense to be instantiated by itself.

Some of the concrete classes that extend `OutputStream` are `FilterOutputStream`, `ByteArrayOutputStream`, `DataOutputStream`, and others. Those classes can be instantiated and they also inherit the generic behaviours from `OutputStream`.

```
1 public abstract class OutputStream implements Closeable, Flushable {
2     // Attributes and methods...
3
4     public abstract void write(int b) throws IOException;
5 }
```

Let's take a look at some code with a customized example of an abstract method. Here, we're exploring the wrong way to instantiate an abstract class:

```
1 public class CustomizedAbstractMethods {
2
3     public static void main(String[] args) {
4         // new Simpson().talk(); This doesn't compile, we can't instantiate abstract\
5 classes
6         new Homer().talk(); // The output will be "D'oh"
7     }
8
9 }
10
11 abstract class Simpson { abstract void talk(); }
12 class Homer extends Simpson {
13     @Override
14     void talk() {
15         System.out.println("D'oh");
16     }
17 }
```

Abstract classes can't be instantiated directly, but we can instantiate the Homer class, which will instantiate the Simpson abstract class and then print "D'oh".

6.12.1 What happens if an abstract class extends another abstract class?

It is possible for one abstract class to inherit from another one. In this case, it isn't necessary for the subclass to implement the abstract methods. This is only required when a concrete class extends an abstract class.

Let's see how this works in code:

```

1  abstract class Hero { abstract void usePower(); }
2  abstract class Marvel extends Hero {}
3
4  class SpiderMan extends Marvel {
5      @Override
6      void usePower() {
7      }
8  }

```

Note that in the Marvel abstract class, it's not necessary to declare the `usePower` method. We only need to do so in the first concrete class in the inheritance chain, which in this case is the `SpiderMan` class. If we don't declare this method in the `SpiderMan` class the code won't compile.

6.12.2 Marvel Attack Polymorphism Challenge

The following challenge is based on the main topics covered in this chapter. In addition to the `InheritanceFinalChallenge` class that will run the program, we have the `SpiderMan` and `IronMan` classes that extend the abstract `Character` class.

There are many concepts at play in this challenge. If you get it right, it means you've really absorbed the contents of this chapter. If not, don't worry; you'll understand why when you run the code and do your own tests.

Therefore, what will happen when running the following code?

```

1  public class InheritanceFinalChallenge {
2      private static int damage;
3      public static void main(String... doYourBest) {
4          Character ironMan = new IronMan();
5          ironMan.attack();
6          ((IronMan)ironMan).protonCannon();    // Line 6
7          Character character = new SpiderMan();
8          character.attack();
9          System.out.println(damage);
10         ((IronMan)character).protonCannon();  // Line 10
11     }
12     static abstract class Character {
13         Character() { damage++; }
14         abstract void attack();
15     }
16     static class IronMan extends Character {
17         Armor armor = new Armor();
18         void protonCannon() {

```

```

19         System.out.println("Boom!");
20     }
21     void attack() { armor.standingLight(); }
22 }
23 static class Armor {
24     void standingLight() { System.out.println("--->>>"); }
25 }
26 static class SpiderMan extends Character {
27     void attack() {
28         new SpiderMan();
29         System.out.println("Spider-Web!");
30     }
31 }
32 }

```

- A. --->>>
 Boom!
 Spider-Web!
 3
 The ClassCastException will be thrown at line 10
- B. --->>>
 The ClassCastException will be thrown at line 6
- C. 2
 The ClassCastException will be thrown at line 10
- D. Boom!
 Spider-Web!
 3
 Boom!

Explanation:

Let's analyze what's happening in the main method of this challenge. First, we create an instance of the IronMan class and assign it to the variable ironMan, of type Character. Then we invoke the attack method from IronMan (demonstrating the concept of polymorphism):

```

1 Character ironMan = new IronMan();
2 ironMan.attack();

```

This method invokes the standingLight method from the Armor class. The resulting output will therefore be:

```
1 --->>>
```

Next we invoke the `protonCannon` method, which is specific to the `IronMan` class. Note that this is only possible if we use casting, because the declared type we used is `Character`. The `Character` class doesn't include this method, so there is no way for the JVM to know it exists at compilation time. We have to take responsibility for this and basically say to the JVM, "I know what I'm doing here, if something bad happens it's my fault." The JVM will then allow us to invoke the method:

```
((IronMan)ironMan).protonCannon();
```

The output from this is:

Boom!

Next we have another example of polymorphism. Here, we instantiate `SpiderMan` and invoke the `attack` method from the `SpiderMan` class:

```
1 Character character = new SpiderMan();
2 character.attack();
```

This prints:

Spider-Web!

The damage variable count will be 3 at this point, because with each instantiation we invoke the default super constructor. Remember that the parent class's constructor will always be invoked first when the child class is instantiated. (If you're wondering why 3, note that another instance of `SpiderMan` is created inside the `attack` method.)

Finally, we invoke the `protonCannon` method again—but the `SpiderMan` instance doesn't have this method, so a `ClassCastException` will be thrown:

```
((IronMan)character).protonCannon();
```

Therefore, the final answer will be:

```
A. --->>>
    Boom!
    Spider-Web!
    3
    The ClassCastException will be thrown at line 10
```

6.13 Factory Method Strategy Discount Real-world Challenge

Suppose you work for a financial company, and you've been tasked with calculating final product prices for customers in an exposed web service. An external system needs to invoke a specific method from your project to reuse some of the logic.

You have to create a method to calculate the prices and expose it as a REST service to the other system. You will receive a String called `command` in your method that indicates which of two types of discounts to apply (`"NORMAL_DISCOUNT"` or `"VIP_DISCOUNT"`), and a product ID that you will be able to use to get the original price of the produce from your system. That method has the goal of applying the appropriate discount to the product the customer is buying. The method will return the final price of the product with the discount applied.

If `"NORMAL_DISCOUNT"` is passed, you need to apply a 20% discount to the price of the received product. If `"VIP_DISCOUNT"` is passed you must apply a 50% discount.

Note that you can use polymorphism in this situation: you can encapsulate the logic for the `"NORMAL_DISCOUNT"` in a specialized class and the logic for the `"VIP_DISCOUNT"` in another class. To do this you will need a generic `Discount` class that can't be instantiated (an abstract class) and that has a common method to give a 10% discount for every product on which you need to invoke it for each input you are receiving on your method.

As a bonus, create a unit test method that passes the following input data and make sure the output will be as it was defined in this situation.

This is the input data you will be receiving:

```
1 "NORMAL_DISCOUNT", 1
2 "VIP_DISCOUNT", 2
3 "NORMAL_DISCOUNT", 3
4 "NORMAL_DISCOUNT", 4
5 "VIP_DISCOUNT", 5
6 "NORMAL_DISCOUNT", 6
```

Create a List populated with the following information in your Product POJO:

```
1 Product(1, "Bat Mobile", 1000)
2 Product(2, "Flying Car", 2000)
3 Product(3, "PS10", 500)
4 Product(4, "Jetpack", 800)
5 Product(4, "Ultima Weapon Sword", 700)
6 Product(5, "Duff Beer", 10)
```

In your logic implementation, you should log the name of the product, the price without any discount, and the price with the discount, and then return the price with the appropriate discount applied.

6.14 Summary

In this chapter you learned how to make good use of inheritance, and when to choose composition over inheritance.

- You also learned about overriding inherited methods and how to use the great power of polymorphism.
- You now know that all Java classes use inheritance because they all extend `Object`, and that if you've ever overridden the `toString`, `equals`, or `hashCode` methods you already made use of polymorphism.
- You understand how access modifiers work in relation to inheritance, and how to prevent inheritance of classes or methods with the `final` keyword or the `private` modifier.
- Although Java doesn't allow inheriting from multiple classes, you've seen how to simulate it by inheriting from interfaces.
- You learned how to work with the `super` reserved word to access a parent class's methods and constructors.
- You also learned how to use abstract classes to design systems that are well organized and easy to maintain.

7 Interfaces

This chapter covers:

- Developing flexible code that depends on interfaces, not instances, and designing interfaces that are reusable
- Creating method implementations in interfaces by using default methods
- Deciding whether to use interfaces or abstract classes
- Using static methods in interfaces
- Simulating multiple inheritance with default methods
- Using private methods in interfaces

A great way to make Java code more flexible is by using interfaces. This reduces coupling in our code, making it easier to change and extend it when necessary.

An interface works like a contract. Any class that implements a given interface contract will have to implement all public abstract methods from that interface. With interfaces, we can apply the concept of polymorphism, which means that we can pass different implementations to the same interface. That's a crucial concept for building code that's easy to extend, because chances are that changes will be only necessary in the implementation of the interface parameter.

Many core Java classes use the concept of interfaces. For example, `ArrayList` implements `List`, `HashSet` implements `Set`, `HashMap` implements `Map`, `ArrayDeque` implements `Deque`, and so on.

7.1 Interfaces and Polymorphism

To demonstrate the flexibility that interfaces provide, let's consider an example. Here, the `addElement` method takes the `List` interface as a parameter. That means we can pass this method an instance of any class that implements that interface:

```
1 public class ListExample {
2
3     public static void main(String... usingListWithFlexibility) {
4         addElement(new ArrayList());
5         addElement(new LinkedList());
6         addElement(new Vector());
7         addElement(new CopyOnWriteArrayList());
8     }
9
10    static void addElement(List list) {
11        list.add("Adding elements with different types of lists.");
12        System.out.println(list + ":" + list.getClass().getSimpleName());
13    }
14
15 }
```

This is a powerful concept, because it means if you need to work with a different List you can simply send the type you need. Now imagine if you needed to implement this same logic without polymorphism—you would have to replicate this code with a specific method for each List type:

```
1 import java.util.ArrayList;
2 import java.util.LinkedList;
3 import java.util.Vector;
4 import java.util.concurrent.CopyOnWriteArrayList;
5
6 public class AddListElementWithoutPolymorphism {
7
8     public static void main(String... addingElementsWithoutPolymorphism) {
9         addElement(new ArrayList());
10        addElement(new LinkedList());
11        addElement(new Vector());
12        addElement(new CopyOnWriteArrayList());
13    }
14
15    static void addElement(ArrayList list) {
16        list.add("Adding elements with different types of lists.");
17        System.out.println(list + ":" + list.getClass().getSimpleName());
18    }
19
20    static void addElement(LinkedList list) {
21        list.add("Adding elements with different types of lists.");
22        System.out.println(list + ":" + list.getClass().getSimpleName());
23    }
```

```
24
25  static void addElement(Vector list) {
26      list.add("Adding elements with different types of lists.");
27      System.out.println(list + ":" + list.getClass().getSimpleName());
28  }
29
30  static void addElement(CopyOnWriteArrayList list) {
31      list.add("Adding elements with different types of lists.");
32      System.out.println(list + ":" + list.getClass().getSimpleName());
33  }
34
35 }
```

Note the repetition—this code is far harder to read and will be much more difficult to maintain. It accomplishes the same thing as the previous version, but requires much more code. This should give you an idea of the power of polymorphism with interfaces.

7.2 Covariant return types

We know that when using polymorphism the method signature—that is, the method name, return type and parameters type—has to be the same. However, does a return type have to be always the same? A method's return type can only be changed if there is an inheritance relationship between the classes, and the superclass has to have the most generic type. For example, if the return type in the superclass is `Number`, the return type in the subclass could be `Integer` because `Integer` inherits from `Number`. This idea that the return type in the subclass must be more specific than the one in the superclass (that is, must vary in the same direction as the subclass relationship) is known as covariance.

In the following example, we're overriding the `move` method from `Dinosaur` in the subclass `Pterodactyl` and returning a `String`. This works because `String` is a child of `Object`. If we tried to use a primitive type in the subclass, for example, the code wouldn't compile because a primitive type is not an `Object`:

```
1  public class CovariantReturn {
2
3      public static void main(String... dinosaurs) {
4          new Dinosaur().move();
5          new Pterodactyl().move();
6      }
7
8  }
9
```

```
10 class Dinosaur {
11     Object move() {
12         return "The Dinosaur is moving";
13     }
14 }
15
16 class Pterodactyl extends Dinosaur {
17     @Override
18     String move() {
19         return "The Pterodactyl is moving in the air";
20     }
21 }
```

The return type from the child class has to be a child of the parent class's return type. This technique won't work with primitive types because there is no inheritance between them.

7.2.1 Jedi Covariant Polymorphism Challenge

In this challenge we have an abstract class and another class that extends it. The concept of covariant types is being used here. What do you think the output will be after the main method is compiled and run? Will the code compile?

```
1 public class YodaForce {
2     public static void main(String... covariantType) {
3         System.out.println(new Yoda().useSaber());
4         System.out.println(new Yoda().attack());
5         System.out.println(new Yoda().lightForce); // Line 6
6     }
7     static abstract class LightForce {
8         int lightForce;
9         abstract Object useSaber();
10        abstract long attack();
11
12        LightForce() {
13            lightForce++;
14        }
15    }
16    static class Yoda extends LightForce {
17        String useSaber() {
18            return "useSaber";
19        }
20    }
```

```

21         long attack() {
22             return 99999;
23         }
24     }
25 }

```

- A. useSaber
99999
3
- B. Compilation error at line 6
- C. useSaber
99999
1
- D. RuntimeException

This code will compile and run fine. The concepts demonstrated here are mostly about polymorphism. In the first method invocation, we are simply invoking the overridden useSaber method:

```
System.out.println(new Yoda().useSaber());
```

Note that we are using a covariant return type for the overridden method: the Yoda subclass's useSaber method returns a String instead of an Object. This method prints "useSaber".

In the second method invocation we are invoking another overridden method, attack:

```
System.out.println(new Yoda().attack());
```

We can't use a covariant type here because there is no inheritance between primitive types; both methods return a long. The output of this method call is 99999.

Finally, we print the lightForce variable that is incremented each time the LightForce class is instantiated:

```
System.out.println(new Yoda().lightForce); // Line 6
```

Even though we instantiate the LightForce class three times, note that the lightForce variable is an instance variable. Therefore, the variable will be reset every time a new instance is created. We have three instantiations, but the result of this variable will be 1 for each of the three instances.

In conclusion, the correct alternative is:

- C. useSaber
99999
1

7.3 Default methods

A problem had to be solved in Java 8. It was necessary to add many new methods in widely used interfaces such as List, Set, and Map because of the use of functional programming in the existing Collection api. Imagine if a public abstract methods were declared in the List interface—a lot of code would break because classes that implement that interface would not correctly implement the new methods, and if those methods had to be added to all those classes the code repetition would be immense.

Java is a language that values backward compatibility, so the decision was made to allow methods with implementations in interfaces. With these new methods, known as default methods, it's possible to provide a default implementation in the interface so that each class is not required to provide its own implementation.

A great example of a default method that's widely used in Java 8 and later is the `forEach` method implemented in the `Iterable` interface. This method goes through each element in a collection, performing the specified action. Let's take a look at the method declaration (don't worry too much about understanding what's happening here now, we'll explore functional programming in more depth in later chapters):

```
1 public interface Iterable<T> {  
2     // Other methods omitted  
3     default void forEach(Consumer<? super T> action) {  
4         Objects.requireNonNull(action);  
5         for (T t : this) {  
6             action.accept(t);  
7         }  
8     }  
9 }
```

We can also provide customized default methods in our own classes. Let's see how that works:

```
1 public class Yoda implements Jedi {  
2  
3     public static void main(String... defaultMethodExample) {  
4         new Yoda().useForce();  
5     }  
6  
7 }  
8  
9 interface Jedi {  
10  
11     default void useForce() {
```

```
12         System.out.println("Using the force!");
13     }
14
15 }
```

As you can see, we don't need to implement the default method; we simply instantiate the Yoda class and use the useForce method. The output will be:

```
"Using the force!"
```

7.4 Abstract classes vs. interfaces

You may be wondering whether interfaces are the same as abstract classes. The answer is no—there are many differences between them. One is that abstract classes can have instance variables that keep state, whereas interfaces can only have constants. Also, more than one interface can be implemented in a class, but just one abstract or concrete class can be extended. Finally, abstract classes have constructors but interfaces do not.

The following code illustrates the differences between an abstract class and an interface. The LightForce abstract class has a constructor and an instance variable, force. The DarkForce interface has a constant, FORCE, and a default method, useForce. In the main method of the program we invoke the constructor of the abstract class and the default method from the DarkForce interface:

```
1  public abstract class LightForce {
2      private int force;
3      LightForce(int force) {
4          this.force = force;
5          System.out.println("The force is being used:" + this.force);
6      }
7
8      public static void main(String... abstractClassesVsInterfaces) {
9          new LukeSkywalker(1099);
10         new DarthVader().useForce();
11     }
12 }
13
14 interface DarkForce {
15     int FORCE = 999;
16
17     default void useForce() {
18         System.out.println("The force is being used:" + FORCE);
19     }
20 }
```

```

21
22 class LukeSkywalker extends LightForce {
23     LukeSkywalker(int force) {
24         super(force);
25     }
26 }
27 class DarthVader implements DarkForce {}

```

The output of this program will be:

The force is being used:1099

The force is being used:999

As you can see, even with default methods, interfaces are very different from classes. A default method can only receive a parameter as a variable, and if we declare a variable in the interface, it has to be a constant. Default methods are a great solution for avoiding code code repetition, however, and also a great feature that enables us to create more sophisticated interfaces with meaningful code.

7.4.1 Absctract Class VS Interface Challenge

In the following challenge, we have an interface and an abstract class. The interface has a default method, useSaber, and in the abstract class we implement the method attack. We are also using the concept of an anonymous inner class in the main method. What do you think will happen when this program is run?

```

1  public class LightVsDarkSide {
2
3      public static void main(String... abstractVsInterface) {
4          Object object = new ObiWan() {
5              int attackPower = 99999;
6              Object jump() {
7                  attackPower = 999999;           // Line 7
8                  return attack(attackPower);
9              }.jump();
10
11          System.out.println(object);
12
13          DarthMaul darthMaul = new DarthMaul() {
14              int defencePower = 99999;           // Line 14
15              public void useSaber() {
16                  System.out.println("DarthMaul loses the saber and defends with the p\
17 ower of " + defencePower);
18              }

```



```
19         };
20
21         darthMaul.useSaber();
22     }
23
24 }
25
26 abstract class ObiWan {
27     Object attack(int attackPower) {
28         return "ObiWan attacks DarthMaul with the power of " + attackPower;
29     }
30 }
31
32 interface DarthMaul {
33     default void useSaber() {
34         System.out.println("DarthMaul uses the saber");
35     }
36 }
```

- A. Compilation error at line 7
- B. Compilation error at line 14
- C. ObiWan attacks DarthMaul with the power of 999999
DarthMaul uses the saber
- D. ObiWan attacks DarthMaul with the power of 999999
DarthMaul loses the saber and defends with the power of 99999



Important: Remember, practice makes it perfect. Therefore, try out the Java Challenge before seeing the answer.

Explanation:

Let's analyze the code to understand what is happening. In the first lines of the main method we are using the concept of anonymous inner classes. As explained before at chapter 4, an anonymous inner class is a class without a name intended to be used just once. It can define static members and class methods, extend another class, or implement an interface.

In our case, the anonymous inner class is extending ObiWan and then declaring the attackPower variable. It then overrides the Object class's jump method, changing the value of attackPower to 999999. The last instruction in the method is the attack method invocation; we pass this the attackPower variable and it prints the String as normal.

Next, we use the same concept of an anonymous inner class to implement the DarthMaul interface. Notice that we are not instantiating an interface or an abstract class; we are using a class with no

name that extends an abstract class or implements an interface. At the beginning of this anonymous inner class we declare a constant called `defencePower`. Then we override the default `useSaber` method, passing the `defencePower` variable into it. Finally, we invoke the `useSaber` method from `DarthMaul`.

In conclusion, the final result will be:

- D. ObiWan attacks `DarthMaul` with the power of 999999
 `DarthMaul` loses the saber and defends with the power of 99999

7.5 Static methods in interfaces

It's possible to use static methods in interfaces, just as we do in normal classes. We simply use the interface name and invoke the static method, as shown here:

```
1 public class StarWarsWarrior {
2
3     public static void main(String... staticMethodOnInterfaces) {
4         Chewbacca.roar();
5     }
6
7 }
8
9 interface Chewbacca {
10
11     static void roar() {
12         System.out.println("Roar!");
13     }
14 }
```

The output in this case will be "Roar!". Just remember that when you use an interface's static or default methods you can't use the access modifier `protected`; if you try this you'll get a compilation error.

Another important thing to understand about static methods is that they can't be overridden. Considering that a static method is a class method, not an instance method, that makes sense.

As the following example demonstrates, the only way to invoke a static method defined in an interface is by referencing the interface name and invoking the method. If you try to invoke the static method by the interface implementation instance, the code won't compile:

```
1 public class LukeSkyWalker implements Jedi {
2
3     public static void main(String... staticMethodShadowing) {
4         Jedi anakinSkyWalker = new LukeSkyWalker();
5         // anakinSkyWalker.useSaber(); This doesn't compile
6         Jedi.useSaber();
7     }
8
9     static void useSaber() {
10        System.out.println("Luke uses his saber");
11    }
12
13 }
14
15 interface Jedi {
16
17     static void useSaber() {
18        System.out.println("Anakin uses his saber");
19    }
20
21 }
```

The above code will print the following: “Anakin uses his saber”

7.6 Simulating multiple inheritance with default methods

Have you ever wondered which interface method will be invoked if a class implements two different interfaces that define the same method? If you don’t include a specific method in the class that implements those two interfaces, the code won’t compile.

The following example demonstrates this. The `Anakin` class implements two interfaces, `Jedi` and `Sith`, that both declare the `useSaber` method:

```

1  public class Anakin implements Jedi, Sith {
2
3      public static void main(String... defaultMethodMultipleInheritance) {
4          new Anakin().useSaber();
5      }
6
7  }
8
9  interface Jedi {
10     default void useSaber() {
11         System.out.println("Jedi's Saber slash");
12     }
13 }
14
15 interface Sith {
16     default void useSaber() {
17         System.out.println("Sith's Saber slash");
18     }
19 }

```

This code will produce the following compilation error:

Error:(3, 8) java: types Jedi and Sith are incompatible; class Anakin inherits unrelated defaults for useSaber() from types Jedi and Sith

What you need to do to make it work is provide an implementation of the useSaber method in Anakin that will remove the ambiguity for the compiler.

```

1  public class Anakin implements Jedi, Sith {
2
3      public static void main(String... defaultMethodMultipleInheritance) {
4          new Anakin().useSaber();
5      }
6
7      @Override
8      public void useSaber() {
9          System.out.println("Anakin's Saber slash");
10         Jedi.super.useSaber();
11         Sith.super.useSaber();
12     }
13
14 }
15
16 interface Jedi {

```

```
17     default void useSaber() {
18         System.out.println("Jedi's Saber slash");
19     }
20 }
21
22 interface Sith {
23     default void useSaber() {
24         System.out.println("Sith's Saber slash");
25     }
26 }
```

The output of this program will be:

```
1 Anakin's Saber slash
2 Jedi's Saber slash
3 Sith's Saber slash
```

This example also demonstrates how it's possible to use the `super` keyword to reference a specific interface method. It's important to mention that we can only do this if we're implementing the interface. In conclusion, as mentioned in the previous chapter, although Java doesn't actually support multiple inheritance there are some ways to simulate this behavior.

7.6.1 Chewbacca Default Attack Challenge

This challenge explores the concepts of polymorphism and static methods. Can you guess what will happen when this program is executed?

```
1 public class ChewbaccaWarrior implements Warrior {
2
3     public static void main(String... doYourBest) {
4         pilotShip();
5         new ChewbaccaWarrior().attack();
6     }
7
8     public void run() {
9         System.out.println("Chewbacca is running");
10    }
11
12    static void pilotShip() {
13        System.out.println("Chewbacca pilots the ship");
14    }
15 }
```

```
16
17 interface Warrior extends Brave {
18     int attackForce = 99999;
19
20     default void attack() {
21         System.out.println("Warrior attack:" + attackForce);
22         run(); // Line 27
23         Brave.super.attack();
24     }
25
26     static void pilotShip() {
27         System.out.println("Warrior pilots the ship");
28     }
29
30     void run();
31 }
32
33 interface Brave {
34     default void attack() {
35         System.out.println("Brave attack");
36     }
37 }
```

- A. Compilation error at line 4
- B. Chewbacca pilots the ship
Warrior attack:99999
Chewbacca is running
Brave attack
- C. Compilation error at line 5
- D. Compilation error at line 27



Important: Remember, to improve your programming skills you need to try out the Java Challenge before seeing the answer.

Explanation:

If you guessed that it will compile fine, you got this challenge right—congratulations! If not, don't worry; you'll learn why as we walk through the code.

The first method we invoke in the main method is the static method `pilotShip`:

```
pilotShip();
```

The `pilotShip` method from the `ChewbaccaWarrior` class will be invoked in this case because we are not explicitly saying that we want to invoke the `pilotShip` method from another class or interface. The JVM will automatically add the class name of the enclosing class, like this:

```
ChewbaccaWarrior.pilotShip();
```

Remember that it's not possible to use polymorphism with static methods, so they always refer to classes, not instances.

Now we're going to explore polymorphism. In the second line we invoke the `attack` method:

```
new ChewbaccaWarrior().attack();
```

This method is declared in both the `Warrior` and `Brave` interfaces. The `Warrior` interface overrides the `attack` method from the `Brave` interface, so the `Chewbacca` class will use the `Warrior` `attack` method and not the `Brave` `attack` method.

Let's take a closer look at the default `attack` method in the `Warrior` interface:

```
1 default void attack() {  
2     System.out.println("Warrior attack: " + attackForce);  
3     run();  
4     Brave.super.attack();  
5 }
```

The first line simply prints the `String` `"Warrior attack: 99999"`. Remember that the `attackForce` constant can't be changed—interfaces don't have state, so they can't have instance variables that are mutable.

In the second line of the `attack` method, we invoke the `run` method. At first glance this seems strange, because it looks like we're invoking a method without a body. Just remember that it's impossible to invoke a default method without an instance, and to have an instance it's necessary to have an implementation of the interface; thus, we can conclude that whenever we invoke this `attack` method we will always have an implementation for the `run` method. That's exactly what is happening in our challenge. The `ChewbaccaWarrior` class is implementing the `Warrior` interface, and it's also implementing the abstract `run` method. Consequently, the code implemented in the `run` method of the `ChewbaccaWarrior` class will be executed. So, this method invocation will print:

```
"Chewbacca is running"
```

The last line of the `Warrior` interface's `attack` method invokes the default `attack` method from the `Brave` interface. This will print:

Brave attack

In conclusion, the right alternative is B.

7.7 Private methods in interfaces

Java 9 introduced the possibility of including private methods in interfaces, for the purpose of avoiding code repetition. Sometimes it's necessary to have some code in the default methods, and it's handy to be able to implement our logic in a private method.

Let's take a look at an example where code repetition would be necessary for a default method:

```
1 public interface HanSolo {
2
3     default void shootWithABlaster() {
4         String weapon = "Blaster";
5         System.out.println("Choose the weapon:" + weapon);
6         System.out.println("Get the " + weapon);
7         System.out.println("Aim the " + weapon);
8         System.out.println("Shoot with the " + weapon);
9     }
10
11     default void shootWithABowcaster() {
12         String weapon = "Bowcaster";
13         System.out.println("Choose the weapon:" + weapon);
14         System.out.println("Get the " + weapon);
15         System.out.println("Aim the " + weapon);
16         System.out.println("Shoot with the " + weapon);
17     }
18
19 }
```

There's a lot of repetition here! Now let's see what happens when we refactor that with the use of a private method. This example demonstrates why private methods are important for interfaces, and how to take advantage of this feature:

```
1 public interface HanSolo {
2     default void shootWithABlaster() {
3         String weapon = "Blaster";
4         shoot(weapon);
5     }
6     default void shootWithABowcaster() {
7         String weapon = "Bowcaster";
8         shoot(weapon);
9     }
10    private void shoot(String weapon) {
```



```
11     System.out.println("Choose the weapon:" + weapon);
12     System.out.println("Get the " + weapon);
13     System.out.println("Aim the " + weapon);
14     System.out.println("Shoot with the " + weapon);
15 }
16 }
```

This is a far simpler and more reusable approach.

7.7.1 Private Jedi Jump Challenge

This challenge explores important concepts relating to private, static, and default interface methods and how to manipulate them. Can you guess what will happen when the main method is executed?

```
1  public class JediAttack {
2      interface Jedi {
3          default int attack() {
4              return jump() + useSaber() + useForce();
5          }
6
7          private int jump() { return 1; }
8          private static int useSaber() { return 3; }
9          private int useForce() { return 5; }
10     }
11     public static void main(String... doYourBest) {
12         System.out.println(new Jedi() {
13             public int jump() { // Line 13
14                 return 2;
15             }
16         }.attack() + Jedi.useSaber()); // Line 16
17     }
18 }
```

- A. 13
- B. 12
- C. Compilation error at line 14
- D. Compilation error at line 16



Important: By analyzing and running the code in your mind, you will absorb the concepts of the Java Challenge.

Explanation:

The first point to analyze is that we are instantiating an anonymous inner class from the Jedi interface, this means, an instance without a name.

Then we are trying to override the `jump()` method in the anonymous inner class. However, note that the `jump()` method declared in the Jedi interface is `private`. Therefore, it's impossible to override this method, it's not reachable outside of this interface. The `jump()` method declared in the anonymous inner class will never be used since it wasn't overridden.

Now, we are going to invoke the `attack` method which invokes the `jump()`, `useSaber` and `useForce()` methods. Let's see what happens in the following invocations:

- The `jump()` method invocation returns 1 since there is no overriding.
- The `useSaber()` method returns 3, also note that this method is `static`. It's possible to use `static` methods in interfaces since Java 8.
- The `useForce()` method returns 5 which is another `private` method.

So far, we have the result of 9 by summing those method return values.

Finally, we will invoke the `static useSaber()` method that returns 3. Therefore, the final result will be:

B) 12

7.8 Command Design Pattern Challenge

Are you familiar with the Command design pattern? With the knowledge you've gained in this chapter, you'll be able to develop code that follows this pattern. The `Thread` class is a great example of the Command pattern—when we start a `Thread`, the logic from the `run` method will be executed. That's the idea of this real-world challenge.

Your goal is to verify whether a product is eligible for a discount. You will receive a `Product` with a name, price, type, and quantity in your `CommandExecutor` and a `List` of three `Commands` to deal with different business requirements.

In one `Command` you will implement logic to verify that the product quantity is at least 2. In the second `Command` you will check that the price of the two products is at least 500. Finally, you will verify that the product type is robotics. If all those conditions are true, you will log a message:

The product `$productName` is eligible for a discount

7.9 Summary

With the knowledge you've gained in this chapter, you will be able to design your systems in a more loosely coupled and easily maintainable way by using interfaces together with the concepts

of polymorphism and overriding methods. Polymorphism is one of the most important and difficult concepts to master in any object-oriented programming language, and understanding it will make it much easier to grasp other concepts.

- You learned that it's possible for an interface to extend another one, which will be helpful in designing your classes. Many classes from the JDK use this concept.
- You know the differences between interfaces and abstract classes and when to use each one.
- You also know how to use default, static, and private methods in interfaces and how to simulate multiple inheritance with default methods.

8 Exceptions

This chapter covers:

- Understanding the difference between checked and unchecked exceptions and when to use each type
- Using the stack trace to identify the root cause of an error
- Handling or declaring checked exceptions in your code
- Using the try, catch, finally flow effectively
- Closing Java resources effectively by using the try-with-resources statement
- Catching multiple exceptions in a single catch block
- Creating customized exceptions that make sense for the application
- Using the “throw early, catch later” principle for code quality

Exceptions—unwanted or unexpected events that arise during execution of a program, disrupting the normal flow of the program’s instructions—are one of the most important concepts to master in Java. As a developer, you’ll need to handle them on a daily basis. When exceptions are handled in the wrong way, bugs are able to hide more easily. This causes developers a lot of stress, so it’s crucial to understand how to use exceptions effectively.

It’s common to see exceptions that won’t help developers when errors occur—for example, generic exceptions with poor log messages. Understanding the key concepts of exceptions and which types of exceptions are most suitable to use in different situations will make all the difference in the software you develop. Successfully absorbing the contents of this chapter will be a huge help to the professional Java developer who wants to deliver high-quality code.

8.1 Checked and unchecked exceptions

There are two main types of exceptions: checked and unchecked. Understanding the difference between them is crucial for handling exceptions effectively.

When we want a class to always handle or declare an exception, we should use a checked exception. These are conditions that are checked at compile time, so they can be handled and the program can continue executing. When we don’t want the application to recover or handle the exception, we should use an unchecked exception. These are checked at runtime and are usually conditions that are outside the control of the program and should cause execution to stop.

Just as `Object` is the superclass of all object types in Java, `Throwable` is the superclass of all exception types (both checked and unchecked). When an exception occurs in a method the method creates an object of type `Throwable`, hence the term “throwing” an exception.

With that basic introduction in mind, let's take a closer look at the details of the different types of exception.

8.1.1 Checked exceptions

Checked exceptions cause a compilation error when thrown by a method, causing program execution to stop unless they are handled by another method. For that reason, many developers say that checked exceptions break the concept of encapsulation (the idea that data and methods that work on that data should be contained within a single unit, such as a class). And in a way they're right, because the method that throws an exception will be know that the other method is throwing an exception within the logic.

In general, you should use checked exceptions when you are dealing with an important business requirement that has to be clear, or when you don't have control over an external resource you are using in your application.

You can see many examples of checked exceptions when working with the well-known JDBC (database manipulation) API from the JDK. Suppose you're working with an external database, and when you try to access it for some reason the database is not running. When the database is down, it makes sense to throw an exception that will let us know what is happening.

With a checked exception, you will know for sure that this exception is being thrown because the compiler will complain if you don't handle the exception. You can also use a customized log message to make the error clear.

Let's see an example of a checked exception in action:

```
1 public static Connection getConnection() {
2     Connection connection = null;
3     try {
4         connection = DriverManager.getConnection(
5             "jdbc:postgresql://localhost:5432/anyDatabase", "user", "password");
6     } catch (SQLException sqlException) {
7         // Some logic here
8         sqlException.printStackTrace();
9     }
10
11     return connection;
12 }
```

There are several different strategies you could use to recover from this error. For example, you could return a Connection from another database, or you could wrap the sqlException in a more generic type. You could also use a generic exception to avoid throwing many different exceptions in a single method, like the InfrastructureException used here:

```
1 public static Connection getConnection() {
2     try {
3         return connection = DriverManager.getConnection(
4             "jdbc:postgresql://localhost:5432/anyDatabase", "user", "password");
5     } catch (SQLException sqlException) {
6         throw new InfrastructureException (sqlException);
7     }
8 }
```

All checked exceptions inherit from `Exception`, which is a subclass of `Throwable`.

8.1.2 Unchecked exceptions

Unchecked exceptions are useful when you want to make the application stop working and don't want to recover from the error condition. They are used for things that might go wrong during runtime of an application and do not result in compilation errors.

All unchecked exceptions inherit from `RuntimeException`, which is a subclass of `Exception`. There are only a few very specific situations where we should handle a `RuntimeException`. Instead, we should try to avoid them by using code logic.

The famous `NullPointerException`, which is widely used in the JDK code, is a great practical example of what a `RuntimeException` should be used. Consider the example of the `toUpperCase` method from the `String` class:

```
1 public String toUpperCase(Locale locale) {
2     if (locale == null) {
3         throw new NullPointerException();
4     }
5     ...
6 }
```

Note that we simply throw the `NullPointerException` without handling it. The `Locale` class is crucial for the logic of this method; if there is no `Locale` (an `Object` representing a specific geographical, political, or cultural region), the intention of the programmer that implemented the `toUpperCase` method is to make the application crash.

Another well-known example is `IllegalArgumentException`, which should be thrown when an unexpected argument is passed to a method. The `File` class from the JDK uses `IllegalArgumentException` when the uniform resource identifier (URI) it's passed is invalid, because there is no way to handle this condition programmatically:

```
1 public File(Uri uri) {  
2     if (!uri.isAbsolute())  
3         throw new IllegalArgumentException("URI is not absolute");  
4     ...  
5 }
```

8.1.3 JVM errors

Other than `Exception` and `RuntimeException` there is the `Error`. This is the most serious type of error that can happen and should never be handled. The various error types in Java all inherit from `Error`, which, like `Exception`, extends the class `Throwable`. An example is the `StackOverflowError`, which indicates that there was a problem with the memory stack. If that happens the application should crash right away.

We can make a `StackOverflowError` happen by creating a recursive method invocation. This method will keep calling itself indefinitely, eventually exhausting the memory resources of the system, if the problem is not caught:

```
1 void pressButton(String employeeName) {  
2     pressButton("Homer");  
3 }
```

8.1.4 The exception hierarchy

Figure 8.1 illustrates the relationship between the various `Exception` and `Error` classes in Java. Note that these are only a few representative examples of the subclasses of the parent types (`Exception`, `Error`, `RuntimeException`, and so on). The important thing to understand is that all checked exceptions inherit from `Exception`, and all unchecked exceptions inherit from `RuntimeException` or `Error`.

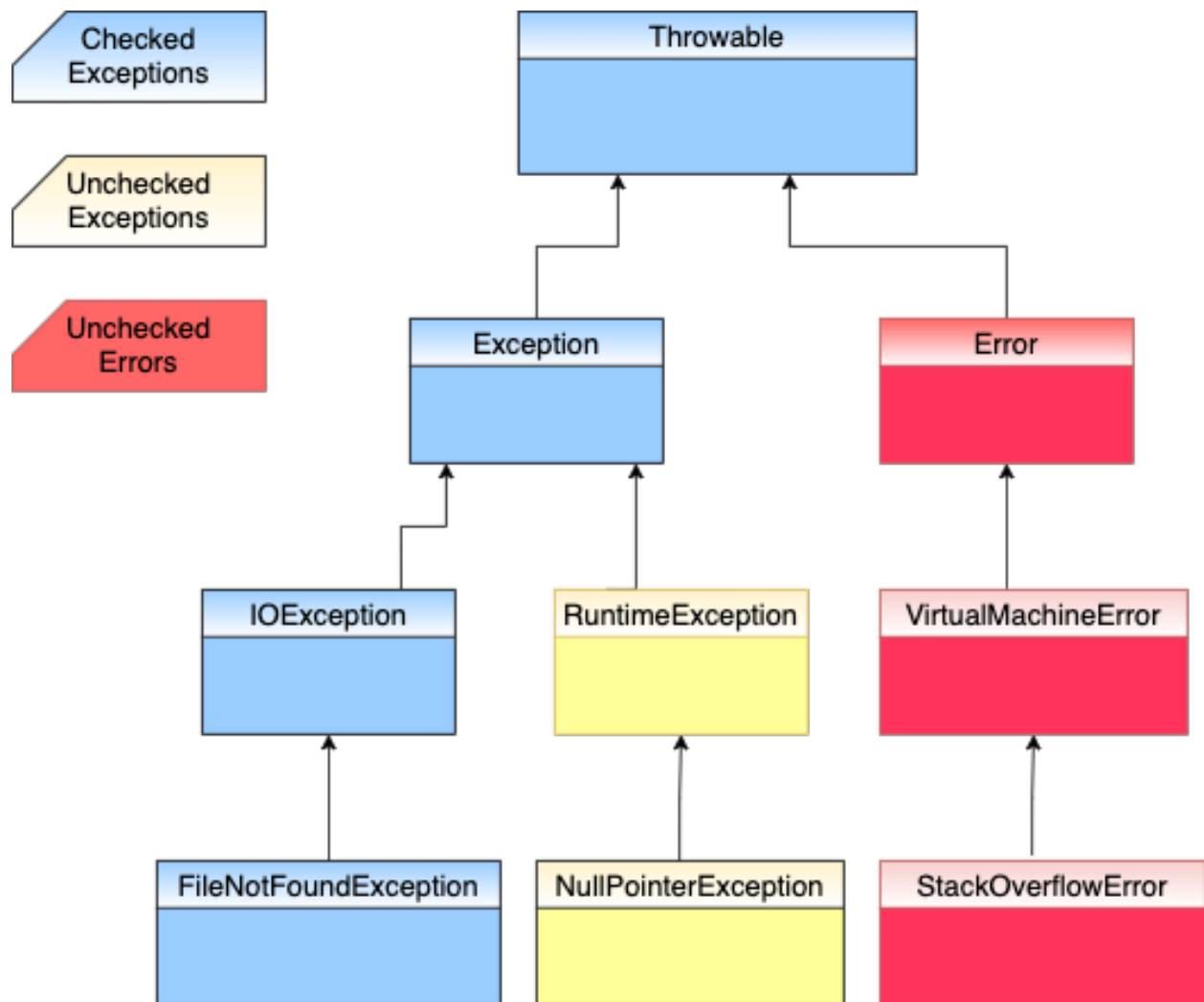


Figure 8.1 The exception hierarchy in Java

As you can see, `SQLException` and `FileNotFoundException` are checked exceptions, whereas `NullPointerException` and `StackOverflowError` are both unchecked exceptions. The class at the top of the exception tree, `Throwable`, is also a checked exception.

8.2 Stack Trace

Perhaps the most important feature of an exception is the stack trace. With this information we can track down the root cause of the error (which method it occurred in), and then figure out what's happening and how to fix it.

Consider the following example, where an exception will be thrown from the `applyDiscount` service method that is being invoked from a controller class (the class that connects with the frontend of an application):


```
1 public class BillingController {
2
3     BillingService billingService;
4
5     BillingController(BillingService billingService) {
6         this.billingService = billingService;
7     }
8
9     public static void main(String... stackTrace) {
10         BillingController controller = new BillingController(new BillingService());
11         controller.applyDiscount("1");
12     }
13
14     void applyDiscount(String customerId) {
15         billingService.applyDiscount(customerId);
16     }
17 }
18
19 class BillingService {
20     void applyDiscount(String customerId) {
21         throw new RuntimeException();
22     }
23 }
```

When the main method is executed, a `RuntimeException` will be thrown and the stack trace will be printed to the console (this is the default behavior whenever an unhandled exception is thrown). In this case, the stack trace of the exception will be the following:

```
1 Exception in thread "main" java.lang.RuntimeException
2     at exceptions.stacktrace.BillingService.applyDiscount(BillingService.java:5)
3     at exceptions.stacktrace.BillingController.applyDiscount(BillingController.java:17)
4     at exceptions.stacktrace.BillingController.main(BillingController.java:13)
```

Note that the stack trace is a list of methods leading from the root method that threw the exception, which is `applyDiscount`, to the last method that received it, which is `main`.

More important exception methods

Other than the `printStackTrace` method there is the `getMessage` method that will inform what error has happened in the application. We can pass this message in the constructor to an `Exception` class or any customized exception.

```
1 public class NoBeerException extends Exception {
2
3     NoBeerException(String message) {
4         super(message);
5     }
6
7     public static void main(String[] args) throws NoBeerException {
8         throw new NoBeerException("All the beer is gone :(");
9     }
10 }
```

Notice that we are passing the exception message into the constructor. That brings us a lot of power to make the error clear and easily fixable.

There is also the `getCause` method that will help you to find out what was the root cause of the exception.

8.3 Handling or Declaring Checked Exceptions

When we work with checked exceptions, we must either handle or declare them. If we don't do that, the code won't compile. There are many checked Exceptions in the JDBC API, for example. We are obliged to either handle the checked exceptions from the JDBC API by using try and catch or throw the exception in the method signature.

When we try to create a `File`, for example, without handling or declaring the `FileNotFoundException`, the code won't compile:

```
1 public static void main(String[] args) {
2     File file = new File("Simpsons.txt");
3     BufferedReader br = new BufferedReader(new FileReader(file));
4 }
```

Here we have two options. Either we handle it with try and catch blocks:

```
1 public static void main(String[] args) {
2     File file = new File("Simpsons.txt");
3     try {
4         BufferedReader br = new BufferedReader(new FileReader(file));
5     } catch (FileNotFoundException e) {
6         e.printStackTrace();
7     }
8 }
```

Or we declare it in the method signature with a throws clause:

```
1 public static void main(String[] args) throws FileNotFoundException {
2     File file = new File("Simpsons.txt");
3     BufferedReader br = new BufferedReader(new FileReader(file));
4 }
```

Remember that this is only necessary with checked exceptions. When we are working with `RuntimeExceptions` or unchecked exceptions we don't need to do anything, because unchecked exceptions in theory shouldn't be handled and instead should be avoided programmatically.

In the example where we handle the exception, note the use of the `printStackTrace` method:

```
e.printStackTrace();
```

This method prints the exception and its stack trace to the standard error stream, which is very useful for debugging.

8.3.1 throws and throw new

We've just seen that the `throws` keyword is used to declare an exception. Related to this is the `throw` keyword, which we can use to throw a particular exception in a method. This is widely used in real applications where we want to wrap a more specific exception in a generic one. For example, in a financial application we might want to create a generic `BusinessException` and a more specific `NoAccountBalanceException`. Let's see what these might look like.

First we'll create the generic exception:

```

1  class BusinessException extends Exception {
2
3      BusinessException() { }
4
5      BusinessException(Exception exception) {
6          super(exception);
7      }
8  }

```

Then we'll create the more specific exception that will extend the generic exception:

```
class NoAccountBalanceException extends BusinessException { }
```

If we have a `BankAccountService` class that implements the logic of a money withdrawal, we can then throw a `NoAccountBalanceException` when the account balance is insufficient to fulfill the withdrawal request:

```

1  class BankAccountService {
2
3      void withdrawMoney(String accountNumber, double moneyAmount) throws NoAccountBal\
4  anceException {
5          // Perform logic
6          throw new NoAccountBalanceException();
7      }
8  }

```

Now consider the controller class that is using the service we created. In this class, we only want to throw the generic `BusinessException`. To make that happen, we need to wrap the `NoAccountBalanceException` in a `BusinessException`:

```

1  public class BankAccountController {
2
3      BankAccountService bankAccountService;
4      BankAccountController(BankAccountService bankAccountService) {
5          this.bankAccountService = bankAccountService;
6      }
7
8      void withdrawMoney(String customerId, double moneyAmount) throws BusinessExcepti\
9  on {
10         try {
11             bankAccountService.withdrawMoney(customerId, moneyAmount);
12         } catch (NoAccountBalanceException noAccountBalanceException) {
13             throw new BusinessException(noAccountBalanceException);
14         }

```

```
15     }  
16  
17 }
```

Knowing how to use the keywords `throw` and `throws` make a great difference in code. Remember that `throws` will be used in a method signature and `throw` will be used directly in code logic.

8.4 try, catch, finally

There are three instructions that are crucial when dealing with exceptions: `try`, `catch`, and `finally`.

The `try` block should be used to define the logic that some exception or error might happen.

The `catch` block is where you provide code to handle the exception. This code is executed only if an exception of the specified type is thrown. You can include whatever logic is appropriate here, but typically the stack trace is printed so it's easier to spot the error and take care of it. You can include more than one `catch` block to handle different exceptions, as you'll see later in this chapter, or you can skip this clause and go straight to the `finally` block.

The `finally` block is always executed when the `try` block exits, as long as it exits normally. If an exception is thrown that's handled by a `catch` clause, it executes after that block has finished executing.

An exception to this rule is if the `try` or `catch` block invokes the `System.exit()` method. In that case, the `finally` block will not be executed.

Including a `finally` block is also optional, but it's a good place to perform cleanup or any other actions that you always want to take place, regardless of whether any exceptions are thrown.

Let's look at a practical example with `try`, `catch`, and `finally`. Here, we're executing simple logic in the `try` block and forcing a `NullPointerException` to be thrown because the `remoteControl` variable is null and we're trying to use the `equals` method afterwards. The `NullPointerException` will be caught in the `catch` block, and at the end the `finally` block will be executed even though there was an exception:

```
1 public class PokerWithTheJoker {  
2  
3     public static void main(String... tryCatchFinally) {  
4         try {  
5             System.out.println("Batman calls the BatMobile");  
6             String remoteControl = null;  
7             remoteControl.equals("BatMobile");  
8         } catch (Exception exception) {  
9             System.out.println("The BatMobile takes a long time to arrive");  
10            exception.printStackTrace();  
11        }  
12    }  
13 }
```

```

11         } finally {
12             System.out.println("Batman is late for playing poker with the Joker");
13         }
14     }
15
16 }

```

The output of this program will be:

```

1  Batman calls the BatMobile
2  The BatMobile takes a long time to arrive
3  java.lang.NullPointerException
4      at exceptions.trycatchfinally.PokerWithTheJoker.main(PokerWithTheJoker.java:9)
5  Batman is late for playing poker with the Joker

```

Earlier I mentioned that there is a way to avoid the finally block being executed, by calling the `System.exit` method. Let's see that in action:

```

1  public class OutlawJoker {
2
3      public static void main(String... avoidingFinallyExecution) {
4          try {
5              // Some logic
6              System.exit(0);
7          } finally {
8              System.out.println("Is the Joker getting caught?");
9          }
10     }
11
12 }

```

This causes the program execution to stop, no matter what, so the code in the finally block will not run. This example also demonstrates the use of try and finally without a catch. This can be useful, for example, if you want to close a specific resource in all cases, even if some unexpected error happens.

8.4.1 Going deeper into the catch block

How do we know whether a catch block will capture an exception or not? The most effective way to answer that is to use the is-a question we explored in chapter 6. If the answer to that question is “yes,” then the exception should be caught. Let's see some practical code examples:

```
1  import java.io.IOException;
2
3  public class CatchThemAll {
4
5      public static void main(String... catchThemAll) {
6          try {
7              throw new NullPointerException();
8          } catch (RuntimeException runtimeException) {
9              runtimeException.printStackTrace();
10         }
11
12         try {
13             throw new IOException();
14         } catch (Exception exception) {
15             exception.printStackTrace();
16         }
17
18         try {
19             throw new StackOverflowError();
20         } catch (Error error) {
21             error.printStackTrace();
22         }
23     }
24
25 }
```

Note that the specialized types `NullPointerException`, `IOException`, and `StackOverflowError` are being caught by the more generalized types `RuntimeException`, `Exception`, and `Error`, respectively. This is because the is-a relationship holds in each case:

- A `NullPointerException` is a `RuntimeException`.
- An `IOException` is an `Exception`.
- A `StackOverflowError` is an `Error`.

Be careful not to catch a more specific type of exception than you throw: if you throw a `RuntimeException` but catch an `IllegalArgumentException`, the more general exception will not be caught. In this case, the is-a relationship does not hold.

8.4.2 Ryu Shoryuken Error Challenge

Now that you know what the `try`, `catch`, and `finally` blocks are, let's explore these concepts in a challenge! Can you guess what the output will be when the main method of this program is executed?

```
1 public class RyuShoryukenException {
2
3     public static void main(String... doYourBest) {
4         try {
5             String ryu = null;
6             ryu.contains("Shoryuken");
7         } catch(Error error) {
8             System.out.println("Akuma Secret Super Art");
9         } finally {
10            System.out.println("Shinkuu Hadouken");
11        }
12
13        System.out.println("Isshun Sengiku");
14    }
15
16 }
```

- A. Shinkuu Hadouken
Isshun Sengiku
- B. java.lang.NullPointerException will be thrown
Shinkuu Hadouken
- C. java.lang.NullPointerException will be thrown
Akuma Secret Super Art
Shinkuu Hadouken
- D. java.lang.NullPointerException will be thrown
Akuma Secret Super Art
Shinkuu Hadouken
Isshun Sengiku



Important: Play around with the Java Challenges code, make code changes, see what happens. More importantly, try out the Java Challenge and only then see the answer.

Explanation:

Let's analyze the try block first. Note that we are trying to access a method of the null ryu variable, which causes the well-known `NullPointerException` to be thrown:


```
1 try {
2     String ryu = null;
3     ryu.contains("Shoryuken");
4 }
```

In the next code block we have the catch statement, but remember that `NullPointerException` is not a subclass of `Error`. A `NullPointerException` is a `RuntimeException`, not an `Error`, so this catch block won't be executed:

```
1 catch(Error error) {
2     System.out.println("Akuma Secret Super Art");
3 }
```

What about the finally block? As you've learned, the finally block will almost always be executed, unless the `System.exit` method is invoked before it is reached. That's not the case here, so even though a `NullPointerException` is thrown in the try block, the finally block will be executed normally:

```
1 finally {
2     System.out.println("Shinkuu Hadouken");
3 }
```

In conclusion, the correct alternative is:

- B. `java.lang.NullPointerException` will be thrown
Shinkuu Hadouken

8.6 try with resources

The `try-with-resources` statement was introduced in Java 7 to reduce the amount of boilerplate code needed to close resources. Before Java 7, we had to use the finally block for this purpose. Let's take a look at a common example from the real world:

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.SQLException;
4
5  public class TryWithFinally {
6
7      public static void main(String... tryWithFinally) throws SQLException {
8          Connection connection = null;
9          try {
10             connection = DriverManager.getConnection("dbURL");
11         } catch (SQLException e) {
12             e.printStackTrace();
13         } finally {
14             connection.close();
15         }
16     }
17
18 }
```

Note that this code is fairly verbose. In addition to the finally block being required to close the connection, we had to declare the connection variable outside the try block for it to be accessible in the finally block. A less verbose option is to use the try-with-resources statement to solve this problem. Here's the same example, making use of this feature:

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.SQLException;
4
5  public class TryWithResources {
6
7      public static void main(String... tryWithResources) {
8          try (Connection connection = DriverManager.getConnection("dbURL")) {
9              // Do the database logic
10         } catch (SQLException e) {
11             e.printStackTrace();
12         }
13     }
14 }
```

In this code we are ensuring that the database resource will be closed in the same way, but this version is much more compact—we can declare the resource in the try statement, and it will be closed automatically when we exit that block. This works with any resource that implements the `AutoCloseable` interface, as you'll see shortly.

8.6.1 Changes in Java 9

A small but important change to the try-with-resources statement was introduced in Java 9. Now we can pass in a resource from outside the try block; we're not required to declare and instantiate the variable inside the try block. Here's an example:

```
1 public class TryWithResourcesJava9 {
2
3     void executeAction(AutoCloseable closeable) {
4         try (closeable) {
5             // Perform your logic
6         } catch (Exception e) {
7             e.printStackTrace();
8         }
9     }
10
11 }
```

As you can see, it's easy to extend this method because we're passing an interface, and we can simply transfer the responsibility for instantiating the class to the code that is invoking this method.

8.6.2 Classes that can be used in the try-with-resources statement

The classes or interfaces that can be used in a try-with-resources statement are the ones that extend the Closeable or AutoCloseable interface. The Connection interface, for example, extends AutoCloseable interfaces:

```
public interface Connection extends Wrapper, AutoCloseable { ... }
```

The OutputStream class is another example; it implements Closeable:

```
public abstract class OutputStream implements Closeable, Flushable { ... }
```

Notice that Closeable extends AutoCloseable:

```
1 public interface Closeable extends AutoCloseable {
2     public void close() throws IOException;
3 }
```

Also note that there is a close method in both interfaces that must be implemented in the class that will have its resource closed.

You can also create your own AutoCloseable class. Let's look at an example:

```
1 public class TheClose {
2
3     public static void main(String... customAutocloseable) {
4         try (Moe moe = new Moe(); Apu apu = new Apu()) { }
5     }
6
7     static class Moe implements AutoCloseable{
8
9         public void close() {
10             System.out.println("Moe closes the Bar");
11         }
12     }
13
14     static class Apu implements Closeable {
15         public void close() {
16             System.out.println("Apu closes the shop");
17         }
18     }
19 }
```

Here we create one `AutoCloseable` class, `Moe`, and one `Closeable` class, `Apu`. Even though these classes implement different interfaces, the behavior will be the same; we can use both of them inside the `try-with-resources` statement.

One other important point to keep in mind about the `try-with-resources` statement is that the resource declared last will be closed first. It works like the Stack data structure, following the “last in, first out” principle. The objects are created in order from left to the right, and the close method is invoked from right to left.

8.6.3 Moe AutoCloseable Bar Challenge

In this challenge we have two POJO classes: `Barney`, which implements `AutoCloseable`, and `Moe`, which implements `Closeable` (and includes some logic operations). We pass both of those classes as parameters to the `try-with-resources` statement. Take a look at the code, and see if you can guess what happens when this program is executed.

```
1  import java.io.Closeable;
2  import java.io.IOException;
3
4  public class ClosingTheBar {
5      static String whoClosedTheBar = "";
6
7      public static void main(String... marvel) {
8          Moe moe = new Moe();
9          executeAction(moe, new Barney());
10         System.out.println(whoClosedTheBar + moe.moeClosedBarCount);
11     }
12
13     private static void executeAction(Closeable moe, AutoCloseable barney) {
14         try (moe; barney) {
15             new Moe().close();    // Line 16
16         } catch (Throwable ignore) {
17             whoClosedTheBar += "?";
18         }
19     }
20
21     static class Barney implements AutoCloseable {
22         public void close() {
23             throw new StackOverflowError();
24         }
25     }
26
27     static class Moe implements Closeable {
28         int moeClosedBarCount = 0;
29         public void close() throws IOException {
30             moeClosedBarCount++;
31             if (moeClosedBarCount == 2) {
32                 whoClosedTheBar += "moe";
33                 throw new IOException();
34             }
35         }
36     }
37
38 }
```

- A. moe?2
- B. StackOverflowError will be thrown and the program will stop
- C. ?1
- D. RuntimeException at line 16



Important: Try out the Java Challenge code, make your code changes, see what happens. More importantly, try out the Java Challenge and only then see the answer.

Explanation:

Let's examine the `executeAction` method, which is where the action is happening. Notice that we are passing `moe` as a `Closeable` instance and `barney` as `AutoCloseable`. The first close method to be executed will be the one at line 16—that's because the close method is executed after the try block execution.

One important observation is that the `try-with-resources` statement will always close the resources from right to left, so `barney` will close first, then `moe`. This means `barney`'s close method will execute first, throwing a `StackOverflowError`, and then `moe`'s close method will be executed. Note that there is logic in this method:

```
1  if (moeClosedBarCount == 2) {  
2      whoClosedTheBar += "moe";  
3      throw new IOException();  
4  }
```

But note also that we are using different instances to invoke the close method. The value of the instance variable `moeClosedBarCount` will never reach 2, because each time an instance is created it will be reset.

The only value that will be concatenated will be "?", and the value of `moeClosedBarCount` will be 1. In conclusion, the final answer to this challenge will be:

C) ?1

8.7 Multi catch

In Java, a try block can be followed by multiple catch blocks. This feature is very useful when we want to have better control over error logs, to make it easier for developers to find out what has gone wrong in the application. It's typically used when working with critical features with complex and unpredictable logic.

In the following example, we are getting a `Connection` from the database, then executing some complex logic. A `RuntimeException` could occur in the `executeComplexLogic` method, in which case the `Exception` catch block will catch it and we'll need to log this information. Likewise, if an `Error` occurs the `Throwable` catch block will capture it and log what happened:

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.SQLException;
4
5  public class DatabaseConnection {
6
7      public static void main(String... multiCatchExample) {
8          try {
9              Connection connection = DriverManager.getConnection("DBURL");
10             executeComplexLogic();
11         } catch (SQLException e) {
12             e.printStackTrace();
13         } catch (Exception e) {
14             e.printStackTrace();
15         } catch (Throwable e) {
16             e.printStackTrace();
17         }
18     }
19
20     private static void executeComplexLogic() { }
21 }
```

Note that when we use multiple catch blocks, the classes we catch must have an inheritance relationship and we must start with the most specific class and proceed to the most generic one. For example, in the preceding code, note that `SQLException` extends `Exception` and `Exception` extends `Throwable`.

This code won't compile, because `Throwable` does not extend `Exception`:

```
1  catch (Throwable e) {
2      e.printStackTrace();
3  } catch (Exception e) {
4      e.printStackTrace();
5  }
```

Also keep in mind that it's not necessary to extend a class directly, as long as the next class caught is at a higher inheritance level. For example, even though `SQLException` doesn't directly extend the `Exception` class, this code will compile:

```

1  catch (SQLException e) {
2      e.printStackTrace();
3  } catch (Exception e) {
4      e.printStackTrace();
5  }

```

There's another way to capture exceptions that was introduced in Java 7. Like the try-with-resources statement, its goal is to reduce the boilerplate code. Instead of creating several catch blocks in the event that you might need to catch many different exceptions, it's possible to create only one catch block that catches more than one type of exception.

In the following example, we will be catching the `ArrayIndexOutOfBoundsException`, `NullPointerException`, and `StackOverflowError` in the same catch block. Note that those exception classes don't have any inheritance relationship, and that's why this code compiles:

```

1  public class MoeFrisk {
2
3      public static void main(String... multicatch) {
4          try {
5              String array[] = null;
6              System.out.println(array[0].contains("shotgun"));
7          } catch (ArrayIndexOutOfBoundsException | NullPointerException | StackOverflowError e) {
8              e.printStackTrace();
9          }
10     }
11 }
12
13 }

```

If we use a class that is a parent class in a multi-catch statement, like in the following example, the code won't compile:

```

1  try {
2      String array[] = null;
3      System.out.println(array[0].contains("shotgun"));
4  } catch (ArrayIndexOutOfBoundsException | Exception | StackOverflowError e) {
5      e.printStackTrace();
6  }

```

In this case we'll get the following compilation error:


```

1 Error:(16, 51) java: Alternatives in a multi-catch statement cannot be related by su\
2 bclassing
3     Alternative java.lang.ArrayIndexOutOfBoundsException is a subclass of alternative \
4 java.lang.Exception

```

A subclass and parent class can't appear together in a multi-catch statement.

8.7.1 Big Bang Theory Exception Chaos Challenge

This challenge explores the concept of catching multiple exception types, both in a multi-catch statement and in separate catch blocks. It will test your understanding of this useful Java feature, and of exception inheritance. Analyze the code carefully and see if you can guess the output.

```

1 public class BigBangTheoryTryCatchFinally {
2     static String s = "";
3     public static void main(String... doYouBest) {
4         try {
5             throw new IllegalArgumentException();
6         } catch (RuntimeException e) {
7             try {
8                 throw new StackOverflowError();
9             } catch (IllegalArgumentException | NullPointerException | IndexOutOfBou\
10 ndsException x) {
11                 s += "Sheldon ";
12             } catch (Error error) {
13                 s += "Penny ";
14             } finally {
15                 s += "Leo ";
16             }
17         } finally {
18             s += "Howard ";
19             try {
20                 throw new VirtualMachineError("Wrong experience formula") { };
21             } catch (Exception | StackOverflowError error) {
22                 s += "Raj ";
23             } catch (Throwable error) {
24                 s += "Melissa ";
25             }
26         }
27         System.out.println(s);
28     }
29 }

```

- A. Sheldon Leo Howard Raj
- B. Sheldon Howard Melissa
- C. Penny Leo Howard Melissa
- D. Sheldon Penny Leo Howard Melissa



Important: Try out the Java Challenge **before** seeing the answer. That's how you will get the best benefit of the Java Challenges.

Explanation:

Let's examine the key points of this challenge. The first exception thrown is an `IllegalArgumentException`:

```
throw new IllegalArgumentException();
```

`IllegalArgumentException` extends `RuntimeException`, so the `RuntimeException` catch block will capture this exception:

```
1 catch (RuntimeException e) {
2     throw new StackOverflowError();
3 }
```

Then a `StackOverflowError` is thrown:

```
throw new StackOverflowError();
```

None of the multi-catch exceptions will be able to capture it:

```
catch (IllegalArgumentException | NullPointerException | IndexOutOfBoundsException x) {
    ... }
```

But the `Error` catch block will capture the `StackOverflowError`, because this class extends `Error`:

```
1 catch (Error error) {
2     s += "Penny ";
3 }
```

Therefore, the first String to be concatenated will be "Penny ".

Then the inner finally block will be executed (recall that this happens even when an exception is thrown), and it will concatenate "Leo ".

The outer finally block will also be executed, and it will concatenate "Howard ".

Then a `VirtualMachineError` will be thrown:

```
throw new VirtualMachineError("Wrong experience formula") { };
```

The first catch block won't be able to capture this exception because there is no inheritance relationship:

```
catch (Exception | StackOverflowError error) { ... }
```

But the second catch block will, because `Throwable` catches all exceptions:

```
1 catch (Throwable error) {  
2     s += "Melissa ";  
3 }
```

Therefore, "Melissa " will be the last String to be concatenated.

In conclusion, the final answer will be:

C. Penny Leo Howard Melissa

8.8 Creating a customized exception

An effective way of helping developers identify exactly what error is occurring in your code is by creating meaningful customized exceptions. Some judgment is required here, though. You shouldn't create a customized exception for every case; instead, you should analyze whether the business requirement is important enough to merit it. If you created a special exception for every case you would have an awful lot of classes to maintain, and that level of code complexity is not necessary.

So when might it be a good idea to create a custom exception? If there is a particular module that is highly important for your system, you might want to create an exception that is focused on it. This will make it easy to recognize problems that occur in this critical component.

If we have a module that handles billing information for a company, for example, we could create an exception called `BillingException` that will represent all other exceptions that might be thrown in this module:

```
1 public class BillingException extends Exception {  
2  
3     public BillingException(String message, Throwable cause) {  
4         super(message, cause);  
5     }  
6  
7 }
```

We could also create a customized unchecked exception by extending `RuntimeException`:

```

1 public class CalculationException extends RuntimeException {
2
3     public CalculationException(String message, Throwable cause) {
4         super(message, cause);
5     }
6
7 }

```

Notice that we are creating a constructor in the BillingException and CalculationException classes. That's important, because it enables us to pass a clear message and cause relating to the exception, making it easier to track down and understand the error that occurred. For example:

- BillingException

```

1 try {
2     // do some logic
3 } catch (SomeException someException) {
4     throw new BillingException("The customer can't be billed.", someException);
5 }

```

- CalculationException:

```

1 try {
2     // do some logic
3 } catch (SomeException someException) {
4     throw new CalculationException("The sum operation has failed", someExcepti\
5 on);
6 }

```

We also could encapsulate the message into our exception, if we wanted to. Also we can even overload the CalculationException constructor if we want to pass a different message. For example:

```

1 public class CalculationException extends RuntimeException {
2     static final String message = "Calculation error";
3     public CalculationException(Throwable cause) {
4         super(this.message, cause);
5     }
6
7     public CalculationException(String message, Throwable cause) {
8         super(message, cause);
9     }
10 }

```

When working with exceptions, we should aim to create clear and meaningful log messages that will make sense to developers.

As suggested earlier, we can structure our business exceptions. It's a good idea to create a generic exception class that will be extended by all the other custom exception classes you need. You can then declare just the generic type in the throws clause instead of listing all the specific exceptions the method throws.

To show how this works, let's suppose we have two more business exceptions:

```

1 class NoDiscountException extends Exception { ... }
2 class NoAccountBalanceException extends Exception { ... }

```

And a method that contains some business logic:

```

1 void calculateCustomerBilling() throws BillingException, NoDiscountException, NoAccountBalanceException {
2
3     // In the logic, this method will throw all three exceptions
4 }

```

Now imagine that this scenario is reproduced in various classes throughout the system. Handling all those different exceptions would be a pain, and the code would be complicated to read and difficult to maintain. It would be far better to create a generic exception type that aggregates all those business requirements—but how can this be done?

Fortunately, there is a solution for this problem: we can simply create a BusinessException class and make all the other business exceptions extend it, like this:

```

1 class BusinessException extends Exception { ... }
2 class NoDiscountException extends BusinessException { ... }
3 class NoAccountBalanceException extends BusinessException { ... }
4 class BillingException extends BusinessException { ... }

```

Then we can remove all the specific exceptions from the throws clause in the previous example and throw only the BusinessException. This approach is far more concise and better organized:

```
void calculateCustomerBilling() throws BusinessException { ... }
```

You can then write code to catch the various exceptions that might be thrown, ensuring that you'll have a detailed and clear log of any exceptions that occur in the system.

8.8.1 No Beer Exception Challenge

This challenge explores several of the concepts introduced in this chapter, including try-with-resources, using multiple catch blocks, and creating custom exceptions. You can test your understanding of these useful Java features by analyzing the code and seeing if you can guess the output.

```

1 public class NoBeerExceptionChallenge {
2     static String finalResult = "";
3
4     public static void main(String... doYourBest) {
5         try (NoBeerException exception = new NoBeerException()) {
6             finalResult += "GetBeer";
7             throw new NoAlcoholError();
8         } catch (Exception exception) {
9             finalResult += "NoWhiskey";
10        } catch (Error error) {
11            finalResult += "NoWine";
12        } finally {
13            try {
14                int i = 1 / 0;
15            } catch (NoBarException noBar) { finalResult += "NoBar";
16            } catch (RuntimeException exception) {
17                finalResult += "NoAlcohol";
18            }
19        }
20
21        System.out.println(finalResult);
22    }
23    static class NoBeerException extends Exception implements AutoCloseable {

```

```

24     public void close() {
25         finalResult += "BarClosing";
26         throw new NoBarException();
27     }
28 }
29 static class NoBarException extends RuntimeException {}
30 static class NoAlcoholError extends Error {}
31 }

```

- A. GetBeerBarClosingNoWineNoAlcohol
- B. BarClosingGetBeerNoWhiskeyNoBar
- C. GetBeerNoWineNoAlcohol
- D. GetBeerNoWhiskeyNoAlcohol



Important: Our brain is like a muscle, the more you practice, the stronger you get. Therefore, try out the Java Challenge **before** seeing the answer.

Explanation:

The first part of the code demonstrates use of the try-with-resources statement. We have three custom exception classes: `NoBeerException` that extends `Exception` and implements `AutoCloseable`, `NoBarException` extending `RuntimeException`, and `NoAlcoholError` extending `Error`.

Remember that the try block will be executed first when using the try-with-resources statement. The first String to be concatenated will therefore be:

GetBeer

Then the close method from `NoBeerException` will be invoked and the second String will be concatenated:

BarClosing

Notice that a `NoBarException` is being thrown in the close method, but a `NoAlcoholError` was thrown as well. What happens here? When an exception is thrown and then the close method from an `AutoCloseable`, for example, throws another exception, the first one will be suppressed. In our case, the `NoBarException` will be suppressed by the `NoAlcoholError`.

If we print the `getSuppressed` method referencing the first position of the array, we will get the following result:

```

1  catch (Error error) {
2      System.out.println(error.getSuppressed()[0]);
3  }

```

...NoBeerExceptionChallenge\$NoBarException

The catch block that will be executed will therefore be:

```

1  catch (Error error) {
2      finalResult += "NoWine";
3  }

```

And the third String that will be concatenated will be:

NoWine

The finally block will always be executed, so next the calculation $1 / 0$ will be performed. But when we try to divide a number by zero in Java, we get the following exception:

```
java.lang.ArithmeticException: / by zero
```

We can conclude that this is a RuntimeException because this error could be avoided with an if statement.

NoBarException extends RuntimeException but has nothing to do with the ArithmeticException. Therefore, the RuntimeException catch block will catch this error and the final String to be concatenated will be:

NoAlcohol

In conclusion, the final answer will be:

A - GetBeerBarClosingNoWineNoAlcohol

8.9 Throw early, catch late

When we are working with business layers, the most suitable action we can take with regard to exceptions is to throw them early and catch them as late as possible, only then logging the error message. It's much cleaner to follow this approach, rather than logging the error information in every layer of the call stack.

The following code prints log messages in the Repository and Service system layers. This makes it more difficult to track down the source of any error that occurs, and should be avoided:

```

1  public class SimpsonRepository {
2
3      List<Simpson> findAll() throws SQLException {
4          try {
5              // Perform action to find all Simpsons
6          } catch (SQLException sqlException) {
7              sqlException.printStackTrace();
8              throw sqlException;
9          }
10     }
11

```



```
12 }
13
14 import java.sql.SQLException;
15 import java.util.List;
16
17 public class SimpsonService {
18
19     SimpsonRepository simpsonRepository = new SimpsonRepository();
20
21     public List<Simpson> findAllSimpsons() {
22         try {
23             return simpsonRepository.findAll();
24         } catch (SQLException sqlException) {
25             sqlException.printStackTrace();
26         }
27     }
28
29 }
```

Instead, it's much better to follow the “throw early, catch late” principle. This will centralize the reporting on the errors that happened, making it easier to find out what went wrong and where. Let's see what the same logic looks like when we apply this principle:

```
1 import java.sql.SQLException;
2 import java.util.List;
3
4 public class SimpsonRepository {
5
6     List<Simpson> findAll() throws SQLException {
7         // Perform action to find all Simpsons
8     }
9
10 }
11
12 public class SimpsonService {
13
14     SimpsonRepository simpsonRepository = new SimpsonRepository();
15
16     public List<Simpson> findAllSimpsons() {
17         try {
18             return simpsonRepository.findAll();
19         } catch (SQLException sqlException) {
20             sqlException.printStackTrace();
21         }
22     }
23 }
```

```
21         }  
22     }  
23  
24 }
```

This code is less verbose and easier to understand. We should try to centralize our exception-handling code as much as possible to facilitate debugging. .

8.10 Real World Exception Creation Challenge

Imagine you're a developer at a financial company and you are given the task of elaborating an exception structure that will enable developers to track down errors more easily and with more clarity.

Your goal is to create a generic checked business exception and some more specific custom exceptions that extend it. Your customized exceptions should show clear error messages indicating the source of the problem when you print the stack trace.

You also need to pass the error code "NO_ACCOUNT_BALANCE" to the frontend application so it can show a clear message to the user.

The generic `BusinessException` class will have a constructor with an error message, `Throwable` cause, and error code (it can be an enum if you prefer). You'll also create two specialized exception classes, `NoAccountBalanceException` and `CentralBankServiceDownException`, both extending `BusinessException`.

Finally, you'll need to create a simple service class named `AccountService` that implements a method named `withdrawAccountMoney` receiving a `customerId` and `moneyAmount`. You don't need to implement the logic in the `withdrawAccountMoney` method; you just need to throw the exceptions you created, catch them as `BusinessExceptions`, and then print the stack trace. That will be enough for this challenger. Over to you!

8.11 Summary

- Exceptions are a crucial concept to master in Java. In this chapter you learned how to use the most suitable exception types for multiple situations, and how to structure your exception-handling code in try, catch, and finally blocks.
- You learned the difference between checked and unchecked exceptions, and that all exception types in Java inherit from `Throwable`.
- You know how to handle and declare exceptions, and how to wrap specific exceptions inside generic ones.
- You learned that it's possible to close Java resources by using the try-with-resources statement instead of using a finally block.

- You understand the options Java provides for catching multiple types of exceptions, and the effects of inheritance on exception handling.
- You learned how to create meaningful customized exceptions with clear log messages when appropriate.
- You also learned how to apply good programming practices when working with exceptions, such as the “throw early, catch late” principle.

9 Lambdas and Functional Interfaces

This chapter covers:

- Understanding why lambdas are important
- Using one line concise functions with lambdas
- Using one line coincide method references
- Understanding when to use lambdas or method references
- Main principles of functional programming

Lambdas, Functional interfaces, and method references are functional programming that helps us to avoid the boiler-plate code. When there wasn't the possibility to use lambdas, we would have to use anonymous inner classes for example. As you might remember in the previous chapters, the syntax from an anonymous inner class is not so friendly.

With lambdas and method references, it's also possible to make better use of Java APIs, such as Stream, Collections, Optional, Files, and so on. Usually, when we use lambdas, we use a one-line function. When we have to use more lines of code, then it's suitable to use method references.

When starting working with lambdas and method references, it will be a bit difficult to fully absorb the concept. However, as we use it, the more familiar we get, and for sure the Java Challenges are a massive help to help you to accomplish that.

9.1 What is Lambda?

We can easily pass a one-line function to a functional interface with lambdas. Lambdas are also part of the functional programming paradigm. In Java, lambdas don't completely follow the functional programming paradigm but still use it optimally.

In simple words, functional programming in its essence is a programming style that shouldn't have any data mutation or side effects in programs. This means when we invoke a function with the same value, the same behavior is expected.

As mentioned above, anonymous classes are verbose. Let's see how we were using that before with the Runnable interface:

```

1 Runnable runnable = new Runnable() {
2     @Override
3     public void run() {
4         System.out.println("Running!!!");
5     }
6 };
7
8 runnable.run();

```

The output will be:

Running!!!

We can convert the above anonymous class code to lambdas in just one line! This is the power of lambdas, it makes the code more clear and easier to maintain:

```

1 Runnable runnable = () -> System.out.println("Running!!!");
2 runnable.run();

```

The output will be the same:

Running!!!

Now that you know why lambdas are powerful, we can go deeper into them and understand how to use lambdas effectively.

9.1.1 Lambdas Shouldn't Change External State

Lambdas in essence are not meant to change outside variables, however, in some cases this is possible. We can change instance or static variables as you can see in the following example:

```

1 import java.util.function.Consumer;
2
3 public class LambdaMutation {
4     private int instanceNumber;
5     private static int staticNumber;
6
7     public static void main(String[] args) {
8         Consumer<Integer> consumer = c -> System.out.println(++new
9         LambdaMutation().instanceNumber + ":" + ++staticNumber);
10        consumer.accept(1);
11    }
12
13 }

```

When running the above code, the output will be:

2:2

As you could see in the code above, we can change instance or static variables in a lambda. As Java is not a purely functional programming language, this is acceptable.

9.1.2 Effective Final Variables

We can change instance and static variables in a Lambda, however, we can't change a local variable. A lambda can only use an effective final local variable.

Effective final variables are variables that are initialized with a value and this value is never changed even though this variable is not final. Let's see an example of an effectively final variable:

```
1 int effectiveFinalVariable = 5;  
2 System.out.println(effectiveFinalVariable);
```

We never changed the code above, therefore, `effectiveFinalVariable` is an effectively final variable.

Let's see how is a non effective final variable now:

```
1 int nonEffectiveFinalVariable = 5;  
2 System.out.println(++nonEffectiveFinalVariable);
```

As you can see we changed the `nonEffectiveFinalVariable`, therefore it's not an effective variable.

Of course, if we pass a local final variable to a lambda expression, it's totally fine since we can't change it. Ideally, we should prefer to use only explicitly final variables in a lambda as a good practice.

Now that you have a good understanding of how local variables work with lambdas, try out the next Java Challenge!

9.1.3 Mysterious Door Lambda Challenge

By running the following code, can you guess what will happen?

```
1  import java.util.Arrays;
2  import java.util.List;
3
4  public class MysteriousDoorLambdaChallenge {
5
6      public static void main(String... theDoors) {
7          int doorNumber = 0;
8          doorNumber++;
9          List<String> doors = Arrays.asList("A", "B", "C");
10         doors.forEach(e -> {
11             System.out.println(e + doorNumber);    // # Line 11
12         });
13     }
14
15 }
```

- A. A0
B1
C2
- B. A1
B2
C3
- C. Compilation Error at line 11
- D. Unpredictable



Important: Try out the Java Challenge **before** seeing the answer. The more Java Challenges you solve, the better you get.

Explanation:

Lambdas can only access variables that are final or effectively final. As mentioned above in its essence should access only immutable data from outside.

Note that the `doorNumber` variable is being changed when we increment it. Therefore, the `doorNumber` variable is not effectively final anymore from that moment. Simply put, effectively final variables are variables that when given a value, this value will be never changed.

In conclusion, in this challenge, there will be a compilation error at line 11 and the final answer will be:

C) Compilation Error at line 11

9.2 Functional Interfaces

There are a couple of predefined functional interfaces in Java that will cover 95% of the cases. In simple words, a functional interface is an interface with only one abstract method. When you have that, then you can use the functional interface functionalities.

Let's get started with the easiest functional interface, the Supplier. As the name suggests, the Supplier interface is responsible to return one value. It supplies one value.

```
Supplier<String> supplier = () -> "Moe";
```

The only method we can invoke is get. Therefore, to get the "Moe" value we can do the following:

```
System.out.println(supplier.get());
```

The output will be:

Moe

To make it clear for you what is a functional interface, you can take a look at how the JDK code defines it:

```
1 @FunctionalInterface
2 public interface Supplier<T> {
3     T get();
4 }
```

If we want to use a Supplier that is specific for the int type, we can use the interface IntSupplier. Note that we don't have to use the generic type here:

```
1 IntSupplier supplier = () -> 777;
2 System.out.println(supplier.getAsInt());
```

The output will be:

777

Then we have the Consumer functional interface. In simple words, the purpose of the Consumer interface is to consume any value we pass.

```
1 Consumer<Integer> consumer = (number) -> System.out.println(number);
2 consumer.accept(777);
```

The output will be:

777

9.2.1 Supplier Consumer Challenge

In the following challenge, we are using IntSupplier and Consumer, also we are increasing some numbers there. Can you guess what will happen when running the following code?


```
1 public class SupplierConsumerChallenge {
2
3     static int value;
4
5     public static void main(String... doYourBest) {
6         IntSupplier valueS = () -> value++;           // Line 5
7         Consumer<Object> oneMoreValue = (Object test) -> value++; // Line 6
8
9         oneMoreValue.accept(2);
10        System.out.println(value + " " + valueS.getAsInt());
11    }
12
13 }
```

Explanation:

- A. Line 5 and line 6 won't compile
- B. 45
- C. 12
- D. 11



Important: Try out the Java Challenge **before** seeing the answer. Run the Java code in your mind and get sharp with Java!

Explanation:

Although a lambda is not supposed to mutate any external state, we still can change a static variable. Therefore, there won't be any compilation error. Lambdas are also lazy, which means that the value won't be incremented unless we invoke the lambda methods `accept` or `getAsInt` in this case.

Then the only moment that the value variable will be incremented is when we invoke the following method. Note that even though we are passing '2' to the `accept` method, we are not doing anything with this number. Therefore after invoking the following method, the value variable will be 1:

```
oneMoreValue.accept(2);
```

Finally, when we print the variable value, we will print 1. Then when we invoke the `valueS.getAsInt()` method we will print 1 as well since the value variable is incremented with the post-increment operator. The value variable will be incremented only after we invoke the `valueS.getAsInt()`. After the `valueS.getAsInt()` invocation, the value variable will have the value of 2:

```
System.out.println(value + " " + valueS.getAsInt());
```

Therefore the final output and the right alternative will be then:

- D. 11

9.2.2 Predicate Functional interface

If we want to use a function that returns a boolean and receives one parameter, the Predicate interface is the one!

Let's see an example of how Predicate can be used:

```
Predicate<String> isYodaJedi = (jedi) -> jedi.equals("Yoda");
```

Since we have our function ready to be invoked, now we can print what happens when we check if the String is equals to "Yoda" with the test method:

```
System.out.println(isYodaJedi.test("Yoda"));
```

Output:

```
true
```

Using and with Predicate

Let's now see the and method. This method joins two conditions and both of them have to be true to be fulfilled.

```
1 Predicate<String> isYodaJedi = (jedi) -> jedi.equals("Yoda");    // #A
2 Predicate<String> isObiWanJedi = (jedi) -> jedi.equals("ObiWan");
3
4 Predicate<String> isJedi = isYodaJedi.and(isObiWanJedi);         // #B
5
6 System.out.println(isJedi.test("Yoda"));                         // #C
```

Output:

```
false
```

Explanation:

- #A: We prepare our Predicate conditions.
- #B: We use join both Predicate conditions with and.
- #C: We use the isJedi Predicate that will check the condition from Yoda and ObiWan. Since we are passing only Yoda to the test method, both conditions won't be fulfilled. Therefore, the output will be false.

Using or with Predicate

There is also the 'or' method that operates just like the || operator in a condition:

```

1 Predicate<String> isYodaJedi = (jedi) -> jedi.equals("Yoda");
2 Predicate<String> isObiWanJedi = (jedi) -> jedi.equals("ObiWan");
3
4 Predicate<String> isJedi = isYodaJedi.or(isObiWanJedi);           // #A
5
6 System.out.println(isJedi.test("Yoda"));                         // #B

```

Output:

true

Explanation:

- #A: Notice that here we are using the or operator.
- #B: Then as we send Yoda to be tested, and since one of the conditions is true, the final result will be also true.

9.2.2 The Function Interface

The purpose of the Function interface is to receive one parameter with the generic type we defined and then return a value according to the generic type we defined.

```

1 Function<String, Integer> transformIntoNumber = (number) -> Integer.parseInt(number);
2 System.out.println(transformIntoNumber.apply("7"));

```

The output will be:

7

We can also compose functions. Let's suppose we want to first multiply a number by 2 and then add 2. We have two ways to do that in Java. We can either use the compose or the andThen function. Let's see the example below:

```

1 Function<Integer, Integer> times2 = (number) -> number * 2;
2 Function<Integer, Integer> add2 = (number) -> number + 2;
3
4 System.out.println(times2.compose(add2).apply(2));
5 System.out.println(times2.andThen(add2).apply(2));

```

The output will be:

8

6

Note that the difference between the compose and andThen functions are the order they are executed. In the compose, the add2 function will be executed and we will get the result of 4 first. Then the times2 function, therefore we have the result of 8.

Then when we invoke the `andThen` method, it's the opposite. The `times2` function will be executed giving us the result of 4 first. Then the `add2` function will be invoked adding 2 and giving us the final result of 6.

9.2.3 BiFunction Interface

The `BiFunction` is very similar to the `Function` interface. As the name suggests, the `BiFunction` receives two parameters instead of one from the `Function`. Let's see how that works in practice:

```
1 BiFunction<Integer, Double, Double> addNumbers = (n1,n2) -> { return n1 + n2; };
2 System.out.println(addNumbers.apply(2, 2.5));
```

The output will be:

4.5

It's also possible to compose a `BiFunction` with a `Function` as the following example demonstrates:

```
1 BiFunction<Integer, Double, Double> addNumbers = (n1,n2) -> n1 + n2;
2 Function<Double, Number> times2 = (number) -> number * 2.0;
3 System.out.println(addNumbers.andThen(times2).apply(2, 2.5));
```

The output will be:

9.0

The order where the functions are executed in the above code example is the following: the `addNumbers` `BiFunction` will run first and then the `times2` `Function` will get the result from the `addNumbers` `BiFunction` and multiply it by 2.0.

Considering the above explanation, what do you think will happen in the following code?

```
1 BiFunction<Integer, Double, Double> addNumbers = (n1,n2) -> n1 + n2;
2 Function<Integer, Number> times2 = (number) -> number * 2.0;
3 System.out.println(addNumbers.andThen(times2).apply(2, 2.5));
```

When we try to compose a `BiFunction` that returns a `Double` as the result with a `Function` that receives an `Integer` as a parameter, the code won't compile. That happens because a `Double` type can't be transformed into `Integer`. Therefore, the `times2` function can't be composed with the `addNumbers` `BiFunction`.

This would be different if the `times2` `Function` was receiving a `Double` as a parameter as the following code:

```
Function<Double, Number> times2 = (number) -> number * 2.0;
```

9.2.4 The BiConsumer Functional Interface

The `BiConsumer` interface will be able to consume two parameters and will return void. Let's see the code example:

```

1 BiConsumer<Integer, Integer> biConsumer = (a, b) -> System.out.println(a + b);
2 biConsumer.accept(2, 2);

```

The output will be:

4

Similar to the other functional interfaces we can also compose functions with BiConsumer.

```

1 BiConsumer<Integer, Integer> add = (a, b) -> System.out.println(a + b);
2 BiConsumer<Integer, Integer> subtract = (a, b) -> System.out.println(a - b);
3
4 add.andThen(subtract).accept(2, 2);

```

The output will be:

```

1 0```
2
3 ### 9.2.5 BiFunction Consumer Challenge
4
5 Now that you learned more about the `Function`, `BiFunction`, and `BiConsumer` inter-
6 faces, it's time to try out the Java Challenge! The following Java Challenge involve\
7 s the operations add, subtraction, and multiplication executed with the above-mentio\
8 ned interfaces!
9
10 Can you guess what will be the output of the following Java Challenge?
11
12 ```java
13 public class BiFunctionConsumerChallenge {
14
15     public static void main(String... doYourBest) {
16         BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
17         Function<Integer, Integer> sub = (a) -> a - 2;
18         Function<Integer, Integer> mult = (a) -> a * 3;
19
20         BiFunction<Integer, Integer, Integer> func =
21             add.andThen(sub).andThen(sub);
22
23         BiConsumer<Integer, Integer> consumer = (a, b)
24             -> System.out.println(a + 2 + b + 2);
25
26         int firstResult = func.apply(2, 2);
27         add.andThen(sub).andThen(mult);
28         int secondResult = add.apply(2, 2);

```

```
29
30     consumer.accept(firstResult, secondResult);
31 }
32
33 }
```

- A. 6
- B. 8
- C. 12
- D. 4



Important: Try out the Java Challenge **before** seeing the answer. Stay consistent, stay solving the Java Challenges, and improving your Java skills.

Explanation:

Let's analyze the part of the code where the action happens. Remember that the functions will only be invoked when we use the `apply` method.

The `firstResult` variable will have the value of 0 at this moment because we invoked the subtraction functions:

```
int firstResult = func.apply(2, 2);
```

The tricky part of this challenge is the following code because note that we are assigning or doing anything with those functions. Therefore, nothing will happen when the following code is executed:

```
add.andThen(sub).andThen(mult);
```

Then when we invoke the following code, we will add 2 to 2 which will give us the result of 4:

```
int secondResult = add.apply(2, 2);
```

Finally, the last piece of code is the consumer, which will receive the `firstResult` and `secondResult` variables and will sum them. In the consumer, we are also adding 2 to each of the variables, giving us the result of 8.

```
consumer.accept(firstResult, secondResult)
```

In conclusion, the correct answer for this Java Challenge is:

- B. 8

9.2.6 The UnaryOperator Functional Interface

The `UnaryOperator` functional interface is very similar to the `Function` interface. The main difference is that the `UnaryOperator` will have to receive a value type and return this same value type. Let's see the code example:

```
UnaryOperator<Integer> unaryOperator = (n) -> n + 2;
```

As you can see in the code above, we can only set one generic type, which in this case is `Integer`. Therefore, we can only receive an `Integer` parameter and only return an `Integer` value.

Now when we invoke the above function:

```
System.out.println(unaryOperator.apply(2));
```

The output will be:

4

9.2.7 The BinaryOperator Functional Interface

The `BinaryOperator` interface is similar to `BiFunction`. The main difference is that `BinaryOperator` takes the same type as parameters and as a return. In the `BiFunction` we can pass different types for parameters and the return value if we wish to.

Notice that it's possible to pass only one generic type to the `BinaryOperator` interface since it works with only one type:

```
1 BinaryOperator<Integer> addNumbers = (x,y) -> { return x + y; };
2 System.out.println(addNumbers.apply(2, 2));
```

The output will be:

4

9.2.8 Custom Functional Interface

There is also a way to create customized functional interfaces. First, we create the functional interface with the `@FunctionalInterface` annotation to make sure this interface has only ONE public abstract method:

```
1 @FunctionalInterface
2 interface Duke {
3     void dance(String song);
4 }
```

Then we invoke the dance method from `Duke`:

```
1 Duke duke = d -> System.out.println("Duke is dancing: " + d);
2 duke.dance("tango");
```

Note that it's also possible to add a private, default, or static method in a functional interface. That's because the compiler will assume we just have to override the only public abstract method. The other methods are there to support the interface. Take a look at the following code:

```
1 @FunctionalInterface
2 interface Duke {
3     void dance(String song);
4
5     private void jump() {
6         System.out.println("Duke is jumping!");
7     }
8
9     default void fly() {
10        System.out.println("Duke is flying!");
11    }
12
13    static void playGame() {
14        System.out.println("Duke is playing Metal Gear!");
15    }
16
17 }
```

Remember that in a functional interface, the limitation we have is that we can't have more than ONE public abstract method. Therefore the following code won't compile:

```
1 @FunctionalInterface
2 interface Duke {
3     void dance(String song);
4     void fly();
5 }
```

Note also that what makes the above code not compile is the `@FunctionalInterface`. It's a good practice to use the `@FunctionalInterface` annotation so we can make it very clear that the interface is functional.

Other important points are that public default and public static were only available in Java 8. Also, private and private static were first introduced in Java 9.

9.2.9 JDK Functional Interfaces

We also have some functional interfaces even before Java 8. The `Runnable` and `Comparator` interfaces are great examples:


```

1  @FunctionalInterface
2  public interface Runnable {
3      public abstract void run();
4  }

```

Let's see now how to use `Runnable` as a functional interface:

```

1  Runnable runnable = () -> System.out.println("Running!");
2  runnable.run();

```

The output will be:

Running!

We can also use a method reference:

```

1  public class RunnableMethodReference {
2      void runWithMethodReference() {
3          System.out.println("Method Reference Run!");
4      }
5
6      public static void main(String... runnableMethodReference) {
7          Runnable runnable = new MethodReferenceChallenge().runnableMethodReference;
8          runnable.run();
9      }
10 }

```

The output of the program above will be:

Method Reference Run!

```

1  @FunctionalInterface
2  public interface Comparator<T> {
3      int compare(T o1, T o2);
4      // Other methods
5  }

```

To use the `Comparator` interface as a lambda:

```

1  Comparator<Integer> comparator = (s1, s2) -> s1.compareTo(s2);
2  System.out.println(comparator.compare(2, 1));

```

The output will be:

1

We can do even better, we can use a method reference making the code cleaner and more concise. The lambda above and the following method reference produce the same result:

```
1 Comparator<Integer> comparator = Integer::compareTo;  
2 System.out.println(comparator.compare(2, 1));
```

The output will be:

```
1
```

9.3 Method Reference

When we master the use of method references, we can make the code a lot cleaner and readable. The idea of method reference is a syntax sugar that reduces boiler-plate code.

There are four kinds of method references:

- Reference to a static method
- Reference to an instance method
- Reference to an instance method
- Reference to a constructor

9.3.1 Reference to a static method

Let's get started with the static method reference. We can explore the famous `String.valueOf` static method. Notice that the `valueOf` method fits into a `Function` because a `Function` receives one parameter and returns one value. That's exactly the contract of the `valueOf` Method:

```
public static String valueOf(Object obj)
```

Therefore, let's assign our method reference to a `Function`:

```
Function<Object, String> valueOfFunction = String::valueOf;  
System.out.println(valueOfFunction.apply(7));
```

The output will be:

```
"7"
```

We could use a lambda expression in the case above too:

```
1 Function<Object, String> valueOfFunction = o -> String.valueOf(o);  
2 System.out.println(valueOfFunction.apply(7));
```

The output will be the same:

```
"7"
```

9.3.2 Reference to an instance method

There are some important differences between the static and instance method references. In this section, we will deconstruct how that works.

The Predicate functional interface needs to get a parameter and return a boolean value. As you can see this is the test method from Predicate:

```
boolean test(T t);
```

Now, we will use the isEmpty String instance method to match this predicate interface. However, note that the isEmpty method doesn't receive any parameter, but how is it possible to match the Predicate interface then?

```
public boolean isEmpty() { ... }
```

Remember one important detail, an instance method can only be invoked with an instance. Therefore, we have to pass an instance of a String to the Predicate method reference anyway.

Let's see the following example:

```
1 Predicate<String> isEmpty = String::isEmpty;  
2 System.out.println(isEmpty.test("Challengers"));
```

The output will be:

```
false
```

To make this concept even more clear, let's see how instance method reference works with a BiFunction. It's the same fundament, we have to match an instance, how many parameters are received, and the return value.

This is the compareTo method from the JDK, and notice this method only receives one parameter and return an int value:

```
public int compareTo(Integer anotherInteger) { ... }
```

Let's see how we can match this method to a BiFunction:

```
1 BiFunction<String, String, Object> compareToBiFunction = Integer::compareTo  
2 System.out.println(compareToBiFunction.apply(7, 1));
```

Since 7 is greater than 1 the output from the code above will be:

```
1
```

Note in the code above that we are matching the compareTo method that receives only one parameter. Therefore, we have to pass this Integer instance when we invoke the apply method.

Also, notice that the number 7 is the Integer instance that will invoke the compareTo method. The key to understanding this concept is that every time we are using instance method reference, we have to pass the instance as the first argument:

```
compareToBiFunction.apply(7, 1);
```

9.3.3 Reference to a Constructor

We are creating a class with a constructor that receives two parameters:

```
1 class Simpson {  
2     String name;  
3     int age;  
4  
5     public Simpson(String name, int age) {  
6         this.name = name;  
7         this.age = age;  
8     }  
9 }
```

Then, we are using the method reference to a constructor matching a BiFunction. This works because in the BiFunction we can receive also two parameters and return a type that in our case is Simpson:

```
1 BiFunction<String, Integer, Simpson> biFunctionSimpson = Simpson::new;  
2 Simpson simpson = biFunctionSimpson.apply("Homer", 35);  
3 System.out.println(simpson.name);
```

The output of the code above will be:

Homer

We could also use a lambda in the example above as the following:

```
1 BiFunction<String, Integer, Simpson> biFunctionSimpson = (n, a) -> new Simpson(n, a);  
2 Simpson simpson = biFunctionSimpson.apply("Homer", 35);  
3 System.out.println(simpson.name);
```

```
1 The result would be the same:  
2 Homer
```

9.3.4 Burns Medicine Lambda Challenge

In the following Java Challenge, we will explore how method references behave with constructors. Also, we are creating a customized functional interface to be used with a method reference. Can you guess what will happen when running the following code?

```

1  public class MisterBurnsGetsMedicine {
2      public static void main(String... doYourBest) {
3          MedicineSupplier medicine = Smithers::new;
4          short shortMedicine = 3;
5          medicine.provideMedicine(1);
6          medicine.provideMedicine(shortMedicine);
7          medicine.provideMedicine(Integer.valueOf(1));
8      }
9      static class Smithers {
10         Smithers(int any) {
11             System.out.printf("Smithers gives:%s to Mister Burns with int \n", any);
12         }
13         Smithers(short any) {
14             System.out.printf("Smithers gives:%s to Mister Burns with short \n", a\
15 ny);
16         }
17         Smithers(Integer any) {
18             System.out.printf("Smithers gives:%s to Mister Burns with Integer \n", a\
19 ny);
20         }
21     }
22     @FunctionalInterface
23     private interface MedicineSupplier {
24         Smithers provideMedicine(int quantity);
25         boolean equals(Object obj);           // #1
26     }
27 }

```

- A. Smithers gives:1 to Mister Burns with int
Smithers gives:3 to Mister Burns with short
Smithers gives:1 to Mister Burns with Integer
- B. Smithers gives:1 to Mister Burns with int
Smithers gives:3 to Mister Burns with int
Smithers gives:1 to Mister Burns with int
- C. It won't compile at // #1
- D. Smithers gives:1 to Mister Burns with short
Smithers gives:3 to Mister Burns with short
Smithers gives:1 to Mister Burns with int



Important: Try out the Java Challenge **before** seeing the answer.

Explanation:

Let's analyze the code:

Let's first analyze the equals method in the MedicineSupplier interface. In a functional interface, we have the rule of having only one public abstract method. However, when we declare the equals method, we are only overriding from the Object class. Therefore, we can have those methods declared in the MedicineSupplier interface.

Now let's go to the main part of the code. The following method reference only works because the provideMedicine method matches exactly with one of the constructors from the Smithers class. Let's see it:

```
MedicineSupplier medicine = Smithers::new;
```

As mentioned, in the following code you will notice that both methods receive an int and return Smithers:

```
1  static class Smithers {
2      Smithers(int any) { System.out.printf("Smithers gives:%s to Mister Burns wit\
3  h int \n", any); }
4      // Other constructors...
5  }
6
7  private interface MedicineSupplier {
8      Smithers provideMedicine(int quantity);
9  }
```

As you can see in the code above, method references are intelligent enough to match the right constructor to the provideMedicine method. Since the method reference will match only one constructor, only the following constructor will be invoked:

```
1  Smithers(int any) {
2      System.out.printf("Smithers gives:%s to Mister Burns with int \n", any);
3  }
```

Therefore all invocations from provideMedicine will execute the Smithers(int any) { ... } constructor. The result is the following:

```
B. Smithers gives:1 to Mister Burns with int
   Smithers gives:3 to Mister Burns with int
   Smithers gives:1 to Mister Burns with int
```

9.3.5 Method References VS Lambda

Whenever possible, it's better to use method references to avoid boilerplate code. Lambdas will be better for more specific cases where we need a bit more freedom in the code.

There are also some interesting differences between Lambdas and Method References. A very important difference is that the constructor of a lambda is invoked the number of times the lambda method is invoked.

A method reference constructor is invoked only once when we assign the method reference to a variable.

Let's see a code example about this case:

```
1 class Morpheus {
2     public Morpheus() {
3         System.out.println("Free your mind");
4     }
5
6     public String learnKungFu() {
7         return "Downloaded Kung Fu";
8     }
9 }
10
11 Supplier methodReferenceSupplier = new Morpheus()::learnKungFu;
```

The output will be:

Free your mind

If we use lambda instead:

```
Supplier lambdaSupplier = () -> new Morpheus().learnKungFu();
```

There will be no output. This happens because lambdas are lazy by essence. Therefore, the Morpheus constructor will be only invoked if we execute the following code:

```
lambdaSupplier.get();
```

Then the output will be:

Free your mind

There are also a few caveats about this difference:

- If we invoke the get method with lambdas more times, the constructor will be invoked again
- If we invoke the get method with method reference more times the constructor will not be invoked again
- If we use a constructor method reference, the constructor won't be invoked

Now that we explored the differences between lambdas and method references, it's time for action!

9.3.6 Lambda VS Method Reference Challenge

In the following code, we are using lambda and a method reference. Your goal is to figure out what is the difference when using lambda or method reference. Therefore, what will be the output of the following code?

```

1  public class LambdaVSMethodReferenceChallenge {
2      public static void main(String... doYourBest) {
3          Runnable universeImpactRunnable = () -> new ChuckNorris().roundHouseKick();
4          Runnable galaxyImpactRunnable = new ChuckNorris()::roundHouseKick;
5          System.out.print("The galaxy is finished = ");
6          universeImpactRunnable.run();
7          universeImpactRunnable.run();
8          galaxyImpactRunnable.run();
9          galaxyImpactRunnable.run();
10     }
11     static class ChuckNorris {
12         private static int numberOfKicks;
13         private int galaxyDamage;
14         ChuckNorris() {
15             this.galaxyDamage = numberOfKicks++;
16         }
17         void roundHouseKick() {
18             System.out.print(this.galaxyDamage);
19         }
20     }
21 }

```

- A. The galaxy is finished = 1234
- B. The galaxy is finished = 0123
- C. The galaxy is finished = 0100
- D. The galaxy is finished = 1200



Important: Try out the Java Challenge **before** seeing the answer.

Explanation:

Let's analyze the code:


```

1 Runnable universeImpactRunnable = () -> new ChuckNorris().roundHouseKick();
2 Runnable galaxyImpactRunnable = new ChuckNorris()::roundHouseKick;

```

There is a crucial difference between lambda and method reference. Lambdas are lazy and they will invoke the class constructor only when the method is invoked. On the other hand, with method reference, the constructor will be invoked right away only where the method reference is assigned, not on the method invocation.

So, at this line the constructor is not invoked:

```
Runnable universeImpactRunnable = () -> new ChuckNorris().roundHouseKick();
```

And, at this line the constructor is invoked:

```
Runnable galaxyImpactRunnable = new ChuckNorris()::roundHouseKick;
```

With that in mind, let's analyze the invoked methods:

```

1 universeImpactRunnable.run();
2 universeImpactRunnable.run();

```

With lambdas, every time the run method is invoked, the constructor will be invoked, which means that the numberOfKicks variable will be increased. So the value from galaxyDamage will be:

12

Then when we invoke those methods:

```

1 galaxyImpactRunnable.run();
2 galaxyImpactRunnable.run();

```

The value will be 0 because remember that the constructor will be only invoked one and it was already invoked at the moment of the method reference declaration.

By knowing that, and also remembering that the post ++ operator increases a value after the line being processed. Then the values will be:

00

The final result will be then:

D) The galaxy is finished = 1200

9.4 Lambda Method Reference Matcher Challenge:

Now that we covered most of the lambda and method reference concepts. We can have the last Java Challenge to help you fully absorb those concepts.

This last challenge is all about you matching lambdas and method references to the main functional interfaces.

Let's complete each of the functional interfaces with lambdas and method references then:

1 - Supplier<String> vaderSupplier = // Complete with A or B

A) () -> "DarthVader";

B) () -> System.out.println("Darth Vader");

2 - Supplier<String> methodReferenceSupplier = // Complete with A or B

A. String::toString;

B. String::new;

3 - Predicate<String> jediPredicate = // Complete with A or B

A. (jedi) -> jedi.equals("jedi");

B. (jedi) -> { return new Object(); }

4 - Predicate<String> methodReferencePredicate = // Complete with A or B

A. String::isEmpty;

B. String::contains;

5 - Consumer<String> beerConsumer = // Complete with A or B

A. (saber) -> "Moe uses the " + saber;

B. (saber) -> System.out.println("Moe uses the " + saber);

6 - Consumer<Integer> methodReferenceConsumer = // Complete with A or B

A. System.out::printf;

B. System.out::println;

7 - Function<String, String> jediFunction = // Complete with A or B

A. (jedi) -> jedi.concat(jedi);

B. (jedi, simpson) -> jedi.concat(simpson);

8 - Function<String, String> methodReferenceFunction = // Complete with A or B

A. new String()::valueOf

B. String::toString;

9 - BiFunction<String, String, Object> jediBiFunction = // Complete with A or B

- A. (jedi, sith, naboo) -> jedi + sith + naboo;
- B. (jedi, sith) -> jedi + sith;

10 - BiFunction<Integer, Integer, Object> methodReferenceBiFunction = // Complete with A or B

- A. Integer::compareTo;
- B. new Integer()::equals;

11 - UnaryOperator<String> simpsonUnaryOperator = // Complete with A or B

- A. (homer) -> homer.concat("Marge");
- B. (marge) -> homer.hashCode() + 7;

12 - UnaryOperator<Integer> methodReferenceUnaryOperator = // Complete with A or B

- A. Integer::valueOf;
- B. String::isBlank;

13 - BinaryOperator<String> moeBiOperator = // Complete with A or B

- A. (moe, homer, beer) -> moe + "gives the beer:" + beer + "to:" + homer;
- B. (moe, beer) -> moe + "gives the beer:" + beer;

14 - BinaryOperator<Double> methodReferenceBiOperator = // Complete with A or B

- A. Double::max;
- B. new Double()::max;

9.4.1 Explanation

1 - The answer is A, Supplier<String> vaderSupplier = () -> "DarthVader"; because when we are using a Supplier, we need to return the value we defined in the generic type. In this case, we need to return a String value. Note that we don't need to use the keyword return in a lambda when we have only one line of code.

2 - The answer is B, String::new because we are supplying a String value, which means we are returning a String. Therefore the String constructor fits in this situation. The toString method won't work because we need an instance to invoke the String::toString method. If we used the following new String()::toString(); then it would work.

3 - The answer is A. Since we are using a Predicate that receives a String and returns a boolean, `(jedi) -> jedi.equals("jedi");` will work just fine. The following `(jedi) -> { return new Object(); }` won't work because we are returning an Object instead of a boolean.

4 - The answer is A. That's because we need an instance to be able to invoke the `isEmpty` method. Also, since the Predicate is expecting a String as a parameter, the `isEmpty` method will receive an object instance and then return a boolean value.

The `String::contains` method would only work if it was static or if we had an instance. The following code would work, `"anyString"::contains` because then we just have to pass a parameter and return a boolean just like the `contains` method is expecting.

5 - The answer is B. The reason is that the following code `(saber) -> System.out.println("Moe uses the " + saber);` is only consuming `saber` and not returning any value. Which matches the Consumer contract. However, the following code `(saber) -> "Moe uses the " + saber;` is returning a value, therefore, it doesn't match the Consumer contract.

6 - The answer is B. Because the `println` method only consumes a parameter and returns void. The `printf` method won't work because there are ambiguous overloaded methods within it. Therefore, the method reference can't guess what method to use.

7 - The answer is A. The following code `(jedi) -> jedi.concat(jedi);` matches the contract of a Function because it receives a value and returns a value too. The following code `(jedi, simpson) -> jedi.concat(simpson);` doesn't match the Function contract because it's receiving two parameters.

8 - The answer is B. Even though the `toString` method doesn't receive a parameter, it needs an instance to be invoked. Therefore the following code `String::toString;` will work because we need to pass the String instance and then the `toString` method will return a String. The following code `new String()::valueOf` won't work because we are passing an instance to the String. If we were doing the following, `String::valueOf`, then this would work just like `String::toString`.

9 - The answer is B. Since in a BiFunction we receive two parameters and return one value, the following code is valid, `(jedi, sith) -> jedi + sith`. The following code `(jedi, sith, naboo) -> jedi + sith + naboo;` is not valid because there are three parameters in this lambda.

10 - The answer is A. The `compareTo` method receives only one parameter but needs an instance to be invoked. Therefore, when we invoke the method reference, we need to pass two parameters, which makes the following code `Integer::compareTo;` match the BiFunction. The `equals` method also receives only one parameter but we are already instantiating a String. Therefore, in the following code `new Integer()::equals;` we need to pass only one value when invoking the method reference. To conclude, the `new Integer()::equals;` doesn't match a BiFunction.

11 - The answer is A. This is because `(homer) -> homer.concat("Marge");` receives a String and returns a String, which matches a UnaryOperator. The following code `(marge) -> homer.hashCode() + 7;` is returning an int instead of a String, which breaks the following contract `UnaryOperator<String> simpsonUnaryOperator`.

12 - The answer is A. The following method reference `Integer::valueOf;` receives an Integer and returns an Integer, therefore, it matches the UnaryOperator contract. The other method reference

`String::isBlank`; doesn't match the contract because it receives a `String` and returns a boolean value.

13 - The answer is B. The following lambda `(moe, beer) -> moe + "gives the beer:" + beer`; receives two `String` parameters and returns one `String`, therefore, it matches the `BinaryOperator<String> moeBiOperator` contract. The following code `(moe, homer, beer) -> moe + "gives the beer:" + beer + "to:" + homer`; doesn't match the contract because it's receiving three parameters.

14 - The answer is A. The `max` method receives two parameters, also notice that this is a static method, therefore, there is no need for an instance. For this reason it matches the `BinaryOperator<Double> methodReferenceBiOperator` contract. The following code `new Double()::max`; won't work because we can't use a static method reference with an instance.

9.5 Summary

- You learned why lambdas and method references are important.
- You know when to use lambdas and method references for their appropriate reasons.
- You learned the lambda and method references syntax.
- You know how to use lambdas with the correspondent functional interfaces.
- You know how to match lambdas and method references to functional interfaces.
- You learned what is an effective final variable.
- You know how to create customized functional interfaces.
- You know how to use older Java classes as functional interfaces.
- You solve many Java Challenges to fix and absorb the concepts of this chapter.

Congratulations, you've finished the Lambdas and Functional Interfaces chapter! Now you understand better how to use the benefits of functional programming in Java! Those concepts will massively improve your code quality!

10 Optional

This chapter covers

- Understanding how to use Optional avoiding NullPointerException
- Wrapping values into an Optional
- Handling Exception with Optional
- Manipulating a present and not present value
- Anti-Patterns from Optional
- Filtering data with Optional
- Transforming data with Optional

The Optional class helps Java developers to avoid the annoying NullPointerException when used correctly. It also helps developers to create more readable code with less code. Instead of using null checks, we can create elegant code by using Optional and achieve more.

According to the [Optional class documentation]:([https://docs.oracle.com/en/java/javase/16/docs/api/java.base/java](https://docs.oracle.com/en/java/javase/16/docs/api/java.base/java/util/Optional.html)

Optional is primarily intended for use as a method **return type** where there is a clear need to represent “no result,” and where using null is likely to cause errors.

10.1 Wrapping a value into Optional

To wrap a value into an Optional, we can use the static method Optional.of:

```
1 String snakeEaterOps = "Secret info";
2 System.out.println(Optional.of(snakeEaterOps).isPresent());
```

Since there is value in the snakeEaterOps variable, the isPresent method returns:
true

Note that the Optional.of method can throw a NullPointerException if a null value is passed to it:

```
1 String snakeEaterOps = null;
2 Optional<String> optSnakeEaterOps = Optional.of(snakeEaterOps);
```

The above method will throw:

```

1 Exception in thread "main" java.lang.NullPointerException
2   at java.base/java.util.Objects.requireNonNull(Objects.java:208)

```

Notice that the Exception is throw in the following method `Objects.requireNonNull(Objects.java:208)` from the JDK code. That's because the `Optional.of` method has a null check for null objects:

```

1 public static <T> T requireNonNull(T obj) {
2     if (obj == null)
3         throw new NullPointerException();
4     return obj;
5 }

```

Therefore, whenever we pass a null value to the `Optional.of` method, the `requireNonNull` check will throw a `NullPointerException`.



Note: Read the JDK code often

It's a very good practice to read the JDK code often so you become better with Java. A few minutes a day consistently will lift your Java skills to a whole new level.

10.1.1 Using `Optional.empty`

To avoid passing null to a method we have the option to create an empty `Optional`:

```

1 Optional<String> emptyOpt = Optional.empty();
2 System.out.println(emptyOpt);

```

It prints:

`Optional.empty`

There is also the option of using the `Optional.ofNullable` method which avoids `NullPointerException` in case a null value is wrapped to it:

```

1 String metalGearName = null;
2 Optional<String> emptyOpt = Optional.ofNullable(metalGearName);
3 System.out.println(emptyOpt);

```

The code above will print:

`Optional.empty`

Therefore, whenever you think the value you want to handle might be null use `Optional.ofNullable` instead of `Optional.of`.

10.2 `isPresent` and `isEmpty`

To check if a value is present with an `Optional` we can do the following:

```
1 Optional<String> optional = Optional.of("Challenger");
2 System.out.println(optional.isPresent());
```

It prints:

true

Let's see what happens when we wrap null in an `Optional.ofNullable` now:

```
1 Optional<String> optional = Optional.ofNullable(null);
2 System.out.println(optional.isPresent());
```

When we wrap a null value in an `Optional.ofNullable` method, the optional variable will wrap the value of `Optional.empty`. Therefore the output will be:

false

Since Java 11, we can also use the `isEmpty` method which as the name suggests, it checks if the value is `Optional.empty`:

```
1 Optional<String> optional = Optional.of("Challenger");
2 System.out.println(optional.isEmpty());
```

It prints:

false

Let's see what happens when we wrap null in an `Optional.ofNullable` now:

```
1 Optional<String> optional = Optional.ofNullable(null);
2 System.out.println(optional.isPresent());
```

It prints:

true



Note

You might be wondering why the `isEmpty` method exists since we could just negate the `optional.isPresent()`. However, in terms of code design, it's more readable to have an explicit method name instead of negating the `isPresent` method. By reading the `isEmpty` method you know right away that you are checking if the value is empty.

10.2.1 OfNullable Spartan Warrior Challenge

The following Java Challenge will focus on the concepts we explored such as the creational `Optional` methods, `Optional.of` and `Optional.ofNullable`. Then we will also explore the `isPresent` and `isEmpty` methods.

Therefore, what will happen when running the following code?


```

1  import java.util.Optional;
2
3  public class OfNullableSpartanWarriorChallenge {
4
5      public static void main(String ... doYourBest) {
6          boolean isWarriorPresent = getOfWarrior("Kratos").isPresent();
7
8          Optional<Object> spartanGhost = Optional.empty();
9          Optional<String> ofNullableWarrior = getOptWarrior();
10         Optional<Object> emptyWarrior = Optional.of("");
11
12         System.out.printf("%s %s %s %s", isWarriorPresent,
13             spartanGhost, ofNullableWarrior, emptyWarrior.isEmpty());
14         System.out.println(getOfWarrior(null));
15     }
16
17     static Optional<String> getOfWarrior(String name) {
18         return Optional.of(name);
19     }
20     static Optional<String> getOptWarrior() {
21         return Optional.ofNullable(null);
22     }
23 }

```

A) true

Exception in thread "main" java.lang.NullPointerException will be thrown

B) false null Optional.empty true

Optional.empty

C. true Optional.empty Optional.empty true
Optional.empty

D. true Optional.empty Optional.empty false
Exception in thread "main" java.lang.NullPointerException will be thrown

Explanation:

At the first line of the main method, we will pass "Kratos" to the getOfWarrior method. Then, the Optional.of method will return an Optional with the "Kratos" value. As a result, the isPresent method will return true.

Then, we assign Optional.empty to spartanGhost.

Invoke the getOptWarrior that returns an Optional.empty since we are using the ofNullable method.

Finally, we pass an empty `String` to the `Optional.of` method. Remember that an empty `String` is different than an empty `Optional`. Therefore, the `emptyWarrior` variable will have an empty `String` in it but will **not** be an `Optional.empty`.

Therefore, when printing the values:

- `isWarriorPresent` will be true.
- `spartanGhost` will be `Optional.empty`
- `ofNullableWarrior` will be `Optional.empty`
- `emptyWarrior.isEmpty()` will be false because there is an empty `String` in this `Optional`.

In conclusion, the correct alternative is:

```
1 D) true Optional.empty Optional.empty false
2     Exception in thread "main" java.lang.NullPointerException will be thrown
```

10.3 Optional Antipatterns

Using `Optional` in the wrong way will make the code confusing. Therefore, it's important to understand what are the anti-patterns so that we can avoid them.

10.3.1 Never Use Optional as a Parameter

To pass a parameter that will receive an `Optional` is in essence an antipattern. That's because we are making explicit that our method is expecting to receive a value or a `null` value. When we do that, we are automatically putting more than one responsibility in our method which makes the method not cohesive.

```
1 void processValue(Optional<String> optValue) {
2     if (optValue.isPresent()) {
3         System.out.println("Do some logic with the value present");
4     } else {
5         System.out.println("Do some other logic with the value not present");
6     }
7 }
```

Notice that there are two main responsibilities in the method above. We will have to create a test flow for each case, when there is a value present and when there isn't. This makes the code confusing.

Instead, it's better to remove the `Optional` type from the method parameter and handle this edge-case scenario wherever this `processValue` method is being invoked:

```

1 void processValue(String value) {
2     System.out.println("Do some logic with the value present");
3 }

```

Wherever we invoke the processValue method, we can do the following:

```

1 Optional.ofNullable("value").ifPresentOrElse(this::processValue,
2     () -> System.out.println("Process logic when there is no value..."));

```

Observe in the method above that we are making clear there are two method flows when there is a value and when there isn't. By doing that, we don't hide features in a method, we make it explicit. Therefore, other programmers will be able to understand the code more easily.

10.3.2 Overloading the Method

When there is more than one parameter in a method and one of those parameters might not have a value, it's also a bad idea to use an Optional parameter:

```

1 public void saveSimpson(String parentName, Optional<Child> optChild) {
2     Child child = optChild.orElseGet(Child::new);
3     // Apply some logic...
4     simpsonDao.save(name, child);
5 }

```

Now, let's suppose a developer invokes the above method passing null in the Optional<Child> child parameter:

```

1 saveSimpson("Homer", null);

```

You probably guessed it, a NullPointerException will be thrown! Therefore, it completely defeats the purpose of the Optional class.

To manage this situation better, we can remove the Optional parameter and handle the null value:

```

1 public void saveSimpson(String parentName, Child child) {
2     Child childToPersist = child != null ? child : new Child();
3     // Apply some logic...
4     simpsonDao.save(name, child);
5 }

```

Another way to handle this situation is to overload the saveSimpson method:

```

1  public void saveSimpson(String parentName) {
2      doSaveSimpson(parentName, new Child("No child"));
3  }
4
5  public void saveSimpson(String parentName, Child child) {
6      Child childToPersist = child != null ? child : new Child("No child");
7      doSaveSimpson(name, child);
8  }
9
10 private void doSaveSimpson(String parentName, Child child) {
11     // Apply some logic...
12     simpsonDao.save(name, child);
13 }

```

By following the above approach, we make our API very clear. Therefore, whoever is using the API will immediately notice the method that only receives the `parentName`. It will be also much easier to test both of the methods.

10.3.3 Don't Use Optional as Serializable

An `Optional` object is not `Serializable` because it's not meant to be used as attributes. Instead, an `Optional` is meant to be used as a **return type** where we can manipulate actions when the value is present or absent.

Also, if we use an `Optional` as a `Serializable`, we would have to unbox the value of each attribute in a class which is quite expensive in terms of performance. Another good reason to make `Optional` not `Serializable` is that we can avoid the following:

```
Map<Optional<List<String>>>
```

In real projects, we have data objects everywhere and if we have the attributes of those classes declared as `Optional`, the code would easily become a mess.

For more information, you can check the notes from Brian Goetz at the following link:

(<http://mail.openjdk.java.net/pipermail/lambda-libs-spec-experts/2013-May/001814.html>)[<http://mail.openjdk.java.net/pipermail/lambda-libs-spec-experts/2013-May/001814.html>]

10.3.4 Using `isPresent` and `get`

The `get` method defeats the purpose of the `Optional` class because if the value is not present, a `NullPointerException` will be thrown:

```

1 String javaMascot = null;
2 Optional<String> optJavaMascot = Optional.ofNullable(javaMascot);
3 String result = optJavaMascot.get(); // NullPointerException here

```

As you can see in the code above, when we have a null value in the Optional and we use the get method, we will get a NullPointerException.

To avoid it, we can use the isPresent method to check the value first:

```

1 String javaMascot = null;
2 Optional<String> optJavaMascot = Optional.ofNullable(javaMascot);
3 if (optJavaMascot.isPresent()) {
4     System.out.println(optJavaMascot.get());
5 } else {
6     System.out.println("No value");
7 }

```

Still, the code above is not optimal, it doesn't add the real benefit of an Optional.

Much better would be to use either the ifPresent method because we can do the same as above with less and more readable code:

```

1 String javaMascot = "Duke";
2 Optional<String> optJavaMascot = Optional.ofNullable(javaMascot);
3 optJavaMascot.ifPresent(System.out::println);

```

If you need to handle the case of having an absent value, you can use the Java 9 method ifPresentOrElse:

```

1 String javaMascot = null;
2 Optional<String> optJavaMascot = Optional.ofNullable();
3 optJavaMascot.ifPresentOrElse(System.out::println,
4     () -> System.out.println("No value"));

```

10.3.5 Takeaways

To summarize the most important points of the Optional anti-patterns, let's check the main points:

- Don't use Optional as a parameter
- Don't use an attribute with the Optional type
- Using elements of a Collection as Optional, e.g List<Optional<String>>. It gets difficult to manipulate data and read code.
- Avoid using isPresent and then the get method

10.4 ifPresent and ifPresentOrElse

Before the `Optional` and `ifPresent` methods were introduced in the Java language, we would have to check if the value is `null` and only then execute our logic:

```
1  if (javaMascot != null) {
2      System.out.println(javaMascot.concat(" Rocks"));
3  }
```

To avoid `NullPointerException`, we are doing a `null` check. With `Optional` we can make it easier. We can use a lambda to do the same as above:

```
1  Optional<String> opt = Optional.ofNullable("Duke");
2  opt.ifPresent(javaMascot -> System.out.println(javaMascot.concat(" Rocks")));
```

The output of both code examples is:

Duke Rocks

When we want to program the case where the value is not present with a lambda, we can use the `ifPresentOrElse` method. The method signature of the `ifPresentOrElse` method is the following:

```
1  public void ifPresentOrElse(Consumer<T> action, Runnable emptyAction)
```

As you can see in the method above, we are receiving a `Consumer` which means that we consume the `Optional` value and perform an action. Otherwise, we run a code action in case the `Optional` value is `null`:

```
1  Optional<String> opt = Optional.ofNullable("Duke");
2
3  opt.ifPresentOrElse(
4      (value) -> System.out.println("The Java mascot is: " + value),
5      () -> System.out.println("No Java Mascot")
6  );
```

The output is:

The Java mascot is: Duke

As mentioned above, we are consuming the value of `javaMascotOpt` in the `Consumer` and printing it. In the `Runnable` we print that there is "No Java Mascot".

When the `Optional` value is empty the `orElse` `Runnable` will be executed:

```
1 Optional<String> javaMascotOptEmpty = Optional.ofNullable(null);
2
3 javaMascotOptEmpty.ifPresentOrElse(
4     System.out::println,
5     () -> System.out.println("No Java Mascot")
6 );
```

The output is:

No Java Mascot

10.5 orElse and orElseGet

In simple words, the `orElse` method will return the value of an `Optional` if the value is present, otherwise, it returns the value from the `orElse` method. This is the implementation of the `orElse` method in the JDK code:

```
1 public T orElse(T other) {
2     return value != null ? value : other;
3 }
```

Let's see the case scenario when we have a value in the `Optional`:

```
1 Optional<String> optYoshiFood = Optional.of("Apple");
2 String yoshiFood = optYoshiFood.orElse("No food :(");
3 System.out.println(yoshiFood);
```

The output is:

Apple

If the `Optional` value is absent, the value we put in the `orElse` method will be returned:

```
1 Optional<String> optYoshiFood = Optional.empty();
2 String yoshiFood = optYoshiFood.orElse("No food :(");
3 System.out.println(yoshiFood);
```

The output is:

No food :(

In the case of the `orElseGet` method, instead of returning a value, it returns the `Supplier` functional interface as the JDK code shows:

```

1 public T orElseGet(Supplier<? extends T> supplier) {
2     return value != null ? value : supplier.get();
3 }

```

Therefore, instead of only returning a value as the `orElse` method, we can use some logic with a lambda expression:

```

1 public class OrElseGet {
2
3     static void printYoshiFoodIfPresent() {
4         Optional<String> optYoshiFood = Optional.empty();
5         String yoshiFood = optYoshiFood.orElseGet
6             (OrElseGet::askFoodWhenNoFoodForYoshi);
7         System.out.println(yoshiFood);
8     }
9
10    static String askFoodWhenNoFoodForYoshi() {
11        String noFoodForYoshi = "No food :(";
12        String askMoreForMario = " Mario needs to get more food!";
13        return noFoodForYoshi.concat(askMoreForMario);
14    }
15
16 }

```

The output is:

No food :(Ask Mario for more

As you can see in the code above, by using a lambda expression we can use some logic in the `Supplier` if we need to. But this is not the only difference between the `orElse` and the `orElseGet` methods.



Prefer using Method Reference when a lambda code has more than one line:

It is a good practice to use a method reference in a functional interface when the code has more than one line. If we used an in place lambda, the code would get confusing. We need to use good judgement whenever we want to use more than one line of code in a lambda expression.

10.5.1 `orElseGet` is Lazy and `orElse` is Not

The `orElseGet` method is lazy because we are using the functional interface `Supplier`. Therefore, in the case the value is present in the `Optional` object, the `printJavaMascot` method won't be executed at all:


```
1 import java.util.Optional;
2
3 class JavaMascotOrElseOrElseGet {
4     public static void main(String[] args) {
5         Optional<String> optJavaMascot = Optional.ofNullable("Duke");
6         optJavaMascot.orElseGet(JavaMascotOrElseOrElseGet::printJavaMascot);
7     }
8
9     static String printJavaMascot() {
10         System.out.println("Duke gets prepared...");
11         return "Duke";
12     }
13 }
```

Nothing will be printed in the output of the program above.



Can the laziness of the `orElseGet` method be useful?

If we are dealing with operations that takes a lot of resources, such as database interactions, the `orElseGet` method is a better option than `orElse`.

The `orElse` method will be always executed no matter if the value is present or absent:

```
1 import java.util.Optional;
2
3 class JavaMascotOrElse {
4     public static void main(String[] args) {
5         Optional<String> optJavaMascot = Optional.ofNullable("Duke");
6         optJavaMascot.orElse(printJavaMascot());
7     }
8
9     static String printJavaMascot() {
10         System.out.println("Duke gets prepared...");
11         return "Duke";
12     }
13 }
```

The output is:

Duke gets prepared...

As you can see in the code above, the `printJavaMascot` method will be executed even if the `Optional` has a value. Therefore, the `orElse` method is more suitable for simpler operations that don't consume much resources.

10.5.2 orElse and orElseGet Java Challenge

In the following challenge, we will explore how the `orElse` and `orElseGet` methods behave and your goal is to find out what will be the output of the following program when running the `main` method. Therefore, which of the following alternatives you think it's the right one?

```

1  import java.util.Optional;
2
3  public class VisionFromOracleChallenge {
4
5      static String finalZionValue = null;
6      static int matrixCount = 0;
7
8      public static void main(String... matrix) {
9          Optional<String> optFromMatrix = Optional.ofNullable(finalZionValue);
10         var agentSmith = "Virus";
11
12         finalZionValue = finalZionValue + Optional.ofNullable(agentSmith).orElse(getVisi\
13 onFromOracle());
14         finalZionValue += optFromMatrix.orElseGet(VisionFromOracleChallenge::getVisionFr\
15 omOracle);
16
17         finalZionValue += matrixCount;
18
19         System.out.println(finalZionValue);
20     }
21
22     static String getVisionFromOracle() {
23         matrixCount++;
24         finalZionValue += "KeyMaker";
25         return "Architect";
26     }
27 }

```

- A. nullVirusArchitect1
- B. nullVirusKeyMakerArchitect1
- C. nullVirusArchitect2
- D. nullKeyMakerVirusKeyMakerArchitect2

Explanation:

The first action we take is to wrap the `finalZionValue` variable in the `optFromMatrix`. Then we assign the "Virus" String to the `agentSmith` variable.

Then, we concatenate the `finalZionValue` that has the value of `null` and encapsulate the `agentSmith` variable in an `Optional`. However, remember that the `orElse` method will be invoked anyway even if the `Optional` value is not `null`. Therefore, the `getVisionFromOracle` method will be invoked and will increment the `matrixCount` variable, will concatenate with "KeyMaker" and will return "Architect" to nothing.

Notice that the `finalZionValue` is receiving a new value because we are using `=`, therefore, even though we concatenated "KeyMaker" in the `finalZionValue`, this value will be replaced with the value of `agentSmith`. This occurs because `finalZionValue + Optional.ofNullable(agentSmith)` will be concatenated **first**!

Therefore, value of the `finalZionValue` at this moment is `nullVirus`.

In the second action, we are using the `optFromMatrix` that is actually empty. Remember that `String` in Java is immutable. Therefore, even though we changed the `finalZionValue` it doesn't matter for the wrapper `optFromMatrix`. Since the `optFromMatrix` is empty, the `orElseGet` method will be invoked. It's important to keep in mind that if `optFromMatrix` wasn't empty, the `orElseGet` method would not be invoked.

Then the `matrixCount` variable will be incremented to 2. The `finalZionValue` will be concatenated again, and this time "Architect" will be returned.

Note that the following code:

```
finalZionValue += optFromMatrix.orElseGet(VisionFromOracleChallenge::getVisionFromOracle);
```

Is the same as the following:

```
finalZionValue = finalZionValue + optFromMatrix.orElseGet(VisionFromOracleChallenge::getVisionFromOracle);
```

Therefore, even though the `finalZionValue` is concatenated in the `getVisionFromOracle` method, the `finalZionValue` will be concatenated with the current value of `nullVirus` **first**! Notice that the "KeyMaker" will be lost because of the Java concatenation execution order.

In conclusion, the final answer will be:

C) `nullVirusArchitect2`

10.6 Handling Exceptions with `orElseThrow`

Sometimes we want to simply throw an `Exception` when we receive a `null` value. For that purpose, we have the `orElseThrow` method that will throw an `Exception` in case a value is `null` in the `Optional` class.

Notice that the `orElseThrow` method receives a `Supplier`, therefore we can use a lambda method that returns or supplies a value.

```

1 String noBeerForHomer = null;
2 Object homer = Optional.ofNullable(noBeerForHomer)
3   .orElseThrow(() -> new IllegalArgumentException("D'oh!"));

```

We can also use a constructor method reference for the `IllegalArgumentException` since it matches the contract of the `Supplier` interface. The `IllegalArgumentException` constructor returns a value and this is exactly what a `Supplier` needs:

```

1 Object homer = Optional.ofNullable(noBeerForHomer)
2   .orElseThrow(IllegalArgumentException::new);

```

As you see in the code above the trade-off of using a method reference is that we can't pass a parameter like we did when we used a lambda. However, if you really want to have a specific Exception message you could create a specific Exception as the following:

```

1 class NoBeerException extends RuntimeException {
2     public NoBeerException() {
3         super("D'oh!");
4     }
5 }

```

Let's use the `NoBeerException` now as a method reference:

```

1 Object homer = Optional.ofNullable(noBeerForHomer)
2   .orElseThrow(NoBeerException::new);

```

Since we are passing "D'oh!" to the `RuntimeException` constructor, now we will see our specific message "D'oh!".

10.6.1 Java 10 `orElseThrow()` method

Java 10 has a handy method to throw an `Exception` when you don't mind about a specific `Exception` or message. This method will always throw the following when the value is `null`:

```
throw new NoSuchElementException("No value present");
```

Let's use this method then:

```
1 Object homer = Optional.ofNullable(noBeerForHomer).orElseThrow();
```

The output will be:

```
Exception in thread "main" java.util.NoSuchElementException: No value present
```

10.6.2 OrElseThrow Lol Challenge

The following Java Challenge explores Exception handling with `Optional`. We are making use of the `orElseThrow` method inside the catch clause. Therefore, by running the code below, can you guess what will happen?

```
1  import java.util.Optional;
2
3  public class OrElseThrowLolChallenge {
4
5      public static void main(String... doYourBest) throws Throwable {
6          String finalOpt = "finalLol";
7          Optional<String> opt = null;
8
9          try {
10             opt = Optional.of(null);
11         } catch (RuntimeException exception) {
12             System.out.println(opt.orElseThrow(() -> new Throwable())); // Line 12
13         } catch (Throwable error) {
14             finalOpt = opt.orElse("lol");
15         }
16
17         System.out.println(finalOpt);
18     }
19
20 }
```

- A. lol
- B. null
- C. NullPointerException will be throw at line 12
- D. finalLol

Explanation:

In the try block, we start trying to encapsulate `null` into an `Optional`. When we use the `Optional.of` method, a `NullPointerException` will be thrown.

Considering that `NullPointerException` is a `RuntimeException`, this catch statement will catch the `NullPointerException`.

Then, notice that we are trying to access the `orElseThrow` method from the `opt` variable but the `opt` variable never got initialized. Therefore, another `NullPointerException` will be thrown. However, pay attention that we are don't have another try for this code line. As a result, the `NullPointerException` will be break the program execution at line 12.

In conclusion, the correct alternative is:

C) `NullPointerException` will be throw at line 12

10.6.3 Filtering Optional Data

It's also possible to filter data in an `Optional` and then check the values in it. Instead of creating an `if` to check if the value is there, we can simply use the `filter` method followed by `isPresent` or `isEmpty`:

```
1 String simpsonName = "Homer";
2 Optional<String> optSimpsonName = Optional.of(simpsonName);
3
4 boolean isHomer = optSimpsonName.filter(s -> s == "Homer").isPresent();
5 System.out.println(isHomer);
6
7 boolean isMarge = optSimpsonName.filter(s -> s == "Marge").isEmpty();
8 System.out.println(isMarge);
```

It will print:

```
1 true
2 false
```

We can also do more sophisticated filtering with `Optional`. Let's see the following code example without `Optional`:

```
1 public static boolean isHomerWithoutOptional(Simpson simpson) {
2     boolean isHomer = false;
3     if (simpson != null && simpson.name.equals("Homer")
4         && simpson.age >= 34 && simpson.age <= 39) {
5         isHomer = true;
6     }
7     return isHomer;
8 }
9
10 public static void main(String[] args) {
11     System.out.println(isHomer(new Simpson("Bart", 10)));
12 }
```

The output is:

false

As you can see, the code above is verbose. With the `map` and `filter` methods from `Optional` we can do the same as the code above with less code.

In the following code, we will first encapsulate the value of `simpson` into an `Optional`. Then we will filter the `Optional` to have only "Homer". We map `Simpson` to return the age with the method reference `Simpson::getAge`. Then in the `filter` method, we will be manipulating the age, therefore, we check if the age is between 34 and 39.

Finally, we check if the value is present.

```

1  public static boolean isHomerWithOptional(Simpson simpson) {
2      Optional<Simpson> optSimpson = Optional.ofNullable(simpson);
3
4      return optSimpson.filter(s -> s.getName().equals("Homer"))
5          .map(Simpson::getAge)
6          .filter(age -> age >= 34 && age <= 39)
7          .isPresent();
8  }
9
10 public static void main(String[] args) {
11     System.out.println(isHomerWithOptional(new Simpson("Homer", 34)));
12     System.out.println(isHomerWithOptional(null));
13 }
```

The output is:

```
true
false
```

10.7 Transforming Optional Data

When we want to transform an `Optional` data, we want to use `map` method. Otherwise, we would have to retrieve a value from an `Optional` by using the `get` method and handle the data with a `null` check like the following:

```

1  Optional<String> simpsonName = Optional.ofNullable("Homer");
2  if (simpsonName.get() != null) {
3      System.out.println(simpsonName.get().concat(" Simpson"));
4  } else {
5      System.out.println("No Value");
6  }
```

As you can see in the code above, the code looks verbose. Now, let's see how to concatenate a `String` from an `Optional` with the `map` method:

```
1 Optional<String> simpsonName = Optional.ofNullable("Homer");
2 String result = simpsonName.map(s -> s.concat(" Simpson")).orElse("No value");
3 System.out.println(result);
```

The output is:

Homer Simpson

Notice that the code is cleaner with the `map` method. Also, we are using the `orElse` method which gives us the possibility to return the value or something else when the value is `null`:

```
1 Optional<String> simpsonName = Optional.ofNullable(null);
2 String result = simpsonName.map(s -> s.concat(" Simpson")).orElse("No value");
3 System.out.println(result);
```

The output is:

No value

We can also encapsulate a `List` into an `Optional` and use the `map` method to transform data. Notice in the following code that we are using the `Collection Factory` method from Java 9, which is a handy way to create a `Collection`. Then, we are using the `map` method that will return the size of the `List`. If the `List` is empty, `0` will be returned.

```
1 List<String> simpsonNames = List.of(
2     "Homer", "Marge", "Bart", "Maggie", "Lisa");
3 Optional<List<String>> simpsons = Optional.of(simpsonNames);
4
5 int size = simpsons
6     .map(List::size)
7     .orElse(0);
```

The output is:

5

It's also possible to chain a `map` and a `filter` with an `Optional`. In the following code, we will map "homer simpson" to `toUpperCase` by using the method reference `String::toUpperCase`. Then we will filter the `simpsonName` to only "HOMER SIMPSON". When invoking the `isPresent` method we will have the value there:


```

1 Optional<String> optSimpson = Optional.of("homer simpson");
2
3 boolean isHomerSimpson = optSimpson
4     .map(String::toUpperCase)
5     .filter(simpsonName -> simpsonName.equals("HOMER SIMPSON"))
6     .isPresent();
7
8 System.out.println(isHomerSimpson);

```

The output is:

true

10.7.1 map vs flatMap

What happens when we want to change a value from an `Optional` inside an `Optional`? Consider the following data class:

```

1 public class Yoshi {
2     private Integer eatenApples;
3     public Yoshi(Integer eatenApples) { this.eatenApples = eatenApples; }
4
5     public Optional<String> getEatenApples() {
6         return Optional.ofNullable(eatenApples);
7     }
8 }

```

Let's try to change the value of the `getEatenApples` with the `map` method. Notice that the `map` method is meant to manipulate the **first `Optional` layer**, not the second one:

```

1 public <U> Optional<U> map(Function<? super T, ? extends U> mapper) {
2     Objects.requireNonNull(mapper);
3     if (!isPresent()) {
4         return empty();
5     } else {
6         return Optional.ofNullable(mapper.apply(value));
7     }
8 }

```

As you can see in the code above, the `else` statement is encapsulating the `Optional` value inside an `Optional.ofNullable`. Therefore, if we try to handle a value that is an `Optional`, we will have something weird like `Optional<Optional<Integer>>`. Let's see that happening in the following code:

```

1 Yoshi yoshi = new Yoshi(7);
2 Optional<Optional<Integer>> optEatenApples = Optional.of(yoshi)
3                                     .map(Yoshi::getEatenApples);
4 System.out.println(optEatenApples);

```

The output is:

```
Optional[Optional[7]]
```

Pay attention that the `map` method is not meant to handle an `Optional` inside `Optional` value. Remember that the `getEatenApples` method returns an `Optional` and the `map` method has an `Optional.ofNullable` encapsulating the value. Therefore, we end up having this odd result: `Optional[Optional[7]]`.

To retrieve the value of `getEatenApples` we would have to use this terrible code:

```

1 Integer eatenApples = optEatenApples.orElseThrow().orElse(0);
2 System.out.println(eatenApples);

```

The code above is confusing and messy. That's because the `map` method shouldn't manipulate a value from an `Optional` inside an `Optional`.

10.7.2 Optional inside an Optional With flatMap

Fortunately the `flatMap` method is meant to manipulate data when there is an `Optional` inside another `Optional`.

Take a brief look into the following JDK code and then check the explanation below:

```

1 public <U> Optional<U> flatMap(Function<? super T, ? extends Optional<? extends U>> \
2 mapper) {
3     Objects.requireNonNull(mapper);
4     if (!isPresent()) {
5         return empty();
6     } else {
7         @SuppressWarnings("unchecked")
8         Optional<U> r = (Optional<U>) mapper.apply(value);
9         return Objects.requireNonNull(r);
10    }
11 }

```

Notice the `else` statement of the code above. We are **not encapsulating the value again into an `Optional` as the `map` method**. Instead, we are transforming the data and then returning the value of the `Optional`:

```
Optional<U> r = (Optional<U>) mapper.apply(value);
```

The following code is a good example of the `flatMap` purpose. We have the `Yoshi` class with the `Optional` `getEatenApples` method:

```

1 Yoshi yoshi = new Yoshi(7);
2 Integer eatenApples = Optional.of(yoshi)
3     .flatMap(Yoshi::getEatenApples).orElse(0);    // #A      \
4
5 System.out.println(eatenApples);

```

The output is:

Optional[7]

- #A: Notice we are encapsulating yoshi into an Optional, then there is the getEatenApples that also returns an Optional. Therefore, we have what we need, an Optional inside an Optional. Then we can manipulate the value of getEatenApples directly avoiding NullPointerException in case yoshi is null.

In the code above, we could return the value of the getEatenApples directly as an Optional. Therefore, to get the value of getEatenApples we just need to use a getter method for our Optional:

```
System.out.println(eatenApples.orElse(0));
```

The output is:

7

10.7.3 Soprano Nullable Gun Challenge

This Java Challenge explores data mapping, filtering, and condition handling with Optional. By analyzing the following code can you guess what will happen when the code is executed?

```

1 import java.util.Optional;
2
3 public class SopranoNullableGunChallenge {
4     public static void main(String... args) {
5         Soprano soprano = new Soprano();
6
7         Optional.of(soprano)
8             .map(Soprano::getName)
9             .map(String::toUpperCase)
10            .filter(n -> n.contains("Tony"))
11            .ifPresentOrElse(System.out::println,
12                () -> System.out.println("No value!"));
13
14        String sopranoGun = Optional.of(soprano)
15            .flatMap(Soprano::getGun)
16            .map(String::toLowerCase)

```

```

17         .filter(g -> g.equals("Beretta"))
18         .or(Optional::empty)
19         .orElse("Colt");
20
21     System.out.println(sopranoGun);
22 }
23
24     static class Soprano {
25         String name = "Tony";
26         String gun = "Beretta";
27         public String getName() { return name; }
28         public Optional<String> getGun() { return Optional.of(gun); }
29     }
30
31 }

```

- A. Tony
Beretta
- B. No value!
Colt
- C. No value!
- D. Tony
Colt

Explanation:

The first detail to pay attention is that we have the static inner class `Soprano` that has a getter to a `String name` and a another one to an `Optional<String>` for `gun`.

Then, we encapsulate the `String name` into an `Optional`:

```
Optional.of(soprano)
```

We map the `soprano` object to the `getName` method which means that we will manipulate the `getName` value instead of the `sopranos` value:

```
.map(Soprano::getName)
```

Now we transform the name `"Tony"` to `"TONY"`:

```
.map(String::toUpperCase)
```

Then, we filter the value of name of `"TONY"` to `"Tony"` which considering that Java is case-sensitive, the `contains` method will return `false`. Therefore, the `filter` method returns `Optional.empty`:

```
.filter(n -> n.contains("Tony"))
```

Now, we invoke the `ifPresentOrElse` method that returns `"Tony"` in case the value is present, or `"No value!"` in case the `Optional` is empty. Considering that the previous `filter` method returned `Optional.empty`, `"No value!"` will be printed:

```

1  .ifPresentOrElse(System.out::println,
2  () -> System.out.println("No value!"));

```

In the second `Optional` statement, we also encapsulate the `soprano` object. Then we use the `flatMap` method. Notice that we are getting only the `getGun` `Optional` layer. This is only possible to accomplish with the `flatMap` method:

```
.flatMap(Soprano::getGun)
```

Now we map the value of the `Optional<String> getGun()` from "Beretta" to lowercase, as a result, it will transform it to "beretta":

```
.map(String::toLowerCase)
```

Then, we filter "beretta" to "Beretta" which returns `Optional.empty`:

```
.filter(g -> g.equals("Beretta"))
```

We use the `or` method, it returns the `Optional` value if the value is present. Otherwise, it returns the logic we implemented in the `Supplier` we are passing, in this case, `Optional::empty`:

```
.or(Optional::empty)
```

Considering that the previous `or` statement received an `Optional.empty`, the `orElse` statement will be invoked:

```
.orElse("Colt");
```

Therefore, the value of `sopranoGun` will be `Colt` and the right answer is:

```

B. No value!
   Colt

```

10.8 Final Yoshi Food Optional Challenge

This is the final Java Challenge of this chapter. We will explore the concepts of each topic we saw. The topics are, `Optional` creation, Exception handling, retrieving data, data filtering, and data transform.

Are you ready for this challenge? So, what will happen when running the following `main` method?

```

1  import java.util.Optional;
2
3  public class YoshiFoodOptionalChallenge {
4
5      public static void main(String... yoshiFood) {
6          Integer yoshiFoodCount;
7          Object isYoshiFoodEmpty;
8          Optional<Integer> optYoshi = Optional.ofNullable(0);
9
10         try {

```

```

11     yoshiFoodCount = optYoshi.orElseThrow();
12 } finally {
13     isYoshiFoodEmpty = Optional.ofNullable(null)
14         .filter(y -> y == null).orElse(Boolean.FALSE);
15 }
16
17 boolean isApple = Optional.of("Apple").flatMap(a -> Optional.of("APPLE"))
18     .equals(Optional.of("APPLE"));
19
20 optYoshi.ifPresentOrElse(System.out::print,
21     Optional.ofNullable(null)::get);
22 System.out.printf("%s %s %s", yoshiFoodCount, isYoshiFoodEmpty, isApple);
23 }
24
25 }

```

- A. NoSuchElementException will be thrown at // #A
- B. Optional[0] NoSuchElementException
- C. 00 false true
- D. 00 true false

Explanation:

First, we create an Optional encapsulating the value of 0:

```
Optional<Integer> optYoshi = Optional.ofNullable(0);
```

Then, we get the value from optYoshi since there is a value there of 0:

```
yoshiFoodCount = optYoshi.orElseThrow();
```

Now in the finally statement, we will encapsulate the value of null into the Optional. Notice that we are encapsulating the value of null into the Optional. The filter method has a check for a null value. If the value is null, the condition won't be even checked.

Therefore, the logic we passed into the filter method will be ignored and the orElse method will return false instead:

```

1 isYoshiFoodEmpty = Optional.ofNullable(null)
2     .filter(y -> y == null).orElse(Boolean.FALSE);

```

In the following code, we encapsulate "Apple" in the Optional. Notice that when we use the flatMap method we will make the Optional flat, this means, instead of having an Optional of an Optional, we will return only one Optional with the value of "APPLE". Therefore, it will be equals to the Optional.of("APPLE") and will return true.

```
1  boolean isApple = Optional.of("Apple").flatMap(a -> Optional.of("APPLE"))  
2                                .equals(Optional.of("APPLE"));
```

To conclude, the right alternative is:

C) 00 false true

10.9 Summary

By reading this chapter, now you can to the following:

- Encapsulate a value into an `Optional`
- Test if a value is present or not with `Optional`
- Consume a value or get another with `Optional`
- Consume a value or throw an `Exception`
- Avoid anti-patterns with an `Optional`
- Filter an `Optional` value
- Transform an 'Optional value

Congratulations! By knowing the `Optional` concepts you are now far more prepared to create high-quality code and deal better with `NullPointerException`. Now it's your time to apply this knowledge in a real-world project!

11 Generics and Object Comparison

This chapter covers:

- Comparing objects with `equals` and `hashCode`
- Easy usage of `equals` and `hashCode` on Lombok
- Why the use of generics is important
- Type-inference generics
- Upper and Lower Bound Generics
- Wildcard Generics

Generics and Collections are one of the most used concepts in Java. Very commonly we will have to deal with arrays of data. List, Set and Map are interfaces that are present in our day-to-day work very often. Therefore, understanding how we can use those Collections effectively makes all the difference to create high-quality code.

11.1 Comparing objects with equals and hashCode

The `equals` and `hashCode` are the methods that will check if an Object is equals to another one. In day-to-day work, we use those two methods very often. Many important core Java classes make use of the `equals` and `hashCode` concept.

The `equals` and `hashCode` methods are declared in the `Object` class in Java. This means that every class we create will inherit the `equals` and `hashCode` methods. However, the implementation of the `equals` and `hashCode` methods in the `Object` class are pretty raw:

```
1 public class Object {  
2     // Other methods ...  
3  
4     @IntrinsicCandidate  
5     public native int hashCode();  
6  
7     public boolean equals(Object obj) {  
8         return (this == obj);  
9     }  
10 }
```


**native modifier:**

Notice that the `Object` class uses the `native` modifier in the `hashCode` method. This modifier indicates that the method implementation will use native code and JNI (Java Native Interface). This modifier is used for lower-level interaction (machine, memory level). When using the `native` keyword, we can implement Java methods using C or C++. One of the main motivations to use the `native` operator is to obtain more performance.

Considering the raw implementation of the `equals` and `hashCode` methods, when we create a custom class, we will have to override those methods. Otherwise, we will be comparing the object instance instead of the object values.

Keep in mind the golden rule that whenever we override the `equals` method, always override the `hashCode` method too. Remember, the `equals` and `hashCode` methods must be always together.

In the following code example, we will override the `equals` method comparing the current instance attribute with the object that is being passed in the `equals` method. In the `hashCode` method we are generating a unique `hashCode` based on the fields of the `SpiderMan` class:

```

1  import java.util.List;
2  import java.util.Objects;
3
4  public class SpiderMan {
5
6      String armorName;
7      String webGadget;
8      List<String> skills;
9
10     @Override
11     public boolean equals(Object o) {
12         if (this == o) {                                // #A
13             return true;
14         }
15         if (o == null || getClass() != o.getClass()) { // #B
16             return false;
17         }
18         SpiderMan spiderMan = (SpiderMan) o;           // #C
19         return Objects.equals(armorName, spiderMan.armorName) && Objects
20             .equals(webGadget, spiderMan.webGadget) && Objects.equals(skills, spiderMan.\
21 skills);                                              // #D
22     }
23
24     @Override
25     public int hashCode() {
26         return Objects.hash(armorName, webGadget, skills); // #E

```

```

27     }
28
29 }

```

- #A: The first check we make in the `equals` method is if both objects are pointing to the same object reference. If this is `true`, we will be sure the attributes will be also the same considering it's the same object.
- #B: Then we will check if the passed object is `null`, if this is `true`, for sure the objects won't be `equals`. This is because to invoke the `equals` method it's necessary to create an instance, otherwise, it's impossible to invoke the `equals` method. The second condition checks if the objects have different class type. If one of those conditions are `true` then the objects are different.
- #C: We cast the passed object to `SpiderMan`.
- #D: Notice we are using the `Objects.equals` method from the `java.util` package that basically avoids `NullPointerException` in case an attribute is `null`:

```

1  public final class Objects {
2      // Other methods
3      public static boolean equals(Object a, Object b) {
4          return (a == b) || (a != null && a.equals(b));
5      }
6  }

```

If we were not using the `Objects.equals` method, we would have to do the `null` checks for each attribute, making the code verbose and confusing. Notice other important point that the `equals` method is meant to check the **total equality of objects**, therefore, we are supposed to check the equality of every attribute.

- #E: Whenever we override the `equals` method, we must also override the `hashCode` method. This is a good practice because it's far more expensive in terms of performance to check the equality of every single attribute, rather than checking the `hashCode` of attributes.

11.1.1 hashCode method

The purpose of the `hashCode` method is to return an `int` number that represents an identity for the object. We can create very sophisticated algorithms for the `hashCode` but in real-world projects, we usually create a `hashCode` based on the attributes of an object.

The great advantage of the `hashCode` method is to get performance. Since the `hashCode` method should use less memory resources than the `equals` method, it will be very helpful to check the equality of the `hashCode` method before checking the `equals` method.

```
1  import java.util.List;
2  import java.util.Objects;
3
4  public class SpiderMan {
5
6      private String suit;
7      private List<String> skills;
8      private int age;
9
10     // equals method...
11
12     @Override
13     public int hashCode() {                // #A
14         return Objects.hash(suit, skills, age); // #B
15     }
16
17 }
```

- #A: Notice that the hashCode method simply returns an int number. We don't receive another object to be compared like the equals method.
- #B: We are using the handy Objects.hash method from the java.util.Objects package. It's better to use this method because then we don't need to replicate the code below, we just reuse it:

```
1  public final class Objects {
2
3      // Other methods...
4      public static int hash(Object... values) {
5          return Arrays.hashCode(values);
6      }
7  }
8
9  public class Arrays {
10
11     // Other methods...
12     public static int hashCode(Object a[]) {
13         if (a == null)
14             return 0;
15
16         int result = 1;
17
18         for (Object element : a)
```

```

19         result = 31 * result + (element == null ? 0 : element.hashCode());
20
21     return result;
22 }
23 }

```

In a real-world project, this is what we will usually do. We will generate the `hashCode` based on the state of the object.

11.1.2 Guidelines for using `equals()` and `hashCode()`

You should only execute an `equals()` method for objects that have the same unique hashcode ID. You should not execute `equals()` when the hashcode ID is different.

Table 1. Hashcode comparisons

If the `hashCode()` comparison ...

Then ...

—
returns true

—
execute `equals()`

returns false

do not execute `equals()`

This principle is mainly used in `Set` or `Hash` collections for performance reasons.

11.1.3 Rules for object comparison

When a `hashCode()` comparison returns `false`, the `equals()` method must also return `false`. If the hashcode is different, then the objects are definitely not equal.

Table 2. Object comparison with `hashCode()`

When the `hashCode` comparison

The `equals()` method should return ...

returns ...

true

true or false

false

false

When the `equals()` method returns `true`, it means that the objects are equal in all values and attributes. In this case, the hashcode comparison must be `true` as well.

Table 3. Object comparison with `equals()`

When the `equals()` method returns ...

The `hashCode()` method should return

true

true

false

true or false

We can also return a constant `int` number that will be always the same. However, by doing that, we won't get the benefit of the `hashCode` method.

11.1.4 Easy equals and hashCode with Lombok

The Lombok library dramatically reduces the boiler-plate code. To generate the equals and hashCode methods automatically, if you are using the Lombok library, you can simply annotate your data object with @EqualsAndHashCode as the following:

```
1 @EqualsAndHashCode
2 public class SpiderMan { ... }
```

As you can see in the code above, it's very easy and simple to implement the equals and hashCode methods with Lombok. Lombok is also very flexible, that means that you can customize the way you use the equals and hashCode methods the way you prefer.

11.1.5 SpiderMan equals & hashCode Challenge

In the following Java Challenge we will explore how the equals and hashCode method should work fundamentally. As discussed before, the aim of those methods is to assure equality of the state of two objects. We are creating four objects and comparing them. Therefore, what do you think it will be the output?

```
1 public class SpiderEqualChallenge {
2     static int equalsCount = 0;
3     public static void main(String... doYourBest) {
4         System.out.println(areEquals(new SpiderMan("Peter"), new SpiderMan("Peter")));
5         SpiderMan symbiote = new SpiderMan("Symbiote") {
6             public int hashCode() { return 43 + 777 + 1; }
7         };
8         System.out.println(areEquals(symbiote, new SpiderMan("Symbiote")));
9         System.out.println(equalsCount);
10    }
11    static boolean areEquals(SpiderMan spiderMan1, SpiderMan spiderMan2) {
12        return spiderMan1.hashCode() == spiderMan2.hashCode()
13            && spiderMan1.equals(spiderMan2);
14    }
15    static class SpiderMan {
16        String name;
17        SpiderMan(String name) {
18            this.name = name;
19        }
20        public boolean equals(Object obj) {
21            final var spiderMan = (SpiderMan) obj;
22            equalsCount++;
```

```
23     return name.equals(spiderMan.name);
24 }
25 public int hashCode() {
26     return (43 + 777);
27 }
28 }
29 }
```

- A. false
true
2
- B. true
false
1
- C. false
false
0
- D. true
true
2

Explanation:

Notice that the `areEquals` method is doing what it's supposed to be done when two objects are being compared with Java. First, there is a comparison with `hashCode` followed by the `equals` check of two objects.

The data class `SpiderMan` that has the `equals` and `hashCode` methods overridden.

Another important point to pay attention is the static `equalsCount` variable that counts how many times the `equals` method was invoked.

Then, in the `main` method, we are invoking the `areEquals` method passing two `SpiderMan` objects with the name of "Peter". Since the value of both objects are the same, the `areEquals` method will return `true`. Notice also that the `equalsCount` variable is being incremented by 1.

In the following statement, we are instantiating another `SpiderMan` with the "Symbiote" name. However, we are also overriding the `hashCode` method passing a different value. Therefore, when we invoke the `areEquals` method, the `hashCode` will be different between the other "Symbiote" object.

When the `hashCode` is different from another object, there is no point to execute the `equals` method. That's because if the `hashCode` method is different from the other object's `hashCode`, the `equals` method has to return `false` as well. Therefore, the `equalsCount` variable won't be incremented.

Considering that the `hashCode` from both objects are different, the result from the `areEquals` method will be `false`.

The last statement will print 1 as the equalsCount.

To conclude, the right answer is:

- B. true
- false
- 1

11.1.6 Takeaways from equals and hashCode

- When overriding the equals method, always override the hashCode method as well. The equals and hashCode method complement each other and should be always together.
- The equals method is responsible to check the full equality of two objects' attributes.
- The hashCode method should execute quicker than the equals method.
- If the equals method returns true, the hashCode method must also return true.
- When creating a data object, always override the equals and hashCode methods.

11.2 Generics

I know that explaining Generics before Collections is not the standard of most Java books. Most books will explain Collections first and then Generics but when I was learning Collections I was confused by the symbol <>. I was using it with Collections but I didn't know what was that exactly.

That's why I decided to not follow what most Java books do and before going through Collections, you can first understand the Generics concept. Then the <> symbol will make sense when using Collections.

As mentioned when we use Collections, we are most of the time also using Generics. The generic type of a Collection is the type between the <>. The generics concept was introduced in Java 1.5.

There is a great definition of Generics in the Java documentation:

In a nutshell, generics enable types (classes and interfaces) to be parameters when defining classes, interfaces and methods.

This means we can define a type of a parameter in a class or interface in a dynamic way. By using generics, we get the benefit of strongly type a parameter dynamically so that we can have more control over the code.

It's far easier for bugs to come up in an application when we don't strongly type our code. Generics make code strongly-typed and secure during compile-time since we don't need to make use of casting for example.

To make the concept of generics clear, let's see some code examples.

11.2.1 List Without Generics

Generics is commonly used with a `Collection` because you are able to control what type will be manipulated in it. The most common `Collection` type is a `List` with the `ArrayList` implementation.

Let's see the code example with a `List` without using generics:

```
1 List list = new ArrayList();           // #A
2 list.add("Duke");                      // #B
3 list.add(777);                         // #C
4 list.add(new Object());
5 list.add(true);
6
7 Integer luckyNumber = (Integer) list.get(1); // #D
8 System.out.println(luckyNumber);
```

Output:

777

Let's analyze the code:

- #A: We instantiate our `List` without a generic type, this means that we can add any element type into our `List`.
- #B: We add a `String`.
- #C: We add an `int` number that will be auto-boxed and wrapped into an `Integer` type.
- #D: When we need to get an array element, we need to cast the type to what we are expecting. That happens because there is no way for the compiler to know what type we inserted in the `List`. In this case, we are getting the second element that contains an `Integer` number 777 and adding it to the list.

Adding any type to the `List` is not good because it's very easy to have the wrong data type in the `List` and cause bugs will be difficult to be tracked.

11.2.2 List With Generics

To avoid the type casting as seen in the code above, we can use generics! Fortunately, we can pass a generic type to the `List` interface as we can see in the following code:

```
public interface List<E> extends Collection<E> { ... }
```

Basically, the `<E>` is the type of the `List` element. Therefore, when using the `List` we need to pass the element's type when instantiating it:


```

1 List<String> list = new ArrayList<>();           // #A
2 list.add("Duke");                               // #B
3
4 String duke = list.get(0);                       // #C
5 System.out.println(duke);

```

Output:

Duke

- #A: Notice that now we are passing the generic type of <String>, therefore, we can only add objects that are typed as String into the List.
- #B: We are adding a String type to the List. Notice that if we try to add the following `list.add(777);` the code wouldn't compile because the List is expecting only String.
- #C: Finally, we retrieve the the first element of the List by the index. Note that we are not using casting because the compiler knows this List will only accept String elements.

**<> operator:**

Before Java 7 we would have to repeat the generic type on the instantiation like the following: `List<String> list = new ArrayList<String>();` which is verbose and unnecessary. Fortunately, since Java 7 we can omit the generic type of the instantiation as the following: `List<String> list = new ArrayList<>();` having the same effect.

11.2.3 Generics don't accept primitive types

If we try to do the following:

```
List<int> listOfInts = new ArrayList<>();
```

The code won't compile, so be aware that we can only use a class type. Another important detail is that if we try the following, the code will work:

```

1 int number = 7;
2 List<Integer> list = new ArrayList<>();
3
4 list.add(number); // This works because of the auto-boxing concept

```

As we saw in previous chapters, Java has a concept of auto-boxing. Therefore, when we add an int type to a List<Integer>, actually, the int type will auto-boxed to Integer implicitly by the compiler.

The same will happen with the other primitive types.

11.2.4 Spider Man Villains Without Generics Challenge

We will explore the use of a `List` without generics in this challenge.

Your goal in this code challenge is to replace code in the `##REPLACE##` area so that it will print "Venom".

Note that more than one alternative might be correct.

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class CollectionWithoutGenericsChallenge {
5
6      public static void main(String[] args) {
7          List list = new ArrayList();
8          list.add("Venom");
9
10         // ##REPLACE##
11
12         System.out.println(superVillain);
13     }
14
15 }
16
17
18 A) String superVillain = list.get(0);
19
20 B) Object superVillain = list.get(0);
21
22 C) var superVillain = list.get(0);
23
24 D) String superVillain = (String) list.get(0);
25
26 **Explanation**:
27
28 In alternative `A`, we can't use the code `String superVillain = list.get(0);` becau\
29 se we are not using casting. Note that there is no way for the compiler to guess wha\
30 t is the variable type since we can put anything into the `list`.
31
32 In alternative `B`, we can use `Object superVillain = list.get(0);` because we are p\
33 utting the `list` value into an `Object` since it can receive any type.
34
35 In alternative `C` we can use `var superVillain = list.get(0);` also because a `var`\

```

```

36  can store any type.
37
38  In alternative `D`, we can use `String superVillain = (String) list.get(0);` because\
39  we are casting the type to `String`.
40
41  Therefore, the correct alternatives are:
42  **B) Object superVillain = list.get(0);**
43  **C) var superVillain = list.get(0);**
44  **D) String superVillain = (String) list.get(0);**
45
46  ## 11.3 Generic Type Inference
47
48  We also have the possibility to create our own generic types. When we want to create\
49  code that is highly reusable and have more control in the data type, we can use gen\
50  erics.
51
52  Let's see the example of using generics in a `class`:
53
54  ```java
55  public class GenericClass<T> {                                // #A
56
57      public T doSomething(T type) {                            // #B
58          System.out.println(type.getClass());                 // #C
59          return type;
60      }
61
62      public static void main(String[] args) {
63          GenericClass<StringBuilder> genericClass = new GenericClass<>(); // #D
64          genericClass.doSomething(new StringBuilder("Generic Builder")); // #E
65
66          new GenericClass<>().doSomething("Generic String"); // #F
67          Integer genericNumber = new GenericClass<Integer>().doSomething(7); // #G
68          System.out.println(genericNumber);
69      }
70
71  }

```

Output:

```

class java.lang.StringBuilder
class java.lang.String
class java.lang.Integer

```

7

Code Analysis:

- #A: This is the syntax to use a generic type at a class level. We use the `<T>` after the class name. Notice that the generic type name can be anything. `T` is commonly used because it's a `T` for Type.
- #B: Once we have the `<T>` generic type declared in the class declaration, we can use the generic type in this class. Notice that we are using the generic type `T` as a return type. We are also using the `T` as the parameter type.
- #C: We print the class type of the generic type.
- #D: We instantiate the `GenericClass` with the generic type of `StringBuilder`.
- #E: Notice that we already passed the `StringBuilder` type to `GenericClass`, this means that the generic type is already set for `StringBuilder`. Therefore, when invoking the `doSomething` method, we can only use the `StringBuilder` type. If we try to pass something else as argument, there will be a compilation error.
- #F: We are instantiating the `GenericClass` again, but now passing a `String` as a parameter. Notice that we are not passing anything in the `<>`, therefore, the generic type applied by default will be `Object`, not `String`.
- #G: We are passing the generic type of `Integer` when we instantiate the `GenericClass`. Therefore, `T` will be `Integer` and we won't have to cast this type. Therefore, we can return the value of the `doSomething` method and print it with no complications.

11.3.1 Generic Type Inference in a Method

If you don't want to declare the generic type in a class level, you can also use it directly in a method:

```

1  public class GenericsMethod {
2
3      public <T> T doSomething(T type) {                // #A
4          System.out.println(type.getClass());
5          return type;
6      }
7
8      public static void main(String[] args) {
9          GenericsMethod genericsMethod = new GenericsMethod();
10         String result = genericsMethod.doSomething("Generics Method");    // #B
11         System.out.println(result);
12
13         result = genericsMethod.<String>doSomething("Method with Generics"); // #C
14         System.out.println(result);
15     }
16
17 }
```

Output:

```
class java.lang.String
```

Method with Generics

class java.lang.String

Method with Generics

Code Analysis:

- #A: Note that instead of declaring the generic type at class level, we are declaring it after the public modifier. Therefore, we will be able to use a generic type as a parameter in the method.
- #B: We are passing the type of String to the T generic type. Notice that since we are using generics at a method level, when we pass a String as a parameter, T will be inferred as String automatically.
- #C: Notice that if we want, we can also pass the method generic type when invoking the method. It's not necessary though since the type we pass as parameter will be already inferred as the generic type. It works but it's redundant.

11.3.2 Generic Type Inference Instance Variable

It's possible to define an instance variable with a generic type. To do so, we need to pass the generic type in the instantiation `GenericInstanceVariable`, then we pass the value in the constructor:

```

1  public class GenericInstanceVariable<T> {           // #A
2
3      T t;                                           // #B
4
5      public GenericInstanceVariable(T t) {         // #C
6          this.t = t;
7      }
8
9      public static void main(String[] args) {
10         GenericInstanceVariable<String> genericInstance =
11             new GenericInstanceVariable<>("T type"); // #D
12
13         String tString = genericInstance.t;        // #E
14         System.out.println(tString);
15     }
16
17 }
```

Output:

T type

Code Analysis:

- #A: Notice that we need to declare the generic type at class level to be able to use it as an instance variable. Another important point is that the `T` type can be used as a constructor or method parameter.
- #B: Since we declared the `T` generic type in the class, we can use `T` as an instance type.
- #C: We can pass the value of our generic type here and then assign to the instance variable.
- #D: It's important to notice here that we are passing the generic type when we use the following `<String>`. If we don't provide the type here, the `T` type can't be inferred when we only pass a value. That's because when we declare the generic type in a class level, the type is supposed to be the same for the whole class.
- #E: We can return the `t` value as a `String` because we defined that in the generic type. That's one of the greatest advantages of using generics, to define a return type during runtime.

11.3.3 Mystique Generic Type Inference Challenge

In the following Java Challenge, we will explore generics at a class, instance variable, constructor and method level. Can you guess what will happen after running the `main` method?

```

1  public class GenericMystiqueChallenge<T> {
2      T t;
3      public GenericMystiqueChallenge(T t) {
4          this.t = t;
5      }
6      public<K> K mystiqueTransform(K k) {
7          return k;
8      }
9      public <T> T mystiqueAttackPower(T t) {
10         return t;
11     }
12     public static void main(String... doYourBest) {
13         GenericMystiqueChallenge<String> genericMystique = new GenericMystiqueChallenge<
14 >("Raven");
15
16         String realName = genericMystique.t; // #A
17         String transformed = genericMystique.mystiqueTransform("Wolverine"); // #B
18         Integer normalForm = genericMystique.mystiqueAttackPower(7); // #C
19
20         System.out.printf("%s %s %s", transformed, realName, normalForm);
21     }
22
23 }
```

A. Compilation error at line // #A

- B. Compilation error at line // #B
- C. Wolverine Raven 7
- D. Compilation error at line // #C

Explanation:

In this Java Challenge we are exploring generics at class, constructor, and method level.

When we instantiate `GenericMystiqueChallenge` we are passing the `<String>` generic type. When we do that, we will pass the `String` type to `T`.

Also note that we are passing "Raven" in the constructor. If we tried to pass anything else there would be a compilation error since we already defined the type of `T` to be `String`.

Therefore, it will be fine to retrieve the `t` variable as `String` since the `T` generic type is `String`.

Then we invoke the `mystiqueTransform` method passing "Wolverine". Notice that the `mystiqueTransform` method receives the generic type of `K`. Since the `K` generic type is being declared at a method level, the type will be inferred without the need of the `<>` operator. Therefore, the variable transformed will have the value of "Wolverine".

Finally, notice that we are using a generic type shadowing. We already have the `T` type declared at class level but we are declaring it again in the `mystiqueAttackPower` method signature since we are using `<T>`.

In conclusion, the correct answer is:

C) Wolverine Raven 7

11.3.4 Declaring Many Generic Types

It's possible to declare as many generics we want, also there is no restrictions for the generics names. There are many good examples in the JDK code with multiple generic types. Let's see some of them:

```
public interface Map<K, V> { ... }
```

```
1 public class HashMap<K,V> extends AbstractMap<K,V>
2     implements Map<K,V>, Cloneable, Serializable {
```

```
public interface BiFunction<T, U, R> { ... }
```

```
public interface BiConsumer<T, U> { ... }
```

Notice the `BinaryOperator` that it's possible to extend `BiConsumer` and override the generic types with another generic type:

```
public interface BinaryOperator<T> extends BiFunction<T, T, T> {
```

As you can see in the above examples, there are plenty of interfaces and classes in the JDK that use multiple generics.

Let's create our own interface that has many generic types:

```

1  public class MultiMapExample {
2
3      interface MultiMap <K,V, R>  { // #A
4          R put(K k, V v);          // #B
5      }
6
7      public static void main(String[] args) {
8          MultiMap<Integer, Double, String> multiMap = (key, value) ->      String.va\
9      lueOf(key + value);              // #C
10         System.out.println(multiMap.put(1, 2.0));    // #D
11     }
12 }

```

Output:

3.0

Code Analysis:

- #A: We are declaring the generic types into the interface, K, V and R.
- #B: Then we are using the generic types in the put method.
- #C: We are passing the type Integer to K, Double to V, and String to R. Then we are passing a lambda expression, basically defining the body of the put method.
- #D: Finally, we invoke the put method passing an Integer 1, Double 2.0 and returns a String which matches exactly the generic types we passed to the MultiMap interface.

11.4 Upper and Lower Bound Generics

To restrict the use of generics, we can define what type it should extend, this is also called as upper bounds generics. To restrict generics to super types we use the concept of lower bound generics.

In the following code, let's see an example where we define the genericity type T that extends String:

```

1  public class UpperBoundGenerics {
2
3      public static void main(String[] args) {
4          String upperBound = concatString("upperBound");
5          System.out.println(upperBound);
6      }
7
8      static <T extends String> String concatString(T t) { // #A
9          return t.concat("Generics");                    // #B
10     }
11
12 }

```


Output:

upperBoundGenerics

Code Analysis:

- #A: Notice the syntax of the upper bound generic type. We are declaring `T` as the generic type and we are also extending `String`. This means that we can receive any type that is or extends a `String`.
- #B: Another important point is that we are using the specific `concat` method from the `String` class. This is only possible because we are making explicit to the compiler that `T` extends `String`, therefore, the `concat` method will be on `t`.

11.4.1 Unbounded Wildcard

When we don't need to store or infer a generic type we can use the unbounded wildcard with `?`.

Unbounded wildcard is useful to create flexible, robust APIs and if you notice, the JDK code has plenty of examples. Let's check the `Files` class from the `nio` package:

```
1 public static Path walkFileTree(Path start, FileVisitor<? super Path> visitor){... }
2
3 public static Path write(Path path, Iterable<? extends CharSequence> lines,
4                          Charset cs, OpenOption... options)
```

Therefore, if you want to create powerful and robust APIs, unbounded generics will be a great help for you.

When we use `?`, we can use the upper bound or lower bound generic type.

Let's see how unbounded type, lower and upper bounds generics work in the following code:

Unbounded type: we can pass a list with any type. `?` will be erased at runtime and will be substituted by the type we are passing:

```
1 List<?> stringList = new ArrayList<String>();
2 List<?> exceptionList = new ArrayList<Exception>();
3 List<?> booleanList = new ArrayList<Boolean>();
4 List<?> consumerList = new ArrayList<Consumer>();
```

Lower bounds generics: we can pass a list that is `Double` or any super type of `Double`:

```
1 List<? super Double> doubleList = new ArrayList<Double>();
2 List<? super Double> numberList = new ArrayList<Number>();
```

Upper bounds generics: we can pass a list that is `Number` or any sub type of `Number`:

```

1 List<? extends Number> numberList = new new ArrayList<Number>();
2 List<? extends Number> doubleList = new ArrayList<Double>();

```

11.4.2 Wildcard Lower Bound Generics

The lower-bounds generic type is more suitable to be used when we will use data as **output**, that means when data will be returned by a method in some way.

If we want to pass a generic type that is the super type of another, then we can use the following code:

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class LowerUnboundedGenerics {
5     public static void main(String[] args) {
6         List<Number> numbers = new ArrayList<>();
7         numbers.add(7.0);
8         numbers.add(3.0);
9         List<? super Double> result = addNumbers(numbers);
10        System.out.println(result);
11    }
12
13    static List<? super Double> addNumbers(List<? super Double> numbers) { // #A
14        numbers.add(1.0); // #B
15        return numbers;
16    }
17 }

```

Output:

[7.0, 3.0, 1.0]

Code Analysis:

- #A: Notice that it's only possible to use lower bound generic type with the unbounded wildcard. Therefore, we can't declare an unbounded generic type, we can only pass it as a type to an existent generic type.
- #B: Another important point is that we can add a Double as a type to the list. We can't add an Integer, Float or any other type to this generic type.

11.4.3 Upper and Lower Bounds God Warriors Challenge

This is a Java code challenge about upper and lower bounds generics.

Your objective is to replace `##REPLACE##` by one of the code alternatives so that the code will compile and print "Everything went fine!".

Which alternatives are correct? **Note that more than one alternative may be correct.**

```

1  public class UpperLowerBoundGenericChallenge {
2
3      interface Zeus {}
4      interface Kratos extends Zeus { }
5      interface Atreus extends Kratos { }
6      static class Warrior<T> {}
7
8      public static void main(String ... doYourBest) {
9          Warrior<Kratos> kratos = new Warrior<>();
10         Warrior<Zeus> zeus = new Warrior<>();
11         Warrior<Atreus> atreus = new Warrior<>();
12
13         // ##REPLACE##
14
15         System.out.println("Everything went fine!");
16     }
17
18     static void fight(Warrior<? extends Zeus> warrior1, Warrior<? super Kratos> warrior\
19 2) {}
20
21 }

```

- A. `fight(kratos, zeus);`
- B. `fight(atreus, new Warrior<>());`
- C. `fight(kratos, atreus);`
- D. `fight(zeus, kratos);`

Explanation:

The code in alternative A will compile fine since `kratos` extends `Zeus` and `zeus` is super of `kratos`.

The code in alternative B will also compile fine since `atreus` extends `zeus` and `new Warrior<>()` will infer the generic type of `Kratos`.

The code in alternative C won't compile because the second `atreus` argument is not super of `Kratos`.

The code in alternative D works fine since the type we see in the wildcard either for `extends` or `super` will always work. For example, in the first warrior we use `<? extends Zeus>`, therefore, `zeus` will work. For `Warrior<? super Kratos>` `kratos` also works fine.

In conclusion, the correct alternatives are:

- A) `fight(kratos, zeus);`
- B) `fight(atreus, new Warrior<>());`
- D) `fight(zeus, kratos);`

11.4.4 Upper Bounds Generics

The upper bounds generics is pretty much the opposite of the lower bounds generics. The upper bounds generic is more suitable to be used when you are performing logic **within** a method and don't want this variable to be used outside of your method.

Let's see how it works in code:

```

1  import java.math.BigDecimal;
2  import java.util.ArrayList;
3  import java.util.List;
4
5  public class UpperUnboundedGenerics {
6      public static void main(String[] args) {
7          List<Integer> intNumbers = new ArrayList<>();
8          intNumbers.add(7);
9          printNumbers(intNumbers);           // #A
10
11         List<BigDecimal> decimalNumbers = new ArrayList<>();
12         decimalNumbers.add(BigDecimal.TEN);
13         printNumbers(decimalNumbers);       // #B
14     }
15     static void printNumbers(List<? extends Number> numbers) { // #C
16         numbers.forEach(System.out::println);
17     }
18 }

```

- #A: We pass a `List` of `Integer` to the `printNumbers` method. This is fine since we are receiving any `List` that extends `Number` in the generic type.
- #B: Here we pass a `List` of `BigDecimal` and that's one of the benefits of using the upperbound generic type, we can pass a whole `List` to a method with some flexibility in the type. We can pass a `List` with any generic type that is a `Number`.
- #C: We are receiving here `List<? extends Number> numbers` which enables us to pass a `List` with a flexible `Number` type. Therefore, we can print any `Number` in this method.

11.4.5 No Polymorphism with Generics

If you try to do the following:

```
List<Number> list = new ArrayList<Integer>();
```

The code won't compile, this is because we can't use polymorphism with generics. To fix the code above we have to use the following:

```
List<Number> list = new ArrayList<>();
```

We can simulate the effect of polymorphism by using bounded generics though. We could also create a method that receives a `List<? extends Number>`:

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class NoPolymorphismGenerics {
5      public static void main(String[] args) {
6          List<Integer> intNumbers = new ArrayList<>();
7          intNumbers.add(7);
8          printNumbers(intNumbers);
9      }
10     public static void printNumbers(List<? extends Number> numbers) { // #A
11         numbers.forEach(System.out::println); // #B
12     }
13 }
```

- #A: We are using a wildcard ? here which gives us the flexibility to pass a `List` that has the generic type of any `Number`.
- #B: We iterate over the `List` of `Number` and print the elements.

Another way to use polymorphism is by adding element by element. If we create a `List<Number>` and add `Integer`, `BigDecimal`, `Long` the code will work:

```
1  List<Number> numbers = new ArrayList<>();
2  numbers.add(Integer.valueOf("7"));
3  numbers.add(BigDecimal.TEN);
4  numbers.add(Long.valueOf("7"));
5  numbers.add(Double.valueOf("7"));
```

Notice that we still can use polymorphism in the element level. That's why we can add any `Number` to the `List`. However, if we want to use polymorphism at the `Collection` level as the previous example, then we have to use bounded generics.

11.4.6 UpperUnbounded MiniMog Generic Challenge

It's time for you to consolidate the concepts of bounded wildcard generics! In the following code challenge, the `addInto` method will be invoked many times with many `Lists` that have different generic types.

What will happen when running the following code?

```

1  import java.math.BigDecimal;
2  import java.util.ArrayList;
3  import java.util.List;
4
5  public class MiniMogWildcardChallenge {
6      public static void main(String... doYourBest) {
7          List<Integer> miniMogHitDamage;
8
9          List<Integer> intDamage = new ArrayList<>();
10         intDamage.add(10);
11         miniMogHitDamage = addInto(intDamage);           // #A
12
13         List<Long> longDamage = new ArrayList<>();
14         miniMogHitDamage.addAll(addInto(longDamage));    // #B
15
16         List<BigDecimal> bigDecimalDamage = new ArrayList<>();
17         bigDecimalDamage.add(BigDecimal.ZERO);
18         miniMogHitDamage.addAll(addInto(bigDecimalDamage));
19
20         miniMogHitDamage.forEach(System.out::println);   // #C
21     }
22     static List addInto(List<? extends Number> list) {
23         List<Number> resultList = new ArrayList<>();
24         resultList.addAll(list);
25         return resultList;
26     }
27 }

```

- A. Exception in thread “main” java.lang.ClassCastException: class java.math.Integer cannot be cast to class java.lang.Number at line // #A
- B. 10
Exception in thread “main” java.lang.ClassCastException: class java.math.Long cannot be cast to class java.lang.Integer at line // #B
- C. 10
0
- D. 10
Exception in thread “main” java.lang.ClassCastException: class java.math.BigDecimal cannot be cast to class java.lang.Integer at line // #C

Explanation:

We first invoke the `addInto` method with the `intDamage` Integer List. Then we add all the Integer elements into the `resultList`. This works because Integer extends Number.

Then we invoke the `addInto` method again with the `longDamage` list. It works fine again because `Long` also extends `Number`.

We invoke the `addInto` method for the last time passing the `bigDecimalDamage` list. It also works fine because `BigDecimal` extends `Number`.

Finally, we try to iterate in the `List<Integer>` but there is a problem, we added an `Integer` and `BigDecimal` elements into it. Therefore, when we try to iterate the elements of this array, we will get a `ClassCastException` because we can't use a `BigDecimal` type in an `Integer List`.

In conclusion, the final answer will be:

D. 10

Exception in thread "main" java.lang.ClassCastException: class java.math.BigDecimal cannot be cast to class java.lang.Integer at line // #C

11.5 Summary

Generics will help you understand better the Java API and will help you design better APIs when needed.

In this chapter, you learned that:

- It makes a generic object strongly typed avoiding type casting.
- It's commonly used to build robust and flexible APIs.
- It organizes code in a way that the object type of a `Collection` is reliable during runtime.
- It's widely used in the JDK code, you will see generics in the `Collection` api, functional interfaces, `Comparable` interface, and so on.
- Generics can be used at a class, variable, constructor or method level.
- A generic type can be declared at a class, constructor, method or instance variable level.
- Only objects can be passed to a generic type, primitive variables are not allowed.

12 Collections

This chapter covers:

- The common uses of a List
- The List implementations such as ArrayList, LinkedList and Vector
- Finding and Sorting elements in a List using Collections
- Using Queue, Stack, Deque, ArrayDeque
- Avoiding ConcurrentModificationException with CopyOnWriteArrayList
- Using unique elements with Set
- The Set implementations such as HashSet, TreeSet, and LinkedHashSet.
- Using key value data with Map
- Using HashMap, LinkedHashMap, Hashtable, TreeMap, and ConcurrentHashMap
- Searching elements in an array

12.1 Collections API

In the following sections, we will explore the most used Collections api.

We will explore List, Set, Queue, Deque and their implementations. To give you a macro-view of them, check out the following diagram:

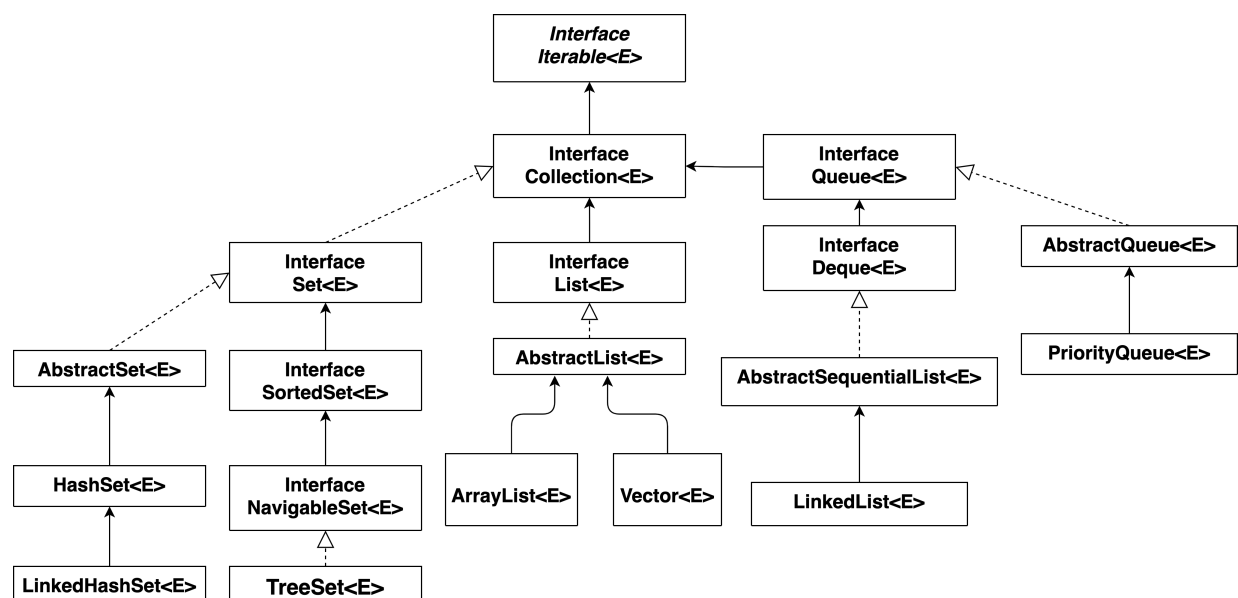


Figure 11.1 Collections API Diagram

Now that you have a brief understanding of the `Collections` api, let's explore their concepts more deeply and of course do some Java code challenges!

12.2 List

The `List` interface is the most used `Collection` interface in the day-to-day work of a Java developer. As the name suggests, we can add many elements in the list, we can access elements quickly by index and elements are ordered in the way they are inserted.

Note that we will also use `ArrayList` as an implementation of `List` here but we will explore it further after `List`.

Let's analyze the following code to see a `List` in practice:

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  List<String> heroes = new ArrayList<>();           // #A
5  heroes.add("Batman");                             // #B
6  heroes.add("Spider-Man");
7  heroes.add("Iron Man");
8
9  System.out.println(heroes);                       // #C
```

Output:

```
1  [Batman, Spider-Man, Iron Man]
```

Code Analysis:

- #A: Note that we are instantiating the `List` with `ArrayList`. By doing that we are using polymorphism. `ArrayList` is the most used `Collection` in real Java projects. We are also passing the generic type of `String` to the list, which means we can only add `String` to it.
- #B: We are adding `String` elements in the list here.
- #C: We are printing the `toString` method from the `List`. Notice that it's nicely formatted. We can easily see all the list elements. Out of curiosity, the `toString` is implemented in the `AbstractCollection` class:

```

1  // You don't need to fully understand the following code, don't worry!
2
3  public abstract class AbstractCollection<E> implements Collection<E> {
4      // Ommitted other methods...
5      public String toString() {
6          Iterator<E> it = iterator();
7          if (! it.hasNext())
8              return "[]";
9
10         StringBuilder sb = new StringBuilder();
11         sb.append('[');
12         for (;;) {
13             E e = it.next();
14             sb.append(e == this ? "(this Collection)" : e);
15             if (! it.hasNext())
16                 return sb.append(']').toString();
17             sb.append(',').append(' ');
18         }
19     }
20 }

```

12.2.1 Iterating over a List with index

One of the greatest advantages of a list is to access elements by index. Therefore, let's see how we can create a for looping accessing the index of each element:

```

1  // Omitted imports and list adding of Batman, Spider-Man, and Iron Man
2  for (int i = 0; i < heroes.size(); i++)           // #A
3      System.out.println(heroes.get(i));           // #B

```

Output:

```

Batman
Spider-Man
Iron Man

```

Code Analysis:

- #A: We create a for looping iterating up until the list size and keep incrementing the i variable.
- #B: Then we get each element by index printing then all the characters.

12.2.2 Iterating over a List with Iterator

Iterating elements of a `List` with `Iterator` is quite verbose.

However, using `Iterator` might be useful when we need to use a `while` looping asking if the next element is present. `Iterator`

Let's see an example of `Iterator`:

```

1 // Omitted imports and list adding of Batman, Spider-Man, and Iron Man
2
3 // Iterator to traverse the list
4 Iterator iterator = heroes.iterator();           // #A
5
6 // Verifies if there is a next element in the iterator
7 while (iterator.hasNext()).                     // #B
8     System.out.print(iterator.next() + " ");    // #C

```

Output:

Batman Spider-Man Iron Man

Code analysis:

- #A: Notice that the `List` has the `iterator` method which returns an `iterator` object capable of iterating over a `list`.
- #B: Then we put the `iterator` in a `while` looping and invoke `hasNext()` to check if there is a next element.
- #C: Finally, we use the `next` method that will effectively get the `list` element and will pass the pointer to the next element.

12.2.3 Iterating over a List with `forEach`

To do the same as mentioned above, we can use the `forEach` or enhanced looping concept. This looping will literally do the same as the `Iterator` but that will happen behind the scenes.

When we decompile the code, we can see that it did the same as above:

```

1 Iterator var2 = heroes.iterator();
2
3 while(var2.hasNext()) {
4     String hero = (String)var2.next();
5     System.out.print(hero + " ");
6 }

```

Therefore, if you don't need any specific feature from the `Iterator` class, you can use the `forEach` statement instead.

Let's see how that works:

```
1 // Omitted imports and list adding of Batman, Spider-Man, and Iron Man
2 for (String hero : heroes) System.out.print(hero + " "); // #A
```

Output:

Batman Spider-Man Iron Man

Code analysis:

- #A: It retrieves String from heroes and puts each of them into a String variable.

12.2.4 Iterating over a List with lambda

To iterate elements with less code, we can also use a lambda expression. In Java 8, the `forEach` method was added in the `List` interface as a default method as you can see in the following code:

```
1 default void forEach(Consumer<? super T> action) {
2     Objects.requireNonNull(action);
3     for (T t : this) {
4         action.accept(t);
5     }
6 }
```

Note that we receive a `Consumer` in the `forEach` method. So, if we want to print all elements using a lambda, we can do the following:

```
1 // Omitted imports and list adding of Batman, Spider-Man, and Iron Man
2 list.forEach(e -> System.out.println(e));
```

Output:

Batman
Spider-Man
Iron Man

To make the code above even better, we can use a method reference to print the elements from the list:

```
list.forEach(System.out::println); // The output will be the same as above
```

12.2.5 List Removing Elements from a List

If we want to remove an element, we can use the following code:

```
1 heroes.remove(0);           // #A By index
2 heroes.remove("Iron Man"); // #B By object
3 System.out.println(heroes);
```

Output:

[Spider-Man]

Code analysis:

- #A: We remove the element by the index. Therefore, we remove the first element that is Batman in that case.
- #B: We remove the element by object value. This means we will remove the "Iron Man", the last element.

12.2.6 Changing Elements from a List

We can also change an element value with the following code:

```
1 heroes.set(0, "Wolverine"); // #A
2 System.out.println(heroes);
```

Output:

Wolverine

Code analysis:

- #A: It changes the element located at the index we are passing. In that case, the index 0 is changed from Spider-Man to Wolverine.

12.2.7 Important Methods of a Collection

Let's see the most used method from the `List` and `Collection` interfaces in detail:

Collection:

- `boolean add(E e)`: Adds an element to a `Collection`
- `boolean remove(Object o)`: Removes an object from a `Collection`
- `boolean contains(Object o)`: Checks if an element is contained in the collection
- `default boolean removeIf(Predicate<? super E> filter) { ... }`: Removes an element based on a condition we pass in the `Predicate` functional interface
- `default Stream<E> stream()`: It's possible to manipulate elements from a collection with a sequential stream. The stream concept is vast and will be covered with more details in the next chapter

12.2.8 Important Methods of a List

List:

- `void add(int index, E element)`: Adds an element based on the index we pass
- `E get(int index)`: Retrieves an element by the index
- `boolean isEmpty()`: Verifies if the list is empty
- `<T> T[] toArray(T[] a)`: Transforms the list to an array
- `boolean containsAll(Collection<?> c)`: Checks if the elements of a Collection is contained in the list
- `boolean addAll(Collection<? extends E> c)`: Adds all elements from another collection into the list
- `void clear()`: It clears the list, it removes all the elements
- `int indexOf(Object o)`: Gets the index of an object of the list
- `int lastIndexOf(Object o)`: Gets the last index of an object. If there are more than one objects that are the same in the list, the last index will be retrieved.
- `static <E> List<E> of(E... elements)`: Creates an unmodifiable list with multiple elements. This method was added in Java 9.

12.3 ArrayList

As mentioned before, the most used implementation of List in real projects is certainly ArrayList. ArrayList has good performance to accessing data with the get and set methods because elements are indexed.

Also, behind the scenes an ArrayList actually stores an array of objects as you can see in the variable `elementData` in the JDK code:

```

1  public class ArrayList<E> extends AbstractList<E> implements List<E>,      RandomAcce\
2  ss, Cloneable, java.io.Serializable {
3      @java.io.Serial
4      private static final long serialVersionUID = 8683452581122892189L;
5      private static final int DEFAULT_CAPACITY = 10;
6      private static final Object[] EMPTY_ELEMENTDATA = {};
7      private static final Object[] DEFAULTCAPACITY_EMPTY_ELEMENTDATA = {};
8
9      transient Object[] elementData;
10     private int size;
11     // Other methods...
12 }
```

An array in any programming language has to have pre-allocated size in memory. Since behind the scenes in the `ArrayList` we use the type `Object[] elementData`, we also have to have an initial capacity. In the case of `ArrayList`, the `DEFAULT_CAPACITY` is used. Therefore, whenever you add more elements than the array's capacity, a new array with the double of the capacity will be created since it's not possible to change an existing array's capacity.

To add elements in an array while the defined capacity is not reached is quick but when the capacity is reached then it's necessary to copy the existing array elements into a new array which makes it a bit slow.

`ArrayList` is not synchronized, which means that it's not safe to use in a multi-thread environment because you might have data collision. However, because `ArrayList` is not synchronized, it's also more performant.

12.3.1 Time Complexity with ArrayList (Big-O Notation)

In programming and algorithms, there is the concept of time-complexity where we can understand how performant is our algorithms.

We won't go in much details about what is Big-O notation but we will address what is the complexity of each `Collection` object:

Method	Complexity	Worst Case Scenario	Explanation
<code>add()</code>	$O(1)$	$O(n)$ when a new array is created with all elements	It's always constant time because of how the array datastructure is defined in memory
<code>get()</code>	$O(1)$	$O(1)$	
<code>remove()</code>	$O(n)$	$O(n)$	We have to check element by element to remove it
<code>indexOf()</code>	$O(n)$	$O(n)$	Runs in linear time, checking each element
<code>contains()</code>	$O(n)$	$O(n)$	Checks each element until finds the value

12.3.2 Immutability with ArrayList

The concept of immutability is very important because the less point of changes in code, the fewer points of bugs. Bugs are stressful and make developers' life miserable. To create immutable objects, this means, objects that don't change state makes your application easier to handle.

Fortunately, there are some ways to create a `List` that is immutable.

12.3.3 List.of

In Java 9, there is the handy `List.of` method that makes our lives easier when creating an array. However, note that it creates an immutable list as you will see in the following example:

```
1 List<String> heroes = List.of("Batman", "Super Man", "Wonder Woman");
2 heroes.add("Robin");
```

When we try to add "Robin" to the heroes list, we will get the following Exception:

```
Exception in thread "main" java.lang.UnsupportedOperationException
```

Therefore, whenever we have an immutable Collection, we can't add, delete or change the list object in any way. If we really need to change the data we would need to create another `ArrayList` object:

```
1 List<String> heroes = List.of("Batman", "Super Man", "Wonder Woman");
2
3 heroes = new ArrayList<>(heroes);
4 heroes.add("Robin");
5 heroes.forEach(System.out::println);
```

12.3.4 Arrays.asList

The `Arrays.asList` method is similar to the `List.of` method and has the same behaviour. It will add elements to it and will return an immutable `List`.

Let's see the following code example:

```
1 List<String> otherHeroes = Arrays.asList("Batman", "Super Man", "Wonder Woman");
2 otherHeroes.add("Robin");
```

We will get the same Exception from the `List.of` method:

```
Exception in thread "main" java.lang.UnsupportedOperationException
```

12.3.5 Collections.unmodifiableList

It's usual to see lists in real Java projects being returned as unmodifiable lists in data objects. This is useful because that prevents programmers to change data when it's not suitable.

Let's see a code example:


```

1  import java.util.ArrayList;
2  import java.util.Collections;
3  import java.util.List;
4
5  public class UnmodifiableList {
6      private static List<String> villains = new ArrayList<>();
7      public static void main(String[] args) {
8          villains.add("Magneto");
9          villains.add("Venom");
10         villains.add("Dr. Octopus");
11
12         villains = getUnmodifiableVillains();
13         villains.add("Carnage");           // #B
14     }
15
16     public static List<String> getUnmodifiableVillains() {
17         return Collections.unmodifiableList(villains);    // #A
18     }
19 }

```

- #A: We use the `Collections.unmodifiableList` method that will return an immutable list. Note that we pass the list as a parameter.
- #B: When we try to add an element to our list, we will get the following Exception:

Exception in thread "main" java.lang.UnsupportedOperationException

When we work with an immutable list, we can't add or delete elements.

12.3.6 List.of Java Challenge

In the following Java Challenge, you will see the `List.of` method introduced on Java 9. With this method, it's possible to add many elements into a list in just one line. Do you know what will happen after running the following code?

```
1  import java.util.List;
2
3  public class ListChallenge {
4
5      public static void main(final String... doYourBest) {
6          final List<String> soldiers = List.of("Tyrion", "Arya", "John");
7
8          if (soldiers.size() > 3) {
9              soldiers.add("Cersei");
10         } else {
11             soldiers.add("Bran");
12         }
13
14         System.out.println(soldiers);
15     }
16
17 }
```

- A. [Tyrion, Arya, John, Bran]
- B. [Tyrion, Arya, John, Cersei]
- C. java.lang.UnsupportedOperationException will thrown
- D. [Tyrion, Arya, John]

There is a crucial concept in the `List.of` method which is immutability. This means that we can't add, change or remove any element from the List when we use the `List.of()` method. Immutable objects make code more reliable and less susceptible to bugs.

Therefore, in the if statement we are checking if the list has more than three elements which is false. Then the else statement will be executed:
`soldiers.add("Bran");`

However, since we are trying to add an element to an immutable list, the following Exception will be thrown:

```
java.lang.UnsupportedOperationException
```

In conclusion, the correct alternative is:

- C) java.lang.UnsupportedOperationException will thrown

12.4 Vector

The `Vector` class is very similar to `ArrayList` with the difference that it is thread-safe. If you are working in a multi-thread environment and you don't want data collision with other thread processes then `Vector` can be a better option.

Since `Vector` is synchronized, it's also less performant than `ArrayList`. `Vector` also implements `List` which means that we can use polymorphism at the same way as an `ArrayList`.

Let's see a code example with `Vector`:

```

1  import java.util.List;
2  import java.util.Vector;
3
4  public class VectorExample {
5      public static void main(String[] args) {
6          List<String> vector = new Vector<>();           // #A
7          vector.add("Batman");
8          vector.add("Joker");
9          vector.add("Robin");
10         vector.add("Joker");
11
12         int indexOfJoker = vector.indexOf("Joker");      // #B
13         int lastIndexOfJoker = vector.lastIndexOf("Joker"); // #C
14
15         System.out.println(lastIndexOfJoker);
16         System.out.println(indexOfJoker);
17     }
18 }

```

Output:

```

1
3

```

- #A: We are instantiating the `List` with a `Vector` making use of polymorphism.
- #B: Then we are using the `indexOf` method that will return the first found element's index number. In our case, the first "Joker" is found at the index 1.
- #C: The `lastIndexOf` method is passing "Joker" and will return 3 because the last position "Joker" is found.

12.5 Deque & Stack

The `Deque` interface has all the capabilities of the legacy `Stack` class from Java 1.0. It's also recommended to use `Deque` instead of `Stack` whenever you need to use `Stack` functionalities.

The concept of stack is easy to absorb if we literally make an analogy of a stack of plates. Let's imagine you are piling up plates, it's much easier to put one plate on the top of the other, right? It's also easier to remove the first plate.

Just imagine how you would pile up your plates in your kitchen one on the top of the other, getting the easiest accessible plate from the pile (Last-in First-out):

```

_/_ <- Put the last plate on the top (Last-in)
_/_
_/_/_ <- Remove the first plate (First-out)
_/_/

```

As the name suggests, a Deque has the behaviours of a double ended queue, which means that we can use it as a stack or a queue. In other words, we can add or delete the first or the last element.

Let's see first how to use a Deque with stack capabilities (Last-in First-out):

```

1  import java.util.Deque;
2  import java.util.LinkedList;
3
4  public class DequeLinkedList {
5
6      public static void main(String[] args) {
7          Deque<String> deque = new LinkedList<>();    // #A
8          deque.push("Wolverine");
9          deque.push("Storm");
10         deque.push("Xavier");
11         deque.push("Juggernaut");                  // #B (Last-in)
12         deque.pop();                                // #C (First-out)
13         System.out.println(deque);
14     }
15 }

```

Output:

[Xavier, Storm, Wolverine]

- #A: Notice we are using the Deque interface as the type of the LinkedList implementation. By doing that, the LinkedList will have capabilities to work as a double-linked stack or queue.
- #B: The push method will insert the element at the top of the stack and also follows the Last-in principle. The push method does the same as the addFirst method. At this point of the code, this is how the deque will be:

[Juggernaut, Xavier, Storm, Wolverine]

- #C: The pop method follows the First-out behaviour which means that it will remove the first element of the stack. The pop method does the same as the removeFirst() method.

12.5.1 Methods Mapping from Deque and Stack

The Deque interface has methods that has different names from Stack but do the same thing. That happened because Stack is a legacy class. To not confuse their use, let's explore them here:

Stack	Deque	Explanation
push(e)	addFirst(e)	Adds an element to the top of the stack
pop()	removeFirst()	Removes the first element of the stack
peek()	getFirst()	Gets the first element of the stack

You can use all of the above-mentioned methods in the Deque interface to keep retro-compatibility with Stack but it's recommended to use the newer methods from Deque instead.

12.5.2 Queue & ArrayDeque

We can use all methods of a Queue when working with a Deque. However, if our intention is to only use Queue capabilities, it's better to make it explicit by declaring our list with Queue.

A Queue is easy to understand if we make the analogy of queue of people in a bank. The first person that will be attended will be the first to get out. It follows the FIFO (First-in First-out) principle as you can see in the following illustration:

```

_____
☒☒☒ | Bank |
/|\ /|\ /|\ |
/\ /\ /\ |
↑
First-in

```

```

_____
☒☒ | Bank |
/|\ /|\ |
/\ /\ |

```

```

☒
/|\
/\
↑
First-out

```

Now that you understand the principles of a queue let's see the code example with the ArrayDeque implementation. ArrayDeque is not thread-safe and it's faster than LinkedList to use queue capabilities:

```

1  import java.util.ArrayDeque;
2  import java.util.Queue;
3
4  public class QueueLinkedList {
5
6      public static void main(String[] args) {
7          Queue<String> heroesQueue = new ArrayDeque<>();           // #A
8          heroesQueue.offer("Wolverine"); // First-in             // #B
9          heroesQueue.add("Storm");                                   // #C
10         heroesQueue.offer("Xavier");
11         System.out.println(heroesQueue);
12         heroesQueue.remove(); // First-out                       // #D
13         heroesQueue.poll();                                       // #E
14         System.out.println(heroesQueue);
15     }
16 }

```

Output:

```

1  [Wolverine, Storm, Xavier]
2  [Xavier]

```

- #A: We can use `LinkedList` as the implementation if we wish to but for the case of a `Queue`, `ArrayDeque` is faster.
- #B: We are using the `offer` method that will simply add an element to the tail, in other words, to the end of the queue. It returns `true` in case the element was inserted in the queue and `false` otherwise depending in the queue elements' capacity.
- #C: We are also using the `add` that will add an element to the tail of the queue similarly to the `offer` method. The difference of the `add` method is that it will throw `IllegalStateException` in case there is no available elements' capacity in the queue.
- #D: The `remove` method will retrieve and remove the head element of the queue, in other words, it removes the first element. It throws `NoSuchElementException` in case the queue is empty. In this case, it will remove "Wolverine".
- #E: The `poll` method will retrieve and remove the head element of the queue, in other words, the first element. The `poll` method doesn't throw `IllegalStateException` as the `remove` method. Instead, it returns `null` in case the queue is empty. In this method invocation, "Storm" will be removed.

To summarize the most important methods of a `Queue`:

Queue	Explanation	Throws Exception due to capacity restrictions?
offer(e)	Adds an element to the tail of the queue	No
add(e)	It's similar to offer	Yes
poll()	Removes the first element (head) of the queue	No
remove()	It's similar to poll	Yes
peek()	Retrieves the first element without removing it	No
element()	It's similar to peek	Yes

12.5.3 Big-O of a Stack and Queue

Since the operations from a Stack and a Queue have the same time-complexity, let's check them together:

Operation	Average and worst case time Complexity	Explanation
add	O(1)	Adds element to the to the tail of the queue
delete	O(1)	Deletes the last stack's element node
access first/last	O(1)	Retrieves the first/last stack's element node
access an element	O(n)	Traverses node by node until the element is found
search a value	O(n)	Traverses node by node until the element's value is found

The space complexity, that means how much space in memory a stack will occupy in average is $O(n)$

12.5.4 Star Trek Deque Challenge

In the following Java code, we will explore the most common methods from a deque with Java. Can you guess what will be the output when running the following main method?

```

1 import java.util.Deque;
2 import java.util.LinkedList;
3
4 public class StarTrekDequeChallenge {
5
6     public static void main(String... confusedCrew) {
7         Deque<String> crew = new LinkedList<>();
8         crew.addFirst("Worf");
9         crew.push("Odo");

```

```

10     crew.addLast("Scott");
11
12     System.out.println(crew.remove());
13     System.out.println(crew.peek());
14     System.out.println(crew.pop());
15     System.out.println(crew.poll());
16 }
17
18 }

```

A. Scott
 Worf
 Worf
 Odo

B)Scott
 Odo
 Odo
 Worf

C. Worf
 Odo
 Odo
 Scott
 D. Odo
 Worf
 Worf
 Scott

Explanation:

The addFirst method will add the element "Worf" as the first element of the Deque.

The push method does the same as the addFirst method, it will add the "Odo" element to the first position and will push the other element to the second position. Therefore, we have "Odo" and "Worf".

The addLast method will add the element to the last position. Therefore, we have no "Odo", "Worf", and "Scott".

Then we will invoke the remove method that will remove the first element, we will also print "Odo".

The peek method will retrieve the first element but it won't remove it. We will then show the value of "Worf" without removing it.

Then we will invoke the `pop` method showing and removing the first element. Therefore, we will show and remove "Worf".

Finally, we will invoke the `pop` method that removes and return the last element of the deque. Then, the element "Scott" will be printed.

In conclusion the correct answer is:

D. Odo
Worf
Worf
Scott

12.5.5 LinkedList

The `LinkedList` class is doubly-linked list that is faster to insert and delete elements and slower to iterate than `ArrayList`. The `LinkedList` data structure is also a graph because it stores data in the first and last `Node` pointers:

```

1 public class LinkedList<E> extends AbstractSequentialList<E>
2     implements List<E>, Deque<E>, Cloneable, java.io.Serializable {
3     transient int size = 0;
4
5     /** Pointer to first node. */
6     transient Node<E> first;
7
8     /** Pointer to last node. */
9     transient Node<E> last;
10    // More attributes and methods...
11 }
```

`LinkedList` is a `Deque` (double ended queue) and is a `Queue`, therefore, we can also use the same capabilities of a `Queue`. A `Deque` can be used at the same way of a `Stack`, it's even recommended to use `Deque` instead of the legacy `Stack` class. Which means that we can use the `LinkedList` class as a `Stack` that follows the LIFO (Last-in first-out) principle. We also can use a `LinkedList` as a `Queue` that follows the FIFO (First-in first-out) principle.

12.6 ConcurrentModificationException

If we use the `ArrayList` implementation and try to add or remove elements while iterating over it, we will have the `ConcurrentModificationException`.

That happens because the `ArrayList`, `Vector`, or `LinkedList` have a fail-fast mechanism that will prevent data collision when multiple threads are accessing the same list Iterator. Instead of taking the risk of causing a bug very difficult to identify, the JDK code will throw `ConcurrentModificationException` to be defensive.

Therefore, whenever we try to add or remove elements with one of the above mentioned list implementations when iterating elements we will get this `ConcurrentModificationException`.

Let's see what happens if we try to remove an element from an `ArrayList` first:

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class RemoveListElement {
5
6      public static void main(String[] args) {
7          List<String> villains = new ArrayList<>();
8          villains.add("Joker");
9          villains.add("Magneto");
10         villains.add("Venom");
11
12         var villainToRemove = "Venom";
13
14         for (String eachVillain: villains) {
15             if (eachVillain.equals(villainToRemove)) {
16                 villains.remove(villainToRemove);
17             }
18         }
19
20         System.out.println(villains);
21     }
22 }

```

Output:

Exception in thread "main" java.util.ConcurrentModificationException

The same would happen if we instantiate our `List` with a `Vector`, `LinkedList`. Therefore, we should avoid the above actions if we are using any of those `List` implementations.

12.6.1 Preventing `ConcurrentModificationException` with `CopyOnWriteArrayList`

The `CopyOnWriteArrayList` completely avoids thread collisions because on every mutative operation such as (add, set, and others) it will make a fresh copy of the underlying array. Therefore it's impossible to get a `ConcurrentModificationException` when using `CopyOnWriteArrayList`:

```

1    // Method add from CopyOnWriteArrayList
2    public boolean add(E e) {
3        synchronized (lock) {
4            Object[] es = getArray();
5            int len = es.length;
6            es = Arrays.copyOf(es, len + 1); // Creates a copy of the array
7            es[len] = e;
8            setArray(es);
9            return true;
10       }
11   }

```

CopyOnWriteArrayList is too costly for common operations, however, it might be better when we want to use many operations such as add or delete while traversing the array than only mutating the array's element.

Let's see now a code example with CopyOnWriteArrayList:

```

1    // Omitted imports
2    public class CopyOnWriteArrayListHeroes {
3        public static void main(String[] args) {
4            List<String> heroes = new CopyOnWriteArrayList<>();
5            heroes.add("Iron Man");
6            heroes.add("Spider Man");
7
8            var heroToRemove = "Iron Man";
9            for (String eachHero: heroes) {
10               if (eachHero.equals(heroToRemove)) {
11                   heroes.remove(heroToRemove);
12                   heroes.add("Batman");
13               }
14           }
15           System.out.println(heroes);
16       }
17   }

```

Output:

[Spider Man, Batman]

As you can see in the example above, we can add and delete elements when iterating the list and nothing will happen. As explained before, this happens because in the add or delete methods a new array copy will be created preventing data collision.

12.6.2 Preventing ConcurrentModificationException with `removeIf`

The easiest way to delete an element from a list given a condition is to use the `removeIf` method.

If you don't need to implement a very complex logic while iterating elements from your array, the `removeIf` method will be usually the best option:

```
1 // Omitted imports
2 public class RemoveIfListElement {
3
4     public static void main(String[] args) {
5         List<String> villains = new ArrayList<>();
6         villains.add("Joker");
7         villains.add("Magneto");
8         villains.add("Venom");
9
10        villains.removeIf(e -> e.equals("Venom"));
11        System.out.println(villains);
12    }
13 }
```

Output:

[Joker, Magneto]

As you can see the code above is more succinct and does the same as the previous examples. Therefore, if you only want to remove an element given a condition, use the `removeIf` method.

12.7 Using Comparable

When we want to sort the elements of a `Collection` we can implement the `Comparable` interface in the element's object.

The `Comparable` interface is already implemented in most of the JDK important classes, such as, `String`, `StringBuilder`, `Integer`, `Double`, `BigDecimal`, and many others. This means that whenever we create a `Collection` with the above-mentioned types, we have the `Comparable` implementation for free.

Let's see how to sort elements of `List of String`:

```
1 List<String> heroes = new ArrayList<>();
2 heroes.add("Spider Man");
3 heroes.add("Iron Man");
4 heroes.add("Doctor Strange");
5
6 Collections.sort(heroes);
7
8 System.out.println(heroes);
```

Output:

[Doctor Strange, Iron Man, Spider Man]

Notice in the code above that we are using the `java.util.Collections` class to sort the list's elements. Another important point is that we could sort the elements from many other JDK classes and we would get the elements sorted.

Let's now analyze the method signature from the `sort` method:

```
public static <T extends Comparable<? super T>> void sort(List<T> list) {}
```

As you can see, the `sort` method can only receive objects that implements `Comparable`. Therefore, if we want to do the following, the code would not compile when we try to invoke the `sort` method:

```
1 List<Object> heroes = new ArrayList<>();
2 heroes.add("Hulk");
3 heroes.add("Captain America");
4 heroes.add("Thor");
5
6 Collections.sort(heroes);
```

As you can see, the `Object` class doesn't implement `Comparable`:

```
public class Object { ... }
```

12.7.1 Implementing Comparable

What if we want to create our own `Comparable` rules? We can do that for any new class we create so that we can easily sort elements.

In the following code, we are creating a `List` with `HeroComparable` elements and then sorting the elements from it.

Now, let's check the code:

```

1  import java.util.ArrayList;
2  import java.util.Collections;
3  import java.util.List;
4
5  public class HeroComparable implements Comparable<HeroComparable> { // #A
6
7      private String name;
8      private String power;
9
10     public HeroComparable(String name, String power) {
11         this.name = name;
12         this.power = power;
13     }
14
15     public static void main(String[] args) {
16         List<HeroComparable> heroes = new ArrayList<>();
17         heroes.add(new HeroComparable("Iron Man", "Repulsor Ray"));
18         heroes.add(new HeroComparable("Spider Man", "Webball"));
19         heroes.add(new HeroComparable("Iron Man", "Laser Beam"));
20         heroes.add(new HeroComparable("Hulk", "Monstrous Swipe"));
21         Collections.sort(heroes);
22         System.out.println(heroes);
23     }
24
25     @Override
26     public int compareTo(HeroComparable o) { // #B
27         int i = name.compareTo(o.name); // #C
28         if (i != 0) return i; // #D
29
30         return power.compareTo(o.power); // #E
31     }
32
33     @Override
34     public String toString() {
35         return "{name='" + name + "' , power='" + power + "'}";
36     }
37 }

```

Output:

```

HeroComparables:[{name='Hulk' , power='Monstrous Swipe'}, {name='Iron Man' ,
power='Laser Beam'}, {name='Iron Man' , power='Repulsor Ray'}, {name='Spider Man' ,
power='Webball'}]

```

- #A: The first important point to notice is that we are implementing the `Comparable` interface. This is required to be able to sort elements by using the `Collections.sort` method.
- #B: The `compareTo` method implementation from the `Comparable` interface.
- #C: We are using the `compareTo` method from the `String` name. It returns 1 if name is greater than `o.name`, 0 if they are equal or -1 if name is lower than `o.name`.
- #D: We just want to return a value here if `i` is different than 0 because then we won't need to compare the next field. If the comparison returns 0 though we will move to the next field comparison.
- #E: We return the comparison from `power` and `o.power` only if the name comparison returns 0.

12.7.2 Easier way to Compare Objects with Google Guava

If we want to rewrite our `compareTo` method with the library Google Guava the code would be that simple:

```

1  @Override
2  public int compareTo(HeroComparableGuava o) {
3      return ComparisonChain.start()
4          .compare(name, o.name, Ordering.natural().nullLast())
5          .compare(power, o.power, Ordering.natural().nullLast()).result();
6  }
```

Notice the important detail that we can also handle the edge-case of having a `null` field value with Guava. We have to make it explicit though, we have to add the third parameter to the `compare` method `Ordering.natural().nullLast()`, then a `null` field won't cause a `NullPointerException`.

12.7.3 Another easy way to compare objects with Apache Commons

Very similarly from the Guava's `ComparisonChain` class, we have the `CompareToBuilder` in the Apache Commons library:

```

1  public int compareTo(HeroComparableApacheCommons o) {
2      return new CompareToBuilder().append(name, o.name)
3          .append(power, o.power).toComparison();
4  }
```

The Apache Commons library handles `null` fields values automatically, we don't need to make it explicit. By default, the `CompareToBuilder` will sort a `null` value first.

Now it's up to you to decide what is the best way for you to sort your data.

12.7.4 Using Comparator

When you don't want to force a contract in your data object, you can use a `Comparator`. A `Comparator` is a handy way for you to sort elements without a need to implement the `compareTo` directly in your data object. It's also useful to use `Comparator` when you want to sort elements in more than one way. Another good reason to use `Comparator` is when you need to order your array by one field.

Let's see the classic way to use `Comparator` with an anonymous inner class:

```

1  import java.util.ArrayList;
2  import java.util.Comparator;
3  import java.util.List;
4
5  public class HeroComparator {
6      private String name;
7      private String power;
8
9      public HeroComparator(String name, String power) {
10         this.name = name;
11         this.power = power;
12     }
13
14     public static void main(String[] args) {
15         List<HeroComparator> heroes = new ArrayList<>();
16         heroes.add(new HeroComparator("SpiderMan", "Web ball"));
17         heroes.add(new HeroComparator("Iron Man", "Laser Beam"));
18         heroes.sort(new Comparator<HeroComparator>() {           // #A
19             @Override
20             public int compare(HeroComparator o1, HeroComparator o2) { // #B
21                 return o1.name.compareTo(o2.name);
22             }
23         });
24         for (var hero: heroes) { System.out.println(hero.name); }
25     }
26 }
```

Output:

```

Iron Man
SpiderMan
```

- #A: It's important to notice that we are using the `sort` method from the `list` interface. Also, we are using the `Comparator` interface as an anonymous inner class, this means that we have an instance without a name implementing the `compare` method. We are also passing the `HeroComparator` as a generic type so we can use this type in the `compare` method.

- #B: The `compare` method has two parameters instead of one of the `compareTo` method from `Comparable`. In essence the `compare` method is very similar to the `compareTo` method, it also returns an `int` number.

12.7.5 Comparator with Lambdas

In the code above, you probably noticed that using an anonymous inner class with the `Comparator` is very verbose.

`Comparator` is a functional interface though:

```
1 @FunctionalInterface
2 public interface Comparator<T> { ... }
```

That means we can use lambdas! We can make the above code much simpler with the same effect:

```
heroes.sort((o1, o2) -> o1.name.compareTo(o2.name));
```

We reduced the code of six to one line! That's the power of lambdas in Java.

But we can make it even better, we can use the helper comparing method from `Comparator`:

```
1 public static <T, U extends Comparable<? super U>> Comparator<T> comparing(
2     Function<? super T, ? extends U> keyExtractor)
3     { ... }
```

The code gets even shorter:

```
heroes.sort(Comparator.comparing(o -> o.name));
```

An important detail to notice when using a lambda is that if you have a `null` field value, you will get a `NullPointerException`. Therefore, you will have to handle this edge-case scenario.

12.7.6 Handling null Values with Comparator

If we want to handle `null` values with `Comparator`, there is the `Comparator.nullsFirst` method for that:

```
1 Comparator<HeroComparatorNullValue> employeeNameComparator
2     = Comparator.comparing(HeroComparatorNullValue::getName,
3     Comparator.nullsFirst(String::compareTo));
```

By using the above code, we can pass a `null` value to the field we are comparing and avoid a `NullPointerException`.

12.7.7 Comparator Java Challenge

In the following challenge, we will explore the use of a comparator with a list using method reference and lambdas. As we've seen before, we can make code more succinct by using the `comparing` method with lambdas and method references. Therefore, it's time to try out the following challenge and consolidate your knowledge!

By analyzing the following code, what do you think will be the output?

```

1  import java.util.ArrayList;
2  import java.util.Comparator;
3  import java.util.List;
4
5  public class JediComparatorChallenge {
6      public static void main(String... doYourBest) {
7          List<Jedi> jediList = new ArrayList<>();
8          jediList.add(new Jedi("Anakin", 10));
9          jediList.add(new Jedi("Luke", 5));
10         jediList.add(new Jedi("Luke", 6));
11         jediList.add(new Jedi("Obi Wan", 7));
12
13         Comparator<Jedi> comparator = Comparator
14             .comparing(Jedi::getName)
15             .thenComparing((a1,a2) -> a2.age.compareTo(a1.getAge()));
16
17         jediList.sort(comparator);
18         jediList.forEach(j -> System.out.println(j.name + ":" + j.age));
19     }
20     static class Jedi {
21         String name;
22         Integer age;
23         public Jedi(String name, Integer age) { this.name = name; this.age = age; }
24         public String getName() { return name; }
25         public Integer getAge() { return age; }
26     }
27 }

```

- A. Anakin:10
Luke:5
Luke:6
Obi Wan:7
- B. Anakin:10
Luke:6

```

    Luke:5
    Obi Wan:7
C. Obi Wan:7
    Luke:6
    Luke:5
    Anakin:10
D. Luke:6
    Luke:5
    Obi Wan:7
    Anakin:10

```

Explanation:

Let's analyze the following code:

```

1 Comparator<Jedi> comparator = Comparator
2     .comparing(Jedi::getName).thenComparing((a1,a2) -> a2.age
3     .compareTo(a1.getAge()));

```

Note in the first comparison that we are passing a method reference for the `getName` method that will a `Jedi` sort by name.

Then we use the `thenComparing` method that will also sort the age of a `Jedi`. However, notice that we are using a lambda that is sorting the age in the reverse:

```
a2.age .compareTo(a1.getAge())
```

Since we are positioning `a2` first, we will also revert the sorting order.

Therefore, the final answer will be:

```

B. Anakin:10
    Luke:6
    Luke:5
    Obi Wan:7

```

12.8 Set

The main purpose of the `Set` interface is to have unique elements in a `Collection`. Another crucial point of a `Set` is that the elements are not ordered, this means that the elements won't be in the same ordered as they are added.

The `Set` interface will heavily rely on the `equals` and `hashCode` methods to decide if elements are the same.

12.9 HashSet

The main implementation of the Set interface is the HashSet class. Behind the scenes a HashSet actually contains a HashMap as you can see in the following JDK code:

```
1 public class HashSet<E> extends AbstractSet<E>
2     implements Set<E>, Cloneable, java.io.Serializable {
3     @java.io.Serial
4     static final long serialVersionUID = -5024744406713321676L;
5
6     private transient HashMap<E, Object> map;
7
8     // Dummy value to associate with an Object in the backing Map
9     private static final Object PRESENT = new Object();
10
11     /**
12      * Constructs a new, empty set; the backing {@code HashMap} instance has
13      * default initial capacity (16) and load factor (0.75).
14      */
15     public HashSet() {
16         map = new HashMap<>();
17     }
18     // Omitted methods
19 }
```

The main purpose of a Set is to add unique elements. We can't also forget that to make it work we need to remember to override the equals and hashCode methods from the element we are working on.

A HashSet offers a constant time performance for the add method (it uses a HashMap behind the scenes). Assuming the size of some array is m - the overall time complexity is Big $O(m)$ (Algorithm time complexity).

Another important point of a HashSet is that it stores elements in a non-ordered way, which means that the order of insertion will be always random.

Let's see HashSet in action:

```

1  import java.util.HashSet;
2  import java.util.Set;
3
4  public class HashSetInAction {
5      public static void main(String[] args) {
6          Set<String> heroes = new HashSet<>();           // #A
7          heroes.add("Batman");                          // #B
8          heroes.add("Batman");                          // #C
9          heroes.add("Doctor Strange");
10         heroes.add("Wolverine");
11
12         System.out.println(heroes);                    // #D
13     }
14 }

```

Output:

[Wolverine, Batman, Doctor Strange]

Let's explore the code above:

- #A: Note that we are instantiating our Set with HashSet. Remember that this is a preferable approach since it decouples the HashSet class from the Set declaration. Because then we are able to use any Set implementation more easily which means the code will be easier to maintain.
- #B: We are adding "Batman" to the HashSet. This works fine since the String class implements the equals and hashCode methods. Remember that the HashSet class will check if the object is the same by using the equals and hashCode methods as you can see in the add implementation of the HashSet.
- #C: Notice that we are trying to add the same String "Batman" but this value will replace the old "Batman". Therefore, we won't have a duplication of "Batman".
- #D: Another crucial point is that a HashSet is not ordered, therefore, the order of insertion is not guaranteed. The result order will be random, it can be any of the following:

[Wolverine, Batman, Doctor Strange]

[Batman, Wolverine, Doctor Strange]

[Doctor Strange, Batman, Wolverine] ...

12.9.1 Borat HashSet equals hashCode Challenge

In this Java code challenge we will explore how the equals and hashCode are used internally in a HashSet. Keep in mind that a HashSet will use the essential concepts of the equals and hashCode methods to make elements insertion more performant.

By knowing that, can you guess what will happen when running the following code?

```

1  import java.util.HashSet;
2  import java.util.Set;
3
4  public class BoratHashSetChallenge {
5      public static void main(String... doYourBest) {
6          Set<Borat> borats = new HashSet<>();
7          borats.add(new Borat(1, "Kazakhstan"));
8          borats.add(new Borat(2, "USAndA"));
9          borats.add(new Borat(2, "England"));
10         borats.forEach(b -> System.out.print(b.id + " " + b.country + " "));
11     }
12     static class Borat {
13         int id;
14         String country;
15         Borat(int id, String country) { this.id = id; this.country = country; }
16
17         public int hashCode() {
18             System.out.println("hashCode:" + this.id + this.country);
19             return this.id;
20         }
21         public boolean equals(Object obj) {
22             System.out.println("equals:" + this.id + this.country);
23             return ((Borat) obj).country.equals(this.country);
24         }
25     }
26 }

```

- A. hashCode:1Kazakhstan
equals:1Kazakhstan
hashCode:2USAndA
equals:2USAndA
hashCode:2England
equals:2England
1 Kazakhstan 2 USAndA 2 England
- B. hashCode:1Kazakhstan
hashCode:2USAndA
1 Kazakhstan 2 USAndA
- C. hashCode:1Kazakhstan
hashCode:2USAndA
hashCode:2England
equals:2England
1 Kazakhstan 2 USAndA 2 England

```
D. hashCode:1Kazakhstan
   hashCode:2USAndA
   hashCode:2England
   1 Kazakhstan 2 USAndA 2 England
```

Explanation:

The key concept of this challenge is that the `HashSet` class relies on the `equals` and `hashCode` methods to decide if an element is the same or not.

Remember that the `hashCode` method will use less memory resources, therefore, it will be compared first. If the `hashCode` of objects are different, there is no necessity to check the `equals` method as well. Then when we insert "Kazakhstan" the `hashCode` method will be invoked but not the `equals` method. This will happen because the `hashCode` from the first and second element insertion are different.

At the third element insertion, note that the `id` is the same. This means that the `equals` method will have to be invoked to make sure if the objects are the same or not. So when we insert the third element, the `hashCode` and `equals` method will be invoked and since the `equals` method will return `false`, the third element will be inserted.

In the end we will print all the inserted elements. In conclusion, the right alternative is:

```
C. hashCode:1Kazakhstan
   hashCode:2USAndA
   hashCode:2England
   equals:2England
   1 Kazakhstan 2 USAndA 2 England
```

12.10 LinkedHashSet

What if we want to work with unique and ordered elements? Then we can use the `LinkedHashSet` implementation.

To talk details, the `LinkedHashSet` documentation says that the performance is likely to be a bit below than `HashSet` because the added expense of a `LinkedList`. There is one exception though, iteration over a `LinkedHashSet` requires time proportional to the size of a set regardless of its capacity.

Iteration over a `HashSet` is likely to be more expensive, requiring time proportional to its capacity.

A linked hash set has two parameters that affect its performance: initial capacity and load factor. They are defined precisely as for `HashSet`.

`LinkedHashSet` is also not synchronized, therefore thread synchronization must be done manually. It's also possible to wrap the `LinkedHashSet` with `Collections.synchronizedSet` to make it thread-safe.

Note that `LinkedHashSet` extends `HashSet`:

```

1 public class LinkedHashSet<E> extends HashSet<E>
2     implements Set<E>, Cloneable, java.io.Serializable { ... }

```

Let's see an example with `LinkedHashSet`:

```

1 import java.util.LinkedHashSet;
2 import java.util.Set;
3
4 public class LinkedHashSetInAction {
5     public static void main(String[] args) {
6         Set<String> heroes = new LinkedHashSet<>(); // #A
7         heroes.add("Batman");
8         heroes.add("Batman"); // #B
9         heroes.add("Doctor Strange");
10        heroes.add("Wolverine");
11        heroes.add("Spider Man");
12
13        System.out.println(heroes); // #C
14    }
15 }

```

Output:

[Batman, Doctor Strange, Wolverine, Spider Man]

- #A: We are instantiating our `Set` with `LinkedHashSet` which gives us the ability of adding elements in the order of insertion.
- #B: The `LinkedHashSet` will replace the value of "Batman" to the new one since it's the same.
- #C: The order is guaranteed to be always the insertion order.

12.10.1 Big-O of a `HashSet` & `LinkedHashSet`

The elements of a `HashSet` and `LinkedHashSet` are stored in an internal map manipulated by hash. Thanks to that, we get a constant time complexity when performing the following operations:

Method	Complexity Average/Worst	Explanation
Case Scenario		
<code>add(e)</code>	$O(1)$	Adds an element into a node
<code>contains(e)</code>	$O(1)$	Finds an element by hash in a <code>HashSet</code> 's map
<code>remove(e)</code>	$O(1)$	Removes an element by hash in the <code>HashSet</code> 's map

To summarize, `HashSet` and `LinkedHashSet` are fast to insert, remove, and search for elements. But

we can't access an element by index like the `ArrayList`.

12.11 TreeSet

To use unique elements that will be automatically sorted we can use `TreeSet`.

The `TreeSet` will also use the `hashCode` and `equals` methods to verify if objects are the same to insert unique elements. The great difference between `HashSet` and `TreeSet` is that the `TreeSet` will rely on the `compareTo` method to sort the elements. Also, the `TreeSet` only accepts `Comparable` objects, therefore whenever we try to add an element that isn't `Comparable` we will get a `java.lang.ClassCastException`.

The `TreeSet` class is also not thread-safe and it has the fail-fast check if we try to modify the set while iterating over. It also throws `ConcurrentModificationException`.

Now let's see what happens when we don't implement `Comparable` in the data class and try to add elements into a `TreeSet`:

```

1  import java.util.Set;
2  import java.util.TreeSet;
3
4  public class TreeSetNonComparable {
5      public static void main(String... args) {
6          Set<Simpson> set = new TreeSet<>();           // #A
7          set.add(new Simpson("Homer "));             // #B
8      }
9      static class Simpson {
10         String name;
11         public Simpson(String name) { this.name = name; }
12         public String toString() { return this.name; }
13     }
14 }

```

Output:

```
java.lang.ClassCastException
```

- #A: There is a warning at this line of code because the `Simpson` class doesn't implement `Comparable`. The restriction to not use an object that is not `Comparable` is not in the instantiation though, so this piece of code will work.
- #B: When we try to add "Homer" we will get a `java.lang.ClassCastException`. This happens because the `add` method implementation will verify if the element implements `Comparable`, and when it doesn't it throws the `Exception`.

12.11.1 Big-O of TreeSet

The `TreeSet` has $O(\log(n))$ time complexity for its operations since the nature of this collection is to keep elements sorted. As mentioned before, the algorithm of a `TreeSet` is the red-black tree.

Let's see the time-complexity of a `TreeSet` in the following table:

Method	Complexity Average/Worst Case Scenario	Explanation
add	$O(\log n)$	Adds an element and sorts <code>TreeSet</code>
contains	$O(\log n)$	It's necessary to traverse the Tree
next	$O(\log n)$	It's necessary to traverse the Tree

12.11.2 Using a Comparable object with TreeSet

Things will be different if we implement `Comparable` in our data class. By doing so, whenever we add an element into the `TreeSet`, the elements will be sorted according to what we programmed in the `compareTo` method:

```

1  import java.util.Set;
2  import java.util.TreeSet;
3
4  public class TreeSetComparable {
5      public static void main(String... doYourBest) {
6          Set<Simpson> set = new TreeSet<>();           // #A
7          set.add(new Simpson("Homer "));             // #B
8          set.add(new Simpson("Bart "));
9          set.add(new Simpson("Maggie "));
10
11         System.out.println(set);
12     }
13     static class Simpson implements Comparable<Simpson> { // #C
14         String name;
15         public Simpson(String name) { this.name = name; }
16         public String toString() { return this.name; }
17
18         @Override
19         public int compareTo(Simpson o) {             // #D
20             return this.name.compareTo(o.name);
21         }
22     }
23 }
```

Output:

[Bart , Homer , Maggie]

- #A: We are instantiating the `TreeSet` object with a `Comparable` `Simpson` this time, therefore, there won't be any warning informing otherwise.
- #B: The `add` method will work fine now that we are implementing `Comparable` in the `Simpson` class.
- #C: Notice that we are implementing `Comparable` in the `Simpson` data class which enables us to sort and compare this data object.
- #D: We are overriding the `compareTo` method comparing the `Simpson`'s name.

To summarize, you want to use `TreeSet` when you want a `Collection` with:

- Unique elements
- Sorted elements

12.11.3 Sort Comparable Simpson Challenge

In the following Java code challenge, you will see the `Set` interface with the `TreeSet` implementation. The core purpose of a `TreeSet` is to sort unique elements according to what is implemented in the `compareTo` method of an `Object`. Note that we are also using the `reverse` method.

Therefore, what will be the output of the following code when running the main method?

```

1  import java.util.*;
2
3  public class SortComparableChallenge {
4      public static void main(String... doYourBest) {
5          Set<Simpson> set = new TreeSet<>();
6          set.add(new Simpson("Homer "));
7          set.add(new Simpson("Marge "));
8          set.add(new Simpson("Lisa "));
9          set.add(new Simpson("Bart "));
10         set.add(new Simpson("Maggie "));
11
12         List<Simpson> list = new ArrayList<>(set);
13         Collections.reverse(list);
14         for (Simpson simpson : list)
15             System.out.print(simpson);
16     }
17
18     static class Simpson implements Comparable<Simpson> {
19         String name;
20         public Simpson(String name) { this.name = name; }
21     }

```

```

22     public int compareTo(Simpson simpson) {
23         return simpson.name.compareTo(this.name);
24     }
25
26     public String toString() { return this.name; }
27 }
28 }

```

- A. Bart Homer Lisa Maggie Marge
- B. Marge Maggie Lisa Homer Bart
- C. Maggie Bart Lisa Marge Homer
- D. Homer Marge Lisa Bart Maggie

Firstly in this Java Challenge we are adding elements into an instance of a `TreeSet`. When we do that, we will sort the elements according to what was implemented in the `compareTo` method.

At the moment we added all the elements in the `TreeSet`, the elements will be already inverted as the following:

Marge Maggie Lisa Homer Bart

You might be wondering why the elements were inverted after adding elements in the `TreeSet`. However, note that in the `compareTo` method from the `Simpson` class, we are inverting the comparison. We are using first the `simpson` parameter, invoking the `compareTo` method and then passing the current instance name. This means that we are actually inverting the sorting of the `TreeSet` elements.

In the following code, we are encapsulating our set into a list because we want to invoke the reverse method:

```
List<Simpson> list = new ArrayList<>(set);
```

Then, when we invoke the reverse method from the List, we will reverse what is already reversed:

```
Collections.reverse(list);
```

Which means that the order of the elements after we print will be the ascending order as the following:

Bart Homer Lisa Maggie Marge

Therefore, the correct alternative of this Java Challenge is:

- A) Bart Homer Lisa Maggie Marge

12.12 Map

Very often in the day-to-day work of a Java developer, it will be necessary to work with a key-value Collection. To accomplish that there is the `Map` interface.

The main implementations of the Map interface are HashMap, LinkedHashMap, and TreeMap. To have a macro-view of the Map hierarchy, let's explore the following diagram:

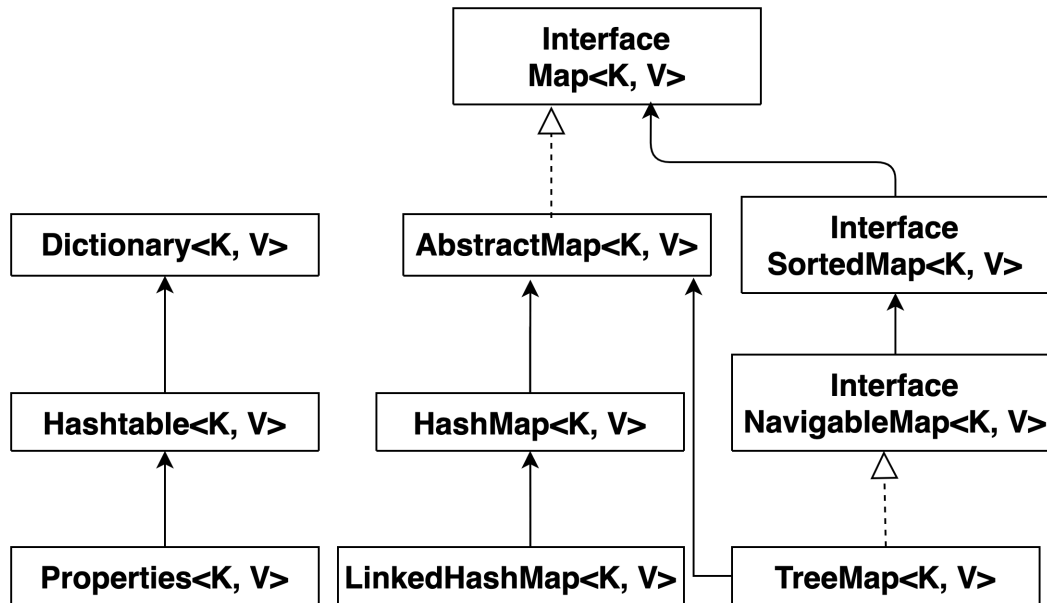


Figure 11.2 Map diagram

12.13 HashMap

The most used Map in Java is the HashMap. It has the capability of adding key value elements and it's also not ordered.

Keep in mind also that a map is a graph under the hood. A HashMap will store data into a Node as you can see in the JDK code:

```

1 public class HashMap<K,V> extends AbstractMap<K,V>
2     implements Map<K,V>, Cloneable, Serializable {
3     // Omitted other attributes and methods
4     transient Node<K,V>[] table;
5 }

```

12.13.1 Adding and removing Map elements

Let's see the basic usage of a HashMap with Java:

```

1  import static java.lang.System.out;
2  import java.util.HashMap;
3  import java.util.Map;
4
5  public class HashMapPutRemove {
6      public static Map<Integer, String> getFinalFantasyCharacters() {
7          Map<Integer, String> map = new HashMap<>();           // #A
8          map.put(1, "Cloud");                                   // #B
9          map.put(2, "Tifa");
10         map.put(3, "Barret");
11         return map;
12     }
13     public static void main(String[] args) {
14         Map<Integer, String> map = getFinalFantasyCharacters();
15
16         out.println(map.remove(2));                             // #C
17         out.println(map.putIfAbsent(2, "Vincent"));             // #D
18         out.println(map.keySet());                             // #E
19
20         for (Map.Entry<Integer,String> entry : map.entrySet()) // #F
21             System.out.println("Key=" + entry.getKey() +      // #G
22                                ", Value = " + entry.getValue());
23     }
24 }

```

Output:

Tifa

null

[1, 2, 3]

Key:1 Value:Cloud Key:2 Value:Vincent Key:3 Value:Barret

Code Analysis:

- #A: Instantiate our Map with a HashMap. We could instantiate this Map with any other Map implementation.
- #B: Puts into the map the key of 1 and value of "Cloud".
- #C: Remove the second key value of the map. In this case, 2 and "Tifa".
- #D: Put an element if the key is not existent in the map. Since we removed the element 2, this method will put the key of 2 and the value of "Vincent". It also returns null because there isn't any element with the key of 2. Therefore, it will also print null.
- #F: The map has the entrySet method that will return the key and value to the Map.Entry. Therefore, we can use the for looping to get each entrySet of the map. This way to iterate a map is a bit verbose though but in the next session we will see a concise way to do so with lambdas.
- #G: We can access the key and value from the current entrySet.

12.13.2 Putting All and Clearing Elements

In case we need to put all elements from a map into another map and clear elements, we have methods to accomplish that.

Let's see a code example:

```

1  public static void main(String[] args) {
2      Map<Integer, String> ffCharacters = new HashMap<>();
3      ffCharacters.put(1, "Cloud");
4      ffCharacters.put(2, "Tifa");
5      Map<Integer, String> moreFFCharacters = new HashMap<>();
6      moreFFCharacters.put(2, "Sephiroth");           // #A
7      moreFFCharacters.put(3, "Zack");
8      moreFFCharacters.put(4, "Aerith");
9
10     ffCharacters.putAll(moreFFCharacters);           // #B
11     ffCharacters.forEach((k, v) -> out.print(" Key: " + k + " Value: " + v));
12
13     ffCharacters.clear();                             // #C
14     ffCharacters.forEach((k, v) -> out.print(" Key: " + k + " Value: " + v));
15 }

```

Output:

Key: 1 Value: Cloud Key: 2 Value: Sephiroth Key: 3 Value: Zack Key: 4 Value: Aerith

Code Analysis:

- #A: Adding "Sephiroth" to the key 2. When we put the same key into a map the value will be replaced. In this case, "Tifa" will be replaced with "Sephiroth".

- #B: Then we use the `putAll` method that will put all the elements into the `ffCharacters` map. Therefore, we will have all the elements from `ffCharacters` and `moreFFCharacters` in the `ffCharacters` map.
- #C: As the name suggests, the `clear` method will remove all the elements from a map. Therefore, when we try to print the values, nothing will be printed.

12.13.3 Getting values from a Map

We can't do much with a map if we can't access its values. Therefore, let's explore the most important methods to retrieve data from a map. In the following code, we will see how to get a value by key, all values, and all keys:

```

1  // Omitted imports and getFinalFantasyCharacters()
2  public class HashMapGetKeyValue {
3      public static void main(String[] args) {
4          Map<Integer, String> map = getFinalFantasyCharacters();
5
6          out.println(map.get(1));           // #A
7          out.println(map.getOrDefault(4, "Vincent")); // #B
8          out.println(map.values());         // #C
9          out.println(map.keySet());         // #D
10     }
11 }

```

Output:

```

Cloud
Vincent
[Cloud, Tifa, Barret]
[1, 2, 3]

```

Code Analysis:

- #A: Retrieve the value of the key of 1. Therefore, it returns "Cloud".
- #B: Returns the default value we pass in case the key is not existent in the map. Therefore, it will return "Vincent".
- #C: We will print all the values contained in the map.
- #D: It will print the keySet of the map. Therefore, the output will be: [1, 2, 3]

12.13.4 Checking if a value is existent in a Map

When we are working with a map, we often will want to check if a value or key is existent. To do so we can use the contains methods. We can also retrieve values with the get method. Let's see how to do those actions in the following code:

```

1  // Omitted imports and getFinalFantasyCharacters()
2  public class HashMapContainsKeySet {
3      public static void main(String[] args) {
4          Map<Integer, String> map = getFinalFantasyCharacters();
5
6          out.println(map.containsKey(3));           // #A
7          out.println(map.containsValue("Cloud"));   // #B
8      }
9  }

```


Output:

true

true

Code Analysis:

- #A: Check if the map has the key of 3. In this case, the key of 3 is there. Therefore it returns true.
- #B: Check if the value of "Cloud" is present in the map object. In this case, "Cloud" is there. Therefore it returns true.

12.13.5 Iterating over a HashMap

There are many ways to iterate over a map, even though we don't need to use all of them it's good to at least know they exist. Therefore, let's see some of the main ways to iterate over a map.

12.13.6 Iterating with Map.Entry and iterator

The most verbose way to iterate over a map is by using iterator explicitly. Note that a map under the hood stores the key values into a Set as you can see in the code below:

```

1 public class HashMap<K,V> extends AbstractMap<K,V>
2     implements Map<K,V>, Cloneable, Serializable {
3
4     // Omitted attributes and methods
5     transient Set<Map.Entry<K,V>> entrySet;
6
7 }
```

Since we are using a Set to store the keys of the HashMap, we have access to the iterator method since a Set implements Iterator. Take a look at the hierarchy from Set up until Iterator:

```

1 public interface Set<E> extends Collection<E> { ... }
2 public interface Collection<E> extends Iterable<E> { ... }
3 public interface Iterable<T> { ... }
```

Let's see now how we use the iterator with map elements:

```

1  Map<Integer, String> avalanche = new HashMap<>();
2  avalanche.put(1, "Cloud");
3  avalanche.put(2, "Barret");
4  avalanche.put(3, "Tifa");
5
6  Iterator<Entry<Integer, String>> it = avalanche.entrySet().iterator(); // #A
7  while (it.hasNext()) { // #B
8      Map.Entry<Integer, String> entry = it.next(); // #C
9      System.out.println("Key: " + entry.getKey() + // #D
10                          "Value: " + entry.getValue());
11 }

```

Output:

Key: 1 Value: Cloud

Key: 2 Value: Barret

Key: 3 Value: Tifa

- #A: We access the `entrySet` from the map and then the iterator. Note that we receive this iterator typed as `<Entry<Integer, String>>`.
- #B: Then once we have the iterator elements, now we can iterate. To accomplish that we can use the `hasNext` method in a `while` loop. This is what the compiler does behind the scenes when we use the enhanced `for` statement as well.
- #C: We get the next element with the `next` method from the iterator.
- #D: Finally, we print the key and value from the entry object.

12.13.7 Iterating with `foreach` and `Map.Entry`

We can improve the code above by using the enhanced `for` looping. Instead of accessing the iterator object explicitly, we can do that implicitly by using the following code:

```

1  // Omitted the adding of elements in the map
2  for (Map.Entry<Integer, String> entry : avalanche.entrySet()) { // #A
3      System.out.println("Key:" + entry.getKey() + // #B
4                          " Value: " + entry.getValue());
5  }

```

Output (Same as above)

- #A: By using the enhanced `for` looping we use the iterator implicitly. When compiling this code though, the iterator will be used behind the scenes. To make it clear, this is the code that will be actually used when compiling this code:

```

1 String result = "";
2 Iterator var2 = avalanche.entrySet().iterator();
3 while(var2.hasNext()) {
4     Entry<Integer, String> entry = (Entry)var2.next();

```

- #B: We print the values as above.

12.13.8 Iterating Map with keys

This is another handy way to iterate a Map, we return the set of keys and get the value associated with the key.

Let's see this example in code:

```

1 // Omitted the adding of elements in the map
2 for (int key : avalanche.keySet()) {                                     // #A
3     System.out.println("Key:" + key + " Value:" + avalanche.get(key)); // #B
4 }

```

- #A: We get the set of keys of the map by using the `keySet` method.
- #B: We print the key directly and then retrieve the value of each key by using the `avalanche.get(key)` method.

Keep in mind to use the `keySet` iteration when you are using only the key, otherwise the `entrySet` method is more efficient in terms of performance.

12.13.9 Iterating with `forEach` from Java 8

The most succinct way to iterate over a map is by using the `forEach` method. That's because we can use a lambda expression and if we have a one-liner logic, for sure this option will be the best.

It's not so recommended to use more than one line of code in a lambda but you still can do that depending on the situation.

Let's see how to use a one-liner `forEach`:

```

1 // Omitted the adding of elements in the map
2 avalanche.forEach((k, v) -> System.out.println("Key:" + k + " Value: " + v));

```

Output (Same as above)

This is how the `forEach` method is declared in the `Map` interface:

```
1 default void forEach(BiConsumer<? super K, ? super V> action) { ... }
```

Note that we receive a `BiConsumer` that enables us to pass two parameters of any type and do some logic without returning any value. That's why in our lambda we pass two parameters, we consume two values on each iteration of the looping.

12.13.10 Computing Map Values

If we want to manipulate a value in a map by key and using lambda, we can use the `compute` method. The `compute` method is handy because we can access an specific key of the map and apply a logic we want:

```
1 // Omitted imports and getFinalFantasyCharacters()
2 public class HashMapCompute {
3     public static void main(String[] args) {
4         Map<Integer, String> ffCharacters = getFinalFantasyCharacters();
5         ffCharacters.compute(1, (k,v) ->                                // #A
6             Optional.ofNullable(v).filter(e -> e.equals("Cloud"))        // #B
7                 .map(e -> e + " with Ultima Weapon").orElse("No weapon")); // #C
8         out.println(ffCharacters.get(1));
9     }
10 }
```

Output:

Cloud with Ultima Weapon

- #A: Notice that we are computing the key and value of `ffCharacters` from of the key 1'.
- #B: We encapsulate the possibly null value we are receiving in the `compute` method. Then we filter the map value to "Cloud" only.
- #C: If the value from the map is "Cloud" we concatenate " with Ultima Weapon", otherwise, "No weapon".

12.13.11 merge method

To merge two maps having more control over it, we can use the `merge` method introduced in Java 8. This method is useful because we can apply some logic on each element even if they are duplicated.

Notice that the `merge` method will be only invoked if the key of the maps are the same. If the keys are different, the key and value from the other map will be put into the merger map:

```

1  import java.util.HashMap;
2  import java.util.Map;
3
4  // Omitted imports
5  public class HashMapMerge {
6
7      public static void main(String[] args) {
8          Map<Integer, String> ffSummons = new HashMap<>();
9          ffSummons.put(1, "Ifrit");
10         ffSummons.put(2, "Bahamut");
11         ffSummons.put(3, "Knights of Round");
12
13         Map<Integer, String> ffParty = new HashMap<>();
14         ffParty.put(1, "Cid");
15         ffParty.put(2, "Yuffie");
16         ffParty.put(4, "Zack");
17
18         ffSummons.forEach((k, v) ->                                // #A
19             ffParty.merge(k, v, (fighter, summon)                  // #B
20                 -> fighter + " invokes " + summon));                // #C
21
22         System.out.println(ffParty);
23     }
24 }

```

Output:

- #A: We iterate over the elements of `ffSummons`, giving us access to the key and value from them.
- #B: On each iteration we invoke the `merge` method in the `ffParty` map. We pass the key and value from `ffSummons` and the last parameter is a `BiConsumer` where we can pass the value from `ffParty` and `ffSummons`. If the key we are passing is existent, then the `merge` method is invoked. Otherwise, the key and value of `ffSummons` will be put into the `ffParty` map.
- #C: We perform the logic of our `BiConsumer`. In this case, we are basically concatenating the value of `ffParty` with `ffSummons`.

12.13.12 Lannister HashMap Java Challenge

The data structure of a `HashMap` is very important to handle key-value elements. To absorb this concept well, we will explore a Java code challenge where we make use of the `equals` and `hashCode` methods.

Therefore, do you know what happens when we put elements into a `HashMap`?

```

1  import java.util.HashMap;
2  import java.util.Map;
3
4  public class LannisterMapChallenge {
5      static int hashCodeCount = 0;
6      static int equalsCount = 0;
7
8      public static void main(String[] doYourBest) {
9          Map<Lannister, String> m = new HashMap<>();
10         m.put(new Lannister("Tyrion"), "I understand the way this game is played");
11         m.put(new Lannister("Jaime"), "The things I do for love...");
12         m.put(new Lannister("Cersei"), "When you play the game of thrones, you win '
13             + "or you die. There is no middle ground.");
14         m.put(new Lannister("Tywin"), "Any man who must say, 'I am the king' is no t
15 ing.");
16
17         System.out.printf("Size: %s equalsCount: %s hashCodeCount: %s", m.size(), eo
18  unt, hashCodeCount);
19     }
20
21     static class Lannister {
22         String name;
23         public Lannister(String name) {this.name = name;}
24         public boolean equals(Object obj) {
25             equalsCount++;
26             return (name.equals(name));
27         }
28         public int hashCode() {
29             hashCodeCount++;
30             return 1 + 8 * 453453 / 77 + 40340403 - 3000000;
31         }
32     }
33 }

```

- A. Size: 2 equalsCount: 2 hashCodeCount: 4
- B. Size: 1 equalsCount: 3 hashCodeCount: 4
- C. Size: 4 equalsCount: 4 hashCodeCount: 3
- D. Size: 3 equalsCount: 1 hashCodeCount: 3

Explanation:

Notice that when we are inserting a key to a HashMap, it will rely on the hashCode and equals methods to define whether or not a key will be inserted or replaced.

Also, notice that the `hashCode` method in the `Lannister` class is always returning the same value. This means that every key in this `HashMap` will have the same `hashCode`. The `equals` method is comparing the same name from `Lannister`, we are not using the `obj` passed by parameter.

The `hashCode` method returns the same number and the `equals` method will always return `true`, which means that the elements in the `HashMap` will be replaced.

Also, when we put "Tyrion" in the `HashMap`, note that only the `hashCode` method will be invoked because this value is used when inserting the key. However, the `equals` method will not be invoked because there isn't any other object to be compared at the moment.

Then, when we put "Jaime" the `hashCode` method will be invoked but also the `equals` method since we have more than 1 element in the `HashMap`. The same will happen for "Cersei" and "Tywin".

In conclusion, the correct alternative is:

B. Size: 1 equalsCount: 3 hashCodeCount: 4

12.14 Hashtable

The `Hashtable` class behaves very similar from a `HashMap` with the main difference that is synchronized, this means it can be used in a multi-thread environment safely.

Notice also that a `Hashtable` doesn't allow `null` values as key. If you pass a `null` value like the following:

```
1 Map<String, String> map = new Hashtable<>();
2 map.put(null, "value");
```

The output will be:

```
1 Exception in thread "main" java.lang.NullPointerException
2     at java.base/java.util.Hashtable.put(Hashtable.java:476)
```

Another important difference is that `Hashtable` extends an obsolete abstract class `Dictionary`. This means we should declare the type of a `Hashtable` as a `Map` and not as a `Dictionary`.

12.15 LinkedHashMap

If you want to use an ordered map, the `LinkedHashMap` implementation will do this job for you. This means that the elements will be ordered as they are inserted.

The `LinkedHashMap` maintains a doubly-linked list running through all of its entries. It's also slightly less performant than a `HashMap`.

Another important point is that the `LinkedHashMap` is not synchronized or thread-safe. This means that data collision may happen when different thread executions are changing data from a `LinkedHashMap`.

The `LinkedHashMap` also has a fail-fast mechanism in case we change modify values while iterating in a loop. That's a defensive approach to prevent hard to track bugs due to data loss.

Another important detail from the `LinkedHashMap` class is that it extends the `HashMap` class and implements a `Map`:

```
1 public class LinkedHashMap<K,V> extends HashMap<K,V> implements Map<K,V>
2 { ... }
```

Let's see the `LinkedHashMap` in action:

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 // Omitted imports
5 public class LinkedHashMapInAction {
6     public static void main(String[] args) {
7         Map<Integer, String> map = new LinkedHashMap<>();           // #A
8
9         for (var i = 0; i <= 100000; i++) {                         // #B
10             map.put(i, "Element:" + i);
11         }
12         out.println(map);
13     }
14 }
```

Output

- #A: We can declare a `LinkedHashMap` with the `Map` interface and make use of polymorphism. The benefit of using polymorphism is the flexibility you have in your code to pass the implementation you find more suitable.
- #B: Notice that we are iterating from 0 to 100000 and the output will be guaranteed in insertion order. If you need your elements to be in insertion order, use `LinkedHashMap`.

12.15.1 Big-o of HashMap, LinkedHashMap, Hashtable, and ConcurrentHashMap

When we use hash functions, we can rapidly put, access, remove, and search elements. However, if there is hash collision the time-complexity will be $O(n)$.

Let's see the time-complexity of `HashMap`, `LinkedHashMap`, and `ConcurrentHashMap` in more details:

Method	Complexity	Worst Case Scenario	Explanation
put()	O(1)	O(n)	O(n) when there is hash collision
get()	O(1)	O(n)	O(n) if there is hash collision
remove()	O(1)	O(n)	O(n) if there is hash collision
contains()	O(1)	O(n)	O(n) if bad hash function is being used

12.16 TreeMap

To have the keys of your map sorted in natural, you can use the `TreeMap`.

A `TreeMap` uses the Red-black tree algorithm based on the `NavigableMap` implementation. The keys of a `TreeMap` must implement `Comparable`, in case we don't do that, we will get an `Exception`.

The `TreeMap` also has the following direct hierarchy:

```

1 public class TreeMap<K,V> extends AbstractMap<K,V>
2     implements NavigableMap<K,V>, Cloneable, java.io.Serializable
3 { ... }
```

A `TreeMap` is also not synchronized, therefore, it will throw `ConcurrentModificationException` in case the map is changed during an iteration as a fail-safe mechanism. However, it's also possible to use a `TreeMap` as a thread-safe object by wrapping in the `Collections.synchronizedSortedMap` method.

Now let's see a common use of a `TreeMap` adding `Comparable` elements and then making sure they were sorted in their natural order:

```

1 import java.util.Map;
2 import java.util.TreeMap;
3
4 public class TreeMapInAction {
5     public static void main(String[] args) {
6         Map<Integer, String> ffSummons = new TreeMap<>();
7         ffSummons.put(3, "Shiva");           // #A
8         ffSummons.put(2, "Cactuar");        // #B
9         ffSummons.put(1, "Siren");
10
11         System.out.println(ffSummons);
12     }
13 }
```

Output:

- #A: The first important to notice is that the key has to implement `Comparable`. As we are passing the key as a primitive `int` number, it will be autoboxed to `Integer` that implements `Comparable` as you can see in the `Integer` class definition:

```
1 public final class Integer extends Number
2     implements Comparable<Integer>, Constable, ConstantDesc { ... }
```

The value doesn't need to implement `Comparable` since values are not sorted, only keys.

- #B: When we put the key of 2, the `TreeMap` will automatically sort the elements in the natural order. Therefore, at that moment we have:

12.16.1 Reversing the natural order with `TreeMap`

```
1 import java.util.Comparator;
2 import java.util.Map;
3 import java.util.TreeMap;
4
5 public class TreeMapReverse {
6     public static void main(String[] args) {
7         Map<Integer, String> ffSummons =
8             new TreeMap<>(Comparator.reverseOrder()); // #A
9         ffSummons.put(1, "Siren");                      // #B
10        ffSummons.put(2, "Cactuar");
11        ffSummons.put(3, "Shiva");
12
13        System.out.println(ffSummons);
14    }
15 }
```

Output:

- #A: If we use the `Comparator.reverseOrder()` instance, we will invert the order of the `Comparable` objects comparison as you can see in the following JDK code:

```

1 private static class ReverseComparator
2     implements Comparator<Comparable<Object>>, Serializable {
3     public int compare(Comparable<Object> c1, Comparable<Object> c2) {
4         return c2.compareTo(c1);
5     }
6 }

```

Therefore, the results will be returned with the key order sorted in the reverse order from the Integer natural order.

- #B: Then, whenever we add an element to the TreeMap they will sorted in the reverse order.

Output:

12.16.2 Exception Cases with Non-comparable Objects with TreeMap

Be aware that if you use an object that is not Comparable you will get an Exception. This happens because as explained before, the TreeMap will use the compareTo method from Comparable to sort the map.

Let's see what happens when the object doesn't implement Comparable:

```

1 import java.util.Map;
2 import java.util.TreeMap;
3
4 public class TreeMapExceptionCase {
5     public static void main(String[] args) {
6         Map<Summon, Integer> ffSummons = new TreeMap<>(); // #A
7         ffSummons.put(new Summon("Carbuncle"), 1); // #B
8         ffSummons.put(new Summon("MiniMog"), 2);
9         ffSummons.put(new Summon("Tonberry"), 3);
10
11         System.out.println(ffSummons);
12     }
13 }
14
15 class Summon { // #A
16     private String name;
17     public Summon(String name) { this.name = name; }
18 }

```

Output:

```

1 Exception in thread "main" java.lang.ClassCastException: class com.javachallengers.c\
2 collections.map.Summon cannot be cast to class java.lang.Comparable (com.javachalleng\
3 ers.collections.map.Summon is in unnamed module of loader 'app'; java.lang.Comparabl\
4 e is in module java.base of loader 'bootstrap')...

```

- #A: We are passing the Summon class that doesn't implement Comparable to the TreeMap key. If you are using the IntelliJ IDE you will probably see a warning informing that the key is not a Comparable object.
- #B: When we try to add the first element to the TreeMap we will get the java.lang.ClassCastException since the put method will try to use a Comparable key but it won't succeed.

12.16.3 Using Customized Comparable Objects with TreeMap

```

1 import java.util.Map;
2 import java.util.TreeMap;
3
4 public class TreeMapCustomizedComparable {
5     public static void main(String[] args) {
6         Map<Summon, String> ffSummons = new TreeMap<>();           // #A
7         ffSummons.put(new Summon("Tonberry"), "Guardian Force");   // #B
8         ffSummons.put(new Summon("MiniMog"), "Guardian Force");
9         ffSummons.put(new Summon("Carbuncle"), "Guardian Force");
10
11         System.out.println(ffSummons);
12     }
13
14     static class Summon implements Comparable<Summon> {           // #C
15         private String name;
16         public Summon(String name) {
17             this.name = name;
18         }
19
20         public int compareTo(Summon o) {                           // #D
21             return this.name.compareTo(o.name);
22         }
23         public String toString() {
24             return "Summon{" + "name='" + name + '\'' + '}';
25         }
26     }
27 }

```

Output:

- #A: There won't be any warning here since the `Summon` class now implements `Comparable`.
- #B: Notice that on each time we put elements into `ffSummons`, the `compareTo` method will be invoked to sort the map keys.
- #C: We are implementing `Comparable` now in the `Summon` class. Notice that we are also using the generic type of `Summon` in the `Comparable`. This is because then we can use `Summon` in the `compareTo` method implementation.
- #D: Here we compare the current instance with another one that is received by parameter. This method will be invoked by the `TreeMap` on every put invocation. Our logic is quite simple here, we are comparing the name of the current instance with the other `Summon` object.

12.16.4 Big-O of a TreeMap

A `TreeMap` will sort the keys of a map when elements are inserted. Therefore, the time-complexity is more costly than a `HashMap` for example. The time-complexity for operations with a `TreeMap` is $O(\log n)$ which means it's less efficient than constant time $O(1)$.

The underlying algorithm used in a `TreeMap` is the Red-Black Tree, the same as used in a `TreeSet`.

Let's see the time-complexity table of a `TreeMap`:

Method	Complexity	Worst Case Scenario
<code>put()</code>	$O(\log n)$	$O(\log n)$
<code>get()</code>	$O(\log n)$	$O(\log n)$
<code>remove()</code>	$O(\log n)$	$O(\log n)$
<code>contains()</code>	$O(\log n)$	$O(\log n)$

12.16.5 Stark Map equals Challenge

In the following Java Code Challenge, you will see a `Map` instantiated with a `LinkedHashMap`. Then we will add some elements, a `Stark` as a key and a number as a value. Note that the `Stark` class has the `equals` and `hashCode` methods overridden.

What will happen when running the following code?

```

1 public class StarkEqualsChallenge {
2     public static void main(String... doYourBest) {
3         Map<Stark, String> map = new LinkedHashMap<>();
4
5         map.put(new Stark("Arya"), "1"); map.put(new Stark("Ned"), "2");
6         map.put(new Stark("Sansa"), "3"); map.put(new Stark("Bran"), "4");
7         map.put(new Stark("Jaime"), "5");
8
9         map.forEach((k, v) -> System.out.print(v));
10    }

```

```

11     static class Stark {
12         String name;
13         Stark(String name) {this.name = name;}
14
15         public boolean equals(Object obj) {
16             return ((Stark)obj).name.length() == this.name.length();
17         }
18
19         public int hashCode() { return 4000 << 2 * 2000 / 10000; }
20     }
21 }

```

- A. 123
- B. 12345
- C. 245
- D. 425

Explanation:

Note that in this Java Challenge we are overriding the equals and hashCode methods, therefore the Map implementations will behave accordingly to what was implemented on those methods.

Another important point to note is the hashCode method that even though it's a bit complex, it's returning always the same value:

```
public int hashCode() { return 4000 << 2 * 2000 / 10000; }
```

Pay attention that the equals method is actually comparing the length of the String. So, objects are going to be the same if the String length is the same.

```

1 Arya has 4 characters - it will be inserted
2 Ned has 3 characters - it will be inserted
3
4 Sansa has 5 characters - it will be inserted
5 Bran has 4 characters - it will replace Arya
6
7 Jaime has 5 characters - it will replace Sansa

```

As Bran has 4 characters and it was the last one to be inserted, it will replace Arya. Therefore the first value is 4.

As Ned is the only one with 3 characters, his value will be printed that is 2.

Then, Sansa has 5 characters, it will be inserted and then Jaime will replace Sansa's value with 5.

Therefore, the correct alternative is:

D) 425

12.17 Elements Searching

What if we want to search for an element in an array? There is a very famous algorithm that does that for us, and this is the **binary search** algorithm. Most programming languages have this algorithm already implemented and in Java this is not different.

An important thing to keep in mind is that the elements must be sorted if we want to use the binary search algorithm. That's because the binary search algorithm will break the array in two and will verify in what side is the searched element.

12.17.1 Searching elements with a non-sorted List

Let's see a code example where we use the `binarySearch` method without sorting the List:

```
1 import java.util.Collections;
2 import java.util.List;
3
4 public class BinarySearchNotSorted {
5     public static void main(String[] args) {
6         List<String> ff8Fighters = List.of("Squall", "Zell", "Quistis");
7         int searchIndex = Collections.binarySearch(ff8Fighters, "Quistis");
8         System.out.println(ff8Fighters.get(searchIndex));
9     }
10 }
```

Output:

Exception in thread "main" java.lang.IndexOutOfBoundsException: Index -1 out of bounds for length 3

As you can see in the code above, we got an Exception because we didn't sort our array. Therefore, the search result is inconsistent and it might not work properly.

12.17.2 Searching elements with a sorted List

As mentioned before, the `binarySearch` method is supposed to work with a sorted list. Therefore, by sorting the list before invoking the `binarySearch` method then we will find the element consistently.

Let's see a code example when we sort the List first:

```

1  // Omitted imports
2  public class BinarySearchSorted {
3      public static void main(String[] args) {
4          List<String> ff8Fighters =
5              new ArrayList<>(List.of("Squall", "Zell", "Quistis"));           // #A
6          Collections.sort(ff8Fighters);                                     // #B
7          int searchIndex = Collections.binarySearch(ff8Fighters, "Quistis"); // #C
8          System.out.println(ff8Fighters.get(searchIndex));
9      }
10 }

```

Output:

Quistis

- #A: Notice that the `List.of` method returns an immutable list which means that we can't change the list. That's why we are wrapping the result of `List.of` into the constructor of an `ArrayList`. Then we can change our array.
- #B: We pass our mutable array to the `Collections.sort` method. Notice that the elements will be sorted in the object reference which means that we don't need to return anything, the sort method will change the `ff8Fighters` list and sort the elements.
- #C: We invoke the `binarySearch` now with the list already sorted and we retrieve the `searchIndex` successfully.

12.17.3 Marvel Binary Search Challenge

The the following Java code challenge, we will explore how the `binarySearch` method works in Java when an element is found and not found.

Can you guess what will happen when running the following code?

```

1  import java.util.Arrays;
2
3  public class MarvelSearchChallenge {
4
5      static String[] marvel = {"Spider-man", "Venom", "Carnage", "Mysterio"};
6
7      public static void main(String[] args) {
8          Arrays.sort(marvel);
9
10         System.out.print(Arrays.binarySearch(marvel, "Xavier") + " ");
11         System.out.print(marvel[Arrays.binarySearch(marvel, "Carnage")] + " ");
12         System.out.print(Arrays.binarySearch(marvel, "Lizard") + " ");
13         System.out.print(Arrays.binarySearch(marvel, "Spider-man"));

```



```
14     }  
15 }
```

- A. -1 Carnage -1 0
- B. -4 2 -1 0
- C. -5 Carnage -2 2
- D. -4 Carnage -1 2

Explanation:

The first important point of this challenge is that the array must be sorted. If it wasn't sorted, we would have the `ArrayIndexOutOfBoundsException`.

Therefore, the array is like this at this point:

```
[Carnage, Mysterio, Spider-man, Venom]
```

So, when we look for `Xavier`, it returns `-5`. That's because `Xavier` is compared with the other elements in natural order. Since `Xavier` comes after all elements alphabetically, it's therefore the fifth element. Notice also that when the element is not found, it doesn't start counting from `0`. It starts from `-1` instead.

Then, we find `Carnage` in the array and we get back the index that is `0`. When we access this index with the `marvel` array, `Carnage` will be printed.

When we try to look for `Lizard`, it returns `-2`. That's because `Lizard` would come after `Carnage`, therefore, the second element.

Finally, we retrieve the index from `Spider-man`, which in our sorted array is the third position. Therefore, it translates to index `2`.

In conclusion, the final answer of this Java Challenge is:

C) -5 Carnage -2 2

12.18 Summary

In this chapter we explored the most important concepts in the `Collections` api to enable you to manipulate array, set, map, queue and stack.

In every real-world system you will have to deal with data all the time, therefore, by knowing the main `Collection` objects in Java will massively help you to create high-quality code and not reinvent the wheel.

In this chapter you learned to use:

- `equals` and `hashCode` methods to check if objects have the same values or not.

- List to use as a type of `ArrayList`, `Vector`, `LinkedList` and `CopyOnWriteArrayList`.

- `ArrayList` for most common use of arrays. Stores an array of objects behind the scenes and it's going to grow the array dynamically.
 - `Vector` when you want your array to have synchronized data in a multi-thread environment.
 - `CopyOnWriteArrayList` when you want to manipulate the array as you wish while traversing it. You won't get `ConcurrentModificationException`.
 - Prevent `ConcurrentModificationException` with effective techniques.
 - Comparing and sorting data from a `List`.
 - `Deque` as a type to use functionalities of a `Stack` and `Queue`.
 - `LinkedList` when you want to make use of `Stack` and `Queue` functionalities.
-
- `Set` to be used as a type for `HashSet`, and `LinkedHashSet`, and `TreeSet`. Also will make elements unique.
 - `HashSet`, it is the most basic implementation of a `Set`. It uses a map behind the scenes to store data. Also inserts unique elements and the elements are not ordered as they are inserted.
 - `LinkedHashSet`, it is very similar to `HashSet` but has the difference that elements will keep the order as they are inserted.
 - `TreeSet`, it inserts unique elements and also sorts them depending on what you configured on your `compareTo` method.
-
- `Map`, used to implement key-value data structures.
 - `HashMap` is the most common implementation of a `Map`. It stores key-values, it accepts `null` as a key and elements are not ordered.
 - `LinkedHashMap` is similar to `HashMap` but elements are ordered as they are inserted.
 - `Hashtable`, it is synchronized which means it can be used in a thread-safe environment and can't use a `null` value as a key.
 - `TreeMap` will use the `compareTo` method to sort elements as they are inserted.
-
- Binary search methods to find elements in a sorted array.

13 Streams

Streams will help you to manipulate data of a `Collection` in a functional and fluid way. You will be able to drastically reduce code if you use streams effectively.

Streams is one of the big features from Java 8, it's not so easy to master it but if we know the most important functionalities of a stream we will be able to be very dangerous.

That's the goal of this chapter, to explore the most used features of a stream so you can use it in your day-to-day work.

// Explain that streams behind the scenes is a pipeline

// Explain that streams are lazy

Creating a Stream

Creating a stream from a List

The Java Collection objects are all prepared to be converted to a stream. In the following code, we can see that we can easily convert a list to a stream by invoking the `stream` method:

```
1 import java.util.List;
2 import java.util.stream.Stream;
3
4 List<String> mavericks = List.of("X ", "Zero ", "Axl ", "Protoman ");
5 Stream<String> maverickStream = mavericks.stream();
6 maverickStream.forEach(System.out::print);
```

Output:

X Zero Axl Protoman

Creating a stream from an Array

Streams are versatile and flexible. We can convert an array to a stream by using the `Stream.of` method:

```

1 // Omitted imports
2 String[] mavericks = new String[]{"X ", "Zero ", "Axl ", "Protoman "};
3 Stream<String> maverickStream = Stream.of(mavericks);
4 maverickStream.forEach(System.out::print);

```

Output:

X Zero Axl Protoman

Alternatively, we can also use the `Arrays.stream` method to have the same output as above:

```

1 // Omitted imports and array creation
2 Stream<String> maverickStream = Stream.of(mavericks);
3 maverickStream.forEach(System.out::print);

```

Creating an empty Stream

When we don't want to return a null stream, we can use the `Stream.empty()` method as the following:

```

1 Stream<String> stream = Stream.empty();

```

Creating a Stream with a Builder

To create streams without the need of a `Collection` or array, we can build our stream in the way we want by using the `Stream.builder` method:

```

1 Stream<String> maverickStream =
2     Stream.<String>builder().add("X ").add("Zero ").add("Axl ").build();
3 maverickStream.forEach(System.out::print);

```

Output:

X Zero Axl

Creating a Stream with generate

It's possible to generate an infinite stream with the `generate` method:

```

1 Stream<String> maverickStream =
2     Stream.generate(() -> "X ");
3 maverickStream.forEach(System.out::print);

```

Output:

XXX...

As you can see, the code above prints X forever and that doesn't make much sense. To make the `generate` method more useful, we need to limit the stream to not print elements forever:

```
1 Stream<String> maverickStream =  
2     Stream.generate(() -> "X ").limit(5);  
3 maverickStream.forEach(System.out::print);
```

Output:

X X X X X

Creating a Stream with iterate

To iterate over numbers we want, we can use the `iterate` method. Notice that we have `0` as the initial number, a lambda function adding `1` to each iteration and the `limit` method to avoid adding elements to the infinity:

```
1 Stream<Integer> iterateStream = Stream.iterate(0, n -> n + 1).limit(10);  
2 iterateStream.forEach(System.out::print);
```

Output:

0123456789

Creating a stream for int, long, and double

To avoid auto-boxing primitive elements to wrappers when using a `Stream<T>` we can use specific stream classes `IntStream`, `LongStream`, and `DoubleStream`.

The `IntStream` and `LongStream` classes are also useful because they have the `range` and `rangeClosed` methods which allows us to create a stream within a specific range of numbers.

Let's see how to use the `range` method, notice that the last argument `5` is exclusive:

```
1 IntStream intStream = IntStream.range(1, 5);  
2 intStream.forEach(System.out::print);
```

Output:

1234

If we want the second argument of the `range` method to be inclusive, we have the `rangeClosed` method to do that:

```
1 IntStream intStream = IntStream.rangeClosed(1, 5);  
2 intStream.forEach(System.out::print);
```

Output:

12345

We can also use `DoubleStream` to generate double numbers as we want:

```
1 DoubleStream.iterate(0.0, i -> i + 1.5).limit(3)
2     .forEach(n -> System.out.print(n + " "));
```

Output:

0.0 1.5 3.0

Creating a stream of a String

There is an interesting concept when we are working with String that is tokenization. Tokenization is useful to separate a String in a determined pattern. We can break a String by a , for example or any other character we wish.

In the following example we will use the `Pattern.compile` method to configure the pattern we want to tokenize. Then, we are invoking the `splitAsStream` method, therefore we will split the String `X, Zero, Axl, Sigma` by the `,`. Then we will have access to each element in separate Strings.

```
1 Pattern.compile(",").splitAsStream("X, Zero, Axl, Sigma")
2     .forEach(System.out::print);
```

Output:

X Zero Axl Sigma

Generating a stream with random numbers

Intermediate Operations

When we are working with streams, we can manipulate the data by using the intermediate operations. We can peek, limit, skip, map, sort and much more.

Let's see how those methods work in practice.

filter()

As the name suggests, the `filter` method will filter the elements of the stream accordingly. We can filter our elements in the way we want by having a data condition.

Let's see some examples to make it clear. Let's see the class we will work on:

```

1 class Maverick {
2     private String name;
3     private double power;
4
5     // Omitted constructor initializing name and power
6     // Omitted toString method
7 }

```

Now, let's see an example where we will filter the mavericks who have the power greater than 8:

```

1 var mavericks = List.of(new Maverick("X", 9), new Maverick("Zero", 10),
2                        new Maverick("Axl", 8));
3 mavericks.stream()
4     .filter(m -> m.power > 8)
5     .forEach(m -> System.out.println(m.name));

```

Output:

X
Zero

We can also filter elements of a stream with different conditions. We can filter by name and power, let's see how that works in code:

```

1 mavericks.stream()
2     .filter(m -> m.name.equals("X") && m.power > 8)
3     .forEach(m -> System.out.println(m.name));

```

Output:

X

One important detail to notice here is that when we use the `filter` method, we are not changing the real mavericks array. We are only filtering it within the stream but there will be no changes in the real array. Therefore, if we print our mavericks array, it will have the same information as before:

```

1 System.out.println(mavericks);

```

Output:

[X, Zero, Axl]

map operations

When we want to transform data in a stream, we can use the `map` method. We can transform data to another type, we can add numbers, concatenate Strings, we can do many things.

Let's see an example when we want to concatenate another String for each element:

```
1 List<String> bojackCharacters = List.of("Bojack", "Diane", "Todd");
2
3 bojackCharacters.stream()
4     .map(b -> b.concat(" and Mr. Peanutbutter"))
5     .forEach(System.out::println);
```

Output:

Bojack and Mr. Peanutbutter
Diane and Mr. Peanutbutter
Todd and Mr. Peanutbutter

Using map to convert data

It's also possible to convert a `String` to `int` if we want to. Then we can manipulate the `int` value in the stream.

The `map` method receives a `Function` and returns a `Stream` as you can see on its method signature:

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper);
```

In the following code, we will create a stream from a list, then we will map it to the length of the `String`, will map to the `Strings` with 5 or more characters, finally we will print the data.

Let's see the code then:

```
1 List<String> bojackCharacters = List.of("Bojack", "Diane", "Todd");
2
3 bojackCharacters.stream()
4     .map(b -> b.length())
5     .filter(l -> l >= 5)
6     .forEach(System.out::println);
```

Notice that in the code above we used a lambda expression instead of a method reference. To simplify the code above, we can use method reference:

```
1 // Same code as above with a method reference:
2     .map(String::length)
```

Output:

6
5

We can also map a `String` to uppercase:


```
1 List<String> bojackCharacters = List.of("Bojack", "Diane", "Todd");
2 bojackCharacters.stream()
3     .map(String::toUpperCase)
4     .forEach(bojackCharacter -> System.out.print(bojackCharacter + " "));
```

Output:

BOJACK DIANE TODD

As you can see we can transform data as we wish. However, it's important to see how the most common examples work, we can sum, multiply, divide, subtract, get the mod number... But to be practical let's see only the multiply operation in code because the other operations will be very similar:

```
1 List<Integer> numbers = List.of(1, 2, 3, 4);
2
3 numbers.stream()
4     .map(n -> n * 2)
5     .forEach(n -> System.out.print(n + " "));
```

Output:

2 4 6 8

distinct

Simply put, the `distinct` function will remove duplicate elements from the stream. Let's see how that works in code:

```
1 List<Integer> numbers = List.of(1, 1, 1, 2, 2, 3, 3, 4, 4, 4, 4);
2 numbers.stream().distinct().forEach(n -> System.out.print(n + " "));
```

Output:

1 2 3 4

Let's see now how the `distinct` method will behave when we create our own object:

```

1  class BojackCharacter {
2      private String name;
3
4      public BojackCharacter(String name) {
5          this.name = name;
6      }
7  }

```

Let's see what happens when we use the above object in the distinct function:

```

1  List<BojackCharacter> bojackCharacters = List.of(
2      new BojackCharacter("Bojack"), new BojackCharacter("Bojack"),
3      new BojackCharacter("Diane"), new BojackCharacter("Diane"));
4
5  bojackCharacters.stream().distinct()
6      .forEach(b -> System.out.print(b.name + " "));

```

Output:

Bojack Bojack Diane Diane

Notice that the elements didn't get distincted. That's because the distinct function relies on the equals and hashCode methods. Therefore, if we implement those methods in the BojackCharacter class, then the elements will be distincted:

```

1  class BojackCharacter {
2      // Omitted field name and constructor
3
4      @Override
5      public boolean equals(Object o) {
6          if (this == o) return true;
7          if (o == null || getClass() != o.getClass()) return false;
8          BojackCharacter that = (BojackCharacter) o;
9          return Objects.equals(name, that.name);
10     }
11
12     @Override
13     public int hashCode() {
14         return Objects.hash(name);
15     }
16 }

```

Now, if we try to run the same code as above, then the elements will be distincted:

```
1 // Omitted List creation
2 bojackCharacters.stream().distinct()
3     .forEach(b -> System.out.print(b.name + " "));
```

Output:

Bojack Diane

Debugging a stream with peek() and IDEs

The peek function is used for debugging what is going on in a stream. It's also possible to debug a stream by adding a breakpoint where you wish. This works for Eclipse and Netbeans. However, if you want a more sophisticated experience to debug streams, you can also use the **Java Stream Debugger** plugin from IntelliJ.

But worry not, the peek method will do the work very well, so you don't really need a sophisticated tool for debugging your streams.

The peek method will simply log data when you need, this means that it will show the data you want as the stream goes on. Let's see how to use the peek method then:

```
1 List<String> bojackList = List.of("Bojack", "Diane", "Todd");
2 bojackList.stream().peek(System.out::println);
```

Output:

Nothing will be printed here...

Why is that? That's because the peek method is an intermediate operation method, this means that nothing will happen unless a terminal operation method is invoked.

It's important to remember that streams are lazy, therefore, something will only happen when there is a method that will effectively do something with the stream.

```
1 List<String> bojackCharacters = Stream.of("BOJACK", "DIANE", "TODD")
2     .map(String::toLowerCase)
3     .peek(e -> System.out.println("Lower Case Elements: " + e))
4     .filter(e -> e.equals("bojack"))
5     .peek(e -> System.out.println("Filtered to bojack: " + e))
6     .collect(Collectors.toList());
7
8 System.out.println(bojackCharacters);
```

Output:

Lower Case Elements: bojack
Filtered to bojack: bojack

Lower Case Elements: diane

Lower Case Elements: todd

[bojack]

Code analysis:

Notice that the `map` method will transform the `String` to lowercase and then will print the information of the current element in the stream pipeline:

Lower Case Elements: bojack

Then, the `filter` method will be invoked filtering a `String` that is equals to `bojack`. In that case, this condition will be true, therefore, the following will be printed:

Filtered to bojack: bojack

The `String` `DIANE` will also be transformed to lowercase. Therefore, the `peek` method will print the following:

Lower Case Elements: diane

However, notice that `diane` will not be equals to `bojack`, therefore, the next `peek` method will have nothing to print. The same will happen to `TODD` and the following will be printed:

Lower Case Elements: todd

Summary of the peek method

As we could see in the code above, the `peek` method will show the value of the current element passing by the stream pipeline. If the element is not present at the point the `peek` method was invoked, then nothing will be printed as we just saw.

If you are unsure about what is going on in your stream pipeline, you can use the `peek` method to check the current pipeline value.

limit()

To limit how many elements we want to see in a stream, we can use the `limit` method.

```
1 Stream.of("Bojack", "Peanutbutter", "Todd", "Diane")
2     .limit(2)
3     .forEach(System.out::println);
```

Output:

Bojack

Peanutbutter

As you can see in the code above, the `limit` method will limit the stream to two elements.

skip()

The skip method will skip as many elements as you want in a stream. You can use skip with the filter method as you can see in the following code:

```
1 Stream.of("Bojack", "Todd", "Peanutbutter", "Princess Carolyn", "Diane")
2     .filter(e -> e.length() > 4).        // #A
3     .skip(2).                             // #B
4     .forEach(e -> System.out.print(e + " "));
```

Output:

Princess Carolyn Diane

- #A: Filtering only the Strings that have more than 4 characters.
- #B: Skips 2 elements that fulfill the filter condition. In this case Bojack and Peanutbutter are skipped.

sorted()

To easily sort elements in a stream you can use the sorted method. Just remember that the object you are using have to implement Comparable so the sorted method can work.

In the following code, we will use a String that already implements Comparable. Therefore, when running the following code, the elements will be sorted in the natural order:

```
1 List<String> zodiacKnights = Arrays.asList("Shaka", "Kanon", "Aldebaran",           \
2                                           "Seya", "Ikki");
3 zodiacKnights.stream().sorted().forEach(e -> System.out.println(e + " "));
```

Output:

Aldebaran Ikki Kanon Seya Shaka

reverseOrder

To reverse the order when using the sorted method in a stream, we can pass Comparator.reverseOrder() in the sorted method.

Also, remember that the Comparator.reverseOrder() will only work if your class implements Comparable:

```

1 List<String> zodiacKnights = Arrays.asList("Aldebaran", "Seya", "Shaka", \
2                                           "Ikki", "Kanon");
3 zodiacKnights.stream().sorted(Comparator.reverseOrder())
4               .forEach(e -> System.out.print(e + " "));

```

Output:

Shaka Seya Kanon Ikki Aldebaran

flatMap operations**Terminal Operations**

To finalize the stream pipeline and perform an action with the stream data, we can use the terminal operation methods. Those are methods that will find, match, count, reduce, collect, group data so you do something meaningful for your application.

An action will only happen in a stream if a terminal operation method is present. Therefore, a stream without a terminal operation method doesn't make sense.

findAny()

The `findAny` method will find any element in the stream, without a particular order:

```

1 List<String> metalGearCharacters = Arrays.asList("Snake", "Ocelot", "Cipher");
2 Optional<String> anyMetalGearCharacter = metalGearCharacters.stream()
3                                           .findAny();
4 System.out.println(anyMetalGearCharacter.orElse(""));

```

Output:

It's not deterministic, it may vary since any element can be found. However, the output will be mostly the first element since we are not working with parallel streams.

findFirst()

As the name suggests, the `findFirst` method will find the first element within what was instructed in the stream pipeline:

```

1 List<String> metalGearCharacters = Arrays.asList("Snake", "Ocelot", "Cipher");
2 Optional<String> firstCharacter = metalGearCharacters.stream().findFirst();
3 System.out.println(firstCharacter.get());

```

Output:

Snake

If we perform operations to filter data in the stream, then the elements will be filtered first and then the first element will be found:

```

1 List<String> metalGearCharacters = Arrays.asList("Snake", "Ocelot", "Otacon");
2 Optional<String> firstCharacter = metalGearCharacters.stream()
3     .filter(e -> e.startsWith("O")).findFirst();
4 System.out.println(firstCharacter.orElse(""));

```

Output:

Ocelot



parallel and findFirst: Notice that if we are working with a parallel stream that are not ordered, the `findFirst` method will still make sure the first element will be retrieved.

parallel

When you need to process large amount of data, using parallel streams might be a good idea. By doing that, you can use multiple processor cores of your machine to process data more quickly. Just remember that if you try to use a `parallel` stream when you are processing a small amount of data, you will lose in performance. That happens because it's costly to manage threads.

Another important detail to keep in mind is that when you use `parallel` streams, the order won't be guaranteed.

Let's see a `parallel` stream in code:

```

1 IntStream numbersStream = IntStream.rangeClosed(0, 15);
2 numbersStream.parallel().forEach(e -> System.out.print(e + " "));

```

Output (Random order):

15 2 5 10 1 8 11 7 0 12 9 14 3 13 4 6

Non-parallel Stream Thread Names

You can see the name of the main thread when you use a non-parallel stream. Notice that only the main thread will be used:

```

1 List<String> letters = Arrays.asList("a", "b", "c", "d", "e");
2 letters.forEach(letter -> System.out.printf("%s Thread: %s ", letter,
3                                           Thread.currentThread().getName()));

```

Output:

a Thread: main b Thread: main c Thread: main d Thread: main e Thread: main

Parallel Stream Thread names

If you use a parallel stream, you will see different threads being used:

```

1 List<String> letters = Arrays.asList("a", "b", "c", "d", "e", "f", "g", "h");
2 letters.parallelStream().forEach(letter -> System.out.printf("%s Thread: %s ", \
3                                           letter, Thread.currentThread().getName()));

```

Output (Random order):

f Thread: main d Thread: ForkJoinPool.commonPool-worker-7 b Thread: ForkJoinPool.commonPool-worker-3 a Thread: ForkJoinPool.commonPool-worker-6 g Thread: ForkJoinPool.commonPool-worker-5 e Thread: ForkJoinPool.commonPool-worker-4 h Thread: ForkJoinPool.commonPool-worker-2 c Thread: ForkJoinPool.commonPool-worker-1

As you can see in the output of the code above, when we are working with parallel streams, we can see different threads processing data.

anyMatch()

The `anyMatch` method will check if any of the stream elements match the condition you want. It returns `true` if the condition is fulfilled:

```

1 Stream<String> knights = Stream.of("Milo", "Aiolos", "Shiryu", "Ikki");
2 boolean anyMatch = knights.anyMatch(s -> s.startsWith("S"));
3 System.out.println(anyMatch);

```

Output:

true

Since Shiryu starts with 'S', the `anyMatch` condition will be fulfilled, therefore, it will return `true`.

allMatch()

The `allMatch` method will compare all elements values of the stream. Therefore, all values need to fulfill the condition:


```
1 Stream<String> knights = Stream.of("Saori", "Shaka", "Shiryu", "Seya");
2 boolean allMatch = knights.allMatch(s -> s.startsWith("S"));
3 System.out.println(allMatch);
```

Output:

true

Because all names from the stream knights start with S, the code above returns true. However, if we change one name of the stream the output will be false:

```
1 Stream<String> knights = Stream.of("Ikki", "Shaka", "Shiryu", "Seya");
2 boolean allMatch = knights.allMatch(s -> s.startsWith("S"));
3 System.out.println(allMatch);
```

Output:

false

noneMatch()

The noneMatch method will check if the condition doesn't match all of the elements. It's pretty much the opposite from the allMatch method:

```
1 Stream<String> knights = Stream.of("Saori", "Shaka", "Shiryu", "Seya");
2 boolean noneMatch = knights.noneMatch(s -> s.startsWith("B"));
3 System.out.println(noneMatch);
```

Output:

true

Since none of the above elements start with B, the noneMatch method will return true. If we pass S instead, the output will be false:

```
1 Stream<String> knights = Stream.of("Shun", "Shaka", "Shiryu", "Seya");
2 boolean noneMatch = knights.noneMatch(s -> s.startsWith("S"));
3 System.out.println(noneMatch);
```

Output:

false

count()

As the name suggests, the count method will count how many elements the stream has after the pipeline process is finished. In the following example, we will have an IntStream with numbers from 1 to 7 and we will filter it to only odd numbers. In the end we will count. Let's see what is the result:

```
1 var oddCount = IntStream.of(1, 2, 3, 4, 5, 6, 7)
2     .filter(i -> i % 2 == 1)
3     .count();
4
5 System.out.println(oddCount);
```

Output:

4

Since the number 1, 3, 5, and 7 are odd, the count result will be 4.

It's also possible to count String elements:

```
1 var saiyanCount = Stream.of("Goku", "Frieza", "Goten", "Gohan")
2     .filter(e -> e.startsWith("Go"))
3     .count();
4
5 System.out.println(saiyanCount);
```

Output:

3

The output is 3 because we are filtering the elements that starts with Go.

min()

The min method of the stream will find the minimum element based on the Comparator rule passed on it. Let's see an example:

```
1 List<Integer> numbers = Arrays.asList(40, 10, 1, 5, 7, 99);
2 Optional<Integer> min = numbers.stream()
3     .min((i, j) -> i.compareTo(j));
4
5 System.out.println(min.get());
```

// Continue here

max()

reduce

Transforming Streams

Converting List to a Map

Converting Map to a List

collect()

Collectors.groupingBy

Average per Group

Sum per Group

Processing Order

Parallel Stream

Summary

13 Newest Features of Java (In Development)

13.1 Introduction to Modules Java 9

13.2 Reserved word var from Java 10

13.3 New switch case statement from Java 14

13.4 Using record from Java 14

13.5 Using Sealed classes Java 15

13.6 Using Pattern Matching Type Checks Java 15

13.7 Text Blocks Java 15

13.8 Summary